

INÍCIO DE SPRINT: Módulo de Reconhecimento e Auditoria (GovSec Scanner)

CONTEXTO DA TAREFA

Vamos iniciar o desenvolvimento prático do módulo de "Asset Discovery & Port Scanning" em Python. Este será o componente fundacional do GovSec Shield, responsável por mapear a infraestrutura da Prefeitura de Betim, identificar portas abertas, coletar banners e gerar dados estruturados que futuramente alimentarão o Zabbix e o Wazuh.

REQUISITOS TÉCNICOS E ARQUITETURAIS

Atuando estritamente sob as regras do "GovSec-AI", sua tarefa é desenhar a arquitetura base deste módulo. O código deve ser de nível Sênior/Enterprise.

1. **Stack:** Python 3.10+, `asyncio` (para concorrência de rede de alta performance), e `pydantic` (para validação rigorosa de dados e configurações).
2. **Design Patterns (SOLID):** O sistema deve ser altamente desacoplado. Crie uma estrutura de diretórios clara que separe:
 - **Core/Config:** Validação de escopo (ex: garantir que só podemos escanear IPs permitidos, evitando varreduras acidentais fora da prefeitura).
 - **Scanners:** A lógica de varredura TCP/UDP.
 - **Analyzers:** A lógica que interpreta o banner/porta e avalia o risco.
 - **Exporters:** Módulos que enviam os resultados (inicialmente para JSON, no futuro para APIs externas).

DIRETRIZ DE SEGURANÇA (Scope Safety)

Implemente obrigatoriamente um mecanismo de "Whitelist de Sub-redes" (CIDR block validation). O scanner deve se recusar a executar se o IP alvo não estiver dentro das sub-redes autorizadas da prefeitura.

ENTREGÁVEIS ESPERADOS NESTA RESPOSTA

Execute o seu "Problem-Solving Methodology" (Análise, Diagnóstico, Plano, Execução e Validação) e entregue:

1. A **Estrutura de Diretórios** sugerida para o projeto (formato tree).
2. O arquivo `requirements.txt` (ou `pyproject.toml`) inicial.

3. O código do arquivo `config.py` ou `core/security.py`, utilizando `pydantic` para validar se um IP alvo está dentro do escopo permitido (Scope Safety).
4. Uma classe abstrata base (`BaseScanner`) no diretório de scanners, utilizando Type Hints estritos e docstrings detalhadas.

Manifesto Arquitetural e Modelagem de Domínio: GovSec Shield (Security OS)

1. Visão e Paradigma: Security Operating System

O GovSec Shield não é um SIEM acoplado a uma IA. É um **Sistema Operacional de Segurança (Security OS)**. As ferramentas externas (Zabbix, Wazuh, Caldera) atuam apenas como *drivers* ou *adaptadores*. O verdadeiro núcleo é a plataforma, que centraliza o domínio, as regras de negócio, a orquestração e a governança. As ferramentas periféricas podem ser substituídas no futuro sem alterar a essência do sistema.

1.1. Metodologia: Adaptive Security Evolution (ASE) via Capacidades

A evolução do sistema não ocorre por "fases" temporais, mas por **Níveis de Maturidade (Capacidades)**. A inteligência e automação só existem sobre uma base de observabilidade estrita:

- **M0 - Arquitetura:** Modelagem de domínio, ADRs, Bounded Contexts. (*Nível Atual*)
- **M1 - Observabilidade:** Coleta de logs, métricas e tracing (OTel).
- **M2 - Monitoramento:** Detecção de anomalias e falhas de infraestrutura.
- **M3 - Correlação:** União de eventos dispersos em incidentes coesos.
- **M4 - Threat Intelligence:** Enriquecimento de incidentes (CVE, EPSS, MITRE).
- **M5 - Assistente IA (Read-Only):** IA interpreta alertas e sugere contexto.
- **M6 - Copiloto:** IA escreve *Commands* e *Playbooks* para aprovação humana.
- **M7 - Automação:** Execução automática de playbooks determinísticos.
- **M8 - Red Team:** Simulação contínua e autônoma de ataques (Caldera/Atomic).
- **M9 - Autonomia:** IA executa respostas ativas reversíveis sob supervisão (Policy Engine).

2. Bounded Contexts (Arquitetura Desacoplada)

O ecossistema é estritamente modular. Nenhum módulo conhece a implementação interna do outro.

- **Core Platform (O Orquestrador):** Conhece apenas o domínio genérico (`SecurityEvent`, `AssetHealthChanged`). Não sabe o que é Wazuh ou Zabbix. Gerencia o fluxo de estados.

- **Security Kernel:** O coração da segurança interna. Centraliza autorização, políticas (OPA/Cedar), criptografia, MFA (FIDO2), rotação de chaves e Vault de *secrets*. Todos os módulos o consultam.
- **Observability:** Diferente de monitoramento. Centraliza Metrics, Logs, Tracing, Health e Telemetry de toda a plataforma (usando OpenTelemetry).
- **Risk Engine:** Motor puramente determinístico. Calcula scores (CVSS, EPSS), impacto e criticidade de ativos. A IA *não* calcula risco, apenas o interpreta.
- **Knowledge Base:** A memória institucional do sistema (Vector DB). Guarda Runbooks, Playbooks, CVEs, decisões arquiteturais (ADRs) e histórico de aprendizado.
- **AI Gateway & Services:** O cérebro modular. Dividido em *Agent Orchestrator*, *Tool Manager*, *Model Router* e LLMs. Roteia requisições dinamicamente (Llama, Gemini, Claude).
- **Connectors (Adapters):** A camada Anti-Corruption (ACL). Ouve as ferramentas específicas (ex: webhook do Zabbix) e traduz para eventos agnósticos do barramento.
- **Edge Layer (API Gateway):** O porteiro. Roteamento, Rate Limit, JWT, OAuth2, RBAC, Cache e auditoria de borda.

3. Padrão de Comunicação (CQRS)

O sistema evita gargalos separando as intenções em barramentos distintos:

- **Command Bus:** Para intenções de mudança de estado (ex: `IsolarHostCommand`).
- **Query Bus:** Para consultas diretas e síncronas (ex: `GetAssetVulnerabilitiesQuery`).
- **Event Bus:** Para reações assíncronas (ex: `IncidentCreatedEvent`).

Regra de Ouro da IA: A IA **nunca** altera o estado do banco. O fluxo obrigatório é: IA -> Gera `Suggestion` -> Converte em `Command` -> `Core` -> Validação (`Policy Engine`) -> Execução -> Gera `Event` -> `Persistence`.

4. Estratégia de Dados (Ops vs. Analytics)

Separação clara entre o estado atual e o histórico massivo:

- **Operational DB (PostgreSQL):** Guarda estado transacional relacional (Ativos, Status de Incidentes, Usuários, Permissões).
- **Analytical DB (OpenSearch / ClickHouse):** Data Lake imutável para telemetria, trilhas de auditoria massivas e eventos brutos.

5. Diagrama C4 (Nível 2: Containers)

C4Context

title Diagrama de Containers (Nível 2) - GovSec Shield (Security OS)

Person(analyst, "Analista SOC", "Opera o dashboard, audita IA e aprova ações críticas.")

System_Boundary(govsec, "GovSec Shield Platform") {
 Container(edge, "Edge Layer (API Gateway)", "Kong / Envoy", "Porteiro da plataforma: Rate Limit, JWT, OPA, Auditoria.")

 Container(frontend, "Dashboard SOC", "React, Tailwind", "Interface única de comando e visualização.")

 Container(core, "Core Platform", "Python, FastAPI", "Regras de negócio, orquestração e máquina de estados.")

 Container(sec_kernel, "Security Kernel", "Go / Rust", "Auth, OPA/Cedar, Vault, Criptografia, Tokens.")

 Container(risk_engine, "Risk Engine", "Python", "Calculador determinístico de CVSS, EPSS e criticidade.")

 Container(ai_gateway, "AI Gateway", "Python, LiteLLM", "Orquestrador de agentes, MCP e roteamento de LLMs.")

 Container(obs, "Observability", "OpenTelemetry", "Métricas, Tracing e Health do ecossistema.")

 ContainerDb(db_ops, "Operational DB", "PostgreSQL", "Transações: Ativos, Incidentes, Roles.")

 ContainerDb(db_analytics, "Analytical DB", "ClickHouse / OpenSearch", "Telemetria massiva e auditoria.")

 ContainerDb(knowledge, "Knowledge Base", "Vector DB (Milvus/Qdrant)", "Memória, Playbooks, ADRs, MITRE.")

 Container(buses, "Message Buses", "Kafka / RabbitMQ", "Command Bus, Query Bus e Event Bus (CQRS).")

 Container(connectors, "Connectors / Adapters", "Python", "ACL: Traduz ferramentas em eventos padronizados.")

}

System_Ext(zabbix, "Zabbix", "Driver de monitoramento infra.")

System_Ext(wazuh, "Wazuh", "Driver de XDR/SIEM.")

System_Ext(caldera, "MITRE Caldera", "Driver de emulação de adversários.")

Rel(analyst, edge, "Acessa via", "HTTPS")

Rel(edge, frontend, "Serve")

Rel(edge, core, "Roteia Commands/Queries", "REST/gRPC")

Rel(edge, sec_kernel, "Valida Permissões", "gRPC")

Rel(core, buses, "Publica Commands / Lê Events")

Rel(core, db_ops, "Lê/Escreve Estado", "SQL")

Rel(core, risk_engine, "Solicita Score")

Rel(ai_gateway, buses, "Lê Events / Envia Suggestions (Commands)")
Rel(ai_gateway, knowledge, "RAG e Memória", "gRPC")

Rel(connectors, buses, "Publica SecurityEvent / InfraEvent")
Rel(zabbix, connectors, "Webhooks/API")
Rel(wazuh, connectors, "Webhooks/API")
Rel(ai_gateway, caldera, "Executa via Tool Manager", "REST")

Rel(core, db_analytics, "Gera trilha de auditoria")
Rel(obs, db_analytics, "Armazena telemetria")

GovSec Shield: Especificação de Domínio e Modelo Canônico (M0.1 & M0.2)

Este documento é a Constituição Técnica da plataforma GovSec Shield. Ele define os princípios irrevogáveis, a Linguagem Ubíqua rigorosa, os modelos canônicos e os contratos que devem ser respeitados por todos os desenvolvedores, agentes de IA e módulos do ecossistema.

1. Princípios Fundamentais do Domínio

Estes princípios são obrigatórios e prevalecem sobre qualquer decisão de implementação técnica.

- **1.1 Domain First:** O domínio governa toda a plataforma. Nenhuma decisão técnica pode alterar a semântica das entidades do domínio. Frameworks, bancos de dados, modelos de IA e ferramentas externas são substituíveis; o domínio, não.
- **1.2 Event First:** Toda alteração de estado relevante deve produzir um evento. Se uma mudança não gera um evento auditável, ela não deve existir.
- **1.3 Security by Default:** Toda funcionalidade nasce assumindo um ambiente hostil. Negar acesso é o comportamento padrão. Toda permissão deve ser concedida explicitamente.
- **1.4 Zero Trust Everywhere:** Nenhum módulo confia em outro módulo por padrão. Toda comunicação interna exige autenticação, autorização e auditoria (Security Kernel).
- **1.5 AI is Advisory:** A Inteligência Artificial nunca é considerada fonte de verdade absoluta. Ela interpreta, sugere e explica, mas **nunca** altera diretamente o estado do domínio sem passar pelo fluxo de comandos e políticas.

2. Invariantes do Domínio

As regras abaixo são absolutas e **nunca** podem ser violadas sob nenhuma circunstância de software:

- Um **Incident** obrigatoriamente possui pelo menos um **SecurityEvent** associado.
- Um **Asset** nunca pode existir sem um identificador único (UUID).
- Um **Command** nunca pode ser executado sem receber autorização do Policy Engine.
- Uma **Suggestion** (IA) nunca modifica estado.
- Um **Event** (Security, Infra, Audit) nunca pode ser editado ou deletado (Append-Only).
- Um **Incident** fechado/encerrado nunca pode voltar para o estado "Novo". Se a ameaça retornar, um novo **Incident** deve ser gerado.
- Todo **Asset** deve possuir um nível de criticidade associado.
- Todo **Command** executado gera obrigatoriamente um **AuditEvent**.
- Todo **Incident** gera uma **Timeline** imutável.
- Toda **Vulnerability** pertence exatamente a um **Asset**.
- Todo **Playbook** possui versionamento estrito.
- Toda resposta automática (Automação) deve possuir um mecanismo de rollback definido.

3. Máquinas de Estado Oficiais

O ciclo de vida das entidades core segue transições rígidas e unidirecionais em seus fluxos normais:

Incident (Incidente)

Novo → Triagem → Em Investigação → Confirmado → Mitigação → Monitoramento → Encerrado

Vulnerability (Vulnerabilidade)

Detectada → Validada → Classificada → Mitigação em Andamento → Mitigada → Aceita (Risco) → Arquivada

Asset (Ativo)

Descoberto → Inventariado → Gerenciado → Monitorado → Comprometido (Opcional) → Isolado (Opcional) → Desativado

4. Ownership de Domínio

Cada entidade possui um "Dono" (Bounded Context). Apenas o módulo Dono tem permissão de escrita sobre a entidade no banco de dados. Os demais módulos interagem via Events ou Commands.

Entidade	Bounded Context Dono (Owner)
Asset	Asset Discovery / Core
Incident	Core Platform
Risk Score	Risk Engine
Suggestion	AI Services / AI Gateway
Playbook	Knowledge Base
Command	Core Platform
Event (Sec/Infra)	Connectors (Produtores) / Observability (Armazenamento)
Audit Log	Security Kernel / Observability

5. Identificadores Globais e Taxonomia Canônica (Canonical Taxonomy)

O sistema inteiro utiliza uma única taxonomia. Ferramentas externas (Connectors) devem traduzir seus dados proprietários para este modelo.

- **Identificadores (Regra de Ouro):** Toda entidade possui um **UUID v4**. **Nunca** utilizar Hostname, IP ou MAC como chaves primárias ou relacionais.
- **Severidade (Severity):** **INFO**, **LOW**, **MEDIUM**, **HIGH**, **CRITICAL**. (Nunca usar "Sev1", "Error", "Level 12").

- **Criticidade (Criticality):** BAIXA, MÉDIA, ALTA, MISSÃO_CRÍTICA.
- **Ambiente (Environment):** PRODUÇÃO, HOMOLOGAÇÃO, DESENVOLVIMENTO, LABORATÓRIO.
- **Confiança da IA (Confidence):** Valor percentual 0% a 100%.

6. Família Canonical Security Model (CSM)

Todos os barramentos operam sobre estruturas canônicas padronizadas.

- **CanonicalEvent:** EventId, Timestamp, Type, Source, CanonicalSeverity, OriginalPayload.
- **CanonicalAsset:** AssetId (UUID), Hostname, IP, MAC, Owner, Criticality, Environment, Tags, Status.
- **CanonicalUser:** UserId (UUID), Identity, Roles, Department, MFA_Status, TrustLevel.
- **CanonicalIncident:** IncidentId, Severity, RiskScore, MITRE_Tags, Status, Timeline, Owner, Assets_Involved.
- **CanonicalCommand:** CommandId, Author, Target, Action, Reason, ApprovalStatus, Rollback_Playbook.
- **CanonicalSuggestion:** SuggestionId, ConfidenceScore, Explanation, Evidence, ProposedCommand, LLM_ModelVersion.

7. Contratos do Ecossistema

7.1. AI Contract (Regras para a IA)

A IA (Agent/LLM) obrigatoriamente deve:

1. Justificar todas as respostas e sugestões.
2. Informar o nível de confiança (Confidence Score).
3. Citar as evidências (logs, alertas) utilizadas na conclusão.
4. Informar limitações ou declarar incerteza quando faltar contexto.
5. **Nunca** inventar informações ausentes (Hallucination Zero).
6. **Nunca** executar comandos ou alterar estado diretamente no banco.
7. Registrar a versão do modelo (ex: Llama-3-70B) e do prompt utilizado no metadado.
8. Produzir respostas reprodutíveis sempre que possível (Low Temperature para análises de segurança).

7.2. Connector Contract (Regras para Integrações)

Todo Connector (ACL) deve:

1. Ser estritamente **idempotente**.
2. Possuir versionamento de schema.

3. Validar o payload de entrada da ferramenta externa.
4. Gerar e publicar **eventos canônicos**.
5. Preservar o payload bruto (**raw_log**) encodado (ex: Base64) para auditoria e *Threat Hunting*.
6. **Nunca** conter regras de negócio, correlação ou cálculo de risco.
7. **Nunca** gravar diretamente no banco de dados operacional.

7.3. Versioning Contract

Todo objeto público, command, event e runbook deve possuir: **Version**, **CreatedAt**, **UpdatedAt**, **SchemaVersion**.

8. Bounded Context Essencial: Asset Discovery

Dedicado à descoberta e gestão do inventário, desvinculado dos conectores passivos.

Responsabilidades de Domínio:

- Descoberta ativa (scanners de rede, Nmap, Python Scanners) e passiva (sniffing ARP, DHCP).
- Consolidação e deduplicação de inventário (ex: resolver um IP que mudou via DHCP para o mesmo UUID de Asset).
- Fingerprinting de sistemas operacionais e serviços (Banner Grabbing).
- Detecção de ativos órfãos (Shadow IT) ou não gerenciados.
- Enriquecimento de contexto para o *Core Platform*.

9. Catálogo de Eventos (Referência M0.7)

A lista exata de eventos de domínio que trafegam no Event Bus.

- **SecurityEventReceived**, **InfrastructureEventReceived**, **TelemetryReceived**
- **IncidentCreated**, **IncidentMerged**, **IncidentClosed**
- **RiskCalculated**
- **AssetDiscovered**, **AssetCompromised**
- **SuggestionGenerated**, **SuggestionApproved**
- **CommandCreated**, **CommandApproved**, **CommandRejected**, **CommandExecuted**
- **PlaybookStarted**, **PlaybookFinished**
- **PolicyAllowed**, **PolicyDenied**, **AuditRecorded**

GovSec Shield: M0.3 — Event Storming e Core Flows (v2.0)

Este documento traduz a linguagem ubíqua e os Bounded Contexts em fluxos orientados a eventos. Ele define a coreografia exata de como os módulos se comunicam, incluindo o tratamento de falhas, idempotência, auditoria e as regras restritas de comunicação síncrona vs assíncrona.

1. Regra Arquitetural de Comunicação (Sync vs Async)

Para evitar que o sistema se torne uma "teia de aranha" de chamadas bloqueantes ou que decisões críticas sofram com latência de filas, estabelecemos a seguinte invariante:

- **Assíncrono (Event/Command Bus via Kafka/RabbitMQ):** Para reações, efeitos colaterais, processamento pesado e notificações (Ex: `IncidentCreated`, `ExecuteActiveResponseCommand`).
- **Síncrono (gRPC): EXCLUSIVO** para decisões que bloqueiam o fluxo de execução e exigem resposta imediata. (Ex: `ValidatePolicyQuery` no Security Kernel, `CalculateRiskQuery` no Risk Engine).

2. Fluxo Principal: Detecção, Triagem, Resposta e Auditoria

Este fluxo demonstra o ciclo completo de um ataque, englobando a deduplicação de alertas, a validação de políticas com trilha de auditoria, e o tratamento de falhas caso a mitigação não ocorra como esperado.

2.1 Diagrama de Sequência

sequenceDiagram
autonumber

actor Hacker
participant WZ as Wazuh (Tool)
participant CON as Connector (ACL)
participant EB as Event Bus
participant CORE as Core Platform
participant RISK as Risk Engine
participant AI as AI Gateway
participant SOC as Dashboard SOC
participant POL as Security Kernel (Policy/Audit)
participant CB as Command Bus

Hacker->>WZ: Executa malware (ex: LSASS dump)
Hacker->>WZ: Executa malware repetidamente (Ruído)

Note over WZ,EB: Fase 1: Ingestão e Idempotência
WZ->>CON: Envia raw alerts (Múltiplos Webhooks)
CON->>CON: Valida payload e gera CorrelationKey (Hash)

CON->>EB: Publica: SecurityEventReceived (Lote deduplicado)

Note over EB,CORE: Fase 2: Triagem e Correlação

EB->>CORE: Consume: SecurityEventReceived

CORE->>CORE: Verifica CorrelationKey (Idempotência/DB)

alt Evento Duplicado (Pertence a Incidente Aberto)

CORE->>CORE: Anexa evento à Timeline (Ignora criação)

else Evento Novo

CORE->>EB: Publica: IncidentCreated (Status: Novo)

end

Note over CORE,RISK: Fase 3: Risco (Síncrono/gRPC - Blocking)

CORE->>RISK: CalculateRiskQuery(AssetID, CVSS)

RISK->>CORE: Retorna Score (Criticidade: ALTA)

CORE->>EB: Publica: RiskCalculated

Note over EB,Al: Fase 4: O Cérebro Advisory

EB->>Al: Consume: RiskCalculated + IncidentCreated

Al->>Al: LLM lê logs, consulta Playbooks

Al->>EB: Publica: SuggestionGenerated (Ação: Isolar Host)

Note over EB,SOC: Fase 5: Validação Humana

EB->>SOC: Push via WebSocket (Alerta + Sugestão IA)

SOC->>CORE: Analista aprova Suggestion (API Call)

Note over CORE,POL: Fase 6: Autorização e Auditoria (Síncrono/gRPC)

CORE->>POL: ValidatePolicyQuery(Token, Action, Target)

POL->>CORE: PolicyAllowed

POL->>EB: Publica: AuditRecorded (Trilha Imutável LGPD)

CORE->>CB: Publica: ExecuteActiveResponseCommand

Note over CB,CON: Fase 7: Execução e Compensação de Falhas

CB->>CON: Consume: ExecuteActiveResponseCommand

alt Sucesso na Execução

CON->>WZ: Aciona API do Wazuh (Isolar IP)

CON->>EB: Publica: CommandExecuted

EB->>CORE: Consume: CommandExecuted

CORE->>EB: Publica: IncidentStatusChanged (Mitigado)

else Falha na Execução (Wazuh Offline / Agente Inacessível)

CON->>EB: Publica: CommandFailed (Motivo: Timeout)

EB->>CORE: Consume: CommandFailed

CORE->>EB: Publica: IncidentStatusChanged (Falha na Contenção)

CORE->>SOC: Alerta Crítico SOC (Requer intervenção manual urgente)

end

3. Dicionário de Transições de Estado (CQRS Atualizado)

3.1 Bounded Context: Connectors (Edge)

- **Responsabilidade Crítica:** Gera a `CorrelationKey` (Hash de deduplicação) baseada em (Asset + Regra + Janela de Tempo) antes de enviar ao Barramento, garantindo o processamento *exactly-once* ou a correlação fácil.
- **Ouve (Commands):** `IsolateAssetCommand`, `BlockIPCommand`.
- **Gera (Events):** `SecurityEventReceived`, `CommandExecuted`, `CommandFailed`.

3.2 Bounded Context: Core Platform (Orquestrador)

- **Ouve (Events):** `SecurityEventReceived`, `CommandExecuted`, `CommandFailed`.
- **Gera (Events):** `IncidentCreated`, `IncidentStatusChanged` (ex: Mitigado vs Falha na Contenção).
- **Queries (Síncronas):** Consulta o *Risk Engine* e o *Security Kernel* via gRPC antes de publicar comandos.

3.3 Bounded Context: Security Kernel (O Guardião)

- **Ouve (Queries Síncronas):** `ValidatePolicyQuery`, `VerifyTokenQuery`.
- **Gera (Events):** `AuditRecorded` (Sempre que uma política de acesso ou comando crítico for invocada, o circuito de auditoria se fecha disparando este evento para o Data Lake imutável).

3.4 Bounded Context: AI Gateway

- **Ouve (Events):** `IncidentCreated`, `RiskCalculated`.
- **Gera (Events):** `SuggestionGenerated`, `ThreatContextEnr`

ADR-001: Seleção do Message Broker (Event-Driven Backbone)

- **Status:** Aprovado
- **Data:** 27 de Julho de 2026
- **Contexto:** GovSec Shield (Security OS)

1. Contexto e Problema

O GovSec Shield opera sob uma Arquitetura Orientada a Eventos (EDA). Ferramentas periféricas (como o Wazuh) podem gerar picos de milhares de alertas por segundo durante um ataque. Precisamos de um barramento de comunicação assíncrona que suporte:

1. **Garantia de Ordenação:** Eventos de um mesmo ativo (**AssetID**) devem ser processados estritamente na ordem cronológica em que ocorreram para que a linha do tempo do incidente faça sentido.
2. **Deduplicação (Idempotência):** Suporte arquitetural para prevenir que 500 alertas do mesmo malware em 10 segundos gerem 500 chamadas de API de contenção idênticas.
3. **Event Replay (Retenção Operacional):** Agentes de IA ou analistas de *Threat Hunting* precisam ser capazes de "reler" eventos do passado recente para correlacionar ataques furtivos (APTs).
4. **Auditoria Imutável:** O barramento deve suportar a natureza *append-only* dos logs legais de auditoria governamental.

2. Opções Consideradas

Opção A: RabbitMQ (AMQP)

- **Prós:** Excelente roteamento flexível (Topic/Direct exchanges), baixíssima latência para mensagens efêmeras, painel de controle nativo maduro.
- **Contras:** É focado em filas efêmeras (*message queue*) e não em um log imutável. Não permite *Event Replay* nativo sem gravar tudo em um banco primeiro. A garantia de ordem falha se houver múltiplos consumidores concorrentes na mesma fila.

Opção B: Apache Kafka (ou Redpanda)

- **Prós:** Opera como um *Distributed Commit Log*. As mensagens são gravadas de forma imutável (append-only) e retidas por tempo configurável. Garante a ordenação estrita dentro de uma partição. Permite escalabilidade horizontal de consumidores sem perder a garantia de ordem (desde que particionado corretamente).
- **Contras:** Infraestrutura mais complexa e consumo maior de recursos (I/O e Memória).

3. Decisão

Nós decidimos utilizar o ecossistema Kafka (implementado via Redpanda para menor overhead operacional e dispensa do Zookeeper/JVM) como o Message Broker padrão do GovSec Shield.

4. Justificativa e Regras de Implementação

A decisão baseia-se na necessidade absoluta de **Retenção (Replay)** e **Ordenação**, vitais para a reconstrução de cronologias de ciberataques:

1. **Ordenação Sagrada (Partition Key = AssetID):** Para garantir que a cronologia de um ataque seja processada corretamente, **todos** os tópicos de eventos usarão obrigatoriamente o **AssetID** como *Partition Key*. Isso garante que todos os eventos do mesmo servidor/usuário caiam na mesma partição e sejam lidos em ordem estrita pelo Core Platform.
2. **Idempotência Real (At-least-once):** O Kafka garante a entrega *at-least-once* (pelo menos uma vez). O mito do *exactly-once* não se aplica a chamadas de APIs externas (ex: isolar host no Wazuh). Portanto, a deduplicação não será mágica do broker: o Core Platform e os Connectors usarão a **CorrelationKey** (Hash do payload gerado na borda) para verificar no banco de dados operacional se a ação já foi tomada, garantindo assim uma idempotência real em nível de aplicação.
3. **Threat Hunting e IA (Retenção Operacional):** Tópicos como **SecurityEvent** terão retenção configurada (ex: 30 dias). Isso permite que a IA "volte no tempo" lendo o próprio tópico Kafka em busca de Indicadores de Comprometimento (IoCs) retrospectivos, atuando como um *buffer* histórico de altíssima performance.
4. **Auditoria Governamental (Retenção de Longo Prazo):** A retenção operacional de 30 dias *não* resolve a conformidade legal. O tópico **AuditRecorded** usará políticas de retenção diferentes (ex: *Log Compaction* infinito ou descarregamento automático via *Tiered Storage* para S3/MinIO após X dias), garantindo a trilha imutável por anos. *(Esta estratégia de armazenamento a frio será detalhada no ADR-003).*

5. Consequências

- **Positivas:** Ganhamos um sistema nervoso central resiliente, capaz de reconstruir o estado do sistema a qualquer momento e escalar o consumo horizontalmente.
- **Negativas:** Exigência de discos de alta performance (SSD/NVMe) nos servidores de mensageria para evitar gargalos de I/O de disco, comum em sistemas *append-only* sob estresse.

ADR-002: Seleção do Banco de Dados Operacional (OLTP)

- **Status:** Aprovado
- **Data:** 27 de Julho de 2026
- **Contexto:** GovSec Shield (Security OS) - Módulo Core e Security Kernel

1. Contexto e Problema

Com a adoção do ecossistema Kafka/Redpanda (ADR-001) e suas garantias de entrega *at-least-once*, a plataforma requer um mecanismo robusto no lado do consumidor para

garantir a idempotência das operações. Se o barramento entregar o mesmo comando de "Isolar Host" duas vezes por falha de rede, a aplicação não pode executá-lo em duplicidade.

Além disso, o GovSec Shield possui requisitos estritos de consistência para:

1. **Máquina de Estados de Incidentes:** Transições de incidentes (ex: **Novo** para **Mitigação**) sofrem de alta concorrência. Se dois analistas ou agentes de IA tentarem atuar no mesmo incidente no mesmo milissegundo, o sistema não pode entrar em estado corrompido.
2. **IAM e RBAC (Security Kernel):** O motor de autorização exige integridade referencial forte, restrições estruturais (constraints) e consultas de altíssima velocidade para validar o acesso a cada comando.

2. Opções Consideradas

- **Opção A: MongoDB (NoSQL Document)**
 - *Prós:* Flexibilidade de schema, gravação rápida de payloads JSON dinâmicos.
 - *Contras:* Falta de *constraints* relacionais nativas robustas para RBAC. Transações multi-documento afetam significativamente a performance e adicionam complexidade desnecessária para deduplicação estrita.
- **Opção B: PostgreSQL (Relacional OLTP)**
 - *Prós:* Transações ACID impecáveis, integridade referencial, recursos avançados de concorrência (ex: **SELECT FOR UPDATE**, locks otimistas/pessimistas), suporte nativo a campos JSONB (unindo o melhor do relacional e NoSQL), e maturidade inquestionável em cenários governamentais.
 - *Contras:* Necessita de gerenciamento rígido de pool de conexões sob alto stress (ex: PgBouncer) e estratégias de *vacuum/cleanup* para tabelas com alta rotatividade (churn).

3. Decisão

Decidimos utilizar o PostgreSQL como o Banco de Dados Operacional padrão para os Bounded Contexts estruturais (Core Platform, Security Kernel, Knowledge Base).

4. Regras de Implementação e Consequências

Esta decisão traz as seguintes regras arquiteturais que todos os desenvolvedores devem seguir:

1. **A Fonte da Idempotência (Tabela de Inbox/Deduplicação):** O Core Platform implementará uma tabela dedicada (ex: **ProcessedEvents** ou **Inbox**) contendo a **CorrelationKey** como restrição **UNIQUE**. Todo processamento de evento que altera estado fará um **INSERT** prévio nesta tabela dentro da mesma transação da

regra de negócio. Se o **INSERT** violar a constraint única, a transação aborta silenciosamente (evento duplicado descartado de forma segura).

2. **Estratégia de TTL (Time-to-Live) da Idempotência:** Tabelas de deduplicação crescem infinitamente. Implementaremos uma estratégia de "janela deslizante" (Sliding Window TTL). O banco rodará rotinas de limpeza periódicas (via extensão **pg_cron** ou *workers* do Core) para expurgar chaves de correlação mais antigas que X dias (ex: 7 dias), baseando-se na premissa de que o barramento de eventos (Kafka) não reenviará mensagens confirmadas mais antigas que esse período.
3. **Concorrência da Máquina de Estados:** Atualizações de status na entidade **Incident** usarão estritamente mecanismos de Lock Otimista (via coluna **Version**) para evitar *Lost Updates*. A IA e os Analistas SOC nunca sobrescreverão dados mutuamente.
4. **JSONB para Dados Variáveis:** Embora o banco seja relacional, o contexto de ferramentas de cibersegurança é dinâmico. O PostgreSQL armazenará o esquema canônico (CSM) em colunas fortemente tipadas, mas payloads originais brutos e metadados flexíveis da IA utilizarão o tipo **JSONB** para indexação ágil.
5. **A Fronteira Analítica (Anti-Pattern Warning): É estritamente proibido** utilizar este banco para armazenar logs brutos do Wazuh, métricas do Zabbix ou a trilha infinita do evento **AuditRecorded**. O PostgreSQL será focado apenas no *Estado Atual* (State). Todo dado de telemetria massiva e armazenamento imutável a frio de longo prazo será responsabilidade do Data Lake Analítico (A ser definido no ADR-003).

5. Consequências

- **Positivas:** Ganhamos um motor de idempotência blindado por *Unique Constraints*, esquemas relacionais perfeitos para o Security Kernel (RBAC), e evitamos race conditions operacionais.
- **Negativas:** Exigirá a implantação de um Pooler de Conexões (PgBouncer) na infraestrutura para evitar esgotamento de conexões quando os conectores do Kafka dispararem múltiplos *workers* concorrentes.

GovSec Shield: M0.4 — Modelo ER do Banco Operacional (PostgreSQL) — v5.4

Este documento define o esquema relacional do Core Platform e Security Kernel. A versão 5.4 fecha a última lacuna de integridade cross-tenant identificada: a relação entre **INCIDENT_TIMELINE** e **PROCESSED_EVENTS**.

1. Diagrama de Entidade-Relacionamento (Mermaid)

```
erDiagram
    TENANT {
```



```
    uuid id PK
    string slug UK
    string name
    string status
    timestamp created_at
}
```

```
USER {
    uuid id PK
    uuid tenant_id PK, FK
    string email UK
    string name
    string mfa_status
    timestamp last_login
    int version
}
```

```
ROLE {
    uuid id PK
    uuid tenant_id FK "Nullable: Role global"
    string name
    string description
}
```

```
RESOURCE_TYPE {
    string name PK
    string description
}
```

```
POLICY {
    uuid id PK
    uuid role_id FK
    uuid tenant_id FK
    string resource_type_name FK "Ref: RESOURCE_TYPE"
    string action
    string effect
}
```

```
USER_ROLE {
    uuid tenant_id PK, FK
    uuid user_id PK, FK
    uuid role_id PK, FK
}
```

```
INCIDENT {
    uuid id PK
    uuid tenant_id PK, FK
    string status
}
```

```
string severity
int risk_score
uuid owner_id FK
uuid asset_id FK
jsonb ai_context
int version
timestamp created_at
timestamp updated_at
}
```

```
INCIDENT_TIMELINE {
  uuid id PK
  uuid tenant_id PK, FK
  uuid incident_id FK
  string event_type
  string correlation_key FK "Ref: PROCESSED_EVENTS (tenant_id, correlation_key)"
  jsonb payload_summary
  timestamp occurred_at
}
```

```
ASSET {
  uuid id PK
  uuid tenant_id PK, FK
  string hostname
  string primary_ip
  string criticality
  string status
  timestamp last_seen
}
```

```
PROCESSED_EVENTS {
  string correlation_key PK
  uuid tenant_id PK, FK
  string event_type
  timestamp processed_at
  timestamp expires_at
}
```

%% Relacionamentos

```
TENANT ||--o{ USER : "possui"
TENANT ||--o{ ASSET : "gerencia"
TENANT ||--o{ INCIDENT : "registra"
TENANT ||--o{ ROLE : "define"
TENANT ||--o{ POLICY : "define"
```

```
ROLE ||--o{ USER_ROLE : "atribuído a"
ROLE ||--o{ POLICY : "define permissões via"
```

RESOURCE_TYPE ||--o{ POLICY : "tipa"

USER ||--o{ USER_ROLE : "possui"

INCIDENT ||--|{ INCIDENT_TIMELINE : "possui histórico"

%% Integridade Cross-Tenant (Compound FKs)

ASSET ||--o{ INCIDENT : "sofre"

USER ||--o{ INCIDENT : "dono"

PROCESSED_EVENTS ||--o{ INCIDENT_TIMELINE : "originou (dedupe)"

2. Invariantes de Negócio e Refinamentos Técnicos

2.1. Multi-Tenancy e Isolamento via RLS

2.1.1. Regra de Conexão: SET LOCAL (Obrigatória)

O uso de PgBouncer exige que a variável de tenant seja definida dentro da transação, nunca na sessão global, para evitar "vazamento" de tenant entre transações reutilizadas pelo Transaction Pooling.

- **Regra:** toda transação deve iniciar com `SET LOCAL app.tenant_id = '...';`.
- **Segurança:** a role `govsec_app_user` deve ter `NOBYPASSRLS` e ser a única utilizada pela aplicação.

2.1.2. Integridade Referencial (Compound FKs)

Para impedir Cross-Tenant Corruption (ex: incidente do Tenant A apontar para asset do Tenant B), usamos FKs compostas.

- **Regra geral:** `FOREIGN KEY (tenant_id, asset_id) REFERENCES ASSET (tenant_id, id)`.
- Todas as tabelas tenant-scoped devem ter `tenant_id` como parte integrante da sua PK ou ser referenciadas via Compound FK que inclua o `tenant_id`.

Aplicações concretas da regra no schema:

Origem

Destino

Constraint

INCIDENT.asset_id	ASSET	FOREIGN KEY (tenant_id, asset_id) REFERENCES ASSET (tenant_id, id)
INCIDENT.owner_id	USER	FOREIGN KEY (tenant_id, owner_id) REFERENCES "USER" (tenant_id, id)
INCIDENT_TIMELINE.incident_id	INCIDENT	FOREIGN KEY (tenant_id, incident_id) REFERENCES INCIDENT (tenant_id, id)
INCIDENT_TIMELINE.correlation_key	PROCESSED_EVENTS	FOREIGN KEY (tenant_id, correlation_key) REFERENCES PROCESSED_EVENTS (tenant_id, correlation_key)
USER_ROLE.user_id	USER	FOREIGN KEY (tenant_id, user_id) REFERENCES "USER" (tenant_id, id)

A última linha fecha a lacuna identificada na revisão da v5.3: sem essa FK composta, uma linha de **INCIDENT_TIMELINE** do Tenant A poderia referenciar (por erro de aplicação) um **correlation_key** pertencente a um evento processado de outro tenant, já que **correlation_key** sozinho não é globalmente único — só é único dentro de um **tenant_id** (conforme a PK composta de **PROCESSED_EVENTS**).

2.1.3. Role Global (Tenant NULL)

As policies de RLS sobre a tabela **ROLE** devem permitir explicitamente o **NULL**:

```
USING (tenant_id = current_setting('app.tenant_id', true)::uuid OR tenant_id IS NULL)
```

2.2. Concorrência (Lock Otimista)

- **Versionamento:** coluna `version` (INT) em `INCIDENT`.
- **Regra de Conflito:** se `version` no update divergir do banco, aborta a operação (HTTP 409). A IA nunca sobrescreve um humano; o humano sempre tem preferência de estado.

2.3. Débitos Técnicos Registrados (Roadmap)

- `POLICY.action`: atualmente string livre. Deve ser migrada para `action_id` FK em uma tabela de lookup no futuro, seguindo o padrão já estabelecido em `RESOURCE_TYPE`.

2.4. Changelog v5.3 → v5.4

- **Cross-Tenant Fix:** adicionada Compound FK (`tenant_id`, `correlation_key`) entre `INCIDENT_TIMELINE` e `PROCESSED_EVENTS`, fechando a última lacuna de isolamento lateral identificada na revisão.
- **Tabela de rastreabilidade:** adicionada tabela explícita (seção 2.1.2) listando todas as Compound FKs do schema, para servir de checklist de implementação e evitar que futuras tabelas fiquem sem a proteção equivalente.

M0.5: Estratégia de Versionamento e Contrato de Envelopamento (v2.2)

Este documento define as regras de evolução e estrutura dos eventos no GovSec Shield. Ele atua como a especificação de contrato do barramento.

1. O Envelope Padrão (Common Header)

Todo evento deve ser encapsulado. A presença do `tenant_id` é obrigatória para eventos de domínio, mas opcional (`null`) para eventos de sistema global ou infraestrutura.

```
{
  "metadata": {
    "event_id": "uuid-v4",
    "trace_id": "uuid-v4",
    "tenant_id": "uuid-v4 | null",
    "occurred_at": "ISO-8601-Timestamp",
    "schema_version": "semver-string",
    "source": "component-name",
    "event_type": "string-enum"
  },
}
```

```
"payload": { ... }  
}
```

- **tenant_id**: Obrigatório para eventos de domínio. `null` permitido apenas para eventos de infraestrutura/plataforma global (ex: `TenantCreated`).
- **schema_version**: Obrigatório (SemVer).
- **event_type**: Enum compartilhado entre producer/consumer.

2. Estratégia de Versionamento: "RecordNameStrategy + CI Enforcement"

A política baseia-se em *Schema Registry Enforcement* com `RecordNameStrategy` para permitir múltiplas versões no mesmo tópico.

- **Registry Strategy**: `RecordNameStrategy` (ou `TopicRecordNameStrategy`). Isso desacopla o nome do *subject* do nome do tópico, permitindo que `security.events.v1` e `security.events.v2` convivam no mesmo tópico Kafka.
- **Política de Compatibilidade**:
 1. `AuditRecorded`: `FULL` (sempre aditivo, proibido remover).
 2. Demais eventos: `FORWARD` (permite novos campos, não altera tipos).
- **Breaking Changes**:
 1. Producer registra um novo Schema no Registry com um novo `RecordName` (Major Version).
 2. Producer envia mensagens com o novo `schema_version`.
 3. Consumidores devem suportar ambas as versões durante a **Janela de Deprecation**.
 4. **Critério de Deprecation**: Um schema/versão antiga só pode ser desativado após o `consumer_lag` do grupo de consumidores antigo atingir **zero** e após um período mínimo de 48h de coexistência.
- **Dispatch**: O consumidor deve rotear a lógica de processamento baseada no campo `metadata.schema_version` do envelope, não baseando-se no schema inferido pelo Registry.

3. Política de Retenção e Conformidade

Evento	Compatibilidade e	Retenção	Política de Evolução
AuditRecorded	FULL	Infinita	Aditiva (Add-only)

SecurityEventReceived	FORWARD	30d	Flexível
IncidentCreated	FORWARD	30d	Flexível
CommandExecuted	FORWARD	30d	Flexível
SuggestionGenerated	FORWARD	vida_do_incident + 7d	Iterativa

Débitos Técnicos

- **DT-001: PII Expurgo (LGPD) no AuditRecorded**
- **DT-002: POLICY.action enum** (Migrar de string livre para lookup centralizado no Security Kernel).

4. Governança

- **Linter de CI:** Bloqueia qualquer PR que altere schemas de forma incompatível com a política definida no Registry.
- **Deployment:** O pipeline de deploy é interrompido se o Schema Registry rejeitar o schema, garantindo que o contrato nunca seja quebrado em tempo de execução.

M0.6: Especificação do Molde Padrão de Contratos de Eventos (v2.2)

Este documento estabelece a estrutura canônica e as regras de construção para todos os esquemas de eventos do barramento Kafka no GovSec Shield. Ele garante a integridade referencial com o Modelo ER (M0.4 v5.4) e a governança de versão estipulada no Contrato de Envolvimento (M0.5 v2.2).

1. Estrutura Estrutural Canônica

Todo evento trafegado na plataforma obedece à separação estrita entre **Metadata** (Envelope), **Data** (Negócio), **References** (Navegabilidade e Relações ER) e **Context** (Metadados de Execução).

```

{
  "metadata": {
    "event_id": "uuid-v4",
    "trace_id": "uuid-v4",
    "tenant_id": "uuid-v4 | null",
    "occurred_at": "ISO-8601-Timestamp",
    "schema_version": "semver-string",
    "source": "component-name",
    "event_type": "string-enum"
  },
  "payload": {
    "data": {
      /* Campos de negócio tipados especificamente para a versão deste evento */
    },
    "references": {
      /* Estritamente tipado por especificação de evento.
         Contém apenas as chaves (FKs) do M0.4 relevantes para este evento
         especificamente.
         NENHUM mapa genérico (map<string, string>) é permitido no schema compilado. */
    },
    "context": {
      /* Metadados operacionais opcionais de suporte */
      "request_id": "uuid-v4 | null",
      "system_version": "string | null"
    }
  }
}

```

2. Invariantes de Design do Template

2.1. Tipagem Estrita e Contratos por Tipo de Evento (**references**)

1. Garantia por Tipo, Não por Mensagem:

- O objeto **references** é estaticamente definido no schema de cada evento.
- Não são permitidos dicionários genéricos de tempo de execução (ex.: **map<string, string>**).
- Exemplo: O schema de **SecurityEventReceived** definirá no Avro/Protobuf exatamente **asset_id** (UUID) e **correlation_key** (String). O schema de **IncidentCreated** definirá **incident_id** (UUID), **asset_id** (UUID) e **owner_id** (UUID | null).
- O Schema Registry valida a presença dos campos definidos no contrato antes do aceite da mensagem.

2. Mapeamento Transversal ao Modelo ER (M0.4 v5.4):

- Qualquer chave declarada dentro de `references` deve obrigatoriamente manter o nome exato e o tipo da chave/atributo correspondente no modelo relacional (ex.: `asset_id`, `incident_id`, `correlation_key`).

2.2. Semântica de Nomenclatura do Bloco `data`

1. Correspondência Relacional 1:1:

- Atributos em `payload.data` que espelham colunas de banco devem utilizar exatamente o mesmo nome em `snake_case` (ex.: `risk_score`, `mfa_status`).

2. Campos Agregados/Derivados:

- Campos calculados ou que agregam múltiplas origens (ex.: `raw_alert_count`, `calculated_cvss_vector`) são permitidos e devem ser expressos em `snake_case`, acompanhados de documentação no registro de schema.

3. Mapeamento SemVer e Compatibilidade de Evolução

Em estrita observância às diretrizes do M0.5 v2.2, a alteração de qualquer campo do envelope ou payload reflete diretamente no atributo `metadata.schema_version` de acordo com a taxonomia abaixo:

Nível SemVer	Condição Trigger no Schema	Ação no Registry / RecordName
MAJOR (X.0.0)	Remoção de campo em <code>data/references</code> , alteração de tipo de dado existente, ou adição de novo campo obrigatório em <code>data/references</code> sem valor default.	Exige registro de novo <code>RecordName</code> (ex: <code>govsec.events.SecurityEventReceivedV2</code>), mantendo o mesmo tópico Kafka (<code>security.events</code>).
MINOR (x.Y.0)	Adição de novo campo em <code>data</code> , <code>references</code> ou <code>context</code> marcado como opcional ou com valor default configurado .	Compatível com o Subject existente sob a regra FORWARD .

PATCH (x.y.Z)	Ajustes de documentação interna do schema, correções de anotações no Registry ou metadata sem alteração de tipos ou estruturas.	Mantém o mesmo Subject e compatibilidade.
-------------------------	---	---

4. Regras de Compatibilidade Estrita (FORWARD Compliance)

- Proibição de Campos Obrigatórios Aditivos:**
 - É estritamente proibido adicionar um novo campo obrigatório (sem valor `default` ou não marcado como `nullable`) em um schema mantendo a mesma versão MAJOR.
- Evolução de Opções e Enums:**
 - Adição de novos valores a um atributo do tipo `enum` em `data` exige que o consumidor trate fallback para valores desconhecidos, ou a alteração deve ser promovida a versão MAJOR.

Especificação de Evento: SecurityEventReceived (v1.1.0)

Este documento especifica o contrato canônico para o evento de ingestão de alertas desnormalizados e deduplicados pela camada de conectores (`Connectors` / `ACL`).

1. Visão Geral do Contrato

- Event Type:** `SECURITY_EVENT_RECEIVED`
- Subject / RecordName:** `govsec.events.SecurityEventReceived`
- Topic Kafka:** `security.events`
- Partition Key:** `payload.references.asset_id`
- Produtores:** Connectors (Wazuh, Zabbix, etc.)
- Consumidores:** Core Platform (Orquestrador)
- Política de Compatibilidade:** `FORWARD`

2. Schema JSON (Estrito - v1.1.0)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
```

```

"$id": "https://govsec.shield/schemas/events/security-event-received-v1.1.0.json",
"title": "SecurityEventReceived",
"type": "object",
"required": ["metadata", "payload"],
"additionalProperties": false,
"properties": {
  "metadata": {
    "type": "object",
    "required": ["event_id", "trace_id", "tenant_id", "occurred_at", "schema_version",
"source", "event_type"],
    "additionalProperties": false,
    "properties": {
      "event_id": { "type": "string", "format": "uuid" },
      "trace_id": { "type": "string", "format": "uuid" },
      "tenant_id": { "type": ["string", "null"], "format": "uuid" },
      "occurred_at": { "type": "string", "format": "date-time" },
      "schema_version": { "type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$" },
      "source": { "type": "string" },
      "event_type": { "type": "string", "enum": ["SECURITY_EVENT_RECEIVED"] }
    }
  },
  "payload": {
    "type": "object",
    "required": ["data", "references", "context"],
    "additionalProperties": false,
    "properties": {
      "data": {
        "type": "object",
        "required": ["raw_alert_count", "canonical_severity", "source_ip"],
        "additionalProperties": false,
        "properties": {
          "raw_alert_id": {
            "type": ["string", "null"],
            "description": "Opcional. ID nativo na origem. Nulo se o conector não possuir ID
nativo."
          },
          "raw_alert_count": {
            "type": "integer",
            "minimum": 1,
            "description": "Campo derivado: Contagem de alertas deduplicados na borda."
          },
          "canonical_severity": {
            "type": "string",
            "enum": ["LOW", "MEDIUM", "HIGH", "CRITICAL"],
            "description": "Campo derivado: Severidade normalizada pelo CSM (distinta da
severidade do incidente)."
          },
          "source_ip": {

```

```

        "type": "string",
        "description": "IP de origem (suporta IPv4 e IPv6).",
    },
    "destination_ip": {
        "type": ["string", "null"],
        "description": "IP de destino (suporta IPv4 e IPv6).",
    }
}
},
"references": {
    "type": "object",
    "required": ["asset_id", "correlation_key"],
    "additionalProperties": false,
    "properties": {
        "asset_id": { "type": "string", "format": "uuid" },
        "correlation_key": { "type": "string" }
    }
},
"context": {
    "type": "object",
    "required": ["system_version"],
    "additionalProperties": false,
    "properties": {
        "request_id": { "type": ["string", "null"], "format": "uuid" },
        "system_version": { "type": "string" }
    }
}
}
}
}
}
}
}
}

```

3. Garantias e Regras

1. **Identidade:** A ausência de `incident_id` em `references` é deliberada; este evento precede a criação do incidente.
2. **IPs:** Não utilizamos `format: ipv4` para evitar *breaking changes* ao introduzir suporte a IPv6.
3. **Rigidez:** O uso de `additionalProperties: false` em todos os níveis garante que nenhum dado inesperado seja persistido no Data Lake.

GovSec Shield

M0.7 — Especificação Canônica do Modelo de Commands (CQRS)

Versão: 1.0.0

Status: Aprovado pelo Comitê de Arquitetura

****Data de Vigência:** 28 de Julho de 2026**
****Classificação:** PÚBLICO INTERNO - CONFIDENCIAL**
****Nível de Controle:** ARQUITETURA FUNDAMENTAL**
****Substitui:** N/A**
****Impacto:** TODOS OS MÓDULOS**

Sumário Executivo

Este documento estabelece a Constituição Técnica dos Commands para o GovSec Shield. Define o modelo canônico, invariantes, fluxos de execução, políticas de segurança, mecanismos de resiliência e padrões de observabilidade que regem todas as operações de escrita no sistema. Como documento fundacional do pilar CQRS da arquitetura, sua conformidade é mandatória para todos os Bounded Contexts, módulos e componentes do ecossistema.

A não conformidade com as especificações aqui contidas constitui violação arquitetural passível de rejeição em revisões de design e impedimento de merges em pipelines de CI/CD.

1. Introdução

1.1 Objetivo

Este documento tem como objetivo:

1. ****Definir Formalmente**** o modelo canônico de Commands do GovSec Shield, estabelecendo padrões rigorosos para toda operação de escrita no sistema.
2. ****Estabelecer Invariantes**** que garantam a integridade, rastreabilidade, segurança e consistência de todas as mutações de estado.
3. ****Orientar a Implementação**** fornecendo especificações técnicas detalhadas que servirão como única fonte de verdade para engenheiros, arquitetos e revisores.
4. ****Garantir Interoperabilidade**** entre Bounded Contexts através de contratos estritos e versionamento controlado.
5. ****Fornecer Base para Ferramentas**** de geração de código, validação estática e análise de conformidade.
6. ****Estabelecer Mecanismos de Resiliência**** que assegurem comportamento previsível sob falhas, picos de carga e cenários adversos.

1.2 Escopo

Este documento abrange:

- **Modelo Canônico:** Estrutura completa do `CanonicalCommand` e todos os seus componentes (metadata, payload, context, execution metadata, references).
- **Invariantes do Domínio:** Conjunto obrigatório de regras que todo Command deve satisfazer antes, durante e após a execução.
- **Taxonomia Oficial:** Sistema classificatório hierárquico de Commands por categoria, domínio e propósito.
- **Catálogo Canônico:** Lista exhaustiva de todos os Commands do sistema com suas propriedades operacionais.
- **Fluxos de Execução:** Caminhos completos desde a recepção do Command até a materialização dos efeitos, incluindo autorização, policy enforcement, execução, auditoria e notificação.
- **Políticas Operacionais:** Regras para idempotência, retry, timeout, rollback, dead letter queue e observabilidade.
- **Segurança:** Mecanismos de Zero Trust, autenticação, autorização, RBAC e OPA/Cedar.
- **Versionamento:** Estratégia SemVer para evolução de Commands.

1.3 Não Escopo

Este documento NÃO abrange:

- **Queries:** O modelo de Queries, apesar de relacionado ao CQRS, é especificado em documento próprio (M0.8 - Especificação Canônica do Modelo de Queries).
- **Eventos:** A estrutura de Domain Events é definida em M0.6 - Contratos Canônicos de Eventos.
- **Sagas e Orquestrações:** Padrões de coreografia e orquestração entre múltiplos Commands serão tratados em M0.9 - Padrões de Saga.
- **Políticas Específicas:** Regras de negócio particulares de cada Bounded Context não são detalhadas, apenas os mecanismos de enforcement.
- **Infraestrutura:** Detalhes de deployment, scaling e configuração de brokers (Kafka/Redpanda) e bancos de dados (PostgreSQL).
- **Integrações Externas:** Protocolos de comunicação com sistemas externos, exceto quando afetam o modelo de Commands.

1.4 Relação com Documentos Anteriores (M0.1 - M0.6)

Este documento é a continuação direta e o refinamento dos seguintes artefatos:

| Documento | Relação |

|-----|-----|

| ****M0.1 - Visão do Sistema**** | Herda os princípios de Security OS, Zero Trust e Security by Default, traduzindo-os em invariantes concretas de Commands. |

| ****M0.2 - Bounded Contexts**** | Define a origem (Owner) e o destino (Executor) de cada Command, respeitando as fronteiras contextuais. |

| ****M0.3 - Event Storming**** | Os Commands documentados neste M0.7 são derivados diretamente das decisões (commands) identificadas no Event Storming. |

| ****M0.4 - Modelo ER**** | Os Commands mapeiam para operações de escrita nas entidades definidas, mantendo consistência referencial. |

| ****M0.5 - Versionamento**** | Aplica a estratégia SemVer aos Commands, incluindo regras específicas para breaking changes no modelo. |

| ****M0.6 - Contratos Canônicos de Eventos**** | Todo Command gera eventos conforme especificado, estabelecendo cadeia causal Command → Event. |

1.5 Princípios Arquiteturais

Os Commands do GovSec Shield são regidos pelos seguintes princípios fundamentais:

1. ****Command First:**** Toda mutação de estado deve ser iniciada por um Command explícito e formal. Mutação implícita ou incidental é proibida.
2. ****Intent-Driven:**** O Command deve expressar claramente a INTENÇÃO do usuário ou sistema, não apenas a operação técnica. Comando é sobre "O QUÊ", não "COMO".
3. ****Immutable Intent:**** Uma vez submetido, o Command é imutável. Sua intenção não pode ser alterada; cancelamentos devem ser Commands separados.
4. ****Policy Before Execution:**** Nenhum Command é executado sem passar pelo Policy Engine. A avaliação de políticas ocorre ANTES da execução.
5. ****Zero Trust By Default:**** Todo Command parte do pressuposto de não confiança. Autenticação, autorização e validação são obrigatórias em TODAS as execuções.
6. ****Idempotency First:**** Todo Command DEVE ser projetado para execução idempotente ou implementar mecanismos de deduplicação robustos.
7. ****Deterministic Execution:**** Dados idênticos de entrada (incluindo contexto de tenant, hora, e estado do sistema) DEVEM produzir resultados idênticos e determinísticos.
8. ****Replay Safe:**** O sistema deve suportar replay seguro de Commands, mesmo em ordens diferentes, sem efeitos colaterais inconsistentes.

9. **Audit All:** TODO Command gera eventos de auditoria completos, incluindo quem, o quê, quando, onde e qual o resultado.

10. **Human Approval When Required:** Commands de alto impacto ou risco elevado requerem aprovação humana explícita antes da execução.

2. Filosofia dos Commands

2.1 Definição Formal

Command é uma estrutura de dados imutável que expressa uma intenção de mutação de estado no sistema. É uma "mensagem de comando" que contém:

- A intenção do ator (usuário, sistema, outro contexto)
- Os dados necessários para realizar a mutação
- O contexto de execução (tenant, origem, correlação)
- As políticas que devem ser avaliadas
- Os metadados para rastreabilidade

Formalmente, um Command é uma tupla:

...

Command = (Intent, Payload, Context, Metadata, Policies)

...

Onde:

- **Intent:** A ação desejada (e.g., `BlockIP`, `CreateIncident`, `ApplyPolicy`)
- **Payload:** Dados específicos para a ação (e.g., IP, CIDR, descrição)
- **Context:** Informações de execução (tenant, origem, correlação)
- **Metadata:** Dados de rastreabilidade (timestamp, versão, idempotência)
- **Policies:** Referências ou avaliações de políticas

2.2 O que NÃO é um Command

1. **Query:** Uma Query NÃO é um Command. Queries recuperam dados, não mutam estado. Não geram eventos, não passam por Policy Engine (apenas autorização de leitura) e não requerem idempotência.

2. **Event:** Um Event NÃO é um Command. Eventos descrevem fatos ocorridos NO PASSADO, representam mutações CONSUMADAS. Não são instruções para fazer algo.

3. **Suggestion:** Uma Suggestion NÃO é um Command. Sugestões são recomendações do sistema (especialmente do AI Advisor) que PODEM gerar Commands, mas não são Commands em si.

4. **Policy Decision:** Uma Policy Decision NÃO é um Command. É o resultado da avaliação de políticas que pode permitir, negar ou condicionar um Command.

5. **Rollback:** Um Rollback NÃO é um Command independente. É uma operação de compensação que pode envolver múltiplos Commands.

2.3 Diferenças Fundamentais

Aspecto	Command	Query	Event	Suggestion	Policy Decision
-----	-----	-----	-----	-----	-----
Intenção	Mudar estado	Ler estado	Registrar fato	Recomendar ação	Avaliar permissão
Mutação	Sim	Não	Já ocorreu	Não	Não
Idempotência	Obrigatória	Opcional	Imutável	Irrelevante	Irrelevante
Policy Engine	Obrigatório	Apenas autorização	Já autorizado	Irrelevante	É o motor
Gera Evento	Sim	Não	Já é evento	Não	Não
Auditoria	Completa	Apenas acesso	Já auditado	Limitada	Completa
Rollback	Possível	N/A	Compensação	N/A	N/A
Timeout	Sim	Sim	N/A	N/A	N/A
Retry	Sim	Sim	N/A	N/A	N/A
Armazenamento	Transitório	N/A	Permanente	Transitório	Transitório

2.4 Separação CQRS - Justificativa Arquitetural

O GovSec Shield adota a separação CQRS (Command Query Responsibility Segregation) como padrão arquitetural fundamental pelos seguintes motivos:

1. **Escalabilidade Independente:** Commands e Queries podem escalar horizontalmente de forma independente, respondendo a padrões de carga distintos.
2. **Modelos Otimizados:** Os modelos de leitura e escrita podem ser otimizados separadamente, sem compromissos. Leitura usa projeções denormalizadas; escrita usa modelos normalizados e invariantes.
3. **Segurança Granular:** Políticas de autorização podem ser diferentes para Commands e Queries, permitindo menor privilégio para leitura e controle mais rigoroso para escrita.
4. **Event Sourcing Ready:** Commands alimentam o Event Store, enquanto Queries leem de projeções construídas a partir do Event Store.
5. **Isolamento de Falhas:** Falhas em Commands (e.g., timeout, concorrência) não afetam a disponibilidade de Queries, e vice-versa.
6. **Auditoria Completa:** Commands fornecem traço completo de "quem mudou o quê e por quê", essencial para conformidade.

7. ****Desempenho:**** Operations de leitura podem utilizar caches e índices agressivos sem comprometer as garantias transacionais da escrita.

3. Princípios Fundamentais

3.1 Command First

****Definição:**** Toda mutação de estado DEVE ser iniciada por um Command. Não é permitida mutação direta de estado por outros mecanismos, exceto durante bootstrapping ou migrações documentadas.

****Racionalização:****

- Estabelece ponto único de entrada para toda mudança, permitindo: auditoria, autorização, validação, policy enforcement e observabilidade.
- Facilita rastreamento de origem de mudanças ("quem fez o quê, quando e por quê?").
- Permite implementação consistente de padrões de resiliência.
- Garante que todas as mutações sejam registradas para eventual replay.

****Exemplo de Conformidade:****

```
```json
// CORRETO
{
 "command": "BlockIPCommand",
 "payload": {
 "ip": "192.168.1.100",
 "cidr": 32
 }
}
```

```
// INCORRETO - Mutação direta
UPDATE blocked_ips SET active = true WHERE ip = '192.168.1.100';
```
```

3.2 Intent Driven

****Definição:**** O Command deve expressar a INTENÇÃO do ator, não a operação técnica. A intenção é o "porquê" da mutação, não apenas o "como".

****Racionalização:****

- Permite evolução do sistema: a implementação técnica pode mudar, a intenção permanece.
- Melhora rastreabilidade: auditores e operadores entendem o propósito.
- Facilita aplicação de políticas baseadas em intenção, não em operações técnicas.
- Permite tradução entre intenções e implementações técnicas através de Handlers.

****Exemplos:****

```
Intenção (Command)	Operação Técnica (Implementação)
`BlockIPCommand`	INSERT/UPDATE em `blocked_ips`
`AddToWhitelistCommand`	INSERT/UPDATE em `whitelist`
`EscalateIncidentCommand`	UPDATE em `incidents.status` + NOTIFY
`ApplyPolicyCommand`	Comprimir política + ATUALIZAR engines
```

****Anti-Pattern (Evitar):****

```
```json
// Ruim - Operação Técnica
{
 "command": "UpdateTableCommand",
 "payload": {
 "table": "incidents",
 "field": "status",
 "value": "escalated"
 }
}
```
```

3.3 Immutable Intent

****Definição:**** Uma vez que um Command é submetido ao sistema, sua intenção NÃO PODE ser alterada. O Command é imutável.

****Racionalização:****

- Evita condições de corrida onde a intenção é alterada durante o processamento.
- Garante que o traço de auditoria seja preciso.
- Permite reprocessamento/replay seguro.
- Evita efeitos colaterais não intencionais.

****Mecanismo:****

- Commands são serializados e imutáveis em transporte (Kafka, gRPC).
- Handlers recebem uma cópia imutável do Command.
- Qualquer modificação na intenção requer novo Command.

3.4 Policy Before Execution

****Definição:**** Nenhum Command é executado sem passar pelo Policy Engine. A avaliação de políticas ocorre ANTES da execução.

****Racionalização:****

- Garante que todas as políticas sejam aplicadas consistentemente.
- Previne execução de operações não autorizadas.
- Permite auditoria de decisões de policy.

- Centraliza regras de segurança e compliance.

****Fluxo:****

...

Command → Policy Engine → [Permitido | Negado | Condicionado]

↓

(Se Permitido ou Condicionado)

↓

Handler

...

3.5 Security by Default

****Definição:**** Todo Command tem segurança máxima por padrão. Qualquer diminuição de segurança deve ser explícita, justificada e aprovada.

****Racionalização:****

- Reduz superfície de ataque.
- Evita esquecimento de verificações de segurança.
- Estabelece cultura de segurança proativa.

****Implementação:****

- Validação de todos os campos.
- Autenticação obrigatória para todo Command.
- Autorização verificada para todo Command.
- Rate limiting por tenant e ator.
- Sanitização de entrada.
- Criptografia de dados sensíveis.

3.6 Zero Trust

****Definição:**** O sistema não confia em nenhuma fonte de Commands, mesmo aquelas "internas" ou "privilegiadas". TODOS os Commands passam por autenticação, autorização e validação.

****Racionalização:****

- Protege contra comprometimento interno ("insider threats").
- Limita danos em caso de breach.
- Segmentação de segurança entre Bounded Contexts.

****Princípios Zero Trust Aplicados:****

1. ****Verifique Explicitamente:**** Autentique e autorize sempre.
2. ****Privilegio Mínimo:**** Comandos têm acesso apenas ao estritamente necessário.
3. ****Assuma Breach:**** Projete como se o sistema já estivesse comprometido.

3.7 Human Approval

****Definição:**** Commands com alto impacto ou risco elevado requerem aprovação humana explícita antes da execução.

****Racionalização:****

- Prevê erros catastróficos.
- Atende requisitos de compliance.
- Fornece camada adicional de segurança.

****Exemplos de Commands que Requerem Aprovação:****

- `DeleteAllIncidentsCommand` (delete em massa)
- `PromoteUserToAdminCommand` (elevação de privilégio)
- `DisableTenantCommand` (desabilitar tenant)
- `ModifyNetworkPolicyCommand` (alterar políticas de rede)
- `ResetSecurityKernelCommand` (reset do kernel)

****Critérios de Classificação:****

- Potencial de dano financeiro > US\$ 100K
- Potencial de violação de compliance
- Impacto em > 10% dos usuários
- Acesso a dados sensíveis (PII, PHI, PCI)

3.8 Deterministic Execution

****Definição:**** Dados idênticos de entrada (Command + Context) produzem resultados idênticos, assumindo estado inicial idêntico.

****Racionalização:****

- Permite replay seguro.
- Facilita testes.
- Reduz bugs intermitentes.
- Apoia idempotência.

****Exceções Permitidas:****

- Timestamps (são gerados no momento da execução, mas devem ser rastreáveis via correlation)
- UUIDs gerados (se forem derivados deterministicamente, e.g., UUID v5)
- Operações de sistema externo (devem ser abstraídas com mocks)

3.9 Replay Safe

****Definição:**** O sistema deve suportar replay de Commands, mesmo em ordem diferente da original, sem efeitos colaterais inconsistentes.

****Racionalização:****

- Suporte a disaster recovery.
- Possibilidade de reprocessamento histórico.
- Análise forense.

****Requisitos para Replay Safe:****

- Idempotência garantida.
- Determinismo (exceto casos documentados).
- Versionamento de Commands.
- Compatibilidade com estado atual.

3.10 Idempotency First

****Definição:**** Todo Command DEVE ser projetado para execução idempotente ou implementar mecanismos de deduplicação robustos.

****Racionalização:****

- Protege contra retry sem duplicação.
- Permite entrega "exactly-once" mesmo com falhas.
- Simplifica recuperação de falhas.

****Estratégias de Idempotência:****

1. ****Naturalmente Idempotente:**** O Command é idempotente por natureza (e.g., ``SetStatusCommand``).
2. ****Idempotency Key:**** Cliente fornece chave única; sistema deduplica.
3. ****Inbox Pattern:**** Sistema registra Commands processados e descarta duplicatas.

4. Linguagem Ubíqua

A Linguagem Ubíqua do GovSec Shield para Commands estabelece termos formais, inequívocos e obrigatórios em toda a comunicação técnica.

4.1 Termos Fundamentais

| Termo | Definição Formal | Exemplo |
|-----------------------------|---|---|
| **Command** | Estrutura de dados imutável expressando intenção de mutação de estado. | <code>`BlockIPCommand`</code> , <code>`CreateIncidentCommand`</code> |
| **Command Handler** | Componente que executa um Command específico, aplicando a lógica de negócio e gerando efeitos. | <code>`BlockIPCommandHandler`</code> |
| **Command Bus** | Middleware que recebe Commands, aplica validação, autorização, policies, e encaminha ao Handler apropriado. | <code>`KafkaCommandBus`</code> , <code>`InMemoryCommandBus`</code> |
| **Command Metadata** | Dados descritivos sobre o Command, não sobre o payload: timestamp, origem, versão, trace, etc. | <code>`metadata.timestamp`</code> , <code>`metadata.tenant`</code> |
| **Executor** | Ator final que executa o Command. Pode ser humano, sistema, ou módulo. | <code>`SecurityKernel`</code> , <code>`Orchestrator`</code> , <code>`User`</code> |
| **Policy Engine** | Componente que avalia políticas (OPA, Cedar) contra o Command e contexto. | <code>`OPAEngine`</code> , <code>`CedarEngine`</code> |
| **Rollback** | Operação de compensação que desfaz os efeitos de um Command executado. | <code>`BlockIPCommand`</code> → <code>`UnblockIPCommand`</code> |

| **Retry** | Tentativa subsequente de executar um Command após falha transitória. | Exponential backoff com max 5 tentativas |

| **Dead Letter Queue (DLQ)** | Fila onde Commands que falharam repetidamente são depositados para análise manual. | `govsec.commands.dlq` |

| **Execution Context** | Informações sobre o ambiente de execução: tenant, origem, correlação, trace. | `{"tenant": "gov-abc", "source": "dashboard"}` |

| **Correlation Key** | Identificador que une múltiplos Commands e eventos relacionados a uma mesma transação ou fluxo. | `"correlation-id": "inc-123-abc"` |

| **Idempotency Key** | Identificador único fornecido pelo cliente para permitir deduplicação. | `"idempotency-key": "1a2b3c4d"` |

| **Tenant** | Unidade de isolamento lógico. Todo Command pertence a um Tenant. | `"tenant": "gov-abc"` |

| **Execution Result** | Resultado da execução do Command: sucesso, falha, pendente, etc. | `{"status": "SUCCESS", "data": {...}}` |

| **Approval** | Aprovação humana para execução de Command de alto risco. | `{"approver": "user@domain.com", "status": "PENDING"}` |

| **Suggestion** | Recomendação do sistema (AI) que pode ser convertida em Command. | `"Suggestion": "Block IP 192.168.1.100"` |

4.2 Termos de Infraestrutura

| Termo | Definição Formal | Exemplo |
|----------------------------|---|---|
| Command Queue | Fila (Kafka/Redpanda) que recebe Commands para processamento assíncrono. | <code>govsec.commands.v1</code> |
| Command Registry | Registro central de todos os Commands suportados, com schema e versionamento. | <code>CommandRegistry.get("BlockIPCommand")</code> |
| Command Validator | Componente que valida sintaxe e semântica do Command antes da execução. | <code>JsonSchemaValidator</code> |
| Command Router | Componente que roteia Command para o Handler ou contexto apropriado. | Roteia por <code>command.name</code> |
| Command Interceptor | Middleware que intercepta Commands para aplicar aspectos transversais. | <code>AuditInterceptor</code> , <code>MetricsInterceptor</code> |
| Command Store | Armazenamento de histórico de Commands para auditoria e replay. | Tabela PostgreSQL <code>command_store</code> |

5. Invariantes do Domínio

5.1 Invariantes Obrigatórias

As seguintes invariantes são MANDATÓRIAS para TODO Command no GovSec Shield:

| ID | Invariante | Descrição | Verificação |
|----------------|------------------|---|----------------------------------|
| INV-001 | UUID Obrigatório | Todo Command possui um UUID único e imutável. | <code>metadata.command_id</code> |

| ****INV-002**** | Tenant Obrigatório | Todo Command pertence a um Tenant válido. |
`metadata.tenant` |
| ****INV-003**** | Correlation Key Obrigatória | Todo Command possui uma correlation key para rastreamento de fluxo. | `metadata.correlation\_id` |
| ****INV-004**** | Gera Audit Event | Todo Command gera pelo menos um evento de auditoria. |
Após execução |
| ****INV-005**** | Deve ser Autorizado | Todo Command deve passar por autorização antes da execução. | Policy Engine |
| ****INV-006**** | Timeout Definido | Todo Command possui um timeout máximo de execução. |
`metadata.timeout` |
| ****INV-007**** | Owner Definido | Todo Command possui um owner/responsável. |
`metadata.owner` |
| ****INV-008**** | Versionamento | Todo Command possui versão semântica. |
`metadata.version` |
| ****INV-009**** | Imutável | Todo Command é imutável após criação. | Serializado como JSON |
|
| ****INV-010**** | Resultado Único | Todo Command gera exatamente um resultado principal. |
`execution\_result` |
| ****INV-011**** | Input Validation | Todo Command deve ter todos os campos de payload validados. | JSON Schema |
| ****INV-012**** | Idempotência | Todo Command deve ser idempotente ou ter mecanismo de deduplicação. | Idempotency Key |
| ****INV-013**** | Rastreabilidade | Todo Command deve ser rastreável até sua origem. |
`metadata.source` |
| ****INV-014**** | Timestamp | Todo Command possui timestamp de criação. |
`metadata.created\_at` |
| ****INV-015**** | Políticas Avaliadas | Todo Command é avaliado pelo Policy Engine. |
`policy\_decision` |

5.2 Invariantes por Categoria

****Invariantes de Segurança:****

- Todo Command com dados sensíveis deve ser criptografado em trânsito.
- Todo Command que modifica privilégios deve passar por auditoria extra.
- Todo Command que modifica políticas de segurança deve ter aprovação humana.

****Invariantes de Resiliência:****

- Commands com taxa de falha > 5% em 1 hora devem ser alertados.
- Commands com latência > SLA devem ser registrados para análise.
- Todo Command deve ter estratégia de retry definida.

****Invariantes de Conformidade:****

- Commands que acessam dados sensíveis (PII, PHI, PCI) devem ser autorizados explicitamente.

- Commands em contexto de compliance (HIPAA, GDPR, PCI-DSS) devem gerar eventos de auditoria reforçados.

5.3 Validação de Invariantes

...

Validator:

1. Recebe Command
2. Valida UUID (INV-001)
3. Valida Tenant (INV-002)
4. Valida Correlation Key (INV-003)
5. Valida Version (INV-008)
6. Valida Payload via JSON Schema
7. Valida Owner (INV-007)
8. Valida Timeout (INV-006)
9. Valida Idempotência (INV-012)
10. Se tudo OK → Prossegue
11. Se falha → Rejeita com erro detalhado

...

6. Modelo Canônico

6.1 Estrutura Completa

O modelo canônico de Command é composto por cinco seções principais:

...

CanonicalCommand

- └─ metadata (Informações do Command)
- └─ payload (Dados da operação)
- └─ context (Ambiente de execução)
- └─ references (Referências a entidades)
- └─ execution_status (Estado pós-execução)

...

6.2 Schema JSON

```json

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "$id": "https://govsecshield.io/schemas/canonical-command.json",
 "title": "Canonical Command",
 "description": "Modelo canônico de Command para o GovSec Shield",
 "type": "object",
 "required": [
 "metadata",
```

```
"payload"
],
"properties": {
 "metadata": {
 "type": "object",
 "required": [
 "command_id",
 "command_name",
 "version",
 "tenant",
 "correlation_id",
 "created_at",
 "source"
],
 },
 "properties": {
 "command_id": {
 "type": "string",
 "format": "uuid",
 "description": "UUID único do Command"
 },
 "command_name": {
 "type": "string",
 "description": "Nome canônico do Command",
 "pattern": "^[A-Z][a-zA-Z0-9]*Command$"
 },
 "version": {
 "type": "string",
 "description": "Versão SemVer do Command",
 "pattern": "^\\d+\\.\\d+\\.\\d+\\.\\d+\\.\\d+$"
 },
 "tenant": {
 "type": "string",
 "description": "ID do Tenant",
 "maxLength": 64
 },
 "correlation_id": {
 "type": "string",
 "description": "ID de correlação para rastreamento",
 "maxLength": 128
 },
 "idempotency_key": {
 "type": "string",
 "description": "Chave para deduplicação",
 "maxLength": 128
 },
 "created_at": {
 "type": "string",
 "format": "date-time",
```

```
 "description": "Timestamp de criação (UTC)"
 },
 "source": {
 "type": "string",
 "description": "Origem do Command (dashboard, api, connector, etc.)",
 "maxLength": 32
 },
 "owner": {
 "type": "string",
 "description": "Responsável pelo Command",
 "maxLength": 128
 },
 "timeout": {
 "type": "integer",
 "description": "Timeout em milissegundos",
 "minimum": 100,
 "maximum": 60000
 },
 "priority": {
 "type": "string",
 "enum": ["LOW", "MEDIUM", "HIGH", "CRITICAL"],
 "description": "Prioridade de execução"
 },
 "requires_approval": {
 "type": "boolean",
 "description": "Indica se requer aprovação humana"
 },
 "trace": {
 "type": "object",
 "description": "Informações de trace (OpenTelemetry)",
 "properties": {
 "trace_id": {"type": "string"},
 "span_id": {"type": "string"},
 "parent_span_id": {"type": "string"}
 }
 }
},
"payload": {
 "type": "object",
 "description": "Dados específicos do Command",
 "additionalProperties": true
},
"context": {
 "type": "object",
 "description": "Contexto de execução",
 "properties": {
 "user": {
```

```

 "type": "object",
 "properties": {
 "id": {"type": "string"},
 "email": {"type": "string"},
 "roles": {"type": "array", "items": {"type": "string"}}
 }
 },
 "client": {
 "type": "object",
 "properties": {
 "ip": {"type": "string"},
 "user_agent": {"type": "string"}
 }
 },
 "environment": {
 "type": "string",
 "enum": ["DEVELOPMENT", "STAGING", "PRODUCTION"]
 }
},
"references": {
 "type": "object",
 "description": "Referências a entidades existentes",
 "properties": {
 "incident_id": {"type": "string"},
 "policy_id": {"type": "string"},
 "asset_id": {"type": "string"},
 "user_id": {"type": "string"}
 },
 "additionalProperties": true
},
"execution_status": {
 "type": "object",
 "description": "Estado pós-execução",
 "properties": {
 "status": {
 "type": "string",
 "enum": ["PENDING", "PROCESSING", "SUCCESS", "FAILURE", "ROLLED_BACK",
"CANCELLED"]
 },
 "started_at": {"type": "string", "format": "date-time"},
 "completed_at": {"type": "string", "format": "date-time"},
 "duration_ms": {"type": "integer"},
 "error": {
 "type": "object",
 "properties": {
 "code": {"type": "string"},
 "message": {"type": "string"},

```



O payload é um objeto JSON que contém os dados específicos para o Command. Deve ser validado por um JSON Schema específico por Command.

**\*\*Exemplo de Payload para BlockIPCommand:\*\***

```
```json
{
  "ip": "192.168.1.100",
  "cidr": 32,
  "reason": "Suspicious activity detected",
  "duration_hours": 24,
  "notify": ["security-team@domain.com"]
}
```
```

#### #### 6.3.3 Context

| Campo         | Tipo   | Obrigatório | Descrição                                                |
|---------------|--------|-------------|----------------------------------------------------------|
| `user`        | Object | Não         | Informações do usuário que originou o Command.           |
| `user.id`     | String | Não         | ID do usuário.                                           |
| `user.email`  | String | Não         | Email do usuário.                                        |
| `user.roles`  | Array  | Não         | Roles do usuário.                                        |
| `client`      | Object | Não         | Informações do cliente (IP, User-Agent).                 |
| `environment` | Enum   | Não         | Ambiente de execução (DEVELOPMENT, STAGING, PRODUCTION). |

#### #### 6.3.4 References

Objeto que contém referências a entidades existentes, facilitando a correlação. Campos são opcionais e dependem do contexto.

**\*\*Exemplo:\*\***

```
```json
{
  "incident_id": "INC-2026-001",
  "policy_id": "POL-12345",
  "asset_id": "ASSET-67890"
}
```
```

#### #### 6.3.5 Execution Status

Preenchido APÓS a execução. Fornece rastreabilidade completa do resultado.

---

## ## 7. Envelope do Command

### ### 7.1 Estrutura do Envelope

O envelope contém o Command serializado com informações adicionais de transporte.

```
```json
{
  "spec_version": "1.0.0",
  "message_type": "COMMAND",
  "message_id": "uuid",
  "timestamp": "2026-07-28T10:30:00Z",
  "correlation_id": "flow-123-abc",
  "tenant": "gov-abc",
  "command": {
    "metadata": {...},
    "payload": {...}
  },
  "signature": {
    "algorithm": "RS256",
    "value": "base64-signature"
  }
}
```
```

### ### 7.2 Campos do Envelope

| Campo            | Tipo     | Obrigatório | Descrição                                       |
|------------------|----------|-------------|-------------------------------------------------|
| `spec_version`   | String   | Sim         | Versão do envelope (SemVer).                    |
| `message_type`   | Enum     | Sim         | Tipo de mensagem ("COMMAND", "EVENT", "QUERY"). |
| `message_id`     | UUID     | Sim         | ID único da mensagem.                           |
| `timestamp`      | ISO-8601 | Sim         | Timestamp de criação do envelope.               |
| `correlation_id` | String   | Sim         | ID de correlação para rastreamento.             |
| `tenant`         | String   | Sim         | ID do Tenant.                                   |
| `command`        | Object   | Sim         | O Command completo.                             |
| `signature`      | Object   | Não         | Assinatura digital para integridade.            |

### ### 7.3 Versionamento do Envelope

| Versão | Mudanças       | Breaking Change | Data       |
|--------|----------------|-----------------|------------|
| 1.0.0  | Versão inicial | -               | 2026-07-28 |

### ### 7.4 Tracing e Observabilidade

O envelope deve incluir informações de tracing (OpenTelemetry):

```

```json
{
  "trace_context": {
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "00f067aa0ba902b7",
    "trace_flags": "01"
  }
}
```

```

### ### 7.5 Schema Version

O schema version indica a versão do esquema do Command. Deve seguir SemVer.

```

```
"schema_version": "1.2.0"
```

```

---

## ## 8. Versionamento

### ### 8.1 Estratégia SemVer

Todos os Commands do GovSec Shield seguem versionamento SemVer (MAJOR.MINOR.PATCH), conforme definido em M0.5.

- **MAJOR:** Mudanças incompatíveis no schema ou comportamento.
- **MINOR:** Adições compatíveis (novos campos opcionais).
- **PATCH:** Correções de bugs ou melhorias internas.

### ### 8.2 Regras de Evolução

| Tipo de Mudança             | Exemplo                      | Nova Versão |
|-----------------------------|------------------------------|-------------|
| Adicionar campo opcional    | `description` opcional       | MINOR       |
| Adicionar campo obrigatório | `duration` obrigatório       | MAJOR       |
| Remover campo               | Remover `legacy_field`       | MAJOR       |
| Renomear campo              | `ip` → `ip_address`          | MAJOR       |
| Mudar tipo de campo         | `status` de string para enum | MAJOR       |
| Mudar enum                  | Adicionar `CRITICAL` ao enum | MINOR       |
| Remover enum                | Remover `LEGACY` do enum     | MAJOR       |
| Mudar validação             | `timeout` de 1-60 para 1-120 | MINOR       |

### ### 8.3 Compatibilidade

**Backward Compatibility (Versões Antigas):**



- Handlers devem aceitar versões antigas de Commands.
- Campos não reconhecidos devem ser ignorados.
- Valores default devem ser usados para campos faltantes.

**\*\*Forward Compatibility (Versões Novas):\*\***

- Handlers antigos devem ignorar campos novos.
- Validação deve ser tolerante a versões futuras.
- Estratégia de "Ignore Unknown Fields".

### ### 8.4 Exemplo de Evolução

**\*\*Versão 1.0.0:\*\***

```
```json
{
  "metadata": {
    "command_name": "BlockIPCommand",
    "version": "1.0.0"
  },
  "payload": {
    "ip": "192.168.1.100",
    "cidr": 32
  }
}
```
```

**\*\*Versão 1.1.0 (MINOR):\*\***

```
```json
{
  "metadata": {
    "command_name": "BlockIPCommand",
    "version": "1.1.0"
  },
  "payload": {
    "ip": "192.168.1.100",
    "cidr": 32,
    "reason": "Suspicious activity" // Novo campo opcional
  }
}
```
```

**\*\*Versão 2.0.0 (MAJOR):\*\***

```
```json
{
  "metadata": {
    "command_name": "BlockIPCommand",
    "version": "2.0.0"
  },

```

```
"payload": {
  "ip_address": "192.168.1.100", // Campo renomeado
  "cidr": 32,
  "reason": "Suspicious activity"
  // duration removido, agora é obrigatório via outro Command
}
}
...
```

9. Taxonomia Oficial

9.1 Categorias de Commands

Os Commands do GovSec Shield são classificados nas seguintes categorias:

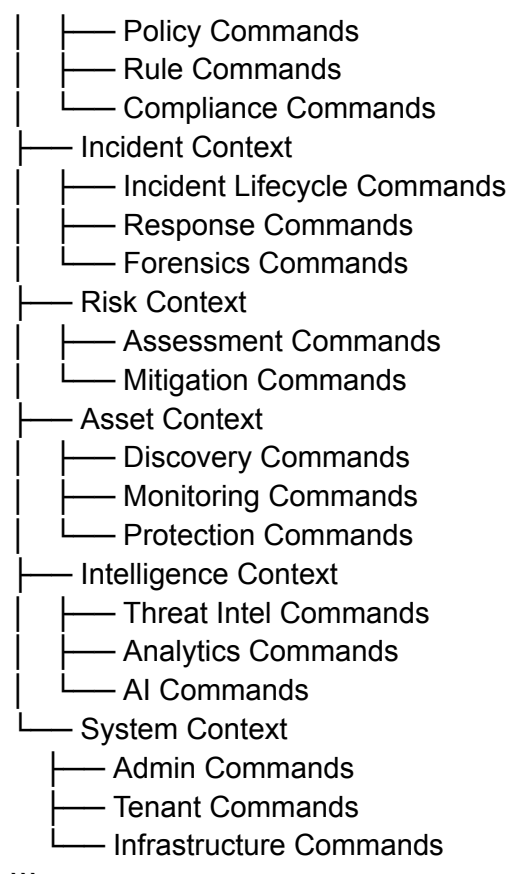
Categoria	Descrição	Exemplos
Discovery Commands	Commands para descoberta de ativos, vulnerabilidades, e configurações.	`DiscoverAssetCommand`, `ScanNetworkCommand`
Monitoring Commands	Commands para monitoramento, coleta de métricas e logs.	`StartMonitoringCommand`, `CollectMetricsCommand`
Risk Commands	Commands para avaliação, gestão e mitigação de riscos.	`AssessRiskCommand`, `MitigateRiskCommand`
Incident Commands	Commands para criação, atualização, escalonamento e resolução de incidentes.	`CreateIncidentCommand`, `EscalateIncidentCommand`
Response Commands	Commands para resposta a incidentes (containment, eradication, recovery).	`ContainThreatCommand`, `IsolateAssetCommand`
Security Commands	Commands para políticas de segurança, regras e configurações.	`ApplyPolicyCommand`, `CreateSecurityGroupCommand`
Knowledge Commands	Commands para gestão de conhecimento, inteligência de ameaças.	`UpdateThreatIntelCommand`, `StoreIndicatorCommand`
AI Commands	Commands para interações com AI Advisor.	`RequestSuggestionCommand`, `ApplyAIActionCommand`
Administration Commands	Commands para administração do sistema.	`CreateTenantCommand`, `UpdateUserRoleCommand`
Infrastructure Commands	Commands para gestão de infraestrutura.	`ProvisionAssetCommand`, `ScaleServiceCommand`

9.2 Hierarquia de Domínios

Cada Command pertence a um Domínio e Subdomínio específico, conforme Bounded Contexts definidos em M0.2.

...

```
Command
└── Security Context
```



10. Catálogo Canônico

10.1 Catálogo Completo de Commands

A tabela a seguir lista todos os Commands do GovSec Shield, com suas propriedades operacionais.

ID	Nome	Descrição	Owner	Executor	Policy	Rollback	Evento Gerado	Timeout (ms)	Retries	Priority	Idempotência	SLA
CMD-001	`CreateIncidentCommand`	Cria um novo incidente	Incident Context	Incident Handler	Policy Engine	`CloseIncidentCommand`	`IncidentCreatedEvent`	5000	3	HIGH	Sim	2s
CMD-002	`EscalateIncidentCommand`	Escalona incidente para nível superior	Incident Context	Incident Handler	Policy Engine	`DeescalateIncidentCommand`	`IncidentEscalatedEvent`	3000	2	HIGH	Sim	1s
CMD-003	`AssignIncidentCommand`	Atribui incidente a uma equipe	Incident Context	Incident Handler	Policy Engine	`UnassignIncidentCommand`	`IncidentAssignedEvent`	3000	2	MEDIUM	Sim	1s

| CMD-004 | `ResolveIncidentCommand` | Marca incidente como resolvido | Incident Context | Incident Handler | Policy Engine | `ReopenIncidentCommand` | `IncidentResolvedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-005 | `CloseIncidentCommand` | Fecha o incidente definitivamente | Incident Context | Incident Handler | Policy Engine | `ReopenIncidentCommand` | `IncidentClosedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-006 | `BlockIPCommand` | Bloqueia um IP específico | Security Context | Security Kernel | Policy Engine | `UnblockIPCommand` | `IPBlockedEvent` | 5000 | 3 | CRITICAL | Sim | 1s |

| CMD-007 | `UnblockIPCommand` | Desbloqueia um IP bloqueado | Security Context | Security Kernel | Policy Engine | `BlockIPCommand` | `IPUnblockedEvent` | 5000 | 3 | HIGH | Sim | 1s |

| CMD-008 | `ApplyPolicyCommand` | Aplica uma política de segurança | Security Context | Policy Engine | Policy Engine | `RevertPolicyCommand` | `PolicyAppliedEvent` | 10000 | 2 | CRITICAL | Sim | 5s |

| CMD-009 | `RevertPolicyCommand` | Reverte uma política aplicada | Security Context | Policy Engine | Policy Engine | N/A | `PolicyRevertedEvent` | 10000 | 2 | CRITICAL | Sim | 5s |

| CMD-010 | `CreateSecurityGroupCommand` | Cria um grupo de segurança | Security Context | Security Kernel | Policy Engine | `DeleteSecurityGroupCommand` | `SecurityGroupCreatedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-011 | `DeleteSecurityGroupCommand` | Remove um grupo de segurança | Security Context | Security Kernel | Policy Engine | N/A | `SecurityGroupDeletedEvent` | 5000 | 2 | HIGH | Sim | 2s |

| CMD-012 | `AddToWhitelistCommand` | Adiciona item à whitelist | Security Context | Security Kernel | Policy Engine | `RemoveFromWhitelistCommand` | `WhitelistUpdatedEvent` | 3000 | 3 | HIGH | Sim | 1s |

| CMD-013 | `RemoveFromWhitelistCommand` | Remove item da whitelist | Security Context | Security Kernel | Policy Engine | `AddToWhitelistCommand` | `WhitelistUpdatedEvent` | 3000 | 3 | HIGH | Sim | 1s |

| CMD-014 | `DiscoverAssetCommand` | Inicia descoberta de ativos | Asset Context | Discovery Handler | Policy Engine | N/A | `AssetDiscoveryStartedEvent` | 30000 | 1 | MEDIUM | Não | 30s |

| CMD-015 | `ScanNetworkCommand` | Escaneia rede em busca de ativos | Asset Context | Discovery Handler | Policy Engine | N/A | `NetworkScanCompletedEvent` | 60000 | 1 | MEDIUM | Não | 60s |

| CMD-016 | `StartMonitoringCommand` | Inicia monitoramento de ativo | Monitoring Context | Monitoring Handler | Policy Engine | `StopMonitoringCommand` | `MonitoringStartedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-017 | `StopMonitoringCommand` | Para monitoramento de ativo | Monitoring Context | Monitoring Handler | Policy Engine | `StartMonitoringCommand` | `MonitoringStoppedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-018 | `CollectMetricsCommand` | Coleta métricas de ativo | Monitoring Context | Metrics Handler | Policy Engine | N/A | `MetricsCollectedEvent` | 10000 | 3 | MEDIUM | Sim | 5s |

| CMD-019 | `AssessRiskCommand` | Avalia risco de um ativo/incidente | Risk Context | Risk Handler | Policy Engine | N/A | `RiskAssessedEvent` | 10000 | 3 | HIGH | Sim | 5s |

| CMD-020 | `MitigateRiskCommand` | Aplica mitigação de risco | Risk Context | Risk Handler | Policy Engine | `RevertMitigationCommand` | `RiskMitigatedEvent` | 15000 | 2 | CRITICAL | Sim | 10s |

| CMD-021 | `ContainThreatCommand` | Contém uma ameaça identificada | Response Context | Response Handler | Policy Engine | `ReleaseContainmentCommand` | `ThreatContainedEvent` | 10000 | 2 | CRITICAL | Sim | 5s |

| CMD-022 | `IsolateAssetCommand` | Isola um ativo comprometido | Response Context | Response Handler | Policy Engine | `ReintegrateAssetCommand` | `AssetIsolatedEvent` | 10000 | 2 | CRITICAL | Sim | 5s |

| CMD-023 | `ReintegrateAssetCommand` | Reintegra um ativo isolado | Response Context | Response Handler | Policy Engine | `IsolateAssetCommand` | `AssetReintegratedEvent` | 10000 | 2 | HIGH | Sim | 5s |

| CMD-024 | `UpdateThreatIntelCommand` | Atualiza inteligência de ameaças | Intelligence Context | Threat Intel Handler | Policy Engine | N/A | `ThreatIntelUpdatedEvent` | 15000 | 3 | MEDIUM | Sim | 10s |

| CMD-025 | `StoreIndicatorCommand` | Armazena um indicador de comprometimento | Intelligence Context | Threat Intel Handler | Policy Engine | N/A | `IndicatorStoredEvent` | 5000 | 3 | MEDIUM | Sim | 2s |

| CMD-026 | `RequestSuggestionCommand` | Solicita sugestão do AI Advisor | AI Context | AI Handler | Policy Engine | N/A | `SuggestionRequestedEvent` | 10000 | 2 | LOW | Não | 10s |

| CMD-027 | `ApplyAIActionCommand` | Aplica ação sugerida pelo AI | AI Context | AI Handler | Policy Engine | `RevertAIActionCommand` | `AIActionAppliedEvent` | 10000 | 2 | HIGH | Sim | 5s |

| CMD-028 | `CreateTenantCommand` | Cria um novo tenant | System Context | Admin Handler | Policy Engine | `DeleteTenantCommand` | `TenantCreatedEvent` | 10000 | 2 | CRITICAL | Sim | 5s |

| CMD-029 | `UpdateUserRoleCommand` | Atualiza role de usuário | System Context | Admin Handler | Policy Engine | `RevertUserRoleCommand` | `UserRoleUpdatedEvent` | 5000 | 3 | HIGH | Sim | 2s |

| CMD-030 | `ProvisionAssetCommand` | Provisiona um novo ativo | System Context | Infrastructure Handler | Policy Engine | `DeprovisionAssetCommand` | `AssetProvisionedEvent` | 30000 | 1 | HIGH | Sim | 30s |

| CMD-031 | `DeprovisionAssetCommand` | Desprovisiona um ativo | System Context | Infrastructure Handler | Policy Engine | N/A | `AssetDeprovisionedEvent` | 30000 | 1 | HIGH | Sim | 30s |

| CMD-032 | `ScaleServiceCommand` | Escala um serviço | System Context | Infrastructure Handler | Policy Engine | N/A | `ServiceScaledEvent` | 30000 | 1 | HIGH | Sim | 30s |

| CMD-033 | `CompilePlaybookCommand` | Compila um playbook de resposta | Response Context | Response Handler | Policy Engine | N/A | `PlaybookCompiledEvent` | 20000 | 2 | HIGH | Sim | 10s |

| CMD-034 | `ExecutePlaybookCommand` | Executa um playbook | Response Context | Response Handler | Policy Engine | `RollbackPlaybookCommand` | `PlaybookExecutedEvent` | 60000 | 1 | CRITICAL | Sim | 60s |

| CMD-035 | `RollbackPlaybookCommand` | Reverte um playbook executado | Response Context | Response Handler | Policy Engine | N/A | `PlaybookRolledBackEvent` | 60000 | 1 | CRITICAL | Sim | 60s |

| CMD-036 | `UpdateComplianceCommand` | Atualiza status de compliance | Security Context | Compliance Handler | Policy Engine | N/A | `ComplianceUpdatedEvent` | 10000 | 3 | HIGH | Sim | 5s |

| CMD-037 | `GenerateReportCommand` | Gera um relatório de segurança | System Context | Report Handler | Policy Engine | N/A | `ReportGeneratedEvent` | 60000 | 1 | MEDIUM | Não | 60s |

| CMD-038 | `AcknowledgeAlertCommand` | Reconhece um alerta | Incident Context | Incident Handler | Policy Engine | N/A | `AlertAcknowledgedEvent` | 3000 | 3 | LOW | Sim | 1s |

11. Ownership

11.1 Modelo de Ownership

Cada Command tem um **Owner** (Dono) e um **Executor** (Executor):

- **Owner:** O Bounded Context ou módulo responsável pela definição e evolução do Command.
- **Executor:** O componente que realmente processa e executa o Command.

...

Command → Owner (Define)
 → Executor (Processa)

...

11.2 Princípios de Ownership

1. **Somente o Owner pode alterar o Command:** Mudanças no schema ou comportamento do Command devem ser aprovadas pelo Owner.
2. **Outros módulos utilizam Command Bus:** Módulos que não são Owner de um Command não podem executá-lo diretamente; devem enviá-lo via Command Bus.
3. **Responsabilidade de Evolução:** O Owner é responsável por manter a compatibilidade e evoluir o Command.
4. **Documentação:** O Owner deve documentar o Command, incluindo propósito, schema e exemplos.

11.3 Matriz de Ownership

Command	Owner	Executor
----- ----- -----		
`CreateIncidentCommand`	Incident Context	Incident Handler
`EscalateIncidentCommand`	Incident Context	Incident Handler
`BlockIPCommand`	Security Context	Security Kernel

```

| `ApplyPolicyCommand` | Security Context | Policy Engine |
| `DiscoverAssetCommand` | Asset Context | Discovery Handler |
| `StartMonitoringCommand` | Monitoring Context | Monitoring Handler |
| `AssessRiskCommand` | Risk Context | Risk Handler |
| `ContainThreatCommand` | Response Context | Response Handler |
| `UpdateThreatIntelCommand` | Intelligence Context | Threat Intel Handler |
| `RequestSuggestionCommand` | AI Context | AI Handler |
| `CreateTenantCommand` | System Context | Admin Handler |
| `ProvisionAssetCommand` | System Context | Infrastructure Handler |

```

11.4 Exemplo de Fluxo com Ownership

...

Module A (Não Owner de BlockIPCommand)

- Envia BlockIPCommand via Command Bus
- Command Bus roteia para Security Context (Owner)
- Security Kernel (Executor) processa
- Gera evento IPBlockedEvent

...

12. Fluxo de Autorização

12.1 Visão Geral

...

[Cliente]

|

v

[Dashboard / API / Connector]

|

v

[Validação Inicial] -> Rejeita se inválido

|

v

[Command Bus]

|

v

[Security Kernel] -> Autentica e autoriza

|

v

[Policy Engine] -> Avalia políticas (OPA/Cedar)

|

v

[Audit] -> Registra decisão

|

v

[Executor] -> Processa o Command

|
v

[Evento] -> Gera eventos

...

12.2 Detalhamento do Fluxo

12.2.1 Dashboard / API / Connector

- Recebe a solicitação do usuário.
- Converte a solicitação em um Command canônico.
- Adiciona metadados (tenant, correlation_id, etc.).
- Envia para o Command Bus.

12.2.2 Command Bus

- Recebe o Command.
- Valida sintaxe e obrigatoriedade dos campos.
- Verifica schema version.
- Roteia para o Security Kernel.

12.2.3 Security Kernel

- Autentica o usuário/ator.
- Verifica se o ator tem permissão para executar este tipo de Command.
- Aplica RBAC.
- Adiciona informações de contexto (IP, User-Agent).
- Encaminha para Policy Engine.

12.2.4 Policy Engine

- Avalia o Command contra políticas OPA/Cedar.
- Retorna uma decisão: PERMIT, DENY, CONDITIONAL.
- Se PERMIT → Prossegue.
- Se DENY → Rejeita o Command.
- Se CONDITIONAL → Aplica condições adicionais.

12.2.5 Audit

- Registra a decisão do Policy Engine.
- Registra o Command completo (para auditoria).
- Registra o resultado da execução.

12.2.6 Executor

- Processa o Command.
- Aplica lógica de negócio.

- Persiste dados.
- Gera eventos.

12.2.7 Eventos

- Domain Event (mudança de estado).
- Audit Event (registro de auditoria).
- Notification Event (notificações).
- Metrics Event (métricas).

12.3 Diagrama de Fluxo de Autorização

```

```mermaid
sequenceDiagram
 participant Client
 participant Dashboard
 participant CommandBus
 participant SecurityKernel
 participant PolicyEngine
 participant Audit
 participant Executor

 Client->>Dashboard: Solicitação
 Dashboard->>CommandBus: Command
 CommandBus->>SecurityKernel: Command
 SecurityKernel->>SecurityKernel: Autenticar
 SecurityKernel->>SecurityKernel: Autorizar (RBAC)
 SecurityKernel->>PolicyEngine: Command + Context
 PolicyEngine->>PolicyEngine: Avaliar OPA/Cedar
 alt Permitido
 PolicyEngine->>Audit: Decisão PERMIT
 PolicyEngine->>Executor: Command
 Executor->>Executor: Processar
 Executor->>Audit: Resultado
 Executor-->>CommandBus: Eventos
 else Negado
 PolicyEngine->>Audit: Decisão DENY
 PolicyEngine-->>CommandBus: Erro
 end
 CommandBus-->>Dashboard: Resposta
 Dashboard-->>Client: Resposta
```

```

12.4 Políticas OPA/Cedar

Exemplo de política OPA para BlockIPCommand:

```

```rego

```

```
package govsec.policies
```

```
default allow = false
```

```
allow {
 input.command_name == "BlockIPCommand"
 input.metadata.tenant == input.user.tenant
 input.user.roles[] == "security_admin"
 input.payload.ip = valid_ip
 not input.payload.ip = blacklisted_ip
}
```

```
valid_ip {

 regex.match(`^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).){3}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$`, input.payload.ip)
}
```

```
blacklisted_ip {
 input.payload.ip = "0.0.0.0"
}
...
```

```

```

## ## 13. Fluxo CQRS

### ### 13.1 Visão Geral do CQRS

O GovSec Shield implementa CQRS com separação completa entre operações de escrita (Commands) e leitura (Queries).

```
...
```

```
[Escrita] [Leitura]
Command → Validation → Handler → Event → Projection → Query
...
```

### ### 13.2 Fluxo Completo de CQRS

```
```mermaid  
flowchart TD  
    A[Cliente] --> B[API Gateway]  
    B --> C[Command Bus]  
    B --> D[Query Bus]  
  
    C --> E[Validação]  
    E --> F[Autorização]  
    F --> G[Policy Engine]
```

```
G --> H[Handler]
H --> I[Domain Events]
I --> J[Event Store]
J --> K[Projeções]
K --> D
```

```
D --> L[Query Handler]
L --> M[Read Model]
M --> B
```

subgraph Escrita

```
C
E
F
G
H
I
J
end
```

subgraph Leitura

```
D
L
M
end
```

subgraph Sincronização

```
J --> K
end
```

...

13.3 Detalhamento do Fluxo

13.3.1 Recepção do Command

1. Cliente envia solicitação para API Gateway.
2. API Gateway valida autenticação (JWT, API Key, etc.).
3. Command é construído a partir da solicitação.
4. Command é enviado para o Command Bus.

13.3.2 Validação

1. Command Bus valida:
 - Schema JSON (campos obrigatórios, tipos).
 - UUID válido.
 - Tenant válido.
 - Correlation ID presente.
2. Se inválido → Rejeita com erro detalhado.

13.3.3 Autorização

1. Security Kernel autentica o ator.
2. Verifica permissões (RBAC).
3. Verifica MFA (se necessário).
4. Verifica zero trust policies.

13.3.4 Policy Engine

1. Policy Engine avalia Command.
2. Verifica políticas de negócio.
3. Verifica políticas de segurança.
4. Retorna decisão (PERMIT/DENY/CONDITIONAL).

13.3.5 Handler

1. Handler recebe Command.
2. Aplica lógica de negócio.
3. Valida invariantes de domínio.
4. Persiste estado no Write Model (PostgreSQL).
5. Prepara eventos.

13.3.6 Persistência

1. Dados são persistidos no banco de dados de escrita (Write Model).
2. Garantias ACID.
3. Transação inclui eventos.

13.3.7 Domain Events

1. Handler gera Domain Events.
2. Events são publicados no Event Store.
3. Events são publicados no Kafka/Redpanda.

13.3.8 Audit Events

1. Evento de auditoria é gerado.
2. Registrado no sistema de auditoria.

13.3.9 Notificações

1. Notificações são enviadas (email, WebSocket, etc.).
2. Alertas são acionados, se necessário.

13.4 Diagrama de Sequência CQRS

```mermaid

sequenceDiagram

participant Client  
participant API  
participant CommandBus  
participant Validator  
participant SecurityKernel  
participant PolicyEngine  
participant Handler  
participant WriteDB  
participant EventStore  
participant Kafka  
participant Projection  
participant ReadDB  
participant QueryBus

Client->>API: HTTP Request  
API->>CommandBus: Send Command  
CommandBus->>Validator: Validate  
Validator-->>CommandBus: OK  
CommandBus->>SecurityKernel: Authorize  
SecurityKernel-->>CommandBus: Authorized  
CommandBus->>PolicyEngine: Evaluate  
PolicyEngine-->>CommandBus: PERMIT  
CommandBus->>Handler: Execute  
Handler->>WriteDB: Save Aggregate  
WriteDB-->>Handler: OK  
Handler->>EventStore: Append Events  
EventStore-->>Handler: OK  
Handler->>Kafka: Publish Events  
Handler-->>CommandBus: OK  
CommandBus-->>API: Result

Kafka->>Projection: Consume Events  
Projection->>ReadDB: Update Projection  
ReadDB-->>Projection: OK

Client->>QueryBus: Query  
QueryBus->>ReadDB: Query  
ReadDB-->>QueryBus: Data  
QueryBus-->>Client: Response

...

---

## ## 14. Idempotência

### ### 14.1 Princípios de Idempotência

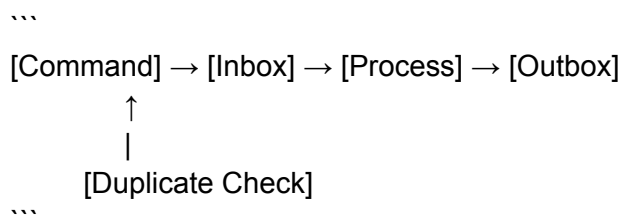
O GovSec Shield garante idempotência através de:

1. **\*\*Naturalmente Idempotente:\*\*** O Command é idempotente por natureza (e.g., `setStatusCommand`).
2. **\*\*Idempotency Key:\*\*** Cliente fornece chave única; sistema deduplica.
3. **\*\*Inbox Pattern:\*\*** Sistema registra Commands processados e descarta duplicatas.

#### ### 14.2 Idempotency Key

- Cliente gera uma chave única (UUID v4 ou v7).
- Inclui no envelope do Command (`metadata.idempotency\_key`).
- O sistema armazena a chave após processamento bem-sucedido.
- Novas tentativas com a mesma chave são detectadas e descartadas.

#### ### 14.3 Inbox Pattern



O Inbox registra:

- `idempotency\_key`
- `command\_id`
- `status` (PENDING, PROCESSING, COMPLETED, FAILED)
- `timestamp`
- `result`

#### ### 14.4 Tabela de Idempotência

```
```sql  
CREATE TABLE idempotency (  
  idempotency_key VARCHAR(128) PRIMARY KEY,  
  command_id UUID NOT NULL,  
  tenant VARCHAR(64) NOT NULL,  
  status VARCHAR(16) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  processed_at TIMESTAMPTZ,  
  result JSONB,  
  expires_at TIMESTAMPTZ NOT NULL,  
  INDEX idx_idempotency_tenant (tenant),  
  INDEX idx_idempotency_expires (expires_at)  
);  
...
```

14.5 TTL e Expiração

- Idempotency Keys expiram após ****24 horas****.
- Commands com mesma chave após expiração são processados novamente.
- Expiração evita crescimento infinito da tabela.

14.6 "Exactly Once" vs "At Least Once"

Conceito	Descrição	Implementação
-----	-----	-----
Exactly Once	Command executado exatamente uma vez.	Idempotency Key + Inbox Pattern
At Least Once	Command executado pelo menos uma vez (pode ter duplicatas).	Retry sem deduplicação

O GovSec Shield implementa ****Exactly Once**** com ****At Least Once**** delivery, utilizando deduplicação para garantir exatamente uma execução.

14.7 Exemplo de Fluxo de Idempotência

```

```mermaid
graph TD
 A[Command Recebido] --> B{Idempotency Key Presente?}
 B -->|Não| C[Processar]
 B -->|Sim| D[Verificar Inbox]
 D --> E{Já Processado?}
 E -->|Sim| F[Retornar Resultado]
 E -->|Não| G[Adicionar Inbox]
 G --> C
 C --> H[Armazenar Resultado]
```

```

15. Retry

15.1 Política de Retry

- ****Número Máximo de Retries:**** 3 (configurável por Command).
- ****Backoff:**** Exponential.
- ****Base de Backoff:**** 100ms.
- ****Multiplicador:**** 2x.
- ****Jitter:**** $\pm 10\%$ para evitar thundering herd.

15.2 Cálculo do Backoff

```

```
Retry 1: 100ms + jitter (10-190ms)
Retry 2: 200ms + jitter (20-380ms)
```

```

Retry 3: 400ms + jitter (40-760ms)
Retry 4: 800ms + jitter (80-1520ms)
...

15.3 Retryable Commands

- Falhas transitórias (timeout, conexão, lock).
- Falhas de recursos (banco de dados temporariamente indisponível).
- Falhas de rede.

15.4 Non-Retryable Commands

- Falhas de validação.
- Falhas de autorização.
- Falhas de integridade (invariantes quebradas).
- Falhas de negócio (regra não satisfeita).
- Deadlock (após tentativas, deve ser escalado).

15.5 Implementação

```
``python
async def execute_with_retry(command, max_retries=3, base_delay=100):
    attempt = 0
    while attempt < max_retries:
        try:
            result = await execute_command(command)
            return result
        except RetryableError as e:
            attempt += 1
            if attempt == max_retries:
                raise MaxRetriesExceeded(e)
            delay = base_delay * (2 ** (attempt - 1))
            delay += random.uniform(-0.1 * delay, 0.1 * delay)
            await asyncio.sleep(delay / 1000)
        except NonRetryableError as e:
            raise
    ...
```

15.6 Retry Queue

Commands que falham são colocados em uma fila de retry com atraso.

```
...
govsec.commands.retry → com delay
...
```

15.7 Monitoramento de Retry

- Métricas: `commands.retry.attempts`, `commands.retry.success`, `commands.retry.failure`.
- Alertas: taxa de retry > 10% deve acionar alerta.
- Dashboard: visualização de retry por Command, tenant, origem.

16. Timeout

16.1 Política de Timeout

Todo Command possui um timeout máximo de execução.

| Categoria | Hard Timeout | Soft Timeout |
|-----------|--------------|--------------|
| ----- | ----- | ----- |
| CRITICAL | 30s | 15s |
| HIGH | 15s | 8s |
| MEDIUM | 10s | 5s |
| LOW | 5s | 3s |

16.2 Hard Timeout

- Tempo máximo de execução.
- Se excedido → Command é cancelado.
- Gera evento de timeout.
- Pode ser retentado (se retryable).

16.3 Soft Timeout

- Alerta interno.
- Aciona logs e métricas.
- Não cancela a execução.

16.4 Cancellation

- Commands podem ser cancelados explicitamente.
- Cancelamento gera `CommandCancelledEvent`.
- Recursos são liberados.

17. Rollback

17.1 Compensating Commands

Rollback é implementado através de **Compensating Commands**:

...

Command → Compensating Command (Rollback)

...

****Exemplo:****

...

BlockIPCommand → UnblockIPCommand
CreateIncidentCommand → CloseIncidentCommand
ApplyPolicyCommand → RevertPolicyCommand
...

17.2 Playbooks de Rollback

- Comandos complexos podem ter playbooks de rollback.
- Playbook define ordem e dependências.
- Execução atômica ou falha.

17.3 Rollback Obrigatório

Commands que exigem rollback obrigatório:

- Commands que afetam segurança (BlockIP, ApplyPolicy).
- Commands que afetam produção (ProvisionAsset, ScaleService).
- Commands que afetam múltiplos sistemas.

17.4 Rollback Proibido

Commands que NÃO podem ter rollback:

- Commands que excluem dados permanentemente (DeleteIncident, DeleteUser).
- Commands que já foram auditados.
- Commands que afetam dados sensíveis (que devem ser preservados).

17.5 Exemplo de Rollback

...

Command: BlockIPCommand
→ Execução: IP bloqueado com sucesso
→ Evento: IPBlockedEvent
→ Rollback: UnblockIPCommand
→ Efeito: IP desbloqueado
→ Evento: IPUnblockedEvent
...

17.6 Diagrama de Rollback

```mermaid

flowchart TD

A[Command] --> B[Execução]

B --> C{Sucesso?}

```
C -->|Sim| D[Evento]
C -->|Não| E[Erro]
E --> F{Rollback Necessário?}
F -->|Sim| G[Executar Compensating Command]
G --> H[Evento de Rollback]
F -->|Não| I[Registrar Falha]
...
```

---

## ## 18. Dead Letter Queue (DLQ)

### ### 18.1 Quando Utilizar

Commands são enviados para DLQ quando:

1. **\*\*Falha Após Máximo de Retries:\*\*** Command falhou após todas as tentativas.
2. **\*\*Falha de Validação Persistente:\*\*** Command consistentemente falha validação.
3. **\*\*Falha de Integridade:\*\*** Command corrompe invariantes do domínio.
4. **\*\*Falha de Negócio:\*\*** Regra de negócio não pode ser satisfeita.
5. **\*\*Command Expirou:\*\*** Timeout excedido repetidamente.

### ### 18.2 Como Reenfileirar

Processo manual ou automatizado:

1. Operador analisa o Command na DLQ.
2. Identifica a causa da falha.
3. Corrige o Command (se necessário).
4. Reenvia o Command para o Command Bus.
5. Ou descarta o Command (se irreparável).

### ### 18.3 Auditoria de DLQ

Todos os Commands em DLQ são auditados:

- Motivo da falha.
- Tentativas realizadas.
- Tempo na DLQ.
- Ações tomadas (reenfileirado, descartado).

### ### 18.4 Monitoramento de DLQ

- Alertas quando DLQ > 10 commands/minuto.
- Dashboard de DLQ.
- Relatórios semanais de análise de DLQ.

### ### 18.5 Estrutura da DLQ

...

govsec.commands.dlq

```
|— tenant: gov-abc
|— command: BlockIPCommand
|— failure_reason: POLICY_DENIED
|— attempts: 3
|— first_failure: 2026-07-28T10:00:00Z
|— last_attempt: 2026-07-28T10:05:00Z
|— status: PENDING_REVIEW
```

...

---

## ## 19. Segurança

### ### 19.1 Zero Trust Architecture

O GovSec Shield implementa Zero Trust em todos os Commands:

```
Princípio Zero Trust	Implementação
Verifique Explicitamente	Autenticação JWT, API Key, ou gRPC mutual TLS para todo
Command.	
Privilégio Mínimo	RBAC com apenas permissões necessárias para cada Command.
Assuma Breach	Segurança em camadas: autenticação, autorização, policy engine,
auditoria.	
Segmentação de Rede	Commands são roteados entre Bounded Contexts via APIs
seguras.	
Monitoramento Contínuo	Commands são monitorados e auditados em tempo real.
```

### ### 19.2 RBAC (Role-Based Access Control)

Permissões definidas por roles:

```
Role	Permissões
`viewer`	Somente Commands de leitura/consulta.
`analyst`	Commands de incidente, risco, monitoramento.
`engineer`	Commands de segurança, asset, infraestrutura.
`security_admin`	Commands de segurança crítica, políticas.
`system_admin`	Todos os Commands (super admin).
```

### ### 19.3 OPA/Cedar

- OPA usado para políticas de autorização.
- Cedar usado para políticas de segurança e compliance.
- Políticas são versionadas e auditadas.

### ### 19.4 MFA (Multi-Factor Authentication)

- Commands críticos exigem MFA.
- MFA via TOTP, WebAuthn, ou SMS.

### ### 19.5 Least Privilege

- Cada Command tem permissões mínimas.
- Princípio de "need-to-know" para dados.

### ### 19.6 Segurança em Commands

| Camada | Mecanismo |  
|-----|-----|  
| Transporte | TLS 1.3, mTLS |  
| Autenticação | JWT, API Key, OAuth2 |  
| Autorização | RBAC, OPA/Cedar |  
| Policy Engine | OPA/Cedar |  
| Auditoria | Eventos de auditoria |  
| Criptografia | Dados sensíveis criptografados |  
| Rate Limiting | Por tenant, ator, e Command |

---

## ## 20. Observabilidade

### ### 20.1 Logs

- Logs estruturados (JSON).
- Níveis: DEBUG, INFO, WARN, ERROR.
- Inclui correlation\_id, command\_id, tenant.
- Logs de entrada e saída de Commands.

### ### 20.2 Tracing

- OpenTelemetry para tracing distribuído.
- Cada Command gera um trace.
- Parent-child relationships.

**\*\*Exemplo de Trace:\*\***

...

Trace: 4bf92f3577b34da6a3ce929d0e0e4736

```
|— Span: API Gateway
| |— Span: Command Bus
| |— Span: Security Kernel
| |— Span: Policy Engine
```

```

|
| └─ Span: Handler
| └─ Span: Write DB
| └─ Span: Event Store
| └─ Span: Kafka
| └─ Span: Query (separado)
...

```

### ### 20.3 Métricas

| Métrica              | Descrição                                  |
|----------------------|--------------------------------------------|
| `commands.received`  | Total de Commands recebidos.               |
| `commands.processed` | Total de Commands processados.             |
| `commands.success`   | Total de Commands bem-sucedidos.           |
| `commands.failure`   | Total de Commands com falha.               |
| `commands.latency`   | Latência de processamento (p50, p95, p99). |
| `commands.timeout`   | Total de Commands com timeout.             |
| `commands.retry`     | Total de retries.                          |
| `commands.dlq`       | Total de Commands na DLQ.                  |
| `commands.by_type`   | Total por tipo de Command.                 |
| `commands.by_tenant` | Total por tenant.                          |

### ### 20.4 OpenTelemetry

- Todos os componentes exportam traces e métricas via OpenTelemetry.
- Collector OpenTelemetry consolida dados.
- Enviado para observability backend (e.g., Grafana, Prometheus, Jaeger).

### ### 20.5 Correlation

- `correlation\_id` une todos os logs, traces, e metrics.
- Permite rastreamento end-to-end.

**\*\*Exemplo de Busca:\*\***

```

```sql
SELECT * FROM logs WHERE correlation_id = 'inc-123-abc-12345'
SELECT * FROM traces WHERE trace_id = '4bf92f3577b34da6a3ce929d0e0e4736'
SELECT * FROM metrics WHERE label_correlation = 'inc-123-abc-12345'
```

```

---

## ## 21. Diagramas Mermaid

### ### 21.1 Diagrama de Sequência - Fluxo Completo do Command

```

```mermaid

```

sequenceDiagram

participant Cliente
participant API
participant CommandBus
participant Validator
participant SecurityKernel
participant PolicyEngine
participant Audit
participant Handler
participant WriteDB
participant EventStore
participant Kafka
participant Projection
participant ReadDB

Cliente->>API: HTTP POST /api/commands
API->>API: Autenticar JWT
API->>CommandBus: Send Command
CommandBus->>Validator: Validate
Validator->>Validator: Schema Validation
Validator-->>CommandBus: OK
CommandBus->>SecurityKernel: Authorize
SecurityKernel->>SecurityKernel: RBAC Check
SecurityKernel-->>CommandBus: Authorized
CommandBus->>PolicyEngine: Evaluate Policies
PolicyEngine->>PolicyEngine: OPA/Cedar
PolicyEngine-->>CommandBus: PERMIT
CommandBus->>Audit: Log Start
CommandBus->>Handler: Execute
Handler->>Handler: Business Logic
Handler->>WriteDB: Save Aggregate
WriteDB-->>Handler: OK
Handler->>EventStore: Append Events
EventStore-->>Handler: OK
Handler->>Kafka: Publish Events
Handler->>Audit: Log Success
Handler-->>CommandBus: Result
CommandBus-->>API: Response
API-->>Cliente: 200 OK

Note over Kafka,Projection: Async Processing
Kafka->>Projection: Consume Events
Projection->>ReadDB: Update Projection
ReadDB-->>Projection: OK

...

21.2 Flowchart - Processamento de Command

```

```mermaid
flowchart TD
 A[Command Recebido] --> B[Validação Inicial]
 B -->|Inválido| C[Rejeitado com Erro]
 B -->|Válido| D[Autenticação]
 D -->|Falha| C
 D -->|Sucesso| E[Autorização RBAC]
 E -->|Falha| C
 E -->|Sucesso| F[Policy Engine]
 F -->|DENY| C
 F -->|PERMIT| G[Log Início]
 G --> H[Handler]
 H --> I[Validação Domínio]
 I -->|Falha| C
 I -->|Sucesso| J[Persistência]
 J --> K[Eventos]
 K --> L[Log Sucesso]
 L --> M[Resposta]

```

```

subgraph Security

```

```

 D

```

```

 E

```

```

 F

```

```

end

```

```

subgraph Processing

```

```

 H

```

```

 I

```

```

 J

```

```

 K

```

```

end

```

```

```

```

21.3 State Diagram - Ciclo de Vida do Command

```

```mermaid
stateDiagram-v2
 [*] --> RECEIVED: Command Recebido
 RECEIVED --> VALIDATING: Validação
 VALIDATING --> VALIDATED: Válido
 VALIDATING --> REJECTED: Inválido
 VALIDATED --> AUTHORIZING: Autorização
 AUTHORIZING --> AUTHORIZED: Autorizado
 AUTHORIZING --> REJECTED: Não Autorizado
 AUTHORIZED --> EVALUATING: Policy Engine
 EVALUATING --> PERMITTED: Permitido
 EVALUATING --> REJECTED: Negado
 PERMITTED --> PROCESSING: Handler

```



PROCESSING --> SUCCESS: Sucesso  
PROCESSING --> FAILED: Falha  
FAILED --> RETRYING: Tentar Novamente  
RETRYING --> SUCCESS: Sucesso  
RETRYING --> FAILED: Falha  
FAILED --> DLQ: Max Retries  
SUCCESS --> COMPLETED: Finalizado  
COMPLETED --> [\*]  
REJECTED --> [\*]  
DLQ --> [\*]  
...

#### ### 21.4 Component Diagram - Command Architecture

```
``mermaid
graph TB
 subgraph "Client Layer"
 Dashboard
 API
 Connector
 end

 subgraph "Gateway Layer"
 APIGateway
 end

 subgraph "Command Bus Layer"
 CommandBus
 Validator
 Router
 end

 subgraph "Security Layer"
 SecurityKernel
 PolicyEngine
 AuditService
 end

 subgraph "Processing Layer"
 IncidentHandler
 SecurityHandler
 RiskHandler
 ResponseHandler
 AIHandler
 SystemHandler
 end

 subgraph "Persistence Layer"
```

```
WriteDB[(Write DB)]
EventStore[(Event Store)]
Kafka[/Kafka/]
end
```

```
Dashboard --> APIGateway
API --> APIGateway
Connector --> APIGateway
```

```
APIGateway --> CommandBus
CommandBus --> Validator
Validator --> Router
Router --> SecurityKernel
SecurityKernel --> PolicyEngine
PolicyEngine --> AuditService
```

```
SecurityKernel --> IncidentHandler
SecurityKernel --> SecurityHandler
SecurityKernel --> RiskHandler
SecurityKernel --> ResponseHandler
SecurityKernel --> AIHandler
SecurityKernel --> SystemHandler
```

```
IncidentHandler --> WriteDB
SecurityHandler --> WriteDB
RiskHandler --> WriteDB
ResponseHandler --> WriteDB
AIHandler --> WriteDB
SystemHandler --> WriteDB
```

```
IncidentHandler --> EventStore
SecurityHandler --> EventStore
RiskHandler --> EventStore
ResponseHandler --> EventStore
AIHandler --> EventStore
SystemHandler --> EventStore
```

```
EventStore --> Kafka
```

```
...
```

```

```

## ## 22. Boas Práticas

### ### 22.1 Design de Commands

1. **\*\*Nomes Expressivos:\*\*** Commands devem ter nomes que expressem intenção clara.
  - Bom: `BlockIPCommand`, `EscalateIncidentCommand`

- Ruim: `UpdateCommand`, `ProcessCommand`
- 2. **\*\*Payload Focado:\*\*** Payload deve conter apenas dados necessários.
  - Evitar dados redundantes.
  - Evitar dados que podem ser inferidos.
- 3. **\*\*Validação Robusta:\*\*** Todos os campos devem ser validados.
  - JSON Schema
  - Validação de negócio
- 4. **\*\*Documentação:\*\*** Todo Command deve ser documentado.
  - Propósito
  - Campos
  - Comportamento

### ### 22.2 Handlers

1. **\*\*Única Responsabilidade:\*\*** Cada Handler deve lidar com um único Command.
2. **\*\*Imutável:\*\*** Handlers devem ser stateless.
3. **\*\*Idempotente:\*\*** Handlers devem ser idempotentes.
4. **\*\*Testável:\*\*** Handlers devem ser facilmente testáveis.

### ### 22.3 Command Bus

1. **\*\*Middleware:\*\*** Usar middleware para aspectos transversais (audit, metrics, retry).
2. **\*\*Falhas Rápidas:\*\*** Rejeitar Commands inválidos rapidamente.
3. **\*\*Rastreabilidade:\*\*** Sempre registrar correlation\_id.

### ### 22.4 Segurança

1. **\*\*Autenticação Sempre:\*\*** Nunca confiar em Commands não autenticados.
2. **\*\*Autorização Granular:\*\*** Cada Command deve ser autorizado explicitamente.
3. **\*\*Políticas Claras:\*\*** Políticas de segurança devem ser explícitas e auditáveis.

### ### 22.5 Performance

1. **\*\*Assíncrono:\*\*** Commands devem ser processados assincronamente sempre que possível.
2. **\*\*Limitado:\*\*** Commands devem ter timeout e retry configurados.
3. **\*\*Escalável:\*\*** Handlers devem ser stateless e horizontalmente escaláveis.

### ### 22.6 Observabilidade

1. **\*\*Logs Estruturados:\*\*** Logs em JSON com campos padronizados.
2. **\*\*Tracing:\*\*** Commands devem gerar traces completos.
3. **\*\*Métricas:\*\*** Métricas devem ser capturadas para todos os Commands.

---

## ## 23. Anti Patterns

### ### 23.1 Práticas Proibidas

Anti-Pattern	Descrição	Por que é Proibido
<b>Executar SQL Diretamente</b>	Handlers executando SQL diretamente no banco de dados.	Quebra abstração, dificulta testes, viola separação de responsabilidades.
<b>Ignorar Policy Engine</b>	Executar Command sem passar pelo Policy Engine.	Viola Zero Trust, compromete segurança.
<b>Executar Command sem Audit</b>	Não gerar eventos de auditoria.	Viola compliance, impossibilita rastreabilidade.
<b>Alterar Estado Diretamente</b>	Mutação de estado sem Command.	Viola CQRS, quebra invariantes.
<b>Commands Síncronos em Operações Demoradas</b>	Commands longos executados sincronicamente.	Degrada performance, aumenta timeout.
<b>Payloads Gigantes</b>	Commands com payloads excessivamente grandes.	Degrada performance, consome memória.
<b>Handlers Stateful</b>	Handlers mantendo estado.	Dificulta escalabilidade, quebra idempotência.
<b>Ignorar Idempotência</b>	Commands sem garantia de idempotência.	Pode causar duplicação de dados.
<b>Rollback Ad-hoc</b>	Rollback sem compensating command formal.	Dificulta rastreabilidade, quebra consistência.

### ### 23.2 Exemplos de Práticas Proibidas

**Proibido: Executar SQL Diretamente**

```
python
PROIBIDO
def execute(self, command: BlockIPCommand):
 cursor.execute("INSERT INTO blocked_ips (ip, cidr) VALUES (%s, %s)", command.ip,
command.cidr)
 cursor.execute("UPDATE incidents SET status = 'blocked' WHERE id = %s",
command.incident_id)
 ...
```

**Correto:**

```
python
CORRETO
def execute(self, command: BlockIPCommand):
 blocked_ip = BlockedIP(ip=command.ip, cidr=command.cidr)
 self.repository.save(blocked_ip)

 incident = self.incident_repository.get(command.incident_id)
```

```
incident.update_status("blocked")
self.incident_repository.save(incident)
...
```

---

## ## 24. Débitos Técnicos

### ### 24.1 Débitos Identificados

ID	Débito	Impacto	Prioridade	Solução Proposta
TD-001	Commands síncronos para operações longas	Performance degradada	HIGH	Migrar para processamento assíncrono com callback
TD-002	Validação de schema não versionada	Quebra de compatibilidade	HIGH	Implementar schema registry
TD-003	Handlers de Command com dependências cíclicas	Acoplamento	MEDIUM	Refatorar para eventos
TD-004	Commands de rollback não implementados	Incapacidade de desfazer	HIGH	Implementar compensating commands
TD-005	DLQ sem processo de reenfileiramento automático	Atraso no processamento	MEDIUM	Implementar reenfileiramento automático
TD-006	Falta de rate limiting por Command	Sobrecarga de recursos	HIGH	Implementar rate limiting por tenant
TD-007	Commands com timeout fixo	Inflexibilidade	LOW	Timeout configurável por Command
TD-008	Métricas de Commands insuficientes	Falta de visibilidade	MEDIUM	Expandir métricas

### ### 24.2 Plano de Mitigação

Débito	Prazo	Responsável
TD-001	Q4 2026	Equipe de Plataforma
TD-002	Q3 2026	Equipe de Arquitetura
TD-003	Q4 2026	Equipe de Segurança
TD-004	Q4 2026	Equipe de Resposta
TD-005	Q1 2027	Equipe de Plataforma
TD-006	Q4 2026	Equipe de Segurança
TD-007	Q2 2027	Equipe de Arquitetura
TD-008	Q4 2026	Equipe de Observabilidade

---

## ## 25. Changelog

### ### 25.1 Versão 1.0.0 (2026-07-28)

#### **\*\*Adições:\*\***

- Especificação completa do modelo canônico de Commands.
- Definição de invariantes do domínio.
- Taxonomia oficial de Commands.
- Catálogo canônico com 38 Commands.
- Fluxos de autorização e CQRS.
- Políticas de idempotência, retry, timeout, rollback, DLQ.
- Diagramas Mermaid detalhados.
- Boas práticas e anti patterns.
- Débitos técnicos identificados.

#### **\*\*Versão Inicial do Documento.\*\***

### **### 25.2 Versões Futuras**

Versão	Data Prevista	Mudanças Planejadas
-----	-----	-----
1.1.0	Q4 2026	Adição de novos Commands, melhorias em métricas
1.2.0	Q1 2027	Suporte a Commands versionados, schema registry
2.0.0	Q2 2027	Breaking changes: novo formato de envelope
2.1.0	Q3 2027	Commands de orquestração

### **### 25.3 Processo de Mudança**

1. Proposta de mudança é submetida ao Comitê de Arquitetura.
2. Revisão técnica e impacto.
3. Aprovação (maioria qualificada).
4. Documento atualizado.
5. Notificação para todas as equipes.
6. Período de transição (se breaking change).

---

## **## Anexos**

### **### Anexo A: Glossário**

Termo	Definição
-----	-----
<b>**Command**</b>	Instrução para mutação de estado.
<b>**Handler**</b>	Componente que executa o Command.
<b>**Command Bus**</b>	Middleware que processa Commands.
<b>**CQRS**</b>	Segregação de Commands e Queries.
<b>**Event Sourcing**</b>	Armazenamento de eventos em vez de estado.
<b>**Idempotência**</b>	Propriedade de execução múltipla ter mesmo efeito.
<b>**Correlation ID**</b>	ID que une múltiplos Commands.
<b>**Tenant**</b>	Unidade de isolamento lógico.
<b>**Policy Engine**</b>	Avalia políticas de segurança/negócio.

**Zero Trust**	Modelo de segurança sem confiança implícita.
**Rollback**	Compensação de Command executado.
**DLQ**	Fila para Commands com falha.

#### ### Anexo B: Referências

- **M0.1** — Visão do Sistema
- **M0.2** — Bounded Contexts
- **M0.3** — Event Storming
- **M0.4** — Modelo ER
- **M0.5** — Versionamento
- **M0.6** — Contratos Canônicos de Eventos
- **RFC 2119**: Key words for use in RFCs to Indicate Requirement Levels
- **SemVer**: Semantic Versioning 2.0.0
- **DDD**: Domain-Driven Design (Eric Evans)
- **CQRS**: Command Query Responsibility Segregation (Martin Fowler)
- **Event Sourcing**: Event Sourcing (Martin Fowler)
- **Zero Trust**: NIST SP 800-207
- **OPA**: Open Policy Agent
- **Cedar**: AWS Cedar Policy Language

#### ### Anexo C: Assinaturas

Papel	Nome	Data	Assinatura
Arquiteto Chefe		2026-07-28	
Comitê de Arquitetura		2026-07-28	
Diretor de Engenharia		2026-07-28	
CISO		2026-07-28	

#### # GovSec Shield

#### ## M0.8 — Modelo Operacional, Capacidades, Governança e Evolução da Plataforma

**Versão:** 1.0.0

**Status:** RATIFICADO PELO COMITÊ DE ARQUITETURA

**Data de Vigência:** 28 de Julho de 2026

**Classificação:** PÚBLICO INTERNO - ESTRATÉGICO

**Nível de Controle:** GOVERNANÇA CORPORATIVA

**Substitui:** N/A

**Impacto:** ESTRATÉGICO, TÁTICO E OPERACIONAL

---

#### ## Sumário Executivo

Este documento estabelece a **Constituição da Evolução** do GovSec Shield. Diferencia-se fundamentalmente de um roadmap temporal ou lista de funcionalidades; trata-se de um **Modelo de Maturidade por Capacidades (Capability Maturity Model)** que define a

trajetória evolutiva da plataforma desde sua fundação arquitetural (M0) até a autonomia supervisionada (M9). Cada estágio de maturidade é uma porta lógica obrigatória, composta por critérios de entrada (DoR), critérios de saída (DoD), dependências rigorosas e KPIs mensuráveis.

A evolução do GovSec Shield não é linear por tempo, mas sim **incremental** por governança. Nenhuma capacidade pode ser ignorada, pulada ou acelerada sem a devida avaliação de riscos e aprovação do Architecture Board. Este documento serve como a referência única para planejamento estratégico, alocação de recursos e certificação de conformidade operacional.

---

## ## 1. Introdução

### ### 1.1 Objetivo

1. **Definir o Modelo de Maturidade Oficial:** Estabelecer uma escada evolutiva (M0 a M9) que oriente o GovSec Shield de um "Sistema Operacional de Segurança" estático para uma "Entidade de Segurança Adaptativa" dinâmica.
2. **Instituir Governança por Capacidades:** Formalizar que todo desenvolvimento deve ser mapeado a uma Capability específica, garantindo que a evolução seja rastreável, segura e auditável.
3. **Operacionalizar a Arquitetura:** Vincular os artefatos técnicos (Commands, Events, Bounded Contexts) às fases de maturidade, criando uma malha de dependências operacionais.
4. **Estabelecer Critérios de Prontidão (DoR/DoD):** Fornecer checklists rigorosos que devem ser atendidos para que uma Capability seja considerada "pronta para evoluir" e "efetivamente entregue".

### ### 1.2 Escopo

Este documento abrange:

- **Modelo de Maturidade (M0–M9):** Detalhamento exaustivo de cada estágio, incluindo pré-requisitos, componentes envolvidos, eventos gerados e KPIs exigidos.
- **Modelo de Capacidades:** Definição e classificação de cada unidade funcional evolutiva.
- **Governança e Decisão:** Processos de Architecture Board, Security Review e gestão de ADRs (Architecture Decision Records) para evolução.
- **Indicadores Operacionais (KPIs/SLOs):** Métricas oficiais que atestam a saúde e eficácia de cada nível de maturidade.
- **Gestão de Riscos:** Matriz de risco inerente a cada salto de capacidade.

### ### 1.3 Não Escopo

- **Detalhamento de Infraestrutura Física:** Provisionamento de hardware, redes subjacentes ou configuração de clusters Kubernetes (serão tratados em documentos de SRE específicos).
- **Código Fonte e Implementação:** Padrões de codificação ou estruturas de repositórios (tratados em guias de engenharia).
- **Recrutamento e Times:** Estratégias de formação de equipes (embora a governança mencione ownership).



- **Modelo Financeiro (FinOps):** Custos específicos de cloud ou licenciamento.

### 1.4 Relação com Documentos Anteriores (M0.1 – M0.7)

| Documento | Relação com M0.8 |

| :--- | :--- |

| **M0.1 (Visão e Paradigma)** | A filosofia de "Security OS" é a base da evolução. M0.8 traduz essa visão em estágios realizáveis. |

| **M0.2 (Bounded Contexts)** | A evolução respeita as fronteiras contextuais. Cada Capability impacta Contextos específicos, mapeados na Matriz de Dependências. |

| **M0.3 (Event Storming)** | As Capacidades são derivadas dos eventos de domínio identificados. M0.8 organiza esses eventos em ondas de maturidade. |

| **M0.4 (Modelo ER)** | A evolução exige migrações de schema. M0.8 define critérios de backward compatibility para mudanças de dados. |

| **M0.5 (Versionamento)** | O versionamento SemVer é a alavanca técnica para evoluir sem quebrar contratos. M0.8 alinha a maturidade operacional à maturidade de versões. |

| **M0.6 (Contratos Canônicos)** | Eventos devem ser estáveis. M0.8 exige que novas capacidades gerem eventos com schemas previamente aprovados. |

| **M0.7 (Modelo de Commands)** | O Catálogo de Commands se expande com a maturidade. M0.8 define *quais* Commands são obrigatórios em *qual* estágio. |

### 1.5 Propósito Operacional

O GovSec Shield não é um projeto finito; é uma plataforma viva. Este documento estabelece o "modo de operação" contínuo, garantindo que:

- **A Segurança evolui com a ameaça:** A capacidade de resposta é aprimorada incrementalmente, sem revoluções destrutivas.
- **A Automação é conquistada:** A automação total (M9) só é alcançada após a observabilidade (M1), correlação (M3) e IA consultiva (M5) estarem maduras.
- **A Governança é o freio de mão:** Nenhuma automação é permitida sem supervisão humana validada por políticas.

---

## 2. Filosofia da Evolução

### 2.1 Capability Driven Development (CDD)

O GovSec Shield adota o **Desenvolvimento Orientado a Capacidades**. Uma Capacidade (Capability) é a menor unidade de valor de negócio que pode ser entregue, medida e governada de forma independente.

- **Por que Capacidades?** Versões (1.0, 2.0) são artefatos de release. Capacidades são artefatos de *valor*. A capacidade "Correlação de Eventos" pode ser entregue em múltiplas versões menores, mas só é considerada ativa quando atinge um patamar de precisão (KPI) definido.
- **Incremental Architecture:** A arquitetura não é "big-bang". Cada capacidade adiciona camadas sobre a base sólida (Hexagonal + Clean Architecture), permitindo rollback granular de funcionalidades se falhas de segurança forem detectadas.

### 2.2 Continuous Evolution vs. Revolução

- **\*\*Proibido Big-Bang:\*\*** Migrações de M0 para M1 que envolvam reescrita total são terminantemente proibidas. A evolução deve ser *\*estruturalmente compatível\** (Strangler Fig Pattern).
- **\*\*Adaptive Security Evolution (ASE):\*\*** Inspirado no NIST SP 800-207 (Zero Trust), a plataforma deve adaptar suas defesas com base no ambiente operacional. Se os KPIs de M3 (Correlação) caírem abaixo do SLO, a plataforma pode reverter automaticamente a execução de certos Commands para o estágio M2, até que a confiabilidade seja restaurada.

### ### 2.3 Governança por Capacidades

Toda nova funcionalidade proposta deve responder a duas perguntas:

1. *\*Qual capacidade esta funcionalidade está aprimorando?\**
2. *\*A capacidade dependente (anterior) está em estado "Operational Excellence"?\**

### ### 2.4 Por que Maturidade, não Cronologia?

Datas são arbitrárias; maturidade é baseada em **\*\*evidências\*\***. A evolução de M3 para M4 (Threat Intelligence) depende de um volume mínimo de dados processados e de uma precisão de correlação mínima. Sem esses KPIs, a IA de inteligência de ameaças irá "alucinar" ou gerar falsos positivos. A maturidade protege o sistema contra a automação prematura.

---

## ## 3. Princípios Fundamentais

| Princípio | Definição Formal | Implicação para M0.8 |

| :--- | :--- | :--- |

| **\*\*Capability First\*\*** | Toda entrega deve ser mapeada a uma Capability do modelo M0–M9. | Funcionalidades órfãs (sem Capability) são rejeitadas no Architecture Review. |

| **\*\*Security by Default\*\*** | A segurança não é uma camada; é a base. Toda nova Capability herda a postura Zero Trust. | Uma nova Capability não pode ser ativada até que o Security Kernel (M0) a valide. |

| **\*\*Observability First\*\*** | "Não se gerencia o que não se mede". | Uma Capability só pode ser declarada "Pronta" (DoD) se 100% de seus traces estiverem expostos via OpenTelemetry. |

| **\*\*Automation Last\*\*** | Automação é o destino, não o ponto de partida. | Nenhuma automação (M7) é permitida antes de M5 (AI Advisory) e M6 (Copilot) estarem validados por humanos. |

| **\*\*AI is Advisory\*\*** | IA no GovSec Shield é consultiva, nunca executiva, até que o estágio M9 seja atingido com supervisão. | Commands gerados por IA (ex:

`ApplyAIActionCommand`) devem sempre passar pelo Policy Engine com flag  
`requires\_approval=true` até M9. |

| **\*\*Policy Before Automation\*\*** | O Policy Engine é o único oráculo que permite a execução. | Mesmo em M9, a Autonomia Supervisionada deve consultar o Policy Engine para cada ação. |

| **\*\*Human Oversight\*\*** | O elemento humano é o juiz final da estratégia. | Todo rollback (M7+) ou decisão de contenção crítica deve permitir intervenção humana em até 5 minutos.

|

| **Fail Safe** | Em caso de falha do sistema, o estado deve retornar ao último estado seguro conhecido. | Mecanismos de Circuit Breaker nas Capabilities de Automação são obrigatórios. |

| **Deterministic Evolution** | Dado o mesmo estado inicial e a mesma sequência de eventos, a evolução da Capability deve ser reproduzível. | Replays de eventos históricos não devem alterar o modelo de maturidade atual. |

| **Backward Compatibility** | A evolução não quebra os consumidores existentes. | Novas versões de Commands (MAJOR) só são permitidas com suporte a múltiplas versões (Polyglot Persistence de schemas). |

---

## ## 4. Linguagem Ubíqua (Estratégica)

| Termo | Definição Formal | Exemplo |

| :--- | :--- | :--- |

| **Capability** | Unidade atômica de valor de negócio, governada por um conjunto de SLOs e KPIs, representando um estágio de maturidade funcional. | `Threat Intelligence Capability (M4)` |

| **Maturity Level** | Estágio numérico (M0–M9) que define o grau de autonomia, correlação e inteligência da plataforma. | `M3 - Correlação` |

| **Baseline** | O estado mínimo aceitável de desempenho para uma Capability ser considerada estável. | `Baseline M2: 95% de uptime do monitoramento.` |

| **Operational Readiness** | Estado em que a plataforma possui procedimentos (runbooks) e ferramentas para manter a Capability em produção. | Aprovação do time de SRE para o novo dashboard. |

| **Operational Excellence** | Estado em que a Capability não apenas funciona, mas excede consistentemente seus SLOs e possui mecanismos de auto-recuperação. | `M3 Excellence: FPR < 1% por 7 dias consecutivos.` |

| **Definition of Ready (DoR)** | Checklist de critérios que devem ser satisfeitos *antes* do início do desenvolvimento de uma Capability. | ADR aprovado, Eventos modelados. |

| **Definition of Done (DoD)** | Checklist de critérios que devem ser satisfeitos *após* o desenvolvimento para liberar a Capability em produção. | Testes integrados, Docs atualizados. |

| **Governance Gate** | Ponto de decisão formal onde o Architecture Board avalia a evolução e autoriza (ou não) a progressão. | Gate entre M4 e M5. |

| **Feature** | Funcionalidade específica que contribui para a realização de uma Capability. | `Feature: Correlação de logs de firewall com incidentes.` |

| **Epic** | Conjunto de Features necessário para alcançar um Maturity Level. | `Epic M4: Ingestão de feeds STIX/TAXII.` |

| **Milestone** | Marco de entrega de uma Capability. | `Milestone M3 atingido em produção.` |

| **Roadmap** | Representação gráfica da evolução baseada em dependências (não em tempo). | `M2 -> M3 -> M4 (paralelo com M5).` |

| **Dependency** | Requisito técnico ou operacional que uma Capability inferior deve atender para que a superior seja iniciada. | `M5 depende de M4 (dados de TI).` |

| **Risk Acceptance** | Decisão formal do CISO de prosseguir com uma Capability mesmo com riscos conhecidos. | Assinatura para prosseguir com M7 apesar de FPR em 2%. |

| **Operational KPI** | Indicador quantitativo de desempenho operacional. | `MTTD (Mean Time to Detect).` |  
| **Operational SLO** | Meta quantitativa para um KPI. | `SLO MTTD < 1 minuto para M3.` |

---

## ## 5. Modelo Oficial de Capacidades

O GovSec Shield define suas Capacidades como uma pilha verticalizada, onde cada camada depende da robustez da camada inferior.

### ### 5.1 Hierarquia e Natureza

- **Capacidades Fundamentais (M0-M2):** Habilitadoras. Sem elas, o sistema não existe ou é cego.
- **Capacidades Cognitivas (M3-M5):** Analíticas. Transformam dados em inteligência.
- **Capacidades Autônomas (M6-M9):** Executivas. Transformam inteligência em ação.

### ### 5.2 Critérios de Evolução

Para evoluir de `Mn` para `M(n+1)`, a plataforma DEVE:

1. **Demonstrar Estabilidade:** A Capabilidade atual deve ter operado em regime de "Operational Excellence" por no mínimo 15 dias corridos.
2. **Cobertura de Dados:** A quantidade de eventos/telemetria processados deve ser estatisticamente significativa (> 1 milhão de eventos/dia).
3. **Validação de Segurança:** O Security Kernel deve atestar que a nova Capabilidade não introduz vetores de ataque.

---

## ## 6. Modelo de Maturidade (M0 – M9)

### ### 6.1 M0 — Arquitetura (Foundation)

- **Objetivo:** Estabelecer a base imutável do Security OS, conforme especificado nos documentos M0.1–M0.7.
- **Descrição:** A plataforma está "viva", mas opera em modo de "inserção passiva" (sink mode). Processa eventos, armazena dados, mas não realiza correlação ativa nem respostas automatizadas. Os Commands básicos (`CreateIncidentCommand`, `BlockIPCommand`) existem, mas requerem acionamento manual via Dashboard.
- **Pré-requisitos:** N/A (Ponto de partida).
- **Critérios de Entrada:** Decisão de patrocínio e arquitetura aprovada.
- **Critérios de Saída:** Todos os Bounded Contexts estão implantados, o Kafka (Redpanda) está operacional, e o fluxo de ingestão de logs está estável.
- **Componentes:** Core Engine, Command Bus, PostgreSQL, Kafka, Security Kernel.
- **Eventos Produzidos:** `SystemBootstrappedEvent`, `IngestionStartedEvent`.
- **Commands Utilizados:** `CreateTenantCommand`, `IngestLogCommand`.
- **KPIs:** Disponibilidade da API > 99.9%. Latência de ingestão < 100ms.
- **SLOs:** MTTR de deploy < 10 min.
- **Riscos:** Vulnerabilidades na camada base.
- **Limitações:** Zero automação. Inteligência artificial ausente.

- **Próxima Capability:** M1 – Observabilidade.

---

#### ### 6.2 M1 — Observabilidade (Observability)

- **Objetivo:** Tornar o sistema 100% transparente internamente. "Ver a si mesmo".
- **Descrição:** Instrumentação completa via OpenTelemetry. Todos os microserviços exportam traces, métricas e logs estruturados. O painel central (Grafana) mostra a saúde de cada Bounded Context.
- **Pré-requisitos:** M0 estável.
- **Critérios de Entrada:** Definição dos Golden Signals para cada serviço.
- **Critérios de Saída:** Cobertura de tracing em 100% das requisições HTTP/gRPC e processamento de Commands.
- **Componentes:** OpenTelemetry Collector, Grafana, Tempo, Prometheus, Loki.
- **Eventos Produzidos:** `TelemetryEmittedEvent`.
- **Commands Utilizados:** N/A (apenas configuração).
- **KPIs:** Cobertura de traces > 99%. Erro de coleta < 0.1%.
- **SLOs:** p95 de latência de exportação < 50ms.
- **Riscos:** Sobrecarga do coletor de telemetria.
- **Limitações:** Apenas monitoramento passivo; sem análise ativa.
- **Próxima Capability:** M2 – Monitoramento.

---

#### ### 6.3 M2 — Monitoramento (Monitoring)

- **Objetivo:** Transformar a telemetria em alertas acionáveis e dashboards de negócio.
- **Descrição:** Criação de regras de alerta (Prometheus Alertmanager). Dashboards específicos por contexto (Incident, Security, Assets). Estabelecimento de "Health Checks" para cada serviço. On-call rotation configurada.
- **Pré-requisitos:** M1 (dados de telemetria disponíveis).
- **Critérios de Entrada:** Definição de alertas críticos (CPU, memória, filas Kafka).
- **Critérios de Saída:** Todos os serviços com dashboards aprovados pela SRE. Testes de alerta (Fire Drill) realizados.
- **Componentes:** Alertmanager, Grafana Dashboards, PagerDuty/Opsgenie.
- **Eventos Produzidos:** `AlertFiredEvent`, `AlertResolvedEvent`.
- **Commands Utilizados:** `AcknowledgeAlertCommand` (humano).
- **KPIs:** Tempo de detecção de falha (MTTD) < 5 min.
- **SLOs:** 100% de alerts críticos disparando em até 1 minuto.
- **Riscos:** Alert fatigue devido a regras mal calibradas.
- **Limitações:** Alertas são isolados; não há correlação entre eles.
- **Próxima Capability:** M3 – Correlação.

---

#### ### 6.4 M3 — Correlação (Correlation)

- **Objetivo:** Unir eventos aparentemente desconexos para formar uma história de incidente coesa.

- **\*\*Descrição:\*\*** Implementação de um Motor de Correlação (CEP - Complex Event Processing). O sistema começa a gerar `CorrelatedIncidentCommand` com base em regras (ex: Falha de autenticação + Acesso a endpoint sensível). Redução massiva de ruído.
- **\*\*Pré-requisitos:\*\*** M2 (Alertas configurados) e M0 (Schema de eventos normalizado).
- **\*\*Critérios de Entrada:\*\*** Conjunto mínimo de 50 regras de correlação definidas (MITRE ATT&CK mapping).
- **\*\*Critérios de Saída:\*\*** Taxa de Falsos Positivos (FPR) na correlação < 5%.
- **\*\*Componentes:\*\*** CEP Engine, Rule Repository (Policy Engine), Knowledge Graph (inicial).
- **\*\*Eventos Produzidos:\*\*** `IncidentCorrelatedEvent`, `FalsePositivelyIdentifiedEvent`.
- **\*\*Commands Utilizados:\*\*** `CreateIncidentCommand` (gerado automaticamente), `EscalateIncidentCommand` (automático baseado em severidade).
- **\*\*KPIs:\*\*** MTTD (Mean Time to Detect) < 60 segundos. Redução de alertas em 80% (consolidação).
- **\*\*SLOs:\*\*** Precisão da correlação > 95%.
- **\*\*Riscos:\*\*** Correlação excessiva gerando incidentes falsos (sobrecarga do analyst).
- **\*\*Limitações:\*\*** Dependente de regras estáticas; não aprende com o tempo.
- **\*\*Próxima Capability:\*\*** M4 – Threat Intelligence.

---

#### ### 6.5 M4 — Threat Intelligence (Inteligência de Ameaças)

- **\*\*Objetivo:\*\*** Enriquecer os incidentes com contexto externo e histórico (IoCs, TTPs).
- **\*\*Descrição:\*\*** Ingestão de feeds STIX/TAXII (governamentais, comerciais e open-source). Enriquecimento automático de IPs, Domínios e Hashes durante a correlação (M3). Atualização da Knowledge Base com pontuação de risco dinâmica.
- **\*\*Pré-requisitos:\*\*** M3 operacional. Feeds de TI contratados/configurados.
- **\*\*Critérios de Entrada:\*\*** Pelo menos 3 feeds externos ingeridos.
- **\*\*Critérios de Saída:\*\*** Cobertura de enriquecimento em 90% dos incidentes gerados.
- **\*\*Componentes:\*\*** TI Aggregator, STIX/TAXII Parser, Knowledge Base (Neo4j/PostgreSQL Vetorial).
- **\*\*Eventos Produzidos:\*\*** `ThreatIntelUpdatedEvent`, `IndicatorMatchedEvent`.
- **\*\*Commands Utilizados:\*\*** `UpdateThreatIntelCommand` (automático), `StoreIndicatorCommand`.
- **\*\*KPIs:\*\*** Cobertura de IoCs conhecidos > 80%. Tempo de atualização de feeds < 5 min.
- **\*\*SLOs:\*\*** Latência de enriquecimento < 500ms por evento.
- **\*\*Riscos:\*\*** Envenenamento de feeds (poisoning) levando a decisões erradas.
- **\*\*Limitações:\*\*** Inteligência é reativa (baseada em listas) e não preditiva.
- **\*\*Próxima Capability:\*\*** M5 – AI Advisory.

---

#### ### 6.6 M5 — AI Advisory (Assistência Consultiva)

- **\*\*Objetivo:\*\*** Utilizar IA (LLMs/ML) para sugerir cursos de ação ao analista, sem executar nada automaticamente.
- **\*\*Descrição:\*\*** O AI Advisor analisa o contexto do incidente (correlação + threat intel) e gera uma `Suggestion` (recomendação). Essa sugestão é apresentada no dashboard. A execução exige clique humano, que dispara o Command correspondente.

- **Pré-requisitos:** M4. Modelos de ML treinados com dados históricos de M3/M4.
- **Critérios de Entrada:** Modelo de linguagem (LLM) aprovado (on-prem ou via AI Gateway) e com políticas de privacidade validadas.
- **Critérios de Saída:** Taxa de Aceitação de Sugestões pelos analistas > 60%.
- **Componentes:** AI Gateway (LLM), Knowledge Base (RAG - Retrieval Augmented Generation), Suggestion Engine.
- **Eventos Produzidos:** `SuggestionGeneratedEvent`, `SuggestionAcceptedEvent` (humano), `SuggestionRejectedEvent`.
- **Commands Utilizados:** `RequestSuggestionCommand` (automático), `ApplySuggestionCommand` (humano) \*- Neste estágio, este Command sempre tem flag `requires_approval=true`.
- **KPIs:** Precisão da sugestão (acurácia) > 70%. Relevância (feedback humano) > 4/5 estrelas.
- **SLOs:** Tempo de geração de sugestão < 5s.
- **Riscos:** Alucinações (hallucinations) da IA gerando sugestões perigosas.
- **Limitações:** A IA é consultiva, não possui contexto de execução em tempo real.
- **Próxima Capability:** M6 – AI Copilot.

---

#### ### 6.7 M6 — AI Copilot (Assistente Colaborativo)

- **Objetivo:** Integrar a IA ao fluxo de trabalho diário do analista como um "par" que interage via linguagem natural e acelera ações manuais.
- **Descrição:** O Copilot entende comandos em linguagem natural (ex: "Mostre todos os assets vulneráveis a CVE-2026-1234 e isole os críticos"). O Copilot traduz isso para queries SQL/Graph e, mediante confirmação do usuário, executa Commands em lote.
- **Pré-requisitos:** M5. Chat interface implementada no Dashboard.
- **Critérios de Entrada:** A interface de chat deve suportar autenticação e auditoria de todas as interações.
- **Critérios de Saída:** 80% dos analistas utilizando a ferramenta semanalmente.
- **Componentes:** Chat UI, Intent Parser, Command Translator.
- **Eventos Produzidos:** `CopilotInteractionEvent`.
- **Commands Utilizados:** Qualquer Command do catálogo M0.7, agora acionado via linguagem natural.
- **KPIs:** Redução do tempo médio de investigação (MTTI) em 40%.
- **SLOs:** Precisão da tradução de intenção > 90%.
- **Riscos:** Comandos mal interpretados (ex: "isolar" vs "observar").
- **Limitações:** Ainda depende do clique humano para Execução de alto risco.
- **Próxima Capability:** M7 – Automação.

---

#### ### 6.8 M7 — Automação (Automation)

- **Objetivo:** Executar playbooks de resposta pré-definidos automaticamente para incidentes de baixa e média criticidade.
- **Descrição:** O Policy Engine, em conjunto com a AI Advisory (M5), decide se um incidente se enquadra em um padrão conhecido (ex: Scanning de porta). O sistema dispara

automaticamente o `ExecutePlaybookCommand`, que isola o ativo sem intervenção humana. Apenas incidentes críticos ou não classificados vão para a fila humana.

- **\*\*Pré-requisitos:\*\*** M6. Playbooks testados em ambiente de staging.
- **\*\*Critérios de Entrada:\*\*** Matriz de Decisão de Automação aprovada pelo CISO.
- **\*\*Critérios de Saída:\*\*** Taxa de Automação (eventos resolvidos sem humano) > 30%.
- **\*\*Componentes:\*\*** Playbook Executor, Orchestrator (Saga), Policy Engine (OPA).
- **\*\*Eventos Produzidos:\*\*** `AutomationTriggeredEvent`, `PlaybookExecutionSucceededEvent`, `PlaybookExecutionFailedEvent`.
- **\*\*Commands Utilizados:\*\*** `ExecutePlaybookCommand`, `IsolateAssetCommand`, `BlockIPCommand` (agora com flag `requires_approval=false` para baixo risco).
- **\*\*KPIs:\*\*** Tempo de Contenção (MTTC) reduzido em 60% (comparado a M0).
- **\*\*SLOs:\*\*** Taxa de sucesso da automação > 95%.
- **\*\*Riscos:\*\*** Efeito dominó (cascade failure) se um playbook mal escrito for executado em larga escala.
- **\*\*Limitações:\*\*** A automação é reativa e baseada em regras estáticas (playbooks).
- **\*\*Próxima Capability:\*\*** M8 – Red Team Autônomo.

---

#### ### 6.9 M8 — Red Team Autônomo (Autonomous Red Team)

- **\*\*Objetivo:\*\*** A plataforma testa proativamente suas próprias defesas.
- **\*\*Descrição:\*\*** O sistema gera cenários de ataque simulado (usando Commands específicos de teste) contra ambientes de staging e, posteriormente, produção (com isolamento). Ele verifica se as correlações (M3), a inteligência (M4) e a automação (M7) detectam e contêm esses ataques. Se falham, o sistema gera tickets de melhoria (Débitos Técnicos).
- **\*\*Pré-requisitos:\*\*** M7. Framework de testes de invasão integrado (ex: Metasploit, custom agents).
- **\*\*Critérios de Entrada:\*\*** Baseline de cobertura MITRE ATT&CK definida.
- **\*\*Critérios de Saída:\*\*** Cobertura MITRE > 70% testada automaticamente.
- **\*\*Componentes:\*\*** Red Team Engine, Simulation Sandbox, Compliance Reporter.
- **\*\*Eventos Produzidos:\*\*** `AttackSimulationStartedEvent`, `DefenseGapIdentifiedEvent`.
- **\*\*Commands Utilizados:\*\*** `SimulateAttackCommand`, `GenerateReportCommand`.
- **\*\*KPIs:\*\*** Cobertura MITRE ATT&CK (aumento contínuo). Tempo de detecção de ataques simulados.
- **\*\*SLOs:\*\*** 100% dos ataques simulados críticos detectados em < 1 minuto.
- **\*\*Riscos:\*\*** Ataques simulados vazarem para produção ou causarem negação de serviço.
- **\*\*Limitações:\*\*** Focado em ataques conhecidos; inovação de ataque (novas táticas) ainda é limitada.
- **\*\*Próxima Capability:\*\*** M9 – Autonomia Supervisionada.

---

#### ### 6.10 M9 — Autonomia Supervisionada (Supervised Autonomy)

- **\*\*Objetivo:\*\*** A plataforma opera como um "SOC Autônomo", gerenciando 95% dos incidentes do início ao fim, com humanos como supervisores estratégicos.
- **\*\*Descrição:\*\*** O sistema não apenas executa playbooks (M7) ou testa (M8), mas também **\*\*adapta suas próprias políticas\*\*** com base nos resultados. Ele propõe novas regras de



correlação, ajusta thresholds de alerta e aprende com os acertos/erros. O analista humano atua como "governador", revisando relatórios agregados e intervindo apenas em "dilemas éticos" ou decisões de alto impacto financeiro/político.

- **Pré-requisitos:** M8. Modelos de Reinforcement Learning (RL) maduros.
- **Críticos de Entrada:** Estrutura de "Human-in-the-loop" para override de decisões.
- **Críticos de Saída:** Confiança do sistema de auto-avaliação > 99%.
- **Componentes:** Self-Healing Engine, Adaptive Policy Engine, Continuous Improvement Loop.
- **Eventos Produzidos:** `PolicyAutoTunedEvent`, `AutonomousDecisionExecutedEvent`.
- **Commands Utilizados:** Todos os Commands existentes, com exceção de `DeleteTenantCommand` (explicitamente bloqueado para AI), operam com `requires\_approval=false`, mas geram eventos de justificativa detalhados.
- **KPIs:** Taxa de Autonomia (resolução sem humano) > 95%. Taxa de Reversão de decisões autônomas < 0.5%.
- **SLOs:** MTTR Geral < 30 segundos.
- **Riscos:** **Existencial** - Viés algorítmico levando a bloqueios generalizados injustos. Por isso, o "Modo Supervisor" humano tem uma "Kill Switch" física (circuit breaker) que reverte toda a automação para M5 (Advisory Only) em menos de 1 minuto.
- **Limitações:** Depende de dados de treino não enviesados. Decisões contextuais (ex: política) ainda requerem validação final.
- **Próxima Capability:** **Loop contínuo de melhoria (M9+)**.

---

## ## 7. Matriz Oficial de Dependências

Capability	Depende de (Pré-requisito)	Impacta (Contextos)	Componentes Críticos	Eventos Chave	Commands Chave	Banco de Dados	Broker
:---	:---	:---	:---	:---	:---	:---	:---
<b>M0 - Arquitetura</b>	N/A	Todos	Command Bus, Security Kernel	`IngestionStartedEvent`	`CreateTenantCommand`	PostgreSQL	Kafka
<b>M1 - Observabilidade</b>	M0	Todos	OTEL Collector, Grafana	`TelemetryEmittedEvent`	-	PostgreSQL (logs)	Kafka
<b>M2 - Monitoramento</b>	M1	SRE, On-call	Alertmanager, PagerDuty	`AlertFiredEvent`	`AcknowledgeAlertCommand`	PostgreSQL	Kafka
<b>M3 - Correlação</b>	M2	Incident, Security	CEP Engine, Policy Engine	`IncidentCorrelatedEvent`	`CreateIncidentCommand` (auto)	PostgreSQL (Read-optimized)	Kafka
<b>M4 - Threat Intel</b>	M3	Intelligence, Security	TI Aggregator, Knowledge Base	`IndicatorMatchedEvent`	`UpdateThreatIntelCommand`	PostgreSQL (Vetorial)	Kafka
<b>M5 - AI Advisory</b>	M4	AI, Incident	AI Gateway, RAG Engine	`SuggestionGeneratedEvent`	`ApplySuggestionCommand` (aprovado)	Knowledge Base (Neo4j)	Kafka
<b>M6 - AI Copilot</b>	M5	Todos (UI)	Chat UI, Intent Parser	`CopilotInteractionEvent`	N/A (geração de Queries)	-	-
<b>M7 - Automação</b>	M6	Response, Security	Orchestrator, Playbook Executor	`AutomationTriggeredEvent`	`ExecutePlaybookCommand` (auto)	PostgreSQL	Kafka

| **M8 - Red Team** | M7 | Security, Assurance | Simulator, MITRE Mapper |  
`DefenseGapIdentifiedEvent` | `SimulateAttackCommand` | PostgreSQL (Testes) | Kafka |  
| **M9 - Autonomia** | M8 | Todos (Governança) | Self-Healing Engine, RL Agent |  
`AutonomousDecisionExecutedEvent` | \*Todos (com exceções)\* | PostgreSQL | Kafka |

---

## ## 8. Governança

### ### 8.1 Architecture Board (AB)

- **Composição:** Arquiteto Chefe, Security Architect, SRE Lead, Product Manager (segurança), CISO (convidado).
- **Atribuições:** Aprovar a transição entre Maturity Levels. Revisar ADRs (Architecture Decision Records). Manter a integridade dos princípios de DDD e Clean Architecture.
- **Quórum:** 4/5 membros para aprovação de M1-M6; 5/5 membros + aval do CISO para M7-M9.

### ### 8.2 Security Review (SRB)

- **Foco:** Garantir que a nova Capability não viola Zero Trust.
- **Output:** Relatório de Segurança contendo análise de threat modeling (STRIDE) e validação de políticas OPA/Cedar.

### ### 8.3 Processo de Mudança (RFC/ADR)

1. **RFC (Request for Change):** Proposta inicial descrevendo a Feature e a Capability alvo.
2. **ADR:** Decisão arquitetural formal aprovada pelo AB.
3. **Implementação:** Segue as checklists de DoR e DoD.
4. **Validação:** Testes de carga, segurança e observabilidade.
5. **Promoção:** Implantação em produção via Progressive Delivery (Canary/Blue-Green).

---

## ## 9. Critérios de Evolução

### ### 9.1 Prontidão para Avançar

Para avançar de `Mn` para `M(n+1)`, os seguintes critérios são MANDATÓRIOS:

1. **KPIs Sustentados:** Os KPIs de `Mn` devem estar verdes por 15 dias consecutivos.
2. **Dívida Técnica Controlada:** O débito técnico específico de `Mn` deve ser inferior a 5% do esforço total de desenvolvimento.
3. **Runbooks Atualizados:** Os runbooks de incidentes para `Mn` devem existir e ter sido testados em drills.
4. **Aprovação do CISO:** Para M5 em diante.

---

## ## 10. Definition of Ready (DoR) – Checklist

Para que o desenvolvimento de uma Capability seja iniciado, o time DEVE atestar os seguintes itens:

- [ ] **\*\*Arquitetura Aprovada:\*\*** ADR finalizado e revisado pelo Architecture Board.
- [ ] **\*\*Modelagem de Domínio:\*\*** Eventos e Commands afetados foram mapeados (M0.6/M0.7).
- [ ] **\*\*Impacto em Dados:\*\*** Schema de banco de dados mapeado e aprovado (M0.4).
- [ ] **\*\*Matriz de Dependências:\*\*** Dependências com outras Capabilities foram validadas.
- [ ] **\*\*Testes Planejados:\*\*** Estratégia de testes (unitário, integração, contrato) definida.
- [ ] **\*\*KPIs Definidos:\*\*** Métricas de sucesso para a nova Capability documentadas.
- [ ] **\*\*Riscos Identificados:\*\*** Threat Modeling realizado e riscos aceitos (ou mitigados).
- [ ] **\*\*Orçamento Alocado:\*\*** Recursos humanos e financeiros disponíveis.

---

## ## 11. Definition of Done (DoD) – Checklist

Para que a Capability seja considerada ENTREGUE em produção, o time DEVE atestar:

- [ ] **\*\*Cobertura de Testes:\*\*** > 90% (Unitário) e 80% (Integração) – estatisticamente.
- [ ] **\*\*Contratos Testados:\*\*** Testes de contrato (Pact) para Commands/Eventos aprovados.
- [ ] **\*\*Observabilidade:\*\*** Traces, Métricas e Logs implementados e visualizáveis no Grafana.
- [ ] **\*\*Documentação:\*\*** Docs atualizadas (arquitetura, onboarding, runbooks).
- [ ] **\*\*Rollback:\*\*** Plano de rollback documentado e testado com sucesso.
- [ ] **\*\*Performance:\*\*** Teste de carga aprovado (latência p99 dentro do SLO).
- [ ] **\*\*Segurança:\*\*** Scan de vulnerabilidades (SAST/DAST) aprovado.
- [ ] **\*\*Versionamento:\*\*** Tags SemVer aplicadas a todos os artefatos (M0.5).
- [ ] **\*\*CI/CD:\*\*** Pipeline de deployment automatizado (GitOps) configurado.

---

## ## 12. KPIs Operacionais

KPI	Definição	M1	M2	M3	M4	M5	M6	M7	M8	M9
:---	:	---	:	---	:	---	:	---	:	---
<b>**MTTD**</b>	Tempo Médio para Detect.	30 min	10 min	1 min	45s	30s	20s	15s	10s	5s
<b>**MTTR**</b>	Tempo Médio para Resolver.	-	60 min	30 min	15 min	10 min	5 min	2 min	1 min	30s
<b>**FPR**</b>	Taxa de Falsos Positivos	-	-	10%	8%	5%	3%	2%	<1%	<0.5%
<b>**Automação Rate**</b>	% resolvido sem humano.	0%	0%	5%	10%	15%	20%	30%	50%	>95%
<b>**Precisão IA**</b>	Acurácia da Sugestão.	-	-	-	70%	75%	85%	90%	99%	
<b>**Cobertura MITRE**</b>	% de técnicas mapeadas.	-	-	20%	40%	50%	60%	70%	80%	95%

---

## ## 13. SLO e SLA

Categoria	SLO (Target)	SLA (Garantido)	Consequência do Não Cumprimento
**\*\*Disponibilidade (API/UI)\*\***	99.99%	99.95%	Crédito financeiro, Plano de Ação.
**\*\*Latência de Ingestão\*\***	p95 < 50ms	p99 < 100ms	Escalonamento horizontal forçado.
**\*\*Latência de Correlação (M3)\*\***	p95 < 500ms	p99 < 1s	Rollback da regra de correlação.
**\*\*Tempo de Geração IA (M5)\*\***	p95 < 2s	p99 < 5s	Falback para modo offline (cache).
**\*\*Tempo de Execução (M7)\*\***	p95 < 1s	p99 < 3s	Desativação da automação para aquele tipo de Command.
**\*\*Durabilidade de Dados\*\***	99.999%	99.99%	Restauração de backup multi-region.

---

## ## 14. Gestão de Riscos

Risco	Probabilidade	Impacto	Estratégia de Mitigação	Aceitação
**\*\*Alucinação IA (M5-M9)\*\***	Média	Crítico	Human-in-the-loop até M9. Políticas de validação de saída (JSON Schema).	Risco residual aceito (M5-M7), mitigado em M8-M9.
**\*\*Cadeia de Automação (M7)\*\***	Baixa	Catastrófico	Circuit Breakers por Tenant. Limite de execuções/minuto.	Não aceito; deve ser resolvido antes de M7.
**\*\*Vazamento de Dados (M4-M9)\*\***	Baixa	Catastrófico	Zero Trust, criptografia em repouso/trânsito, auditoria rígida.	Risco residual inerente, mas controlado.
**\*\*Falha de Dependência (Ex: Kafka down)\*\***	Média	Alto	Replicação multi-zona, fallback para cache local.	Risco aceito com plano de BCP.
**\*\*Envenenamento de TI (M4)\*\***	Média	Alto	Validação de fonte (trusted feeds only), verificação de reputação.	Risco aceito com monitoramento de anomalias.
**\*\*Viés Algorítmico (M9)\*\***	Média	Médio	Auditoria de decisões semanais por comitê de ética.	Risco mitigado pela supervisão humana.

---

## ## 15. Observabilidade

### ### 15.1 OpenTelemetry (OTel)

Todos os serviços devem exportar:

- **\*\*Traces:\*\*** 100% das requisições do Command Bus e Handlers.
- **\*\*Metrics:\*\*** Golden Signals (Latência, Tráfego, Erros, Saturação).
- **\*\*Logs:\*\*** Estruturados (JSON), com níveis dinâmicos (INFO/DEBUG/ERROR).

### ### 15.2 Estratégia de Alertas

- **\*\*Crítico:\*\*** PagerDuty aciona on-call.
- **\*\*Alto:\*\*** Slack/Teams com notificação.
- **\*\*Médio:\*\*** Dashboard highlight.
- **\*\*Baixo:\*\*** Registro em log para análise de tendência.

### ### 15.3 Health Checks

- **\*\*Liveness:\*\*** API `/healthz` (verifica se o serviço responde).
- **\*\*Readiness:\*\*** API `/ready` (verifica dependências como banco e Kafka).
- **\*\*Deep Health:\*\*** (M3+) Verifica atraso na fila de processamento.

---

## ## 16. Segurança Operacional

- **\*\*Zero Trust:\*\*** Toda comunicação entre Contextos é autenticada via mTLS.
- **\*\*Least Privilege:\*\*** As credenciais da pipeline de CI/CD para deploy de novas Capabilities possuem escopo limitado ao Bounded Context correspondente.
- **\*\*RBAC:\*\*** A evolução das Capabilities requer permissões específicas (`capability:write`, `capability:approve`).
- **\*\*MFA:\*\*** Exigido para qualquer ação de aprovação de Gate (M5+).
- **\*\*LGPD/GDPR:\*\*** Dados sensíveis (PII) ingeridos no M0 devem ser anonimizados para uso em M5/M9 (LLMs). A base de conhecimento não deve armazenar PII diretamente.

---

## ## 17. Roadmap Oficial (Baseado em Dependências)

```mermaid

graph LR

M0[Arquitetura] --> M1[Observabilidade]

M1 --> M2[Monitoramento]

M2 --> M3[Correlação]

M3 --> M4[Threat Intel]

M4 --> M5[AI Advisory]

M5 --> M6[AI Copilot]

M6 --> M7[Automação]

M7 --> M8[Red Team Autônomo]

M8 --> M9[Autonomia Supervisionada]

M3 -.-> M5

M4 -.-> M8

M5 -.-> M9

```

**\*\*Nota:\*\*** As setas tracejadas indicam dependência de dados, não de progressão linear. Ex: M3 fornece dados para treinar M5, mas M5 pode evoluir conceitualmente em paralelo à M4.

---

## ## 18. Diagramas Mermaid

### ### 18.1 Fluxo de Evolução e Governança

```

```mermaid
flowchart TD
    A[Proposta de Feature] --> B{Arquitetura Board}
    B -->|Aprovada| C[DoR - Checklist]
    C -->|OK| D[Desenvolvimento]
    D --> E[Testes & Observabilidade]
    E --> F[DoD - Checklist]
    F -->|OK| G[Security Review]
    G -->|Aprovado| H[Canary Deploy]
    H --> I{Monitoramento KPIs}
    I -->|SLO Alcançado| J[Promoção para Producao]
    I -->|SLO Não Alcançado| K[Análise de Falha]
    K -->|Hotfix| D
    K -->|Rollback| L[Maturidade Anterior]
    J --> M[Capability Ativa]
    M --> N[Próxima Capability]
```

```

### ### 18.2 Matriz de Maturidade Operacional

```

```mermaid
quadrantChart
    title Mapa de Maturidade GovSec Shield
    x-axis "Reativo" --> "Proativo"
    y-axis "Manual" --> "Autônomo"
    quadrant-1 "Autonomia Supervisionada (M9)"
    quadrant-2 "Red Team (M8)"
    quadrant-3 "Base Operacional (M0-M2)"
    quadrant-4 "Correlação e IA (M3-M7)"
    M0-Arquitetura: [0.1, 0.1]
    M2-Monitoramento: [0.2, 0.3]
    M3-Correlação: [0.4, 0.5]
    M5-AI Advisory: [0.6, 0.6]
    M7-Automação: [0.7, 0.8]
    M9-Autonomia: [0.9, 0.95]
```

```

### ### 18.3 Estado de Transição (Ciclo de Vida da Capability)

```

```mermaid
stateDiagram-v2
    [*] --> Proposed: RFC Submetido
    Proposed --> UnderReview: Architecture Board
    UnderReview --> Rejected: Nao Aprovado
    UnderReview --> Planned: DoR Aprovado
    Planned --> InDevelopment: Inicio Implementacao
    InDevelopment --> Testing: DoD Parcial
    Testing --> Staging: Deploy Homolog
```

```

Staging --> Production: Canary & KPIs  
Production --> Operational Excellence: SLOs Sustentados  
Operational Excellence --> Evolved: Avançar M(n+1)  
Operational Excellence --> Rollback: Falha Crítica  
Rollback --> Planned: Correção  
Rejected --> [\*]  
Evolved --> [\*]

...

---

## ## 19. Boas Práticas

1. **\*\*Evolua a Observabilidade Primeiro:\*\*** Antes de adicionar uma nova fonte de dados, garanta que os pipelines de telemetria estão dimensionados para recebê-la.
2. **\*\*Versionamento de Políticas:\*\*** Toda mudança no Policy Engine (OPA/Cedar) associada a uma nova Capability deve ser versionada junto com o código.
3. **\*\*Testes de Carga Regulares:\*\*** Ao atingir M7+, execute testes de carga semanais simulando picos de 10x o tráfego normal.
4. **\*\*Chaos Engineering:\*\*** (M8+) Utilize o Red Team para causar falhas deliberadas (ex: matar pods Kafka) para validar a resiliência da automação.
5. **\*\*Feedback Humano Estruturado:\*\*** (M5+) Analistas devem classificar as sugestões de IA (Útil/Não útil) para alimentar o ciclo de ML.
6. **\*\*Documentação como Código:\*\*** Atualize a documentação arquitetural (ADR) simultaneamente ao pull request da Capability.

---

## ## 20. Anti Patterns

| Anti-Pattern | Descrição | Por que é Proibido |  
| :--- | :--- | :--- |  
| **\*\*Automatizar antes de Observar\*\*** | Implementar M7 (Automação) sem ter M1/M2 consolidados. | Você não saberá se a automação falhou ou causou danos. "Voo cego". |  
| **\*\*Adicionar IA sem Métricas\*\*** | Colocar LLM em produção sem definir KPIs de precisão (M5). | A IA pode estar alucinando, corroendo a confiança do analista. |  
| **\*\*Ignorar KPIs para Avançar\*\*** | Pular de M3 para M4 com FPR em 20%. | A TI enriquecerá dados ruins, amplificando o ruído. Garbage in, gospel out. |  
| **\*\*Pular Capabilidade\*\*** | Tentar implementar M9 (Autonomia) partindo de M2. | A plataforma não tem base de dados histórica (M3/M4) nem validação humana (M5/M6), resultando em decisões catastróficas. |  
| **\*\*Quebrar Contratos\*\*** | Alterar o schema de um Evento/Command em um novo MAJOR sem suportar a versão antiga (M0.5). | Quebra a correlação (M3) e a inteligência (M4) que dependem de dados históricos. |  
| **\*\*Ignorar Governança\*\*** | Implementar uma Feature sem passar pelo Architecture Board. | Gera "Shadow IT" arquitetural, fragmentando o modelo DDD e criando dívida técnica impagável. |

| **\*\*Overfitting de IA\*\*** | Treinar o AI Advisor apenas com dados do último mês. | O modelo não generaliza bem para novas táticas de ataque (M8/M9). |

---

## ## 21. Débitos Técnicos

| ID | Débito | Capability Relacionada | Impacto | Plano de Mitigação | Prioridade |  
| :--- | :--- | :--- | :--- | :--- | :--- |  
| **\*\*TD-OPS-01\*\*** | Falta de "Playbook as Code" versionado. | M7 | Dificuldade em auditar mudanças em automação. | Implementar repositório Git para Playbooks com CI/CD. | Alta |  
| **\*\*TD-OPS-02\*\*** | Modelos de IA (M5) com viés para dados de rede, ignorando dados de identidade. | M5, M9 | Decisões de bloqueio podem ser imprecisas para usuários. | Balanceamento de dataset de treino. | Média |  
| **\*\*TD-OPS-03\*\*** | Testes de carga da Camada de Correlação (M3) não cobrem picos de feriados. | M3 | Risco de queda de performance em eventos sazonais. | Simulação de Black Friday/Eventos Cibernéticos. | Média |  
| **\*\*TD-OPS-04\*\*** | Ausência de "Kill Switch" granular (por Tenant) para M7-M9. | M9 | Rollback de automação afeta todo o sistema. | Implementar feature flags por Tenant. | Alta |  
| **\*\*TD-OPS-05\*\*** | Dependência de um único feed de Threat Intel (M4). | M4 | Ponto único de falha (SPOF) para enriquecimento. | Adicionar feed secundário (redundância). | Alta |

---

## ## 22. Changelog

### ### 22.1 Versão 1.0.0 (2026-07-28)

- **\*\*Criação Inicial:\*\*** Documento fundacional do Modelo Operacional e de Maturidade.
- **\*\*Definição do Modelo M0-M9:\*\*** Estabelecimento formal dos 10 estágios de evolução.
- **\*\*Definição de KPIs e SLOs:\*\*** Métricas oficiais para cada nível.
- **\*\*Governança:\*\*** Criação dos processos de Architecture Board e Security Review.
- **\*\*Roadmap:\*\*** Mapeamento de dependências e transições.
- **\*\*DoR/DoD:\*\*** Checklists oficiais para desenvolvimento.

### ### 22.2 Mudanças Futuras (Previstas)

| Versão | Data Prevista | Mudança Planejada |  
| :--- | :--- | :--- |  
| 1.1.0 | Q1 2027 | Inclusão de M10 - "Autonomia Estratégica" (planejamento de capacidade preditivo). |  
| 1.2.0 | Q3 2027 | Refinamento dos KPIs de M8/M9 baseado em dados de produção. |  
| 2.0.0 | Q1 2028 | Revisão do Modelo de Maturidade para incorporar exigências da nova regulamentação (ex: AI Act). |

---

## ## Anexos

### ### Anexo A: Glossário de Termos Estratégicos



| Termo | Definição |  
| :--- | :--- |  
| **\*\*Golden Signals\*\*** | Latência, Tráfego, Erros, Saturação (Google SRE). |  
| **\*\*Strangler Fig Pattern\*\*** | Estratégia de substituição incremental de funcionalidades. |  
| **\*\*Circuit Breaker\*\*** | Padrão que interrompe chamadas a serviços com falha para evitar cascadeamento. |  
| **\*\*Progressive Delivery\*\*** | Implantação gradual (Canary, A/B Test, Feature Flags). |  
| **\*\*STIX/TAXII\*\*** | Padrões para troca de inteligência de ameaças cibernéticas. |  
| **\*\*RAG\*\*** | Retrieval-Augmented Generation - técnica para enriquecer prompts de IA com dados da base de conhecimento. |

### ### Anexo B: Referências Normativas

- **\*\*NIST SP 800-207\*\***: Zero Trust Architecture.
- **\*\*Google SRE Book\*\***: Site Reliability Engineering.
- **\*\*TOGAF 9.2\*\***: The Open Group Architecture Framework (Capability-Based Planning).
- **\*\*MITRE ATT&CK Framework\*\***: Matriz de Táticas e Técnicas.
- **\*\*M0.1 a M0.7\*\***: Documentos base do GovSec Shield.

---

**\*\*Fim do Documento M0.8\*\***

\*Este documento é a carta magna da evolução do GovSec Shield. A não conformidade com os estágios de maturidade, a tentativa de automação prematura ou a violação dos critérios de DoR/DoD caracterizam violação grave da governança corporativa, sujeita a revisão pelo CISO e pelo Comitê Executivo.\*

# GovSec Shield

## M0.8 — Modelo Operacional, Capacidades, Governança e Evolução da Plataforma

**\*\*Versão\*\***: 1.0.0

**\*\*Status\*\***: RATIFICADO PELO COMITÊ DE ARQUITETURA

**\*\*Data de Vigência\*\***: 28 de Julho de 2026

**\*\*Classificação\*\***: PÚBLICO INTERNO - ESTRATÉGICO

**\*\*Nível de Controle\*\***: GOVERNANÇA CORPORATIVA

**\*\*Substitui\*\***: N/A

**\*\*Impacto\*\***: ESTRATÉGICO, TÁTICO E OPERACIONAL

---

### ## Sumário Executivo

Este documento estabelece a **\*\*Constituição da Evolução\*\*** do GovSec Shield. Diferencia-se fundamentalmente de um roadmap temporal ou lista de funcionalidades; trata-se de um **\*\*Modelo de Maturidade por Capacidades (Capability Maturity Model)\*\*** que define a trajetória evolutiva da plataforma desde sua fundação arquitetural (M0) até a autonomia supervisionada (M9). Cada estágio de maturidade é uma porta lógica obrigatória, composta por critérios de entrada (DoR), critérios de saída (DoD), dependências rigorosas e KPIs mensuráveis.

A evolução do GovSec Shield não é linear por tempo, mas sim **incremental** por governança. Nenhuma capacidade pode ser ignorada, pulada ou acelerada sem a devida avaliação de riscos e aprovação do Architecture Board. Este documento serve como a referência única para planejamento estratégico, alocação de recursos e certificação de conformidade operacional.

---

## ## 1. Introdução

### ### 1.1 Objetivo

1. **Definir o Modelo de Maturidade Oficial:** Estabelecer uma escada evolutiva (M0 a M9) que oriente o GovSec Shield de um "Sistema Operacional de Segurança" estático para uma "Entidade de Segurança Adaptativa" dinâmica.
2. **Instituir Governança por Capacidades:** Formalizar que todo desenvolvimento deve ser mapeado a uma Capability específica, garantindo que a evolução seja rastreável, segura e auditável.
3. **Operacionalizar a Arquitetura:** Vincular os artefatos técnicos (Commands, Events, Bounded Contexts) às fases de maturidade, criando uma malha de dependências operacionais.
4. **Estabelecer Critérios de Prontidão (DoR/DoD):** Fornecer checklists rigorosos que devem ser atendidos para que uma Capability seja considerada "pronta para evoluir" e "efetivamente entregue".

### ### 1.2 Escopo

Este documento abrange:

- **Modelo de Maturidade (M0–M9):** Detalhamento exaustivo de cada estágio, incluindo pré-requisitos, componentes envolvidos, eventos gerados e KPIs exigidos.
- **Modelo de Capacidades:** Definição e classificação de cada unidade funcional evolutiva.
- **Governança e Decisão:** Processos de Architecture Board, Security Review e gestão de ADRs (Architecture Decision Records) para evolução.
- **Indicadores Operacionais (KPIs/SLOs):** Métricas oficiais que atestam a saúde e eficácia de cada nível de maturidade.
- **Gestão de Riscos:** Matriz de risco inerente a cada salto de capacidade.

### ### 1.3 Não Escopo

- **Detalhamento de Infraestrutura Física:** Provisionamento de hardware, redes subjacentes ou configuração de clusters Kubernetes (serão tratados em documentos de SRE específicos).
- **Código Fonte e Implementação:** Padrões de codificação ou estruturas de repositórios (tratados em guias de engenharia).
- **Recrutamento e Times:** Estratégias de formação de equipes (embora a governança mencione ownership).
- **Modelo Financeiro (FinOps):** Custos específicos de cloud ou licenciamento.

### ### 1.4 Relação com Documentos Anteriores (M0.1 – M0.7)

| Documento | Relação com M0.8 |

| :--- | :--- |

| **\*\*M0.1 (Visão e Paradigma)\*\*** | A filosofia de "Security OS" é a base da evolução. M0.8 traduz essa visão em estágios realizáveis. |

| **\*\*M0.2 (Bounded Contexts)\*\*** | A evolução respeita as fronteiras contextuais. Cada Capability impacta Contextos específicos, mapeados na Matriz de Dependências. |

| **\*\*M0.3 (Event Storming)\*\*** | As Capacidades são derivadas dos eventos de domínio identificados. M0.8 organiza esses eventos em ondas de maturidade. |

| **\*\*M0.4 (Modelo ER)\*\*** | A evolução exige migrações de schema. M0.8 define critérios de backward compatibility para mudanças de dados. |

| **\*\*M0.5 (Versionamento)\*\*** | O versionamento SemVer é a alavanca técnica para evoluir sem quebrar contratos. M0.8 alinha a maturidade operacional à maturidade de versões. |

| **\*\*M0.6 (Contratos Canônicos)\*\*** | Eventos devem ser estáveis. M0.8 exige que novas capacidades gerem eventos com schemas previamente aprovados. |

| **\*\*M0.7 (Modelo de Commands)\*\*** | O Catálogo de Commands se expande com a maturidade. M0.8 define *\*quais\** Commands são obrigatórios em *\*qual\** estágio. |

### ### 1.5 Propósito Operacional

O GovSec Shield não é um projeto finito; é uma plataforma viva. Este documento estabelece o "modo de operação" contínuo, garantindo que:

- **\*\*A Segurança evolui com a ameaça:\*\*** A capacidade de resposta é aprimorada incrementalmente, sem revoluções destrutivas.
- **\*\*A Automação é conquistada:\*\*** A automação total (M9) só é alcançada após a observabilidade (M1), correlação (M3) e IA consultiva (M5) estarem maduras.
- **\*\*A Governança é o freio de mão:\*\*** Nenhuma automação é permitida sem supervisão humana validada por políticas.

---

## ## 2. Filosofia da Evolução

### ### 2.1 Capability Driven Development (CDD)

O GovSec Shield adota o **\*\*Desenvolvimento Orientado a Capacidades\*\***. Uma Capacidade (Capability) é a menor unidade de valor de negócio que pode ser entregue, medida e governada de forma independente.

- **\*\*Por que Capacidades?\*\*** Versões (1.0, 2.0) são artefatos de release. Capacidades são artefatos de *\*valor\**. A capacidade "Correlação de Eventos" pode ser entregue em múltiplas versões menores, mas só é considerada ativa quando atinge um patamar de precisão (KPI) definido.
- **\*\*Incremental Architecture:\*\*** A arquitetura não é "big-bang". Cada capacidade adiciona camadas sobre a base sólida (Hexagonal + Clean Architecture), permitindo rollback granular de funcionalidades se falhas de segurança forem detectadas.

### ### 2.2 Continuous Evolution vs. Revolução

- **\*\*Proibido Big-Bang:\*\*** Migrações de M0 para M1 que envolvam reescrita total são terminantemente proibidas. A evolução deve ser *\*estruturalmente compatível\** (Strangler Fig Pattern).

- **Adaptive Security Evolution (ASE):** Inspirado no NIST SP 800-207 (Zero Trust), a plataforma deve adaptar suas defesas com base no ambiente operacional. Se os KPIs de M3 (Correlação) caírem abaixo do SLO, a plataforma pode reverter automaticamente a execução de certos Commands para o estágio M2, até que a confiabilidade seja restaurada.

#### 2.3 Governança por Capacidades

Toda nova funcionalidade proposta deve responder a duas perguntas:

1. \*Qual capacidade esta funcionalidade está aprimorando?\*
2. \*A capacidade dependente (anterior) está em estado "Operational Excellence"?\*

#### 2.4 Por que Maturidade, não Cronologia?

Datas são arbitrárias; maturidade é baseada em **evidências**. A evolução de M3 para M4 (Threat Intelligence) depende de um volume mínimo de dados processados e de uma precisão de correlação mínima. Sem esses KPIs, a IA de inteligência de ameaças irá "alucinar" ou gerar falsos positivos. A maturidade protege o sistema contra a automação prematura.

---

### 3. Princípios Fundamentais

| Princípio | Definição Formal | Implicação para M0.8 |

| :--- | :--- | :--- |

| **Capability First** | Toda entrega deve ser mapeada a uma Capability do modelo M0–M9. | Funcionalidades órfãs (sem Capability) são rejeitadas no Architecture Review. |

| **Security by Default** | A segurança não é uma camada; é a base. Toda nova Capability herda a postura Zero Trust. | Uma nova Capability não pode ser ativada até que o Security Kernel (M0) a valide. |

| **Observability First** | "Não se gerencia o que não se mede". | Uma Capability só pode ser declarada "Pronta" (DoD) se 100% de seus traces estiverem expostos via OpenTelemetry. |

| **Automation Last** | Automação é o destino, não o ponto de partida. | Nenhuma automação (M7) é permitida antes de M5 (AI Advisory) e M6 (Copilot) estarem validados por humanos. |

| **AI is Advisory** | IA no GovSec Shield é consultiva, nunca executiva, até que o estágio M9 seja atingido com supervisão. | Commands gerados por IA (ex: `ApplyAIActionCommand`) devem sempre passar pelo Policy Engine com flag `requires\_approval=true` até M9. |

| **Policy Before Automation** | O Policy Engine é o único oráculo que permite a execução. | Mesmo em M9, a Autonomia Supervisionada deve consultar o Policy Engine para cada ação. |

| **Human Oversight** | O elemento humano é o juiz final da estratégia. | Todo rollback (M7+) ou decisão de contenção crítica deve permitir intervenção humana em até 5 minutos. |

|

| **Fail Safe** | Em caso de falha do sistema, o estado deve retornar ao último estado seguro conhecido. | Mecanismos de Circuit Breaker nas Capabilities de Automação são obrigatórios. |

| **Deterministic Evolution** | Dado o mesmo estado inicial e a mesma sequência de eventos, a evolução da Capability deve ser reproduzível. | Replays de eventos históricos não devem alterar o modelo de maturidade atual. |  
| **Backward Compatibility** | A evolução não quebra os consumidores existentes. | Novas versões de Commands (MAJOR) só são permitidas com suporte a múltiplas versões (Polyglot Persistence de schemas). |

---

## ## 4. Linguagem Ubíqua (Estratégica)

| Termo | Definição Formal | Exemplo |

| :--- | :--- | :--- |

| **Capability** | Unidade atômica de valor de negócio, governada por um conjunto de SLOs e KPIs, representando um estágio de maturidade funcional. | `Threat Intelligence Capability (M4)` |

| **Maturity Level** | Estágio numérico (M0–M9) que define o grau de autonomia, correlação e inteligência da plataforma. | `M3 - Correlação` |

| **Baseline** | O estado mínimo aceitável de desempenho para uma Capability ser considerada estável. | `Baseline M2: 95% de uptime do monitoramento.` |

| **Operational Readiness** | Estado em que a plataforma possui procedimentos (runbooks) e ferramentas para manter a Capability em produção. | Aprovação do time de SRE para o novo dashboard. |

| **Operational Excellence** | Estado em que a Capability não apenas funciona, mas excede consistentemente seus SLOs e possui mecanismos de auto-recuperação. | `M3 Excellence: FPR < 1% por 7 dias consecutivos.` |

| **Definition of Ready (DoR)** | Checklist de critérios que devem ser satisfeitos *antes* do início do desenvolvimento de uma Capability. | ADR aprovado, Eventos modelados. |

| **Definition of Done (DoD)** | Checklist de critérios que devem ser satisfeitos *após* o desenvolvimento para liberar a Capability em produção. | Testes integrados, Docs atualizados. |

| **Governance Gate** | Ponto de decisão formal onde o Architecture Board avalia a evolução e autoriza (ou não) a progressão. | Gate entre M4 e M5. |

| **Feature** | Funcionalidade específica que contribui para a realização de uma Capability. | `Feature: Correlação de logs de firewall com incidentes.` |

| **Epic** | Conjunto de Features necessário para alcançar um Maturity Level. | `Epic M4: Ingestão de feeds STIX/TAXII.` |

| **Milestone** | Marco de entrega de uma Capability. | `Milestone M3 atingido em produção.` |

| **Roadmap** | Representação gráfica da evolução baseada em dependências (não em tempo). | `M2 -> M3 -> M4 (paralelo com M5).` |

| **Dependency** | Requisito técnico ou operacional que uma Capability inferior deve atender para que a superior seja iniciada. | `M5 depende de M4 (dados de TI).` |

| **Risk Acceptance** | Decisão formal do CISO de prosseguir com uma Capability mesmo com riscos conhecidos. | Assinatura para prosseguir com M7 apesar de FPR em 2%. |

| **Operational KPI** | Indicador quantitativo de desempenho operacional. | `MTTD (Mean Time to Detect).` |

| **Operational SLO** | Meta quantitativa para um KPI. | `SLO MTTD < 1 minuto para M3.` |

---

## ## 5. Modelo Oficial de Capacidades

O GovSec Shield define suas Capacidades como uma pilha verticalizada, onde cada camada depende da robustez da camada inferior.

### ### 5.1 Hierarquia e Natureza

- **\*\*Capacidades Fundamentais (M0-M2):\*\*** Habilitadoras. Sem elas, o sistema não existe ou é cego.
- **\*\*Capacidades Cognitivas (M3-M5):\*\*** Analíticas. Transformam dados em inteligência.
- **\*\*Capacidades Autônomas (M6-M9):\*\*** Executivas. Transformam inteligência em ação.

### ### 5.2 Critérios de Evolução

Para evoluir de `Mn` para `M(n+1)`, a plataforma DEVE:

1. **\*\*Demonstrar Estabilidade:\*\*** A Capabilidade atual deve ter operado em regime de "Operational Excellence" por no mínimo 15 dias corridos.
2. **\*\*Cobertura de Dados:\*\*** A quantidade de eventos/telemetria processados deve ser estatisticamente significativa (> 1 milhão de eventos/dia).
3. **\*\*Validação de Segurança:\*\*** O Security Kernel deve atestar que a nova Capabilidade não introduz vetores de ataque.

---

## ## 6. Modelo de Maturidade (M0 – M9)

### ### 6.1 M0 — Arquitetura (Foundation)

- **\*\*Objetivo:\*\*** Estabelecer a base imutável do Security OS, conforme especificado nos documentos M0.1–M0.7.
- **\*\*Descrição:\*\*** A plataforma está "viva", mas opera em modo de "inserção passiva" (sink mode). Processa eventos, armazena dados, mas não realiza correlação ativa nem respostas automatizadas. Os Commands básicos (`CreateIncidentCommand`, `BlockIPCommand`) existem, mas requerem acionamento manual via Dashboard.
- **\*\*Pré-requisitos:\*\*** N/A (Ponto de partida).
- **\*\*Critérios de Entrada:\*\*** Decisão de patrocínio e arquitetura aprovada.
- **\*\*Critérios de Saída:\*\*** Todos os Bounded Contexts estão implantados, o Kafka (Redpanda) está operacional, e o fluxo de ingestão de logs está estável.
- **\*\*Componentes:\*\*** Core Engine, Command Bus, PostgreSQL, Kafka, Security Kernel.
- **\*\*Eventos Produzidos:\*\*** `SystemBootstrappedEvent`, `IngestionStartedEvent`.
- **\*\*Commands Utilizados:\*\*** `CreateTenantCommand`, `IngestLogCommand`.
- **\*\*KPIs:\*\*** Disponibilidade da API > 99.9%. Latência de ingestão < 100ms.
- **\*\*SLOs:\*\*** MTTR de deploy < 10 min.
- **\*\*Riscos:\*\*** Vulnerabilidades na camada base.
- **\*\*Limitações:\*\*** Zero automação. Inteligência artificial ausente.
- **\*\*Próxima Capability:\*\*** M1 – Observabilidade.

---

### ### 6.2 M1 — Observabilidade (Observability)

- **Objetivo:** Tornar o sistema 100% transparente internamente. "Ver a si mesmo".
- **Descrição:** Instrumentação completa via OpenTelemetry. Todos os microserviços exportam traces, métricas e logs estruturados. O painel central (Grafana) mostra a saúde de cada Bounded Context.
- **Pré-requisitos:** M0 estável.
- **Critérios de Entrada:** Definição dos Golden Signals para cada serviço.
- **Critérios de Saída:** Cobertura de tracing em 100% das requisições HTTP/gRPC e processamento de Commands.
- **Componentes:** OpenTelemetry Collector, Grafana, Tempo, Prometheus, Loki.
- **Eventos Produzidos:** `TelemetryEmittedEvent`.
- **Commands Utilizados:** N/A (apenas configuração).
- **KPIs:** Cobertura de traces > 99%. Erro de coleta < 0.1%.
- **SLOs:** p95 de latência de exportação < 50ms.
- **Riscos:** Sobrecarga do coletor de telemetria.
- **Limitações:** Apenas monitoramento passivo; sem análise ativa.
- **Próxima Capability:** M2 – Monitoramento.

---

### ### 6.3 M2 — Monitoramento (Monitoring)

- **Objetivo:** Transformar a telemetria em alertas acionáveis e dashboards de negócio.
- **Descrição:** Criação de regras de alerta (Prometheus Alertmanager). Dashboards específicos por contexto (Incident, Security, Assets). Estabelecimento de "Health Checks" para cada serviço. On-call rotation configurada.
- **Pré-requisitos:** M1 (dados de telemetria disponíveis).
- **Critérios de Entrada:** Definição de alertas críticos (CPU, memória, filas Kafka).
- **Critérios de Saída:** Todos os serviços com dashboards aprovados pela SRE. Testes de alerta (Fire Drill) realizados.
- **Componentes:** Alertmanager, Grafana Dashboards, PagerDuty/Opsgenie.
- **Eventos Produzidos:** `AlertFiredEvent`, `AlertResolvedEvent`.
- **Commands Utilizados:** `AcknowledgeAlertCommand` (humano).
- **KPIs:** Tempo de detecção de falha (MTTD) < 5 min.
- **SLOs:** 100% de alerts críticos disparando em até 1 minuto.
- **Riscos:** Alert fatigue devido a regras mal calibradas.
- **Limitações:** Alertas são isolados; não há correlação entre eles.
- **Próxima Capability:** M3 – Correlação.

---

### ### 6.4 M3 — Correlação (Correlation)

- **Objetivo:** Unir eventos aparentemente desconexos para formar uma história de incidente coesa.
- **Descrição:** Implementação de um Motor de Correlação (CEP - Complex Event Processing). O sistema começa a gerar `CorrelatedIncidentCommand` com base em regras (ex: Falha de autenticação + Acesso a endpoint sensível). Redução massiva de ruído.
- **Pré-requisitos:** M2 (Alertas configurados) e M0 (Schema de eventos normalizado).

- **\*\*Critérios de Entrada:\*\*** Conjunto mínimo de 50 regras de correlação definidas (MITRE ATT&CK mapping).
- **\*\*Critérios de Saída:\*\*** Taxa de Falsos Positivos (FPR) na correlação < 5%.
- **\*\*Componentes:\*\*** CEP Engine, Rule Repository (Policy Engine), Knowledge Graph (inicial).
- **\*\*Eventos Produzidos:\*\*** `IncidentCorrelatedEvent`, `FalsePositiveIdentifiedEvent`.
- **\*\*Commands Utilizados:\*\*** `CreateIncidentCommand` (gerado automaticamente), `EscalateIncidentCommand` (automático baseado em severidade).
- **\*\*KPIs:\*\*** MTTD (Mean Time to Detect) < 60 segundos. Redução de alertas em 80% (consolidação).
- **\*\*SLOs:\*\*** Precisão da correlação > 95%.
- **\*\*Riscos:\*\*** Correlação excessiva gerando incidentes falsos (sobrecarga do analyst).
- **\*\*Limitações:\*\*** Dependente de regras estáticas; não aprende com o tempo.
- **\*\*Próxima Capability:\*\*** M4 – Threat Intelligence.

---

#### ### 6.5 M4 — Threat Intelligence (Inteligência de Ameaças)

- **\*\*Objetivo:\*\*** Enriquecer os incidentes com contexto externo e histórico (IoCs, TTPs).
- **\*\*Descrição:\*\*** Ingestão de feeds STIX/TAXII (governamentais, comerciais e open-source). Enriquecimento automático de IPs, Domínios e Hashes durante a correlação (M3). Atualização da Knowledge Base com pontuação de risco dinâmica.
- **\*\*Pré-requisitos:\*\*** M3 operacional. Feeds de TI contratados/configurados.
- **\*\*Critérios de Entrada:\*\*** Pelo menos 3 feeds externos ingeridos.
- **\*\*Critérios de Saída:\*\*** Cobertura de enriquecimento em 90% dos incidentes gerados.
- **\*\*Componentes:\*\*** TI Aggregator, STIX/TAXII Parser, Knowledge Base (Neo4j/PostgreSQL Vetorial).
- **\*\*Eventos Produzidos:\*\*** `ThreatIntelUpdatedEvent`, `IndicatorMatchedEvent`.
- **\*\*Commands Utilizados:\*\*** `UpdateThreatIntelCommand` (automático), `StoreIndicatorCommand`.
- **\*\*KPIs:\*\*** Cobertura de IoCs conhecidos > 80%. Tempo de atualização de feeds < 5 min.
- **\*\*SLOs:\*\*** Latência de enriquecimento < 500ms por evento.
- **\*\*Riscos:\*\*** Envenenamento de feeds (poisoning) levando a decisões erradas.
- **\*\*Limitações:\*\*** Inteligência é reativa (baseada em listas) e não preditiva.
- **\*\*Próxima Capability:\*\*** M5 – AI Advisory.

---

#### ### 6.6 M5 — AI Advisory (Assistência Consultiva)

- **\*\*Objetivo:\*\*** Utilizar IA (LLMs/ML) para sugerir cursos de ação ao analista, sem executar nada automaticamente.
- **\*\*Descrição:\*\*** O AI Advisor analisa o contexto do incidente (correlação + threat intel) e gera uma `Suggestion` (recomendação). Essa sugestão é apresentada no dashboard. A execução exige clique humano, que dispara o Command correspondente.
- **\*\*Pré-requisitos:\*\*** M4. Modelos de ML treinados com dados históricos de M3/M4.
- **\*\*Critérios de Entrada:\*\*** Modelo de linguagem (LLM) aprovado (on-prem ou via AI Gateway) e com políticas de privacidade validadas.
- **\*\*Critérios de Saída:\*\*** Taxa de Aceitação de Sugestões pelos analistas > 60%.



- **Componentes:** AI Gateway (LLM), Knowledge Base (RAG - Retrieval Augmented Generation), Suggestion Engine.
- **Eventos Produzidos:** `SuggestionGeneratedEvent`, `SuggestionAcceptedEvent` (humano), `SuggestionRejectedEvent`.
- **Commands Utilizados:** `RequestSuggestionCommand` (automático), `ApplySuggestionCommand` (humano) \*- Neste estágio, este Command sempre tem flag `requires_approval=true`.
- **KPIs:** Precisão da sugestão (acurácia) > 70%. Relevância (feedback humano) > 4/5 estrelas.
- **SLOs:** Tempo de geração de sugestão < 5s.
- **Riscos:** Alucinações (hallucinations) da IA gerando sugestões perigosas.
- **Limitações:** A IA é consultiva, não possui contexto de execução em tempo real.
- **Próxima Capability:** M6 – AI Copilot.

---

#### ### 6.7 M6 — AI Copilot (Assistente Colaborativo)

- **Objetivo:** Integrar a IA ao fluxo de trabalho diário do analista como um "par" que interage via linguagem natural e acelera ações manuais.
- **Descrição:** O Copilot entende comandos em linguagem natural (ex: "Mostre todos os assets vulneráveis a CVE-2026-1234 e isole os críticos"). O Copilot traduz isso para queries SQL/Graph e, mediante confirmação do usuário, executa Commands em lote.
- **Pré-requisitos:** M5. Chat interface implementada no Dashboard.
- **Critérios de Entrada:** A interface de chat deve suportar autenticação e auditoria de todas as interações.
- **Critérios de Saída:** 80% dos analistas utilizando a ferramenta semanalmente.
- **Componentes:** Chat UI, Intent Parser, Command Translator.
- **Eventos Produzidos:** `CopilotInteractionEvent`.
- **Commands Utilizados:** Qualquer Command do catálogo M0.7, agora acionado via linguagem natural.
- **KPIs:** Redução do tempo médio de investigação (MTTI) em 40%.
- **SLOs:** Precisão da tradução de intenção > 90%.
- **Riscos:** Comandos mal interpretados (ex: "isolar" vs "observar").
- **Limitações:** Ainda depende do clique humano para Execução de alto risco.
- **Próxima Capability:** M7 – Automação.

---

#### ### 6.8 M7 — Automação (Automation)

- **Objetivo:** Executar playbooks de resposta pré-definidos automaticamente para incidentes de baixa e média criticidade.
- **Descrição:** O Policy Engine, em conjunto com a AI Advisory (M5), decide se um incidente se enquadra em um padrão conhecido (ex: Scanning de porta). O sistema dispara automaticamente o `ExecutePlaybookCommand`, que isola o ativo sem intervenção humana. Apenas incidentes críticos ou não classificados vão para a fila humana.
- **Pré-requisitos:** M6. Playbooks testados em ambiente de staging.
- **Critérios de Entrada:** Matriz de Decisão de Automação aprovada pelo CISO.
- **Critérios de Saída:** Taxa de Automação (eventos resolvidos sem humano) > 30%.

- **Componentes:** Playbook Executor, Orchestrator (Saga), Policy Engine (OPA).
- **Eventos Produzidos:** `AutomationTriggeredEvent`, `PlaybookExecutionSucceededEvent`, `PlaybookExecutionFailedEvent`.
- **Commands Utilizados:** `ExecutePlaybookCommand`, `IsolateAssetCommand`, `BlockIPCommand` (agora com flag `requires_approval=false` para baixo risco).
- **KPIs:** Tempo de Contenção (MTTC) reduzido em 60% (comparado a M0).
- **SLOs:** Taxa de sucesso da automação > 95%.
- **Riscos:** Efeito dominó (cascade failure) se um playbook mal escrito for executado em larga escala.
- **Limitações:** A automação é reativa e baseada em regras estáticas (playbooks).
- **Próxima Capability:** M8 – Red Team Autônomo.

---

### ### 6.9 M8 — Red Team Autônomo (Autonomous Red Team)

- **Objetivo:** A plataforma testa proativamente suas próprias defesas.
- **Descrição:** O sistema gera cenários de ataque simulado (usando Commands específicos de teste) contra ambientes de staging e, posteriormente, produção (com isolamento). Ele verifica se as correlações (M3), a inteligência (M4) e a automação (M7) detectam e contêm esses ataques. Se falham, o sistema gera tickets de melhoria (Débitos Técnicos).
- **Pré-requisitos:** M7. Framework de testes de invasão integrado (ex: Metasploit, custom agents).
- **Crítérios de Entrada:** Baseline de cobertura MITRE ATT&CK definida.
- **Crítérios de Saída:** Cobertura MITRE > 70% testada automaticamente.
- **Componentes:** Red Team Engine, Simulation Sandbox, Compliance Reporter.
- **Eventos Produzidos:** `AttackSimulationStartedEvent`, `DefenseGapIdentifiedEvent`.
- **Commands Utilizados:** `SimulateAttackCommand`, `GenerateReportCommand`.
- **KPIs:** Cobertura MITRE ATT&CK (aumento contínuo). Tempo de detecção de ataques simulados.
- **SLOs:** 100% dos ataques simulados críticos detectados em < 1 minuto.
- **Riscos:** Ataques simulados vazarem para produção ou causarem negação de serviço.
- **Limitações:** Focado em ataques conhecidos; inovação de ataque (novas táticas) ainda é limitada.
- **Próxima Capability:** M9 – Autonomia Supervisionada.

---

### ### 6.10 M9 — Autonomia Supervisionada (Supervised Autonomy)

- **Objetivo:** A plataforma opera como um "SOC Autônomo", gerenciando 95% dos incidentes do início ao fim, com humanos como supervisores estratégicos.
- **Descrição:** O sistema não apenas executa playbooks (M7) ou testa (M8), mas também **adapta suas próprias políticas** com base nos resultados. Ele propõe novas regras de correlação, ajusta thresholds de alerta e aprende com os acertos/erros. O analista humano atua como "governador", revisando relatórios agregados e intervindo apenas em "dilemas éticos" ou decisões de alto impacto financeiro/político.
- **Pré-requisitos:** M8. Modelos de Reinforcement Learning (RL) maduros.
- **Crítérios de Entrada:** Estrutura de "Human-in-the-loop" para override de decisões.

- **Critérios de Saída:** Confiança do sistema de auto-avaliação > 99%.
- **Componentes:** Self-Healing Engine, Adaptive Policy Engine, Continuous Improvement Loop.
- **Eventos Produzidos:** `PolicyAutoTunedEvent`, `AutonomousDecisionExecutedEvent`.
- **Commands Utilizados:** Todos os Commands existentes, com exceção de `DeleteTenantCommand` (explicitamente bloqueado para AI), operam com `requires\_approval=false`, mas geram eventos de justificativa detalhados.
- **KPIs:** Taxa de Autonomia (resolução sem humano) > 95%. Taxa de Reversão de decisões autônomas < 0.5%.
- **SLOs:** MTTR Geral < 30 segundos.
- **Riscos:** **Existencial** - Viés algorítmico levando a bloqueios generalizados injustos. Por isso, o "Modo Supervisor" humano tem uma "Kill Switch" física (circuit breaker) que reverte toda a automação para M5 (Advisory Only) em menos de 1 minuto.
- **Limitações:** Depende de dados de treino não enviesados. Decisões contextuais (ex: política) ainda requerem validação final.
- **Próxima Capability:** **Loop contínuo de melhoria (M9+)**.

---

## ## 7. Matriz Oficial de Dependências

| Capability                  | Depende de (Pré-requisito) | Impacta (Contextos)    | Componentes Críticos            | Eventos Chave                     | Commands Chave                      | Banco de Dados              | Broker |
|-----------------------------|----------------------------|------------------------|---------------------------------|-----------------------------------|-------------------------------------|-----------------------------|--------|
| <b>M0 - Arquitetura</b>     | N/A                        | Todos                  | Command Bus, Security Kernel    | `IngestionStartedEvent`           | `CreateTenantCommand`               | PostgreSQL                  | Kafka  |
| <b>M1 - Observabilidade</b> | M0                         | Todos                  | OTEL Collector, Grafana         | `TelemetryEmittedEvent`           | -                                   | PostgreSQL (logs)           | Kafka  |
| <b>M2 - Monitoramento</b>   | M1                         | SRE, On-call           | Alertmanager, PagerDuty         | `AlertFiredEvent`                 | `AcknowledgeAlertCommand`           | PostgreSQL                  | Kafka  |
| <b>M3 - Correlação</b>      | M2                         | Incident, Security     | CEP Engine, Policy Engine       | `IncidentCorrelatedEvent`         | `CreateIncidentCommand` (auto)      | PostgreSQL (Read-optimized) | Kafka  |
| <b>M4 - Threat Intel</b>    | M3                         | Intelligence, Security | TI Aggregator, Knowledge Base   | `IndicatorMatchedEvent`           | `UpdateThreatIntelCommand`          | PostgreSQL (Vetorial)       | Kafka  |
| <b>M5 - AI Advisory</b>     | M4                         | AI, Incident           | AI Gateway, RAG Engine          | `SuggestionGeneratedEvent`        | `ApplySuggestionCommand` (aprovado) | Knowledge Base (Neo4j)      | Kafka  |
| <b>M6 - AI Copilot</b>      | M5                         | Todos (UI)             | Chat UI, Intent Parser          | `CopilotInteractionEvent`         | -                                   | -                           | -      |
| <b>M7 - Automação</b>       | M6                         | Response, Security     | Orchestrator, Playbook Executor | `AutomationTriggeredEvent`        | `ExecutePlaybookCommand` (auto)     | PostgreSQL                  | Kafka  |
| <b>M8 - Red Team</b>        | M7                         | Security, Assurance    | Simulator, MITRE Mapper         | `DefenseGapIdentifiedEvent`       | `SimulateAttackCommand`             | PostgreSQL (Testes)         | Kafka  |
| <b>M9 - Autonomia</b>       | M8                         | Todos (Governança)     | Self-Healing Engine, RL Agent   | `AutonomousDecisionExecutedEvent` | *Todos (com exceções)*              | PostgreSQL                  | Kafka  |

---

## ## 8. Governança

### ### 8.1 Architecture Board (AB)

- **Composição:** Arquiteto Chefe, Security Architect, SRE Lead, Product Manager (segurança), CISO (convidado).
- **Atribuições:** Aprovar a transição entre Maturity Levels. Revisar ADRs (Architecture Decision Records). Manter a integridade dos princípios de DDD e Clean Architecture.
- **Quórum:** 4/5 membros para aprovação de M1-M6; 5/5 membros + aval do CISO para M7-M9.

### ### 8.2 Security Review (SRB)

- **Foco:** Garantir que a nova Capability não viola Zero Trust.
- **Output:** Relatório de Segurança contendo análise de threat modeling (STRIDE) e validação de políticas OPA/Cedar.

### ### 8.3 Processo de Mudança (RFC/ADR)

1. **RFC (Request for Change):** Proposta inicial descrevendo a Feature e a Capability alvo.
2. **ADR:** Decisão arquitetural formal aprovada pelo AB.
3. **Implementação:** Segue as checklists de DoR e DoD.
4. **Validação:** Testes de carga, segurança e observabilidade.
5. **Promoção:** Implantação em produção via Progressive Delivery (Canary/Blue-Green).

---

## ## 9. Critérios de Evolução

### ### 9.1 Prontidão para Avançar

Para avançar de `Mn` para `M(n+1)`, os seguintes critérios são MANDATÓRIOS:

1. **KPIs Sustentados:** Os KPIs de `Mn` devem estar verdes por 15 dias consecutivos.
2. **Dívida Técnica Controlada:** O débito técnico específico de `Mn` deve ser inferior a 5% do esforço total de desenvolvimento.
3. **Runbooks Atualizados:** Os runbooks de incidentes para `Mn` devem existir e ter sido testados em drills.
4. **Aprovação do CISO:** Para M5 em diante.

---

## ## 10. Definition of Ready (DoR) – Checklist

Para que o desenvolvimento de uma Capability seja iniciado, o time DEVE atestar os seguintes itens:

- [ ] **Arquitetura Aprovada:** ADR finalizado e revisado pelo Architecture Board.
- [ ] **Modelagem de Domínio:** Eventos e Commands afetados foram mapeados (M0.6/M0.7).
- [ ] **Impacto em Dados:** Schema de banco de dados mapeado e aprovado (M0.4).

- [ ] **\*\*Matriz de Dependências:\*\*** Dependências com outras Capabilities foram validadas.
- [ ] **\*\*Testes Planejados:\*\*** Estratégia de testes (unitário, integração, contrato) definida.
- [ ] **\*\*KPIs Definidos:\*\*** Métricas de sucesso para a nova Capability documentadas.
- [ ] **\*\*Riscos Identificados:\*\*** Threat Modeling realizado e riscos aceitos (ou mitigados).
- [ ] **\*\*Orçamento Alocado:\*\*** Recursos humanos e financeiros disponíveis.

---

## ## 11. Definition of Done (DoD) – Checklist

Para que a Capability seja considerada ENTREGUE em produção, o time DEVE atestar:

- [ ] **\*\*Cobertura de Testes:\*\*** > 90% (Unitário) e 80% (Integração) – estatisticamente.
- [ ] **\*\*Contratos Testados:\*\*** Testes de contrato (Pact) para Commands/Eventos aprovados.
- [ ] **\*\*Observabilidade:\*\*** Traces, Métricas e Logs implementados e visualizáveis no Grafana.
- [ ] **\*\*Documentação:\*\*** Docs atualizadas (arquitetura, onboarding, runbooks).
- [ ] **\*\*Rollback:\*\*** Plano de rollback documentado e testado com sucesso.
- [ ] **\*\*Performance:\*\*** Teste de carga aprovado (latência p99 dentro do SLO).
- [ ] **\*\*Segurança:\*\*** Scan de vulnerabilidades (SAST/DAST) aprovado.
- [ ] **\*\*Versionamento:\*\*** Tags SemVer aplicadas a todos os artefatos (M0.5).
- [ ] **\*\*CI/CD:\*\*** Pipeline de deployment automatizado (GitOps) configurado.

---

## ## 12. KPIs Operacionais

| KPI                 | Definição                  | M1     | M2     | M3     | M4     | M5     | M6    | M7    | M8    | M9    |
|---------------------|----------------------------|--------|--------|--------|--------|--------|-------|-------|-------|-------|
| **MTTD**            | Tempo Médio para Detect.   | 30 min | 10 min | 1 min  | 45s    | 30s    | 20s   | 15s   | 10s   | 5s    |
| **MTTR**            | Tempo Médio para Resolver. | -      | 60 min | 30 min | 15 min | 10 min | 5 min | 2 min | 1 min | 30s   |
| **FPR**             | Taxa de Falsos Positivos   | -      | -      | 10%    | 8%     | 5%     | 3%    | 2%    | <1%   | <0.5% |
| **Automação Rate**  | % resolvido sem humano.    | 0%     | 0%     | 5%     | 10%    | 15%    | 20%   | 30%   | 50%   | >95%  |
| **Precisão IA**     | Acurácia da Sugestão.      | -      | -      | -      | -      | 70%    | 75%   | 85%   | 90%   | 99%   |
| **Cobertura MITRE** | % de técnicas mapeadas.    | -      | -      | -      | 20%    | 40%    | 50%   | 60%   | 70%   | 80%   |
|                     |                            |        |        |        |        |        |       |       | 95%   |       |

---

## ## 13. SLO e SLA

| Categoria                    | SLO (Target) | SLA (Garantido) | Consequência do Não Cumprimento    |
|------------------------------|--------------|-----------------|------------------------------------|
| **Disponibilidade (API/UI)** | 99.99%       | 99.95%          | Crédito financeiro, Plano de Ação. |
| **Latência de Ingestão**     | p95 < 50ms   | p99 < 100ms     | Escalonamento horizontal forçado.  |

| **\*\*Latência de Correlação (M3)\*\*** | p95 < 500ms | p99 < 1s | Rollback da regra de correlação. |  
| **\*\*Tempo de Geração IA (M5)\*\*** | p95 < 2s | p99 < 5s | Falback para modo offline (cache). |  
| **\*\*Tempo de Execução (M7)\*\*** | p95 < 1s | p99 < 3s | Desativação da automação para aquele tipo de Command. |  
| **\*\*Durabilidade de Dados\*\*** | 99.999% | 99.99% | Restauração de backup multi-region. |

---

## ## 14. Gestão de Riscos

| Risco | Probabilidade | Impacto | Estratégia de Mitigação | Aceitação |  
| :--- | :--- | :--- | :--- | :--- |  
| **\*\*Alucinação IA (M5-M9)\*\*** | Média | Crítico | Human-in-the-loop até M9. Políticas de validação de saída (JSON Schema). | Risco residual aceito (M5-M7), mitigado em M8-M9. |  
| **\*\*Cadeia de Automação (M7)\*\*** | Baixa | Catastrófico | Circuit Breakers por Tenant. Limite de execuções/minuto. | Não aceito; deve ser resolvido antes de M7. |  
| **\*\*Vazamento de Dados (M4-M9)\*\*** | Baixa | Catastrófico | Zero Trust, criptografia em repouso/trânsito, auditoria rígida. | Risco residual inerente, mas controlado. |  
| **\*\*Falha de Dependência (Ex: Kafka down)\*\*** | Média | Alto | Replicação multi-zona, fallback para cache local. | Risco aceito com plano de BCP. |  
| **\*\*Envenenamento de TI (M4)\*\*** | Média | Alto | Validação de fonte (trusted feeds only), verificação de reputação. | Risco aceito com monitoramento de anomalias. |  
| **\*\*Viés Algorítmico (M9)\*\*** | Média | Médio | Auditoria de decisões semanais por comitê de ética. | Risco mitigado pela supervisão humana. |

---

## ## 15. Observabilidade

### ### 15.1 OpenTelemetry (OTel)

Todos os serviços devem exportar:

- **\*\*Traces:\*\*** 100% das requisições do Command Bus e Handlers.
- **\*\*Metrics:\*\*** Golden Signals (Latência, Tráfego, Erros, Saturação).
- **\*\*Logs:\*\*** Estruturados (JSON), com níveis dinâmicos (INFO/DEBUG/ERROR).

### ### 15.2 Estratégia de Alertas

- **\*\*Crítico:\*\*** PagerDuty aciona on-call.
- **\*\*Alto:\*\*** Slack/Teams com notificação.
- **\*\*Médio:\*\*** Dashboard highlight.
- **\*\*Baixo:\*\*** Registro em log para análise de tendência.

### ### 15.3 Health Checks

- **\*\*Liveness:\*\*** API `/healthz` (verifica se o serviço responde).
- **\*\*Readiness:\*\*** API `/ready` (verifica dependências como banco e Kafka).
- **\*\*Deep Health:\*\*** (M3+) Verifica atraso na fila de processamento.

---

## ## 16. Segurança Operacional

- **Zero Trust:** Toda comunicação entre Contextos é autenticada via mTLS.
- **Least Privilege:** As credenciais da pipeline de CI/CD para deploy de novas Capabilities possuem escopo limitado ao Bounded Context correspondente.
- **RBAC:** A evolução das Capabilities requer permissões específicas (`capability:write`, `capability:approve`).
- **MFA:** Exigido para qualquer ação de aprovação de Gate (M5+).
- **LGPD/GDPR:** Dados sensíveis (PII) ingeridos no M0 devem ser anonimizados para uso em M5/M9 (LLMs). A base de conhecimento não deve armazenar PII diretamente.

---

## ## 17. Roadmap Oficial (Baseado em Dependências)

```
```mermaid
graph LR
  M0[Arquitetura] --> M1[Observabilidade]
  M1 --> M2[Monitoramento]
  M2 --> M3[Correlação]
  M3 --> M4[Threat Intel]
  M4 --> M5[AI Advisory]
  M5 --> M6[AI Copilot]
  M6 --> M7[Automação]
  M7 --> M8[Red Team Autônomo]
  M8 --> M9[Autonomia Supervisionada]

  M3 -. -> M5
  M4 -. -> M8
  M5 -. -> M9
```
```

**Nota:** As setas tracejadas indicam dependência de dados, não de progressão linear. Ex: M3 fornece dados para treinar M5, mas M5 pode evoluir conceitualmente em paralelo à M4.

---

## ## 18. Diagramas Mermaid

### ### 18.1 Fluxo de Evolução e Governança

```
```mermaid
flowchart TD
  A[Proposta de Feature] --> B[Arquitetura Board]
  B --> C[Aprovada]
  C --> D[DoR - Checklist]
  D --> E[OK]
  E --> F[Desenvolvimento]
  F --> G[Testes & Observabilidade]
```

```

E --> F[DoD - Checklist]
F -->|OK| G[Security Review]
G -->|Aprovado| H[Canary Deploy]
H --> I{Monitoramento KPIs}
I -->|SLO Alcançado| J[Promoção para Producao]
I -->|SLO Não Alcançado| K[Análise de Falha]
K -->|Hotfix| D
K -->|Rollback| L[Maturidade Anterior]
J --> M[Capability Ativa]
M --> N[Próxima Capability]
...

```

18.2 Matriz de Maturidade Operacional

```

```mermaid
quadrantChart
 title Mapa de Maturidade GovSec Shield
 x-axis "Reativo" --> "Proativo"
 y-axis "Manual" --> "Autônomo"
 quadrant-1 "Autonomia Supervisionada (M9)"
 quadrant-2 "Red Team (M8)"
 quadrant-3 "Base Operacional (M0-M2)"
 quadrant-4 "Correlação e IA (M3-M7)"
 M0-Arquitetura: [0.1, 0.1]
 M2-Monitoramento: [0.2, 0.3]
 M3-Correlação: [0.4, 0.5]
 M5-AI Advisory: [0.6, 0.6]
 M7-Automação: [0.7, 0.8]
 M9-Autonomia: [0.9, 0.95]
...

```

### ### 18.3 Estado de Transição (Ciclo de Vida da Capability)

```

```mermaid
stateDiagram-v2
    [*] --> Proposed: RFC Submetido
    Proposed --> UnderReview: Architecture Board
    UnderReview --> Rejected: Nao Aprovado
    UnderReview --> Planned: DoR Aprovado
    Planned --> InDevelopment: Inicio Implementacao
    InDevelopment --> Testing: DoD Parcial
    Testing --> Staging: Deploy Homolog
    Staging --> Production: Canary & KPIs
    Production --> OperationalExcellence: SLOs Sustentados
    OperationalExcellence --> Evolved: Avancar M(n+1)
    OperationalExcellence --> Rollback: Falha Critica
    Rollback --> Planned: Correcao
    Rejected --> [*]

```


Evolved --> [*]

...

19. Boas Práticas

1. ****Evolua a Observabilidade Primeiro:**** Antes de adicionar uma nova fonte de dados, garanta que os pipelines de telemetria estão dimensionados para recebê-la.
2. ****Versionamento de Políticas:**** Toda mudança no Policy Engine (OPA/Cedar) associada a uma nova Capability deve ser versionada junto com o código.
3. ****Testes de Carga Regulares:**** Ao atingir M7+, execute testes de carga semanais simulando picos de 10x o tráfego normal.
4. ****Chaos Engineering:**** (M8+) Utilize o Red Team para causar falhas deliberadas (ex: matar pods Kafka) para validar a resiliência da automação.
5. ****Feedback Humano Estruturado:**** (M5+) Analistas devem classificar as sugestões de IA (Útil/Não útil) para alimentar o ciclo de ML.
6. ****Documentação como Código:**** Atualize a documentação arquitetural (ADR) simultaneamente ao pull request da Capability.

20. Anti Patterns

| Anti-Pattern | Descrição | Por que é Proibido |

| :--- | :--- | :--- |

| ****Automatizar antes de Observar**** | Implementar M7 (Automação) sem ter M1/M2 consolidados. | Você não saberá se a automação falhou ou causou danos. "Voo cego". |

| ****Adicionar IA sem Métricas**** | Colocar LLM em produção sem definir KPIs de precisão (M5). | A IA pode estar alucinando, corroendo a confiança do analista. |

| ****Ignorar KPIs para Avançar**** | Pular de M3 para M4 com FPR em 20%. | A TI enriquecerá dados ruins, amplificando o ruído. Garbage in, gospel out. |

| ****Pular Capabilidade**** | Tentar implementar M9 (Autonomia) partindo de M2. | A plataforma não tem base de dados histórica (M3/M4) nem validação humana (M5/M6), resultando em decisões catastróficas. |

| ****Quebrar Contratos**** | Alterar o schema de um Evento/Command em um novo MAJOR sem suportar a versão antiga (M0.5). | Quebra a correlação (M3) e a inteligência (M4) que dependem de dados históricos. |

| ****Ignorar Governança**** | Implementar uma Feature sem passar pelo Architecture Board. | Gera "Shadow IT" arquitetural, fragmentando o modelo DDD e criando dívida técnica impagável. |

| ****Overfitting de IA**** | Treinar o AI Advisor apenas com dados do último mês. | O modelo não generaliza bem para novas táticas de ataque (M8/M9). |

21. Débitos Técnicos

ID	Débito	Capability Relacionada	Impacto	Plano de Mitigação	Prioridade
---	---	---	---	---	---
TD-OPS-01	Falta de "Playbook as Code" versionado.	M7	Dificuldade em auditar mudanças em automação.	Implementar repositório Git para Playbooks com CI/CD.	Alta
TD-OPS-02	Modelos de IA (M5) com viés para dados de rede, ignorando dados de identidade.	M5, M9	Decisões de bloqueio podem ser imprecisas para usuários.	Balanceamento de dataset de treino.	Média
TD-OPS-03	Testes de carga da Camada de Correlação (M3) não cobrem picos de feriados.	M3	Risco de queda de performance em eventos sazonais.	Simulação de Black Friday/Eventos Cibernéticos.	Média
TD-OPS-04	Ausência de "Kill Switch" granular (por Tenant) para M7-M9.	M9	Rollback de automação afeta todo o sistema.	Implementar feature flags por Tenant.	Alta
TD-OPS-05	Dependência de um único feed de Threat Intel (M4).	M4	Ponto único de falha (SPOF) para enriquecimento.	Adicionar feed secundário (redundância).	Alta

22. Changelog

22.1 Versão 1.0.0 (2026-07-28)

- ****Criação Inicial:**** Documento fundacional do Modelo Operacional e de Maturidade.
- ****Definição do Modelo M0-M9:**** Estabelecimento formal dos 10 estágios de evolução.
- ****Definição de KPIs e SLOs:**** Métricas oficiais para cada nível.
- ****Governança:**** Criação dos processos de Architecture Board e Security Review.
- ****Roadmap:**** Mapeamento de dependências e transições.
- ****DoR/DoD:**** Checklists oficiais para desenvolvimento.

22.2 Mudanças Futuras (Previstas)

Versão	Data Prevista	Mudança Planejada
---	---	---
1.1.0	Q1 2027	Inclusão de M10 - "Autonomia Estratégica" (planejamento de capacidade preditivo).
1.2.0	Q3 2027	Refinamento dos KPIs de M8/M9 baseado em dados de produção.
2.0.0	Q1 2028	Revisão do Modelo de Maturidade para incorporar exigências da nova regulamentação (ex: AI Act).

Anexos

Anexo A: Glossário de Termos Estratégicos

Termo	Definição
---	---
Golden Signals	Latência, Tráfego, Erros, Saturação (Google SRE).
Strangler Fig Pattern	Estratégia de substituição incremental de funcionalidades.
Circuit Breaker	Padrão que interrompe chamadas a serviços com falha para evitar cascadeamento.
Progressive Delivery	Implantação gradual (Canary, A/B Test, Feature Flags).

| ****STIX/TAXII**** | Padrões para troca de inteligência de ameaças cibernéticas. |
| ****RAG**** | Retrieval-Augmented Generation - técnica para enriquecer prompts de IA com dados da base de conhecimento. |

Anexo B: Referências Normativas

- ****NIST SP 800-207****: Zero Trust Architecture.
- ****Google SRE Book****: Site Reliability Engineering.
- ****TOGAF 9.2****: The Open Group Architecture Framework (Capability-Based Planning).
- ****MITRE ATT&CK Framework****: Matriz de Táticas e Técnicas.
- ****M0.1 a M0.7****: Documentos base do GovSec Shield.

****Fim do Documento M0.8****

Este documento é a carta magna da evolução do GovSec Shield. A não conformidade com os estágios de maturidade, a tentativa de automação prematura ou a violação dos critérios de DoR/DoD caracterizam violação grave da governança corporativa, sujeita a revisão pelo CISO e pelo Comitê Executivo.

GovSec Shield

M0.9 — Engineering Playbook, Product Backlog e Plano Mestre de Desenvolvimento

****Versão:**** 1.0.0

****Status:**** RATIFICADO PELO COMITÊ DE ARQUITETURA E ENGENHARIA

****Data de Vigência:**** 28 de Julho de 2026

****Classificação:**** PÚBLICO INTERNO - ENGENHARIA

****Nível de Controle:**** EXECUÇÃO TÁTICA

****Substitui:**** N/A

****Impacto:**** TODAS AS EQUIPES DE DESENVOLVIMENTO, SRE, QA, SECURITY E PRODUCT MANAGEMENT

Sumário Executivo

Este documento é a ****Constituição da Engenharia**** do GovSec Shield. Diferencia-se dos artefatos arquiteturais (M0.1–M0.8) por focalizar exclusivamente no ****como construir****, não no ****o que construir**** ou ****por que construir****. Ele estabelece os processos, padrões, fluxos de trabalho, critérios de qualidade e estrutura de backlog que regerão toda a atividade de desenvolvimento, desde a concepção de uma Feature até sua entrega em produção.

Este documento é mandatório para todas as equipes de engenharia, garantindo consistência, qualidade, segurança e eficiência ao longo de todo o ciclo de vida do produto. A não conformidade com as seções aqui descritas constitui violação do processo de engenharia, sujeita a bloqueio de pull requests e impedimento de deploys.

1. Introdução

1.1 Objetivo

1. **Definir a Estrutura do Backlog:** Estabelecer a hierarquia oficial de artefatos (Capabilities, Epics, Features, Stories) e organizar o Product Backlog de forma a refletir a maturidade definida no M0.8.
2. **Operacionalizar a Arquitetura:** Traduzir as decisões arquiteturais (M0.1–M0.8) em tarefas executáveis, sprints e marcos de entrega.
3. **Estabelecer Padrões de Engenharia:** Definir fluxos de trabalho de Git, CI/CD, testes, qualidade de código, revisão de código e documentação.
4. **Criar a Estratégia de Execução:** Elaborar um Plano Mestre de Sprints (incluindo Sprint Zero) que guie a equipe até a entrega do MVP e posteriormente da plataforma Enterprise.
5. **Instituir Mecanismos de Governança do Código:** Estabelecer critérios rigorosos de Definition of Ready (DoR) e Definition of Done (DoD), bem como processos de Code Review e gestão de débito técnico.

1.2 Escopo

Este documento abrange:

- **Organização do Produto:** Hierarquia de backlog (Capability → Epic → Feature → Story → Task).
- **Product Backlog e Roadmap:** Lista exaustiva de Epics e a ordem de implementação baseada em dependências.
- **Plano Mestre de Sprints:** Detalhamento de sprints, incluindo Sprint Zero, sprints de fundação, sprints de capacidades, sprints de integração e sprints de refinamento.
- **Estratégia DevSecOps:** Pipeline de CI/CD, estratégia Git, qualidade de código, testes e segurança (SAST/DAST).
- **Governança de Engenharia:** Code Review, Documentation as Code, ADRs (Architecture Decision Records), gestão de débito técnico.
- **Métricas e KPIs:** Indicadores de desempenho da engenharia (lead time, deployment frequency, MTTR).

1.3 Não Escopo

- **Definição de Arquitetura:** Toda decisão arquitetural já foi tomada em M0.1–M0.8. Este documento não altera nem adiciona novas decisões arquiteturais.
- **Estratégia de Negócios:** Pricing, go-to-market, vendas ou estratégias de produto (exceto a priorização do backlog, que reflete diretrizes de negócio já aprovadas).
- **Detalhes de Infraestrutura Física:** Provisionamento de hardware ou configuração de rede profunda (scope de SRE).

1.4 Relação com Documentos Anteriores (M0.1 – M0.8)

| Documento | Relação com M0.9 |

| :--- | :--- |

| **M0.1 (Visão e Paradigma)** | M0.9 transforma a visão em um plano de ação. A filosofia de "Security OS" é traduzida em Features e Stories. |

| **M0.2 (Bounded Contexts)** | A estrutura de squads e ownership no M0.9 é alinhada aos Bounded Contexts. |

M0.3 (Event Storming)	As histórias derivadas do Event Storming são mapeadas como Features e Tasks neste documento.
M0.4 (Modelo ER)	A implementação do modelo de dados (migrações, schemas) é planejada nas sprints e sujeita a testes de integração.
M0.5 (Versionamento)	A estratégia de releases e versionamento SemVer do M0.5 é formalizada no pipeline de CI/CD e nas políticas de branching.
M0.6 (Contratos Canônicos)	Os testes de contrato (Pact) para eventos são uma exigência de DoD no M0.9.
M0.7 (Modelo de Commands)	A implementação dos Commands é o principal conteúdo das Epics e Stories.
M0.8 (Modelo Operacional)	O roadmap de M0.9 é diretamente derivado dos estágios de maturidade (M0–M9) definidos em M0.8.

1.5 Objetivos da Engenharia

1. **Entregar o MVP (M0 + M1 + M2)** em um ciclo de desenvolvimento de 10 sprints.
2. **Atingir M3 (Correlação)** no segundo ciclo (sprints 11–20).
3. **Estabelecer Dívida Técnica Zero** para funcionalidades entregues (com exceção de débitos planejados e aprovados).
4. **Manter um lead time médio de Feature < 5 dias úteis.**
5. **Garantir que 100% do código de produção passe por SAST, DAST e análise de dependências.**

2. Filosofia da Engenharia

Os princípios abaixo são a base de toda decisão de engenharia no GovSec Shield.

| Princípio | Definição | Implicação Prática |

| :--- | :--- | :--- |

| **Engineering First** | A excelência técnica não é negociável. Qualidade, manutenibilidade e performance são priorizadas sobre prazos arbitrários. | Metas de cobertura de teste e complexidade ciclomática são *hard gates* no pipeline. |

| **Quality First** | A qualidade é responsabilidade de todos, não apenas do QA. | TDD (Test-Driven Development) é fortemente encorajado. Todo bug crítico gera um novo teste de regressão. |

| **Architecture First** | A arquitetura (DDD, Hexagonal, CQRS) guia a implementação, e não o contrário. | Qualquer desvio arquitetural deve ser justificado em um ADR e aprovado pelo Architecture Board. |

| **Domain First** | O código deve refletir a linguagem ubíqua do domínio. | Nomes de classes, métodos e variáveis devem espelhar termos do domínio (ex: ``Incident``, ``BlockIPCommand``). |

| **Security First** | Segurança é uma propriedade não-funcional de primeira classe, não um anexo. | A pipeline de CI/CD inclui etapas obrigatórias de security scanning. A configuração de políticas OPA é code-reviewed. |

| **Documentation First** | Documentação é tão importante quanto o código. Ela deve ser criada junto com o código, não depois. | ADRs são criados antes da implementação. READMEs e diagramas são atualizados no mesmo PR. |

| **Automation First** | Tudo que pode ser automatizado (testes, deploys, provisionamento) deve ser automatizado. | Pipelines de CI/CD são declarativos e executam 100% das verificações sem intervenção manual. |

| **Everything as Code** | Configuração, políticas, infraestrutura, documentação e pipelines são versionados como código. | Uso de GitOps (ArgoCD), Terraform, e políticas OPA/Cedar em repositórios de código. |

| **Observability by Default** | Nenhuma nova funcionalidade é considerada completa sem métricas, logs e traces. | Todos os serviços exportam OpenTelemetry; dashboards são criados no mesmo PR que a funcionalidade. |

| **Infrastructure as Code** | A infraestrutura (rede, bancos, clusters) é provisionada via código versionado. | Uso de Terraform, Kubernetes manifests (Helm) no mesmo repositório ou em um repo de infra dedicado. |

3. Organização do Produto

A hierarquia oficial do produto é a seguinte:

...

Vision (Visão do Produto)

- └─ Capability (Capacidade) → Mapeada para M0.8
 - └─ Epic (Epopéia) → Grande iniciativa
 - └─ Feature → Funcionalidade entregável
 - └─ Story → História de usuário
 - └─ Task → Tarefa atômica para dev
 - └─ Bug → Defeito em produção
 - └─ Spike → Investigação técnica
 - └─ Technical Debt → Dívida técnica planejada
 - └─ Hotfix → Correção urgente em produção

...

Nível	Descrição	Exemplo	Responsável
Vision	Objetivo de longo prazo do produto.	"Tornar o GovSec Shield um SOC autônomo."	Product Manager
Capability	Maturidade funcional (M0.8).	"M3 - Correlação."	Architecture Board
Epic	Iniciativa grande, multi-sprint, que contribui para uma Capability.	"Implementar Engine de Correlação CEP."	Tech Lead / PM
Feature	Funcionalidade específica dentro de uma Epic.	"Correlação de falhas de autenticação."	Time de desenvolvimento
Story	História de usuário (quem, o quê, por quê).	"Como Analista, quero ver a correlação no dashboard."	Desenvolvedor
Task	Tarefa técnica atômica (ex: criar endpoint, escrever teste).	"Criar endpoint GET /correlation/rules."	Desenvolvedor
Bug	Defeito relatado.	"Incidente com ID 123 não está correlacionando."	Desenvolvedor (prioridade alta)

Spike	Investigação para reduzir incerteza técnica.	"Testar performance do CEP com 10k eventos/s."	Desenvolvedor (time-boxed)
Technical Debt	Melhoria planejada para reduzir dívida técnica.	"Refatorar módulo de integração de logs."	Time de engenharia
Hotfix	Correção crítica para produção.	"Hotfix: Corrigir SQL injection na API."	Time de engenharia (imediato)

4. Organização do Product Backlog

4.1 Estrutura do Backlog

O Product Backlog é organizado em **Capabilities** (alinhadas ao M0.8). Dentro de cada Capability, as **Epics** são priorizadas. Cada Epic contém **Features** e **Stories**.

Regra de Priorização:

1. **Dependências Técnicas:** Features que são pré-requisitos para outras são priorizadas primeiro (ex: M0 antes de M1).
2. **Valor de Negócio:** Features que entregam maior redução de risco ou valor operacional para o SOC têm prioridade.
3. **Risco:** Features com alto risco técnico ou de segurança são priorizadas para serem executadas em sprints iniciais.

4.2 Política de Backlog

- O Backlog é **groomed** (refinado) semanalmente pelo Product Manager, Tech Leads e Architecture Board.
- Toda Feature deve ter uma **Definition of Ready (DoR)** preenchida antes de entrar em uma Sprint.
- Dívida técnica documentada é tratada como backlog, com prioridade definida baseada em impacto.

5. Catálogo Oficial de Epics

Este catálogo lista todas as Epics planejadas para o GovSec Shield, organizadas por Capability. Cada Epic inclui: Objetivo, Owner, Dependências, Features-chave e Critérios de Aceite.

5.1 Epic: Core Platform (M0 - Arquitetura)

- **Objetivo:** Estabelecer a base do Security OS, implementando o Command Bus, Security Kernel, e infraestrutura de mensageria.
- **Owner:** Squad Core Platform.
- **Dependências:** N/A.
- **Features-chave:**
 - Setup do Cluster Kubernetes.
 - Deploy do Redpanda (Kafka).
 - Deploy do PostgreSQL (com particionamento por tenant).

- Implementação do Command Bus (gRPC + Kafka).
- Implementação do Security Kernel (autenticação JWT, RBAC básico).
- Implementação do Policy Engine (OPA) com políticas padrão.
- Criação dos Commands base: ``CreateIncidentCommand``, ``IngestLogCommand``, ``BlockIPCommand`` (stubs).
- Setup do OpenTelemetry Collector.
- ****Critérios de Aceite:****
 - O pipeline de deploy (CI/CD) está funcional.
 - A API Health Check (``/healthz``, ``/ready``) retorna status OK para todos os serviços.
 - Um comando ``CreateTenantCommand`` é executado com sucesso e gera um evento ``TenantCreatedEvent``.
 - O Security Kernel rejeita Commands não autenticados.

5.2 Epic: Observability Foundation (M1 - Observabilidade)

- ****Objetivo:**** Implementar a instrumentação completa e dashboards.
- ****Owner:**** Squad SRE/Observability.
- ****Dependências:**** Core Platform.
- ****Features-chave:****
 - Instrumentação de todos os serviços com OpenTelemetry SDK (Java, Python, Go).
 - Exportação de traces para Tempo/Jaeger.
 - Exportação de métricas para Prometheus.
 - Exportação de logs estruturados para Loki.
 - Criação de dashboards Golden Signals no Grafana.
 - Configuração de alertas básicos (CPU, memória, latência).
- ****Critérios de Aceite:****
 - Cobertura de tracing em 100% das requisições HTTP/gRPC.
 - Todos os dashboards estão disponíveis e mostram dados em tempo real.
 - Alerta de "Serviço Down" dispara e notifica no Slack/PagerDuty.

5.3 Epic: Monitoring and Alerting (M2 - Monitoramento)

- ****Objetivo:**** Configurar alertas proativos e runbooks.
- ****Owner:**** Squad SRE/Observability.
- ****Dependências:**** Observability Foundation.
- ****Features-chave:****
 - Definição de regras de alerta para cada Bounded Context.
 - Configuração do Alertmanager com rotas para equipes on-call.
 - Criação de runbooks para alertas comuns.
 - Integração com PagerDuty/Opsgenie.
 - Implementação do ``AcknowledgeAlertCommand``.
- ****Critérios de Aceite:****
 - Teste de alerta (Fire Drill) bem-sucedido.
 - O runbook para o alerta "Kafka Lag" existe e é acessível via dashboard.
 - O ``AcknowledgeAlertCommand`` atualiza o status do alerta no Grafana.

5.4 Epic: Correlation Engine (M3 - Correlação)

- ****Objetivo:**** Construir o motor de correlação de eventos (CEP).
- ****Owner:**** Squad Security Engine.
- ****Dependências:**** Core Platform, Monitoring and Alerting.

- ****Features-chave:****
 - Implementação do processamento de streams (Flink/Kafka Streams).
 - Definição de regras de correlação (ex: falha de login + acesso a dado sensível).
 - Criação de interface para definir regras (UI) e versionar regras (Git).
 - Implementação do ``CreateIncidentCommand`` automático com base em correlação.
 - Métricas de correlação (incidentes gerados, falsos positivos).
- ****Critérios de Aceite:****
 - Geração de incidente automatizado para cenários de teste (ex: brute force).
 - Taxa de falsos positivos < 10% durante testes de carga.
 - O tempo de correlação (desde o evento até o incidente) é < 1 segundo.

5.5 Epic: Threat Intelligence (M4 - Threat Intel)

- ****Objetivo:**** Integrar feeds de inteligência de ameaças.
- ****Owner:**** Squad Intelligence.
- ****Dependências:**** Correlation Engine.
- ****Features-chave:****
 - Ingestão de feeds STIX/TAXII (MISP, AlienVault, etc.).
 - Enriquecimento automático de eventos com dados de TI (IP, domínio, hash).
 - Atualização da Knowledge Base (PostgreSQL vetorial).
 - Implementação do ``UpdateThreatIntelCommand``.
- ****Critérios de Aceite:****
 - Pelo menos 2 feeds externos estão ingeridos.
 - Um evento contendo um IP malicioso conhecido tem o campo ``threat_score`` preenchido.
 - O tempo de enriquecimento é < 500ms.

5.6 Epic: AI Advisory (M5 - AI Advisory)

- ****Objetivo:**** Implementar a camada de IA consultiva.
- ****Owner:**** Squad AI.
- ****Dependências:**** Threat Intelligence, Knowledge Base.
- ****Features-chave:****
 - Deploy do AI Gateway (LLM on-prem ou via API).
 - Configuração do RAG (Retrieval-Augmented Generation) com a Knowledge Base.
 - Implementação do ``RequestSuggestionCommand`` e do ``SuggestionGeneratedEvent``.
 - Integração da sugestão no Dashboard SOC.
- ****Critérios de Aceite:****
 - Para um incidente, o sistema gera uma sugestão em < 5s.
 - A sugestão inclui referências à Knowledge Base.
 - O analista pode aceitar/rejeitar a sugestão.

5.7 Epic: Automation & Playbooks (M7 - Automação)

- ****Objetivo:**** Executar playbooks de resposta automatizados.
- ****Owner:**** Squad Response.
- ****Dependências:**** Correlation Engine, AI Advisory.
- ****Features-chave:****
 - Implementação do ``ExecutePlaybookCommand``.
 - Criação de playbooks padrão (ex: isolamento de IP, bloqueio de domínio).
 - Integração com Policy Engine para autorização de automação.

- Implementação de Circuit Breaker para automação.
- ****Critérios de Aceite:****
 - Um playbook de isolamento é executado automaticamente para um incidente de baixa criticidade.
 - O sistema gera um evento `AutomationTriggeredEvent`.
 - A taxa de sucesso da automação é > 95% em testes.

5.8 Epic: Dashboard SOC (UI)

- ****Objetivo:**** Fornecer a interface do analista.
- ****Owner:**** Squad Frontend.
- ****Dependências:**** Core Platform, Correlation Engine, AI Advisory.
- ****Features-chave:****
 - Tela de incidentes (listagem, detalhes, timeline).
 - Visualização de correlação (gráfico de relações).
 - Interface de Chat para AI Copilot (M6).
 - Painel de KPIs (MTTD, MTTR, etc.).
- ****Critérios de Aceite:****
 - O dashboard é responsivo e carrega em < 2s.
 - O analista pode executar Commands diretamente da tela de incidente (ex: `BlockIPCommand`).
 - A timeline mostra eventos e ações em ordem cronológica.

5.9 Epic: Identity & Access Management (IAM)

- ****Objetivo:**** Gerenciar usuários, roles e permissões.
- ****Owner:**** Squad Core Platform.
- ****Dependências:**** Core Platform.
- ****Features-chave:****
 - Integração com SSO (OAuth2/SAML).
 - Gestão de RBAC por tenant.
 - Implementação de MFA.
 - Implementação do `UpdateUserRoleCommand`.
- ****Critérios de Aceite:****
 - Usuário com role `viewer` não pode executar `BlockIPCommand`.
 - Usuário com role `security\_admin` pode executar todos os Commands de segurança.
 - MFA é exigido para Commands críticos (ex: `ApplyPolicyCommand`).

5.10 Epic: Connectors & Integrations

- ****Objetivo:**** Ingerir dados de fontes externas (firewalls, SIEMs, cloud providers).
- ****Owner:**** Squad Integrations.
- ****Dependências:**** Core Platform.
- ****Features-chave:****
 - Conector para AWS CloudTrail.
 - Conector para Azure Activity Logs.
 - Conector para Palo Alto Firewall.
 - Normalização de logs para o formato canônico.
 - Implementação do `IngestLogCommand`.
- ****Critérios de Aceite:****
 - Logs da AWS são ingeridos e normalizados em < 5 minutos.

- O sistema processa 10k logs/s sem perda.

5.11 Epic: Governance & Compliance

- **Objetivo:** Garantir que o sistema atenda a requisitos regulatórios.
- **Owner:** Squad Security/Compliance.
- **Dependências:** Core Platform, IAM, Observability.
- **Features-chave:**
 - Auditoria de todos os Commands (audit trail).
 - Geração de relatórios de compliance (LGPD/GDPR).
 - Implementação de políticas de retenção de dados.
- **Critérios de Aceite:**
 - O sistema gera um relatório de auditoria para um incidente em < 10s.
 - Os logs de auditoria são imutáveis e armazenados por 7 anos.

5.12 Epic: Deployment & DevSecOps

- **Objetivo:** Automatizar a entrega contínua e a segurança do pipeline.
- **Owner:** Squad SRE/DevOps.
- **Dependências:** Core Platform.
- **Features-chave:**
 - Pipeline CI/CD (GitHub Actions/GitLab CI).
 - Infraestrutura como Código (Terraform).
 - GitOps (ArgoCD).
 - SAST/DAST integrados.
 - Análise de dependências (Snyk).
- **Critérios de Aceite:**
 - O pipeline de CI/CD executa em < 10 minutos.
 - 100% dos PRs passam por SAST e análise de dependências.
 - O deploy é feito via ArgoCD (GitOps).

6. Product Roadmap

O roadmap abaixo é baseado em dependências técnicas, não em datas. Ele define a ordem de implementação das Epics.

```
```mermaid
```

```
graph LR
```

```
 subgraph Fase 1: Fundação
```

```
 A[Core Platform] --> B[IAM]
```

```
 A --> C[Deployment & DevSecOps]
```

```
 A --> D[Connectors & Integrations]
```

```
 end
```

```
 subgraph Fase 2: Observabilidade e Monitoramento
```

```
 D --> E[Observability Foundation]
```

```
 E --> F[Monitoring and Alerting]
```

```
 end
```

```
 subgraph Fase 3: Correlação e Inteligência
```

```

 F --> G[Correlation Engine]
 G --> H[Threat Intelligence]
 H --> I[Dashboard SOC]
end
subgraph Fase 4: IA e Automação
 H --> J[AI Advisory]
 J --> K[Automation & Playbooks]
end
subgraph Fase 5: Governança
 A --> L[Governance & Compliance]
end
...

```

**\*\*Ordem de Execução das Epics (Prioritizada):\*\***

1. Core Platform (inclui IAM e Connectors base).
2. Deployment & DevSecOps.
3. Observability Foundation.
4. Monitoring and Alerting.
5. Correlation Engine.
6. Dashboard SOC (paralelo com 5).
7. Threat Intelligence.
8. AI Advisory.
9. Automation & Playbooks.
10. Governance & Compliance (contínuo).

---

## ## 7. Plano Mestre de Sprints

O Plano Mestre de Sprints define sprints de 2 semanas. A duração total prevista para o MVP (M0 + M1 + M2) é de 10 sprints. Para a versão Enterprise (M3 + M4 + M5 + M7), são previstas mais 12 sprints, totalizando 22 sprints.

### ### 7.1 Sprint Zero (Setup Inicial)

- **\*\*Nome:\*\*** Sprint Zero – Foundation Setup.
- **\*\*Objetivo:\*\*** Preparar o ambiente de desenvolvimento, pipelines e ferramentas.
- **\*\*Capabilities atendidas:\*\*** N/A.
- **\*\*Epics:\*\*** Deployment & DevSecOps.
- **\*\*Features/Histórias:\*\***
  - Setup de repositórios Git com branch protection.
  - Setup de CI/CD (GitHub Actions).
  - Configuração de ambiente de desenvolvimento local (Docker Compose).
  - Criação de templates de PR e issues.
  - Setup de linters, formatadores (Prettier, Black, etc.) e pre-commit hooks.
  - Definição de padrões de commit (Conventional Commits).
  - Criação de ADR template.
  - Setup de ferramentas de segurança (Snyk, Trivy) no pipeline.
- **\*\*Entregáveis:\*\***

- Repositórios configurados.
- Pipeline de CI básica (build + test) funcional.
- Documentação de setup (README).
- Template de ADR.
- **Critérios de Aceite:**
  - Um commit de teste passa pelo pipeline e gera artefatos.
  - O ambiente local roda com `docker-compose up`.
  - O PR template contém checklists de DoR.

### ### 7.2 Sprints 1–4: Core Platform (MVP - M0)

- **Sprint 1:** Core Services e Messaging.
  - **Objetivo:** Implementar os serviços base e o Kafka.
  - **Features:**
    - Deploy do Redpanda (Kafka) via Helm.
    - Deploy do PostgreSQL.
    - Criação do serviço de autenticação (JWT).
    - Implementação do Command Bus (gRPC).
  - **Dependências:** Sprint Zero.
  - **Entregáveis:** API de autenticação; producer/consumer Kafka funcional.
- **Sprint 2:** Security Kernel e Command Handlers base.
  - **Objetivo:** Implementar o Security Kernel e Commands base.
  - **Features:**
    - Implementação do Security Kernel (RBAC).
    - Implementação do Policy Engine (OPA) com políticas de exemplo.
    - Implementação do `CreateTenantCommand`.
    - Implementação do `IngestLogCommand`.
  - **Dependências:** Sprint 1.
  - **Entregáveis:** Um tenant pode ser criado e logs podem ser ingeridos.
- **Sprint 3:** Integração e Testes de Contrato.
  - **Objetivo:** Garantir que os serviços interagem corretamente.
  - **Features:**
    - Testes de contrato (Pact) entre Command Bus e Handlers.
    - Testes de integração com PostgreSQL e Kafka.
    - Implementação do `BlockIPCommand` (stub).
    - Implementação do `CreateIncidentCommand` (stub).
  - **Dependências:** Sprint 2.
  - **Entregáveis:** Suite de testes de contrato passando 100%.
- **Sprint 4:** Connectors Base e Onboarding.
  - **Objetivo:** Permitir a ingestão de dados de fontes externas.
  - **Features:**
    - Conector para Syslog (UDP/TCP).
    - Conector para arquivos de log (tail).
    - Normalização básica de logs para o formato canônico.
  - **Dependências:** Sprint 3.
  - **Entregáveis:** Dois conectores (syslog e file) operacionais.

### ### 7.3 Sprints 5–6: Observabilidade (MVP - M1 e M2)

- **Sprint 5:** Instrumentação e Dashboards (M1).

- **Objetivo:** Implementar observabilidade completa.
- **Features:**
  - Instrumentação de todos os serviços com OpenTelemetry.
  - Deploy do Prometheus, Loki, Tempo.
  - Configuração de dashboards Golden Signals no Grafana.
- **Dependências:** Core Platform (Sprint 4).
- **Entregáveis:** Grafana com dados de métricas e logs.
- **Sprint 6:** Alertas e Runbooks (M2).
  - **Objetivo:** Configurar alertas proativos.
  - **Features:**
    - Definição de regras de alerta (CPU, memória, lag do Kafka).
    - Configuração do Alertmanager com rotas para PagerDuty.
    - Criação de runbooks para alertas comuns.
    - Implementação do `AcknowledgeAlertCommand`.
  - **Dependências:** Sprint 5.
  - **Entregáveis:** Sistema de alertas funcional; runbooks documentados.
  - **Críticos:** MVP entregue ao final desta sprint.

### ### 7.4 Sprints 7–10: Correlação e Dashboard (M3 - Versão Enterprise Inicial)

- **Sprint 7:** Motor de Correlação (CEP).
  - **Objetivo:** Implementar o processamento de streams.
  - **Features:**
    - Deploy do Flink ou Kafka Streams.
    - Implementação de pipeline de correlação.
    - Definição de regras de correlação (ex: brute force).
  - **Dependências:** M2 (Sprint 6).
  - **Entregáveis:** Sistema de correlação processando eventos.
- **Sprint 8:** Integração com Incident Management.
  - **Objetivo:** Gerar incidentes a partir da correlação.
  - **Features:**
    - Integração do `CreateIncidentCommand` automático.
    - Implementação do `EscalateIncidentCommand`.
    - Criação da timeline de incidentes.
  - **Dependências:** Sprint 7.
  - **Entregáveis:** Incidentes gerados automaticamente.
- **Sprint 9:** Dashboard SOC (UI).
  - **Objetivo:** Fornecer a interface do analista.
  - **Features:**
    - Tela de listagem de incidentes.
    - Tela de detalhes do incidente.
    - Visualização da correlação (gráfico).
    - Botões de ação (BlockIP, etc.).
  - **Dependências:** Sprint 8.
  - **Entregáveis:** Dashboard funcional para analistas.
- **Sprint 10:** Refinamento e Testes de Carga.
  - **Objetivo:** Estabilizar a correlação e testar performance.
  - **Features:**
    - Testes de carga (10k eventos/s).

- Otimização de queries e regras de correlação.
- Ajuste de KPIs.
- **Dependências:** Sprint 9.
- **Entregáveis:** Relatório de performance; sistema estável.

#### ### 7.5 Sprints 11–14: Threat Intelligence (M4)

- **Sprint 11:** Ingestão de Feeds STIX/TAXII.
- **Sprint 12:** Enriquecimento de Eventos.
- **Sprint 13:** Knowledge Base e Vetorização.
- **Sprint 14:** Dashboard de TI e Métricas.

#### ### 7.6 Sprints 15–18: AI Advisory e Copilot (M5, M6)

- **Sprint 15:** AI Gateway e LLM Setup.
- **Sprint 16:** RAG e Suggestion Engine.
- **Sprint 17:** Integração com Dashboard (Chat).
- **Sprint 18:** Refinamento da IA (Precisão).

#### ### 7.7 Sprints 19–22: Automação e Playbooks (M7)

- **Sprint 19:** Playbook Engine e Executor.
- **Sprint 20:** Automação de Baixo Risco.
- **Sprint 21:** Circuit Breaker e Supervisão.
- **Sprint 22:** Testes Finais e Release Enterprise.

---

### ## 8. Sprint Zero (Detalhamento)

A Sprint Zero é crítica para a qualidade do desenvolvimento futuro. Ela inclui:

#### ### 8.1 Setup de Repositórios

- Criação de repositórios (monorepo ou multi-repo, decisão do Architecture Board: **monorepo** para facilitar coordenação).
- Configuração de `CODEOWNERS` para cada Bounded Context.
- Configuração de branch protection (exigir PR, status checks, etc.).

#### ### 8.2 CI/CD Pipeline

- Configuração do GitHub Actions com stages:
  - `lint`: ESLint, Prettier, Black, etc.
  - `test`: Unit tests.
  - `build`: Compilação e criação de imagens Docker.
  - `security`: Snyk, Trivy, GitLeaks.
- Configuração do ArgoCD para deploy contínuo.

#### ### 8.3 Padrões de Código

- Definição de `.editorconfig` e linters.
- Definição de pre-commit hooks (husky, pre-commit).
- Template de PR com checklist.
- Template de commit (Conventional Commits).

### ### 8.4 Documentação

- Criação de README com instruções de setup.
- Template ADR (`docs/adr/0000-template.md`).
- Template RFC (`docs/rfc/0000-template.md`).

### ### 8.5 Ambientes

- Configuração de ambiente de desenvolvimento (`docker-compose.dev.yml`).
- Configuração de ambiente de staging (namespace Kubernetes).
- Configuração de ambiente de produção (separação de secrets).

---

## ## 9. Estratégia Git

### ### 9.1 Modelo de Branching

O GovSec Shield adota um modelo baseado em **Trunk-Based Development** com **Release Branches**.

- **main**: Branch principal, sempre em estado deployável. Todo código em `main` passou por CI e testes.
- **feature/\***: Branches para desenvolvimento de novas Features. São criados a partir de `main` e mergeados via PR.
- **release/vX.Y.Z**: Branches de preparação para release. Permitem estabilização e correções sem bloquear `main`.
- **hotfix/\***: Branches para correções urgentes em produção. Baseados na tag de release.
- **chore/\***: Tarefas de manutenção (deps, refactor).

### ### 9.2 Naming Convention

- **feature/GOV-123-description** (GOV é o ID da issue)
- **hotfix/GOV-456-description**
- **release/v1.2.0**

### ### 9.3 Commit Convention (Conventional Commits)

...

<type>[optional scope]: <description>

[optional body]

[optional footer]

...

**Tipos permitidos:**

- **feat**: Nova funcionalidade.
- **fix**: Correção de bug.
- **docs**: Documentação.
- **style**: Formatação (sem alteração de lógica).



- `refactor`: Refatoração.
- `perf`: Melhoria de performance.
- `test`: Adição/refatoração de testes.
- `chore`: Tarefas de manutenção (build, deps).
- `ci`: Mudanças no CI/CD.

**\*\*Exemplo:\*\***

...

feat(correlation): implementar regra de brute force

- Adiciona detecção de múltiplas falhas de login
- Gera IncidentCorrelatedEvent
- Atualiza dashboard com nova métrica

GOV-123

...

### ### 9.4 Pull Request Process

1. O desenvolvedor cria uma branch `feature/GOV-XXX`.
2. Desenvolve e faz commits com mensagens semânticas.
3. Abre um PR para `main`.
4. O PR deve passar por todos os checks de CI (build, test, lint, security).
5. Pelo menos **\*\*2 approving reviews\*\*** de membros do time (incluindo um Tech Lead).
6. O PR é mergeado usando **\*\*Squash and Merge\*\*** para manter o histórico limpo.

### ### 9.5 Merge Policy

- **\*\*Não é permitido:\*\*** Merge direto para `main` sem PR.
- **\*\*Requer:\*\*** Status checks verdes.
- **\*\*Requer:\*\*** Revisão aprovada.
- **\*\*Linear History:\*\*** Uso de rebase ou squash para manter histórico linear.

---

## ## 10. Estratégia DevSecOps

### ### 10.1 Princípios DevSecOps

- **\*\*Security as Code:\*\*** Políticas de segurança são versionadas e revisadas como código.
- **\*\*Shift-Left:\*\*** Segurança é aplicada o mais cedo possível no ciclo de desenvolvimento.
- **\*\*Pipeline de Segurança:\*\*** SAST, DAST, análise de dependências e scanning de containers são etapas obrigatórias do pipeline.

### ### 10.2 Pipeline DevSecOps

O pipeline de CI/CD inclui as seguintes etapas obrigatórias:

1. **\*\*Lint e Format:\*\*** Verificação de estilo de código.
2. **\*\*Static Code Analysis (SAST):\*\*** Semgrep, CodeQL.
3. **\*\*Dependency Scanning:\*\*** Snyk, OWASP Dependency Check.
4. **\*\*Secret Scanning:\*\*** GitLeaks, TruffleHog.

5. **Build:** Compilação e construção de imagem Docker.
6. **Container Scanning:** Trivy, Anchore (verifica vulnerabilidades na imagem).
7. **Artifact Scanning:** Verificação de assinatura e integridade.
8. **Deploy (Staging):** Implantação em ambiente de staging.
9. **Dynamic Analysis (DAST):** OWASP ZAP contra o ambiente de staging.
10. **Performance Test:** Testes de carga.
11. **Deploy (Production):** Implantação em produção via GitOps (ArgoCD).

### ### 10.3 Segurança de Secrets

- **Secrets** nunca são armazenados no código.
- **Uso do HashiCorp Vault** (ou AWS Secrets Manager) para gerenciar secrets.
- O CI/CD injeta secrets via variáveis de ambiente (criptografadas).

---

## ## 11. Estratégia CI/CD

### ### 11.1 Pipeline de CI

A pipeline de integração contínua (CI) é executada em cada pull request e push para `main`.

**Stages (GitHub Actions):**

1. **Checkout:** Código fonte.
2. **Setup:** Configurar linguagem (Go, Python, Java).
3. **Cache:** Dependências.
4. **Lint:** Verificação de estilo e análise estática.
5. **Unit Test:** Execução de testes unitários com cobertura.
6. **Integration Test:** Testes de integração com bancos de dados e mensageria (usando containers).
7. **Contract Test (Pact):** Verificação de contratos entre serviços.
8. **Security Scan (SAST & Deps):** Snyk e CodeQL.
9. **Build:** Compilar binário ou imagem Docker.
10. **Package:** Publicar artefatos (imagem Docker para registry).

### ### 11.2 Pipeline de CD (Continuous Delivery)

A entrega contínua (CD) é gerenciada via **ArgoCD** (GitOps).

1. **Staging:** Após merge em `main`, o ArgoCD sincroniza automaticamente o ambiente de staging.
2. **Smoke Tests:** Testes de sanidade executados no staging.
3. **Performance Tests:** Testes de carga executados periodicamente.
4. **Production:** O deploy em produção é acionado via **promoção manual** (ou automática com approvals) através do ArgoCD.
5. **Rollback:** Em caso de falha, o ArgoCD pode reverter para a versão anterior (via rollback de manifestos).

### ### 11.3 Health Checks e Observabilidade no Pipeline

- Após o deploy, o pipeline executa um health check (`/healthz`) para garantir que o serviço está ativo.

- As métricas de erro (5xx) são monitoradas; se excederem o threshold, o deploy é automaticamente revertido.

---

## ## 12. Estratégia de Testes

Tipo de Teste	Ferramenta	Obrigatório?	Cobertura Mínima	Executado em
Unitários	JUnit (Java), pytest (Python), go test	Sim	80% (linhas)	CI (PR)
Integração	Testcontainers, Docker Compose	Sim	70% (casos críticos)	CI (push main)
Contrato	Pact (Consumer-Driven)	Sim	100% (contratos definidos)	CI (PR)
Performance	k6, JMeter	Sim	-	Staging (semanal)
Carga	k6	Sim	-	Staging (mensal)
Stress	k6	Não (planejado)	-	Pré-release
Resiliência	Chaos Mesh, Gremlin	Não (M8+)	-	Pré-release
Segurança (SAST)	CodeQL, Semgrep	Sim	-	CI (PR)
Segurança (DAST)	OWASP ZAP	Sim	-	Staging (pós-deploy)
E2E	Cypress, Playwright	Sim	Fluxos críticos	CI (push main)
Mutation Testing	PITest (Java), Mutmut (Python)	Não	-	Ocasional

### ### 12.1 Testes de Contrato (Pact)

Para garantir a comunicação entre Bounded Contexts, os testes de contrato são obrigatórios para todo novo Command/Evento.

- **Consumer:** O cliente do Command (ex: Dashboard) define expectativas.
- **Provider:** O Handler (ex: Security Kernel) verifica se atende às expectativas.
- **Falha no Pact:** Bloqueia o merge.

### ### 12.2 Testes de Performance e Carga

- **Metas:**
  - Ingestão de 10.000 eventos/segundo.
  - Correlação de 1.000 eventos/segundo.
  - Latência p95 < 100ms para Commands críticos.
- **Execução:** Simula cenários de pico (ex: ataque DDoS) e verifica degradação.

---

## ## 13. Qualidade de Código

### ### 13.1 Padrões Obrigatórios

- **SOLID:** Todos os módulos devem seguir os princípios SOLID.
- **DDD:** Entidades, Value Objects, Aggregates, Repositories, Services e Domain Events devem ser claramente identificados.
- **Hexagonal/Ports & Adapters:** Lógica de negócio isolada de infraestrutura.
- **Dependency Injection:** Uso de injeção de dependência (Spring, Guice, Wire) para desacoplamento.

### ### 13.2 Análise Estática

- **Complexidade Ciclomática:** Máximo de 10 por método.
- **Duplicação de Código:** Máximo de 3% de duplicação.
- **Code Smells:** Zero code smells críticos (bloqueantes) no SonarQube.
- **Cobertura:** 80% de cobertura de linha (branches: 70%).

### ### 13.3 Tipagem

- **Linguagens com tipagem estática:** Go, Java, TypeScript (frontend).
- **Python:** Uso de type hints (mypy) em modo estrito.
- **TypeScript:** Uso de `strict` mode.

---

## ## 14. Code Review

### ### 14.1 Checklist de Code Review

O revisor deve verificar:

- **Arquitetura:** O código segue o modelo Hexagonal/DDD? Há vazamento de infraestrutura para o domínio?
- **Segurança:** Há injeção de SQL, exposição de dados sensíveis, vulnerabilidades OWASP Top 10?
- **Performance:** O código faz uso eficiente de recursos (evita N+1, loops desnecessários)?
- **Legibilidade:** O código é claro, bem nomeado e comentado? É fácil entender a intenção?
- **Acoplamento:** Há acoplamento desnecessário entre Contextos? O uso de eventos é adequado?
- **Testes:** Há testes para os novos caminhos? A cobertura aumentou ou diminuiu?
- **Documentação:** O README, ADR ou comentários foram atualizados?
- **Versionamento:** Os Commands/Eventos estão versionados corretamente? Há breaking changes?

### ### 14.2 Papéis no Code Review

- **Author:** O desenvolvedor que criou o PR.
- **Reviewer (pelo menos 1):** Desenvolvedor sênior ou Tech Lead, com conhecimento no domínio.
- **Security Reviewer (obrigatório para alterações no Security Kernel ou Policy Engine):** Membro do Security Team.

### ### 14.3 Tempo de Revisão

- O reviewer deve fornecer feedback em até **24 horas úteis**.
- O autor deve responder aos comentários e atualizar o PR em até **48 horas**.

---

## ## 15. Documentação

### ### 15.1 Documentos Obrigatórios

- **\*\*ADR (Architecture Decision Record):\*\*** Para toda decisão arquitetural significativa. Armazenado em `docs/adr/`.
- **\*\*RFC (Request for Comments):\*\*** Para propostas de grandes mudanças (ex: novo Bounded Context). Armazenado em `docs/rfc/`.
- **\*\*README:\*\*** Cada serviço/módulo deve ter um README com instruções de setup, build e execução.
- **\*\*Runbooks:\*\*** Procedimentos operacionais para SRE (ex: "Como reiniciar o serviço X", "Como escalar o Kafka").
- **\*\*Playbooks:\*\*** Procedimentos de resposta a incidentes (ex: "Como bloquear um IP manualmente").
- **\*\*Threat Models:\*\*** Modelo de ameaças (STRIDE) para cada serviço crítico.

### ### 15.2 Documentação como Código

- Toda documentação deve ser escrita em Markdown e versionada no repositório.
- Diagramas devem ser gerados a partir de código (Mermaid, PlantUML) para facilitar atualizações.

---

## ## 16. Gestão de Débito Técnico

### ### 16.1 Classificação

- **\*\*Crítico:\*\*** Impede a entrega ou afeta a segurança/produção. Deve ser pago imediatamente.
- **\*\*Alto:\*\*** Afeta a performance ou manutenibilidade de forma significativa. Deve ser pago na próxima sprint.
- **\*\*Médio:\*\*** Afeta a manutenibilidade, mas não bloqueia entregas. Deve ser agendado.
- **\*\*Baixo:\*\*** Melhorias cosméticas ou de estilo. Pode ser ignorado.

### ### 16.2 Registro

- Débito técnico é registrado como **\*\*Issues\*\*** no sistema de rastreamento (Jira), com label `tech-debt`.
- Inclui: descrição, impacto, estimativa e classe.

### ### 16.3 Pagamento

- 20% da capacidade de cada sprint é dedicada ao pagamento de débito técnico.
- Débitos críticos são tratados como hotfixes.

---

## ## 17. Releases

### ### 17.1 Estratégia de Versionamento

- **\*\*SemVer:\*\*** MAJOR.MINOR.PATCH (conforme M0.5).
- **\*\*Alpha:\*\*** `vX.Y.Z-alpha.N` (pré-lançamento, instável).
- **\*\*Beta:\*\*** `vX.Y.Z-beta.N` (estável, testes em produção).
- **\*\*RC (Release Candidate):\*\*** `vX.Y.Z-rc.N` (candidato a estável).

- **Stable/LTS:** `vX.Y.Z` (versão estável, suporte estendido).

### 17.2 Pipeline de Release

1. **Feature Freeze:** Branches de feature não são mais aceitas para a release.
2. **Criação da Release Branch:** `release/vX.Y.Z`.
3. **Estabilização:** Correções de bugs e ajustes de performance.
4. **Testes Finais:** Suite completa de testes (incluindo performance e segurança).
5. **Tag:** Criação da tag `vX.Y.Z`.
6. **Release Notes:** Documentação das mudanças (breaking changes, novas features).

### 17.3 Hotfix

1. **Branch:** `hotfix/GOV-XXX` a partir da tag de produção.
2. **Fix:** Correção e testes.
3. **Merge:** Merge para `main` e para a release branch ativa (se houver).

---

## 18. Definition of Ready (DoR) – Checklist

Uma Feature/Story só pode entrar em uma Sprint se atender a TODOS os itens abaixo:

- [ ] **Critérios de Aceite:** Claramente definidos e acordados com o Product Manager.
- [ ] **Dependências:** Todas as dependências externas estão resolvidas ou acordadas.
- [ ] **Arquitetura:** O design preliminar (ADR) está aprovado (se houver mudança arquitetural).
- [ ] **Modelagem:** Os Commands e Events afetados foram identificados e revisados.
- [ ] **Testes:** O plano de testes (unitários, integração) está definido.
- [ ] **KPIs:** Os KPIs a serem medidos foram definidos.
- [ ] **Segurança:** Uma revisão inicial de segurança foi realizada (threat modeling).
- [ ] **Estimativa:** A história foi estimada (pontos de história ou tempo).
- [ ] **UI/UX:** Se aplicável, os protótipos de UI estão aprovados.
- [ ] **Documentação:** O esforço de documentação foi considerado.

---

## 19. Definition of Done (DoD) – Checklist

Uma Feature/Story só é considerada CONCLUÍDA se atender a TODOS os itens abaixo:

- [ ] **Código:** O código está implementado e segue os padrões de qualidade.
- [ ] **Testes Unitários:** Cobertura mínima de 80% (linhas) e todos os testes passam.
- [ ] **Testes de Integração:** Criados e passando (se aplicável).
- [ ] **Testes de Contrato (Pact):** Criados e passando.
- [ ] **Pipeline CI:** Todos os estágios do pipeline (build, lint, test, security) passam.
- [ ] **Revisão de Código:** Aprovada por pelo menos 2 revisores.
- [ ] **Análise Estática:** Sem code smells críticos no SonarQube.
- [ ] **Documentação:** README, ADR (se aplicável), e docs de API atualizados.
- [ ] **Runbooks/Playbooks:** Atualizados (se houver mudança operacional).

- [ ] **\*\*Integração:\*\*** A funcionalidade foi integrada com os sistemas dependentes.
- [ ] **\*\*Observabilidade:\*\*** Métricas, logs e traces foram configurados e testados.
- [ ] **\*\*Performance:\*\*** Testes de performance básicos (se aplicável) passaram.
- [ ] **\*\*Segurança:\*\*** Sem vulnerabilidades críticas no SAST/DAST.
- [ ] **\*\*Deploy:\*\*** A funcionalidade está em produção (ou staging para validação).
- [ ] **\*\*Smoke Test:\*\*** Teste de sanidade executado no ambiente alvo com sucesso.
- [ ] **\*\*Versionamento:\*\*** Commands/Eventos versionados corretamente.

---

## ## 20. Matriz de Responsabilidades (RACI)

| Atividade | Architecture Board | Tech Lead | Desenvolvedor | QA | SRE | Security | Product Manager |

Atividade	Architecture Board	Tech Lead	Desenvolvedor	QA	SRE	Security	Product Manager
<b>**Definir Arquitetura**</b>	R	C	C	I	I	C	I
<b>**Criar ADR**</b>	A	R	C	I	I	C	I
<b>**Planejar Sprint**</b>	I	C	C	C	C	I	R
<b>**Desenvolver Feature**</b>	I	C	R	C	I	I	I
<b>**Escrever Testes Unitários**</b>	I	C	R	A	I	I	I
<b>**Realizar Code Review**</b>	I	R	C	I	I	C	I
<b>**Revisar Segurança**</b>	C	C	C	I	I	R	I
<b>**Realizar Testes de Integração**</b>	I	C	R	A	I	I	I
<b>**Gerenciar Pipeline CI/CD**</b>	I	C	C	I	R	C	I
<b>**Realizar Deploy**</b>	I	C	I	I	R	C	I
<b>**Gerenciar Backlog**</b>	I	C	I	I	I	I	R
<b>**Gerenciar Riscos**</b>	A	C	I	I	I	C	I
<b>**Aprovar Release**</b>	R	C	I	I	C	C	C

**\*\*Legenda:\*\***

- **\*\*R\*\*** = Responsável (Responsible) – quem executa.
- **\*\*A\*\*** = Aprovador (Accountable) – quem responde pelo resultado.
- **\*\*C\*\*** = Consultado (Consulted) – quem fornece input.
- **\*\*I\*\*** = Informado (Informed) – quem é mantido atualizado.

---

## ## 21. KPIs da Engenharia

| KPI | Definição | Meta | Frequência |

| :--- | :--- | :--- | :--- |

| **\*\*Lead Time\*\*** | Tempo desde o início do desenvolvimento até a entrega em produção. | < 5 dias | Semanal |

| **\*\*Cycle Time\*\*** | Tempo desde o início do desenvolvimento até o PR ser aprovado. | < 3 dias | Semanal |

| **\*\*Deployment Frequency\*\*** | Número de deploys em produção por dia. | > 1 deploy/dia | Diário |

**MTTR (Mean Time to Recover)**	Tempo médio para recuperar de uma falha em produção.	< 30 min	Mensal
**Change Failure Rate**	% de deploys que causam falha em produção.	< 5%	Mensal
**Test Coverage**	Cobertura de código por testes unitários.	> 80%	Diário
**Technical Debt Ratio**	% do esforço de desenvolvimento dedicado a débito técnico.	< 10%	Mensal
**Code Quality (Sonar)**	Número de code smells críticos e bugs.	Zero	Diário
**Security Vulnerabilities**	Número de vulnerabilidades críticas e altas (SAST/DAST).	Zero	Diário
**PR Review Time**	Tempo médio para revisar um PR.	< 24h	Semanal

---

## ## 22. Diagramas Mermaid

### ### 22.1 Hierarquia do Backlog

```
``mermaid
graph TD
 Vision[Vision: Security OS] --> Cap[Cap[M3: Correlação]]
 Cap --> Epic[Epic[Epic: Correlation Engine]]
 Epic --> F1[F1[Feature: CEP Core]]
 Epic --> F2[F2[Feature: Rule Management]]
 F1 --> S1[S1[Story: Implement Flink Job]]
 F1 --> S2[S2[Story: Create Correlation API]]
 S1 --> T1[T1[Task: Setup Flink Cluster]]
 S1 --> T2[T2[Task: Write Unit Tests]]
 S1 --> B[Bug: Fix event deserialization]
```

### ### 22.2 Pipeline CI/CD

```
``mermaid
flowchart LR
 A[Commit] --> B[CI: Lint & Test]
 B --> C[CI: Security Scan]
 C --> D[CI: Build]
 D --> E[CD: Deploy Staging]
 E --> F[Performance Test]
 F --> G[DAST Scan]
 G --> H{Aprovado?}
 H -->|Sim| I[CD: Deploy Production]
 H -->|Não| J[Rollback]
```

### ### 22.3 Git Flow

```
``mermaid
```



```
gitGraph
 commit id: "init"
 branch feature/GOV-123
 checkout feature/GOV-123
 commit id: "feat: add rule engine"
 commit id: "test: add unit tests"
 checkout main
 merge feature/GOV-123 tag: "v1.0.0"
 branch hotfix/GOV-456
 checkout hotfix/GOV-456
 commit id: "fix: CVE-2026-1234"
 checkout main
 merge hotfix/GOV-456 tag: "v1.0.1"
...
```

### ### 22.4 Sprint Flow

```
```mermaid
flowchart TD
  A[Backlog Grooming] --> B[Sprint Planning]
  B --> C[Development]
  C --> D[Daily Standup]
  D --> C
  C --> E[Code Review]
  E --> F[Testing]
  F --> G[Deploy Staging]
  G --> H[Sprint Review]
  H --> I[Retrospective]
```
...
```

---

## ## 23. Boas Práticas

1. **Test-Driven Development (TDD):** Escreva o teste antes do código. Isso garante cobertura e design claro.
2. **Commits Pequenos e Frequentes:** Facilita revisão e rollback.
3. **Mantenha o README Atualizado:** O README é a porta de entrada para novos membros.
4. **Revisão de Código como Mentoria:** Use a revisão para compartilhar conhecimento.
5. **Documento Decisões:** Use ADRs para capturar o "porquê" das decisões arquiteturais.
6. **Automatize Tudo:** Se dói fazer manualmente, automatize.
7. **Segurança em Camadas:** Nunca confie apenas em uma camada de segurança.
8. **Monitore Sempre:** Instrumente seu código com métricas desde o dia 1.
9. **Fail Fast:** Falhe rapidamente em desenvolvimento, para não falhar em produção.
10. **Cultura de Blameless Postmortems:** Foque em aprender, não em culpar.

---

## ## 24. Anti Patterns

| Anti-Pattern | Descrição | Por que é Proibido |  
| :--- | :--- | :--- |  
| **\*\*Pular Revisão\*\*** | Fazer merge de PR sem revisão. | Introduz bugs e dívida técnica. |  
| **\*\*Merge Direto na Main\*\*** | Fazer merge de feature branch sem PR. | Viola o processo de qualidade. |  
| **\*\*Deploy sem Testes\*\*** | Deploy em produção sem executar testes. | Risco de quebrar produção. |  
| **\*\*Ignorar ADR\*\*** | Implementar mudança arquitetural sem ADR. | Gera decisões não rastreáveis. |  
| **\*\*Criar Acoplamento\*\*** | Acoplar Contextos desnecessariamente. | Viola DDD, dificulta evolução. |  
| **\*\*Duplicação de Código\*\*** | Copiar e colar código. | Dificulta manutenção, gera bugs. |  
| **\*\*Hardcode\*\*** | Colocar valores fixos no código. | Quebra configurabilidade, riscos de segurança. |  
| **\*\*Secrets no Código\*\*** | Commit de senhas, chaves, tokens. | Violação crítica de segurança. |  
| **\*\*Commits Gigantes\*\*** | Commits com centenas de arquivos. | Dificulta revisão e rollback. |  
| **\*\*Features Incompletas\*\*** | Entregar funcionalidades pela metade. | Gera débito técnico e confusão. |

---

## ## 25. Roadmap Futuro

- **\*\*Q1 2027:\*\*** Implementar M8 (Red Team Autônomo) e M9 (Autonomia Supervisionada).  
- **\*\*Q3 2027:\*\*** Migrar para um modelo de **\*\*multi-região ativo-ativo\*\*** para alta disponibilidade.  
- **\*\*Q1 2028:\*\*** Integrar com **\*\*sovereign cloud\*\*** para atender regulamentações governamentais.  
- **\*\*Q4 2028:\*\*** Adicionar **\*\*suporte a inteligência artificial explicativa (XAI)\*\*** para transparência de decisões.

---

## ## 26. Débitos Técnicos Planejados

| ID | Débito | Prioridade | Plano de Pagamento |  
| :--- | :--- | :--- | :--- |  
| TD-ENG-01 | Migrar de JDK 17 para JDK 21 (LTS) para aproveitar Virtual Threads. | Alta | Sprint 15 |  
| TD-ENG-02 | Refatorar o módulo de conectores para usar um framework unificado (ex: Apache Camel). | Média | Sprint 20 |  
| TD-ENG-03 | Melhorar a cobertura de testes de integração para 90%. | Média | Contínuo |  
| TD-ENG-04 | Substituir a autenticação JWT por OAuth2/OIDC com suporte a refresh tokens. | Alta | Sprint 12 |

| TD-ENG-05 | Implementar um sistema de feature flags (LaunchDarkly) para deploy contínuo. | Alta | Sprint 18 |

---

## ## 27. Changelog

### ### 27.1 Versão 1.0.0 (2026-07-28)

- **Criação Inicial:** Documento fundacional da engenharia do GovSec Shield.
- **Definição da Hierarquia de Backlog:** Capabilities, Epics, Features, Stories.
- **Catálogo de Epics:** Lista completa de 12 Epics com descrições.
- **Plano Mestre de Sprints:** Detalhamento de 22 sprints, incluindo Sprint Zero.
- **Estratégias:** Git, DevSecOps, CI/CD, Testes, Qualidade de Código, Code Review.
- **Governança:** Matriz RACI, KPIs, DoR/DoD.
- **Diagramas:** Roadmap, Pipeline, Git Flow, Backlog Hierarchy.

### ### 27.2 Mudanças Futuras

| Versão | Data | Mudança |

| :--- | :--- | :--- |

| 1.1.0 | Q4 2026 | Atualização do catálogo de Epics com base em feedback da implementação. |

| 2.0.0 | Q1 2027 | Incorporação de práticas de engenharia para M8/M9 (Chaos Engineering, Reinforcement Learning). |

---

**\*\*Fim do Documento M0.9\*\***

\*Este documento é a bíblia da engenharia do GovSec Shield. Qualquer desvio deve ser justificado e aprovado pelo Architecture Board. A qualidade do produto final depende da estrita adesão a estes padrões e processos.\*

# GovSec Shield

## M1.0 — Technical Standards & Engineering Standards

**\*\*Versão:\*\* 1.0.0**

**\*\*Status:\*\* RATIFICADO PELO COMITÊ DE ARQUITETURA E ENGENHARIA**

**\*\*Data de Vigência:\*\* 28 de Julho de 2026**

**\*\*Classificação:\*\* PÚBLICO INTERNO - ENGENHARIA**

**\*\*Nível de Controle:\*\* EXECUÇÃO TÁTICA**

**\*\*Substitui:\*\* N/A**

**\*\*Impacto:\*\* TODAS AS EQUIPES DE DESENVOLVIMENTO E REVISORES DE CÓDIGO**

---

## ## Sumário Executivo

Este documento estabelece os **\*\*Padrões Técnicos e de Engenharia\*\*** do GovSec Shield. Trata-se da especificação normativa e obrigatória para toda implementação de código,

desde a estrutura de diretórios até a formatação de logs, passando por padrões de nomenclatura, tratamento de erros, persistência, observabilidade e segurança. Todo Pull Request, toda nova funcionalidade, todo módulo ou conector deve estar em conformidade estrita com este documento.

A não conformidade acarreta a rejeição automática da Pull Request e a necessidade de correção antes da integração.

---

## ## 1. Introdução

### ### 1.1 Objetivo

1. **\*\*Uniformizar o Código:\*\*** Garantir que todos os módulos do GovSec Shield sigam a mesma estrutura, estilo e convenções, independentemente do time que os desenvolveu.
2. **\*\*Garantir Qualidade:\*\*** Estabelecer padrões mínimos de qualidade (cobertura de testes, complexidade, segurança) que são verificados automaticamente no pipeline de CI/CD.
3. **\*\*Facilitar a Manutenção:\*\*** Reduzir a curva de aprendizado para novos membros da equipe e permitir que qualquer desenvolvedor possa trabalhar em qualquer parte do sistema com previsibilidade.
4. **\*\*Reforçar a Arquitetura:\*\*** Garantir que as decisões arquiteturais (DDD, Hexagonal, CQRS, Event Sourcing Ready) sejam refletidas no código de forma consistente.

### ### 1.2 Escopo

Este documento abrange:

- **\*\*Estrutura de Repositórios:\*\*** Organização de diretórios e pacotes.
- **\*\*Convenções de Código:\*\*** Nomenclatura de classes, métodos, variáveis, etc.
- **\*\*Padrões de Linguagem (Python):\*\*** Tipagem, async, Pydantic, etc.
- **\*\*APIs e Comunicação:\*\*** FastAPI, OpenAPI, versionamento, erros.
- **\*\*Persistência:\*\*** SQLAlchemy, Repository Pattern, Unit of Work.
- **\*\*Eventos e Mensageria:\*\*** Schemas, versionamento, idempotência.
- **\*\*Testes e Qualidade:\*\*** Frameworks, cobertura, análise estática.
- **\*\*Segurança e Observabilidade:\*\*** Integração com Vault, OpenTelemetry, logs.
- **\*\*CI/CD e Git:\*\*** Pipelines, revisões, releases.

### ### 1.3 Não Escopo

- **\*\*Arquitetura de Alto Nível:\*\*** Bounded Contexts, Commands, Eventos – já definidos em M0.x.
- **\*\*Roadmap e Plano de Sprints:\*\*** Já cobertos pelo M0.9.
- **\*\*Infraestrutura Detalhada:\*\*** Kubernetes, redes, etc. (capa de SRE).

### ### 1.4 Referências e Documentos Relacionados

- **\*\*M0.1:\*\*** Visão e Paradigma
- **\*\*M0.2:\*\*** Bounded Contexts
- **\*\*M0.3:\*\*** Event Storming
- **\*\*M0.4:\*\*** Modelo ER
- **\*\*M0.5:\*\*** Versionamento
- **\*\*M0.6:\*\*** Contratos Canônicos

- **M0.7:** Modelo Canônico de Commands
- **M0.8:** Modelo Operacional e Capacidades
- **M0.9:** Engineering Playbook e Plano Mestre de Desenvolvimento

---

## ## 2. Filosofia da Engenharia

Os princípios a seguir são fundamentais e orientam todas as decisões de implementação.

| Princípio | Definição | Aplicação no GovSec Shield |

| :--- | :--- | :--- |

| **Clean Code** | Código legível, com nomes significativos, funções pequenas e baixa complexidade. | Funções/métodos com no máximo 20 linhas; nomes expressivos em linguagem ubíqua. |

| **Clean Architecture** | Separação em camadas concêntricas: Domínio, Aplicação, Infraestrutura, Interface. | O domínio não conhece infraestrutura; dependências apontam para dentro. |

| **DDD (Domain-Driven Design)** | O código reflete o modelo de domínio e a linguagem ubíqua. | Entidades, Agregados, Value Objects, Repositórios, Serviços de Domínio e Eventos de Domínio são primeira classe. |

| **SOLID** | Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. | Cada classe tem uma única razão para mudar; dependemos de abstrações. |

| **KISS (Keep It Simple, Stupid)** | Prefira soluções simples a soluções complexas. | Evitar abstrações prematuras; começar simples e refatorar quando necessário. |

| **DRY (Don't Repeat Yourself)** | Evite duplicação de lógica e conhecimento. | Extrair lógica comum para funções/módulos compartilhados (core/shared). |

| **YAGNI (You Aren't Gonna Need It)** | Não implemente funcionalidades que não são necessárias hoje. | Nada de "over-engineering"; apenas o necessário para atender aos critérios de aceite. |

| **Composition over Inheritance** | Prefira composição a herança para reutilização. | Usar interfaces e injeção de dependências; evitar hierarquias profundas de herança. |

| **Explicit is Better than Implicit** (Zen of Python) | O código deve ser claro e explícito. | Evitar magia (ex: metaclasses) sem necessidade; favorecer legibilidade. |

| **Fail Fast** | Detectar erros o mais cedo possível. | Validações em entrada de dados; asserts em pré-condições; exceções específicas. |

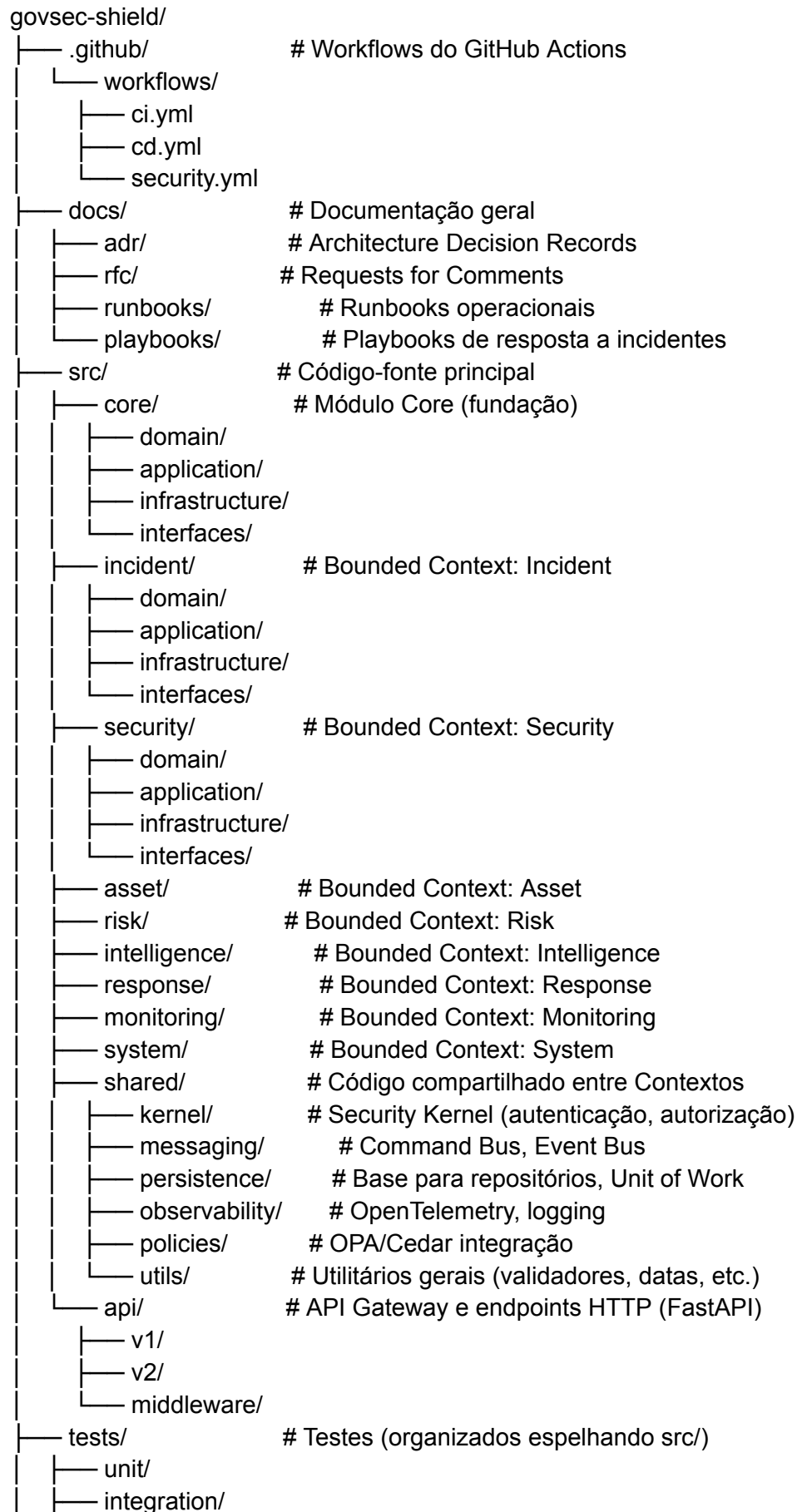
| **Security by Default** | Segurança não é um add-on; é uma propriedade inerente. | Todas as APIs exigem autenticação, todos os inputs são sanitizados, todos os dados sensíveis são criptografados. |

---

## ## 3. Estrutura Oficial do Repositório

O GovSec Shield adota um **monorepo** para facilitar a coordenação entre Contextos. A estrutura de diretórios é padronizada conforme abaixo.

...



```

| | | contract/
| | | performance/
| | |
| | | scripts/ # Scripts auxiliares (migrações, seed, etc.)
| | | configs/ # Configurações (YAML, JSON)
| | | | |
| | | | | dev/
| | | | | staging/
| | | | | prod/
| | | | |
| | | | | docker/ # Dockerfiles e docker-compose
| | | | | | |
| | | | | | | compose/
| | | | | | | services/
| | | | | | | Dockerfile.*
| | | | | | |
| | | | | | | deploy/ # Manifests Kubernetes / Helm
| | | | | | | | |
| | | | | | | | | base/
| | | | | | | | | overlays/
| | | | | | | | | helm/
| | | | | | | | |
| | | | | | | | | tools/ # Ferramentas de desenvolvimento (linters, hooks)
| | | | | | | | | pyproject.toml # Configuração do Poetry (dependências, build)
| | | | | | | | |
| | | | | | | | | ...

```

### ### 3.1 Responsabilidade de Cada Diretório

| Diretório | Responsabilidade |

| :--- | :--- |

| `src/<context>/domain/` | Entidades, Value Objects, Agregados, Repositórios (interfaces), Serviços de Domínio, Eventos de Domínio, Políticas. **\*\*Não depende de nada externo.\*\*** |

| `src/<context>/application/` | Use Cases, Command Handlers, Query Handlers, DTOs, Mappers, Serviços de Aplicação. **\*\*Depende apenas do domínio.\*\*** |

| `src/<context>/infrastructure/` | Implementações concretas de repositórios, clientes HTTP/grpc, adaptadores de mensageria, persistência (SQLAlchemy models). **\*\*Depende de application e domain.\*\*** |

| `src/<context>/interfaces/` | Adaptadores de entrada (API REST, gRPC, CLI, consumidores de eventos). **\*\*Depende de application.\*\*** |

| `src/shared/` | Código reutilizável por múltiplos contextos. Não deve conter lógica de domínio específica. |

| `src/api/` | Roteamento FastAPI, middlewares, handlers de exceção, OpenAPI. |

---

## ## 4. Convenções de Nomenclatura

### ### 4.1 Classes, Interfaces e Tipos

| Tipo | Convenção | Exemplo |

| :--- | :--- | :--- |

| **\*\*Classe\*\*** | PascalCase (substantivo) | `Incident`, `BlockIPCommandHandler` |

| **\*\*Interface (Protocol)\*\*** | PascalCase + sufixo `Protocol` (opcional) |

| `IncidentRepositoryProtocol` |

| **\*\*Enum\*\*** | PascalCase, valores em UPPER\_CASE | `class IncidentSeverity(Enum):  
CRITICAL = "critical" |

| **\*\*DTO\*\*** | PascalCase + sufixo `DTO` | `CreateIncidentDTO` |

| **\*\*Value Object\*\*** | PascalCase (imutável) | `EmailAddress`, `IPAddress` |  
| **\*\*Aggregate\*\*** | PascalCase (raiz) | `IncidentAggregate` |  
| **\*\*Repository\*\*** | Interface: PascalCase + `Repository`; Implementação: + `Impl` |  
`IncidentRepository`, `PostgresIncidentRepository` |  
| **\*\*Factory\*\*** | PascalCase + `Factory` | `IncidentFactory` |  
| **\*\*Builder\*\*** | PascalCase + `Builder` | `IncidentBuilder` |  
| **\*\*Service (Domain)\*\*** | PascalCase + `Service` | `RiskAssessmentService` |  
| **\*\*Command\*\*** | PascalCase + `Command` (conforme M0.7) | `BlockIPCommand` |  
| **\*\*Command Handler\*\*** | PascalCase + `CommandHandler` | `BlockIPCommandHandler` |  
| **\*\*Query\*\*** | PascalCase + `Query` | `ListIncidentsQuery` |  
| **\*\*Event\*\*** | PascalCase + `Event` (conforme M0.6) | `IncidentCreatedEvent` |  
| **\*\*Policy\*\*** | PascalCase + `Policy` | `CompliancePolicy` |  
| **\*\*Validator\*\*** | PascalCase + `Validator` | `IPAddressValidator` |  
| **\*\*Exception\*\*** | PascalCase + `Error` | `IncidentNotFoundError`, `ValidationError` |

#### ### 4.2 Módulos e Pacotes

- Nomes em **\*\*snake\_case\*\***, sempre no singular.
- Ex: `incident`, `security`, `asset`, `risk`, `shared.kernel`, `shared.messaging`.

#### ### 4.3 Arquivos

- Nomes em **\*\*snake\_case\*\***.
- Ex: `block\_ip\_command.py`, `incident\_repository.py`, `models.py`.

#### ### 4.4 Funções e Métodos

- **\*\*snake\_case\*\***, verbos que indicam ação.
- `get\_incident\_by\_id`, `block\_ip`, `create\_tenant`, `handle\_event`.
- Métodos de validação: `validate\_\*`.
- Métodos de fábrica: `create\_\*`, `from\_\*`.
- Métodos de transformação: `to\_dto`, `to\_entity`.

#### ### 4.5 Variáveis e Constantes

- Variáveis: **\*\*snake\_case\*\*** descritiva.
- Constantes: **\*\*UPPER\_SNAKE\_CASE\*\*** em nível de módulo.
- Nomes de variáveis de uma letra são proibidos (exceto para índices em loops simples).

#### ### 4.6 Ambientes, Secrets e Configurações

- Variáveis de ambiente: **\*\*UPPER\_SNAKE\_CASE\*\***, prefixo `GOVSEC\_`.
- Ex: `GOVSEC\_DB\_URL`, `GOVSEC\_KAFKA\_BOOTSTRAP`.
- Secrets (Vault): caminho hierárquico `/secret/govsec/<environment>/<service>/<key>`.

---

## ## 5. Padrões Python

O GovSec Shield utiliza **\*\*Python 3.12+\*\*** como linguagem primária para serviços de backend.

### ### 5.1 Type Hints Obrigatórios



- Todo parâmetro e retorno de função/método **\*\*deve\*\*** ser anotado com type hints.
- Uso de `typing` (List, Dict, Optional, Union, etc.) ou tipos nativos (list, dict, etc. – Python 3.9+).
- **\*\*Modo estrito:\*\*** `mypy --strict` é executado no pipeline.

```
```python
def get_incident(incident_id: UUID, tenant: str) -> Incident | None:
    ...
```
```

### ### 5.2 Async First

- Operações I/O-bound (HTTP, banco de dados, mensageria) devem ser assíncronas (`async/await`).
- Uso de `httpx.AsyncClient`, `asyncpg`, `aiokafka`, etc.
- **\*\*Proibido\*\*** uso de `time.sleep()`; usar `asyncio.sleep()`.
- Context managers assíncronos (`async with`) para recursos.

### ### 5.3 Pydantic (>=2.0)

- **\*\*Todos os DTOs, Schemas de API, Schemas de Eventos e Validation Models\*\*** devem usar Pydantic.
- `model\_validator` para validações cross-campo.
- `field\_validator` para validações individuais.
- Configuração `extra = "forbid"` para evitar campos inesperados.
- Uso de `BaseModel` para serialização/desserialização.

```
```python
from pydantic import BaseModel, Field, field_validator
```

```
class BlockIPCommandDTO(BaseModel):
    ip: str = Field(..., pattern=r"^\d{1,3}\.\d{3}\d{1,3}$")
    cidr: int = Field(..., ge=0, le=32)
    tenant: str

    @field_validator("ip")
    def validate_ip(cls, v):
        # validação adicional
        return v
```
```

### ### 5.4 Dataclasses vs Pydantic

- **\*\*Pydantic\*\*** para entrada/saída de APIs e eventos (validação, serialização).
- **\*\*Dataclasses\*\*** para objetos internos simples que não exigem validação.
- **\*\*Preferência:\*\*** Pydantic por consistência.

### ### 5.5 Enums

- Usar `enum.Enum` ou `enum.StrEnum` (Python 3.11+).
- Valores em strings para facilitar serialização JSON.

```

python
class IncidentStatus(StrEnum):
 OPEN = "open"
 IN_PROGRESS = "in_progress"
 RESOLVED = "resolved"
 CLOSED = "closed"

```

### ### 5.6 Abstract Base Classes (ABC) e Protocols

- **ABC** para hierarquias claras de herança (ex: repositórios).
- **Protocol** (Python 3.8+) para duck typing estrutural (ex: interfaces de serviços).

```

python
from abc import ABC, abstractmethod

class IncidentRepository(ABC):
 @abstractmethod
 async def save(self, incident: Incident) -> None:
 ...

```

### ### 5.7 Generics

- Usar `typing.Generic` e `TypeVar` para repositórios, serviços e utilitários reutilizáveis.

```

python
T = TypeVar("T", bound=Entity)

class GenericRepository(ABC, Generic[T]):
 @abstractmethod
 async def get(self, id: UUID) -> T | None:
 ...

```

### ### 5.8 Context Managers

- Usar `contextlib` para gerenciamento de recursos (conexões de banco, sessões HTTP).
- `async with` para gerenciadores assíncronos.

### ### 5.9 Dependency Injection

- **Injeção manual** via construtor (preferida) ou via frameworks (ex: FastAPI DI).
- Evitar injeção de dependência mágica; favorecer transparência.

```

python
class CreateIncidentHandler:
 def __init__(self, repo: IncidentRepository, event_bus: EventBus):
 self._repo = repo
 self._event_bus = event_bus
 ...

```

#### ### 5.10 Logging

- Usar `logging.getLogger(__name__)` em cada módulo.
- Logs estruturados (JSON) via `python-json-logger` ou similar.
- Níveis: DEBUG (desenvolvimento), INFO (ações normais), WARNING (recuperáveis), ERROR (falhas), CRITICAL (falhas graves).

#### ### 5.11 Tratamento de Erros

- Usar hierarquia de exceções (ver seção 7).
- Nunca usar `except:` sem especificar o tipo; usar `except Exception as e` com log.
- Lançar exceções específicas do domínio.

#### ### 5.12 Performance e Concorrência

- Evitar loops com I/O bloqueante dentro de loops; usar `asyncio.gather` para paralelismo.
- Usar semáforos (`asyncio.Semaphore`) para limitar concorrência em recursos.
- **Proibido** usar threads (GIL) para tarefas I/O; usar `async`.

#### ### 5.13 Thread Safety

- Código `async` é single-threaded por event loop; mas ao usar threads (ex: CPU-bound), usar `concurrent.futures` com cuidado.
- Evitar estado compartilhado mutável entre tarefas.

#### ### 5.14 AsyncIO

- **Event loop** gerenciado pela aplicação (FastAPI já gerencia).
- Usar `asyncio.create_task` para fire-and-forget ou tarefas em background.
- Em testes, usar `pytest-asyncio`.

---

### ## 6. Estrutura das APIs

#### ### 6.1 FastAPI

- Framework oficial para APIs REST.
- Documentação OpenAPI automaticamente gerada em `/docs`.
- Versionamento via prefixo na URL: `/api/v1/...`, `/api/v2/...`.

#### ### 6.2 Padrões REST

| Método | Recurso         | Descrição                             |
|--------|-----------------|---------------------------------------|
| GET    | /incidents      | Listar incidentes (com paginação)     |
| GET    | /incidents/{id} | Obter detalhes de um incidente        |
| POST   | /incidents      | Criar um incidente                    |
| PUT    | /incidents/{id} | Atualizar um incidente (substituição) |
| PATCH  | /incidents/{id} | Atualização parcial                   |
| DELETE | /incidents/{id} | Excluir um incidente                  |

#### ### 6.3 Paginação

- **Offset/Limit** (para coleções grandes, usar cursor-based para performance).
- Parâmetros: `skip` (offset) e `limit` (padrão 20, máximo 100).

- Resposta inclui metadados: `total`, `skip`, `limit`, `next`.

#### ### 6.4 Filtros e Sorting

- Filtros via query parameters: `?status=open&severity=critical`.
- Sorting: `?sort\_by=created\_at&order=desc`.
- Múltiplos campos: `sort\_by=created\_at,-updated\_at` (hífen para desc).

#### ### 6.5 Headers Obrigatórios

| Header             | Descrição                                              |
|--------------------|--------------------------------------------------------|
| :---               | :---                                                   |
| `X-Correlation-ID` | ID de correlação para rastreamento end-to-end          |
| `X-Tenant-ID`      | ID do tenant (obrigatório para todos os endpoints)     |
| `X-User-ID`        | ID do usuário autenticado (geralmente extraído do JWT) |
| `Trace-ID`         | (Opcional) ID de trace para integração OpenTelemetry   |

#### ### 6.6 Status Codes

| Code | Descrição                                               |
|------|---------------------------------------------------------|
| :--- | :---                                                    |
| 200  | OK (sucesso)                                            |
| 201  | Created (recurso criado)                                |
| 204  | No Content (deleção bem-sucedida)                       |
| 400  | Bad Request (erro de validação)                         |
| 401  | Unauthorized (não autenticado)                          |
| 403  | Forbidden (não autorizado)                              |
| 404  | Not Found (recurso não encontrado)                      |
| 409  | Conflict (conflito de estado, ex: versão desatualizada) |
| 422  | Unprocessable Entity (erro de validação semântica)      |
| 429  | Too Many Requests (rate limiting)                       |
| 500  | Internal Server Error (erro inesperado)                 |

#### ### 6.7 Error Responses (RFC 7807)

Todos os erros devem seguir o formato RFC 7807 (Problem Details for HTTP APIs).

```
```json
{
  "type": "https://govsecshield.io/errors/validation",
  "title": "Validation Error",
  "status": 422,
  "detail": "O campo 'ip' não é um endereço IPv4 válido.",
  "instance": "/api/v1/incidents",
  "correlation_id": "abc-123"
}
```
```

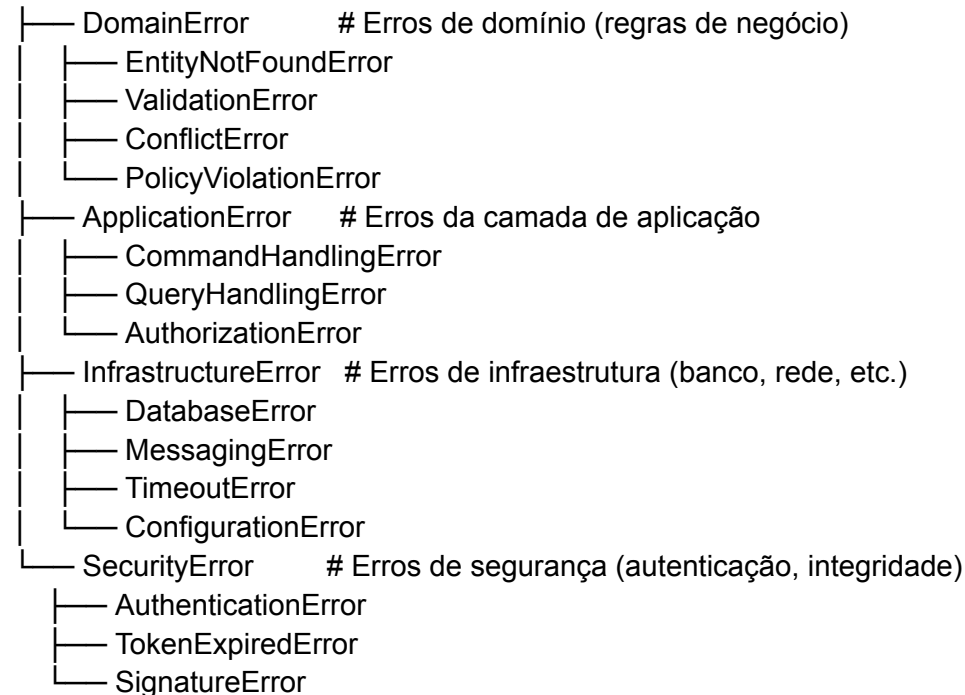
---

### ## 7. Tratamento de Erros

### ### 7.1 Hierarquia de Exceções

...

BaseGovSecError (abc.ABC)



...

### ### 7.2 Domain Exceptions

- Lançadas quando uma regra de negócio é violada.
- Devem incluir detalhes suficientes para o cliente entender o problema.

```
```python
class ValidationError(DomainError):
    def __init__(self, field: str, reason: str):
        super().__init__(f"Validation failed on {field}: {reason}")
        self.field = field
        self.reason = reason
...
```
```

### ### 7.3 Timeouts

- Usar `asyncio.timeout` para operações com timeout.
- Timeout padrão: 5 segundos para operações curtas, 30 segundos para operações pesadas.
- Timeout gera `TimeoutError` que deve ser tratado e convertido em resposta HTTP 504 ou 408.

### ### 7.4 Retries

- Para operações com falhas transitórias (rede, banco), implementar retry com backoff exponencial.
- Usar bibliotecas como `tenacity` ou `backoff` com decoração.

```

python
import tenacity

@tenacity.retry(
 stop=tenacity.stop_after_attempt(3),
 wait=tenacity.wait_exponential(multiplier=1, min=1, max=10)
)
async def call_external_service():
 ...

```

### ### 7.5 Circuit Breaker

- Para chamadas a serviços externos, usar padrão Circuit Breaker (ex: `aiohttp` + `circuit-breaker`).
- Estados: CLOSED (permite chamadas) → OPEN (bloqueia) → HALF-OPEN (testa).
- Configuração típica: falha 5 vezes em 60 segundos abre o circuito.

### ### 7.6 Fallback

- Fornecer fallback para operações críticas (ex: cache local, dados padrão).
- Registrar no log e alertar via métrica.

### ### 7.7 Dead Letter Queue (DLQ)

- Commands/Eventos que falham repetidamente vão para DLQ (conforme M0.7).
- A DLQ deve ser monitorada e auditada.

---

## ## 8. Logging

### ### 8.1 Estrutura dos Logs

Logs devem ser emitidos em formato **JSON estruturado** com os campos mínimos obrigatórios:

| Campo            | Tipo          | Descrição                                       |
|------------------|---------------|-------------------------------------------------|
| `timestamp`      | ISO-8601      | Data/hora em UTC                                |
| `level`          | string        | `INFO`, `WARNING`, `ERROR`, etc.                |
| `logger`         | string        | Nome do logger (ex: `src.incident.handlers`)    |
| `message`        | string        | Mensagem legível                                |
| `correlation_id` | string (UUID) | ID de correlação (extraído do header ou gerado) |
| `tenant`         | string        | ID do tenant                                    |
| `user_id`        | string        | ID do usuário (se autenticado)                  |
| `command`        | string        | Nome do Command (se aplicável)                  |
| `event`          | string        | Nome do Event (se aplicável)                    |
| `asset`          | string        | ID do asset (se aplicável)                      |
| `incident`       | string        | ID do incidente (se aplicável)                  |
| `duration_ms`    | float         | Duração da operação (se relevante)              |
| `error`          | object        | Detalhes do erro (se houver)                    |

| `extra` | object | Campos adicionais (contexto) |

### ### 8.2 Mascaramento de Dados (LGPD/GDPR)

- Dados sensíveis (PII) como emails, IPs (em alguns contextos), senhas, tokens **devem** ser mascarados nos logs.
- Uso de filtros de logging que substituem valores sensíveis por `[REDACTED]`.
- Política: **Nunca** logar senhas, tokens de autenticação, ou números de documentos.

```
```python
# Exemplo de filtro
class SensitiveDataFilter(logging.Filter):
    def filter(self, record):
        # substitui padrões de email, etc.
        return True
```
```

### ### 8.3 Trace e Span

- Integrar com OpenTelemetry: cada log deve incluir `trace\_id` e `span\_id` para correlação.

---

## ## 9. Observabilidade

### ### 9.1 OpenTelemetry

- **Obrigatório** para todos os serviços.
- Exportar traces para Jaeger/Tempo, métricas para Prometheus, logs para Loki.

### ### 9.2 Metrics (Prometheus)

| Métrica                       | Tipo      | Descrição                                             |
|-------------------------------|-----------|-------------------------------------------------------|
| http_requests_total           | Counter   | Total de requisições HTTP por rota, método, status    |
| http_request_duration_seconds | Histogram | Latência por rota                                     |
| commands_processed_total      | Counter   | Commands processados por tipo, status (success/error) |
| events_emitted_total          | Counter   | Eventos emitidos por tipo                             |
| queue_lag                     | Gauge     | Lag do Kafka por tópico/partição                      |
| db_connection_pool_size       | Gauge     | Tamanho do pool de conexões                           |
| error_rate                    | Gauge     | Taxa de erro para serviços críticos                   |

### ### 9.3 Tracing

- **100% de amostragem** para Commands e eventos críticos.
- Uso de propagação W3C Trace Context.
- Anotar spans com atributos relevantes: `command.name`, `tenant`, `correlation\_id`, `user\_id`.

### ### 9.4 Health Checks

- **Liveness:** `/healthz` – retorna 200 se o serviço está vivo (sem verificar dependências).

- **Readiness:** `/ready` – retorna 200 se o serviço está pronto para receber tráfego (verifica banco, Kafka, etc.).
- **Startup:** `/startup` – (opcional) para verificações de inicialização.

#### ### 9.5 Dashboards e Alertas

- Dashboards padrão no Grafana para Golden Signals de cada serviço.
- Alertas configurados para:
  - Erros 5xx > 1% em 5 min.
  - Latência p95 > SLO.
  - Lag do Kafka > 1000 mensagens.
  - Serviço indisponível por > 1 min.

---

### ## 10. Segurança

#### ### 10.1 Secrets Management

- **Nunca** armazenar secrets em código ou variáveis de ambiente planas.
- Usar **HashiCorp Vault** para armazenar e rotacionar secrets.
- O aplicativo obtém secrets via API do Vault ou sidecar injector.
- **Rotação automática:** suportada para credenciais de banco, chaves de API, JWT signing keys.

#### ### 10.2 JWT e OAuth2

- Autenticação via **JWT** assinado com RS256 ou ES256.
- O token contém: ``user_id``, ``tenant``, ``roles``, ``exp``, ``iat``.
- Validação: verificar assinatura, expiração, e claims mínimas.
- Suporte a **OAuth2** para integração com providers externos (Google, Microsoft, etc.).

#### ### 10.3 RBAC (Role-Based Access Control)

- Cada endpoint/Command deve ter uma permissão associada.
- O Security Kernel verifica se o usuário possui a role necessária.
- Estrutura de roles conforme M0.8: ``viewer``, ``analyst``, ``engineer``, ``security_admin``, ``system_admin``.

#### ### 10.4 MFA (Multi-Factor Authentication)

- Exigido para ações críticas: criação de tenant, promoção de usuário, alteração de políticas.
- Implementação via TOTP (Google Authenticator) ou WebAuthn.

#### ### 10.5 TLS e mTLS

- **TLS 1.3** obrigatório para todas as conexões externas.
- **mTLS** para comunicação entre serviços (East-West), com certificados gerenciados pelo Vault ou Istio.

#### ### 10.6 Rate Limiting

- Por endpoint, por tenant, por usuário.
- Limites: 100 req/min para usuários padrão, 1000 req/min para administradores.
- Headers: ``X-RateLimit-Limit``, ``X-RateLimit-Remaining``, ``X-RateLimit-Reset``.



### ### 10.7 Headers de Segurança (OWASP)

| Header                      | Valor                                 | Descrição                         |
|-----------------------------|---------------------------------------|-----------------------------------|
| :---                        | :---                                  | :---                              |
| `Strict-Transport-Security` | `max-age=31536000; includeSubDomains` | Força HTTPS                       |
| `X-Content-Type-Options`    | `nosniff`                             | Previne MIME sniffing             |
| `X-Frame-Options`           | `DENY`                                | Previne clickjacking              |
| `Content-Security-Policy`   | `default-src 'self'`                  | Política de segurança de conteúdo |
| `X-XSS-Protection`          | `1; mode=block`                       | (Compatibilidade)                 |
| `Referrer-Policy`           | `strict-origin-when-cross-origin`     | Política de referrer              |

### ### 10.8 Prevenção de Vulnerabilidades

| Tipo                     | Medida                                                                                  |
|--------------------------|-----------------------------------------------------------------------------------------|
| :---                     | :---                                                                                    |
| <b>SQL Injection</b>     | Uso de SQLAlchemy ORM com parâmetros parametrizados; nunca concatenar strings.          |
| <b>XSS</b>               | Escapar saídas em templates (FastAPI já trata JSON, mas UI deve usar escape).           |
| <b>SSRF</b>              | Validar e restringir URLs para chamadas externas; usar lista de permissões.             |
| <b>XXE</b>               | Desabilitar processamento de entidades externas em parsers XML.                         |
| <b>Command Injection</b> | Nunca usar `os.system` ou `subprocess` com entradas não sanitizadas; usar APIs seguras. |

---

## ## 11. Persistência

### ### 11.1 SQLAlchemy (ORM)

- **Ferramenta oficial:** SQLAlchemy 2.0+ (modo assíncrono com `asyncpg`).
- **Models:** Definidos separadamente das entidades de domínio (camada de infraestrutura).
- **Mapping:** Uso de `Mapped` e `mapped\_column` para type hints.

```
```python
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, UUID

class IncidentModel(Base):
    __tablename__ = "incidents"
    id: Mapped[UUID] = mapped_column(UUID(as_uuid=True), primary_key=True)
    title: Mapped[str] = mapped_column(String(255))
    ...
```
```

### ### 11.2 Alembic (Migrações)

- Todas as mudanças de schema devem ser gerenciadas via migrações Alembic.
- As migrações são versionadas no repositório (`src/<context>/infrastructure/migrations`).
- **Política:** Nunca alterar uma migração já mergeada; criar uma nova.

### ### 11.3 Repository Pattern

- Camada de domínio define a interface do repositório.
- Camada de infraestrutura implementa o repositório usando SQLAlchemy.

```
```python
# domain/repositories.py
class IncidentRepository(ABC):
    @abstractmethod
    async def save(self, incident: Incident) -> None:
        ...

# infrastructure/repositories.py
class PostgresIncidentRepository(IncidentRepository):
    async def save(self, incident: Incident) -> None:
        model = IncidentMapper.to_model(incident)
        self.session.add(model)
        await self.session.flush()
...```
```

11.4 Unit of Work (UoW)

- Padrão UoW para gerenciar transações e garantir consistência.
- UoW é injetado em Handlers e Services.
- Abstrai commit/rollback.

```
```python
class UnitOfWork(ABC):
 incident_repo: IncidentRepository
 async def commit(self):
 ...
 async def rollback(self):
 ...
...```
```

### ### 11.5 Transactions

- Operações que envolvem múltiplos repositórios devem ser feitas dentro de uma única transação (UoW).
- Usar `async with` para gerenciar transações.

```
```python
async with unit_of_work as uow:
    incident = uow.incident_repo.get(incident_id)
    incident.close()
    uow.incident_repo.save(incident)
    await uow.commit()
...```
```

11.6 Optimistic Locking

- Para prevenir atualizações concorrentes, usar **versionamento** (campo `version` inteiro).
- Ao salvar, verificar se a versão no banco é a mesma que a versão da entidade.

```
```sql
UPDATE incidents SET status = 'closed', version = version + 1
WHERE id = :id AND version = :version
```
```

11.7 Soft Delete

- Para dados que não podem ser removidos fisicamente (ex: incidentes), usar `deleted_at` (timestamp).
- Queries devem filtrar `deleted_at IS NULL` por padrão.

11.8 JSONB

- Usar `JSONB` (PostgreSQL) para dados semiestruturados (ex: metadados, contextos, extensões).
- Indexar campos do JSONB quando necessário (índices GIN).

11.9 Índices

- Cada tabela deve ter índices apropriados para queries comuns.
- Índices compostos para filtros combinados (ex: `(tenant, status, created_at)`).
- Revisão de índices com `EXPLAIN` para queries lentas.

12. Eventos

12.1 Modelagem

- Eventos seguem o formato canônico definido no M0.6.
- Cada evento possui: `event_id`, `event_type`, `version`, `tenant`, `correlation_id`, `timestamp`, `payload`.

12.2 Versionamento

- Eventos são versionados via SemVer.
- Novos campos opcionais: MINOR; campos obrigatórios novos ou remoções: MAJOR.
- Schema Registry (ex: Apicurio) pode ser utilizado para gerenciar versões.

12.3 Envelope e Serialização

- Envelope: contém metadados + payload serializado.
- Suporte a **JSON Schema** para validação e **Avro** para eficiência em alto volume.

```
```json
{
 "id": "uuid",
 "type": "IncidentCreatedEvent",
 "version": "1.0.0",
 "tenant": "gov-abc",
 "correlation_id": "corr-123",

```

```
"timestamp": "2026-07-28T10:30:00Z",
"payload": {
 "incident_id": "inc-123",
 "title": "..."
}
}
...
```

#### ### 12.4 Idempotência

- Eventos são idempotentes por natureza (fatos).
- Consumidores devem tratar duplicatas usando o `event\_id`.

#### ### 12.5 Partition Key

- Para Kafka, a **partition key** deve ser o `tenant` (ou `tenant + correlation\_id`) para garantir ordenação por tenant.

#### ### 12.6 Retry e Replay

- Consumidores com falha devem usar retry com backoff.
- Replay de eventos: possível para reprocessamento, mas com garantia de idempotência.

---

### ## 13. Testes

#### ### 13.1 Frameworks

Tipo	Framework
Unitários	`pytest`, `pytest-asyncio`
Integração	`pytest` + `testcontainers` (PostgreSQL, Kafka, etc.)
Contrato (Pact)	`pact-python`
Mutation	`mutmut` ou `pytest-mutation`
Performance	`k6`, `locust`
Chaos	`chaostoolkit`

#### ### 13.2 Cobertura Mínima

- **Unitários:** 80% de cobertura de linhas (branches: 70%).
- **Integração:** 100% dos fluxos críticos (ex: criação de incidente, correlação).
- **Contrato:** 100% dos contratos entre serviços (Pact).

#### ### 13.3 Fixtures e Mocks

- Usar `pytest.fixture` para objetos comuns.
- Mocks: `unittest.mock` ou `pytest-mock`; evitar mocks excessivos que tornem os testes frágeis.
- Preferir **fakes** (implementações leves) a mocks para repositórios.

#### ### 13.4 Testes de Integração

- Usar `testcontainers` para levantar PostgreSQL e Kafka em memória para testes.
- Testar a integração real entre Command Bus, Handler e banco.

#### ### 13.5 Testes de Contrato (Pact)

- **Consumer** define as expectativas da API/evento.
- **Provider** verifica se atende às expectativas.
- Executados no CI; falha bloqueia o merge.

#### ### 13.6 Mutation Testing

- Semanalmente, executar mutation testing para avaliar a qualidade dos testes.
- Objetivo: reduzir mutantes sobreviventes.

#### ### 13.7 Testes de Performance e Carga

- Executados no ambiente de staging com `k6`.
- Cenário: 10k requisições/segundo para APIs de ingestão.
- Métricas: latência p95, throughput, erros.

#### ### 13.8 Chaos Engineering

- A partir do M8, introduzir chaos tests: falhas de rede, picos de latência, falhas de pods.
- Ferramenta: `chaostoolkit` ou `Gremlin`.

---

### ## 14. Qualidade

#### ### 14.1 Ferramentas

| Ferramenta | Uso |

| :--- | :--- |

| **Black** | Formatação automática de código |

| **Ruff** | Linting rápido e substituição do Flake8 |

| **Mypy** | Verificação estática de tipos (modo estrito) |

| **Bandit** | Análise de segurança em código Python |

| **Safety** | Verificação de vulnerabilidades em dependências |

| **SonarQube** | Análise contínua de qualidade (code smells, duplicação, complexidade) |

#### ### 14.2 Complexidade Ciclomática

- **Máximo**: 10 por função/método.
- Acima disso, deve-se refatorar (extrair funções, simplificar lógica).

#### ### 14.3 Maintainability Index

- **Meta**: > 65 (A) para todos os módulos.
- Monitorado no SonarQube.

#### ### 14.4 Technical Debt Ratio

- **Meta**: < 5% do esforço total estimado.
- Débitos são categorizados e priorizados conforme M0.9.

---

### ## 15. Documentação

### ### 15.1 ADR (Architecture Decision Record)

- Documento em Markdown no formato:
  - **Title**
  - **Status** (Proposed, Accepted, Superseded)
  - **Context**
  - **Decision**
  - **Consequences**
- Armazenado em `docs/adr/` com numeração sequencial.

### ### 15.2 RFC (Request for Comments)

- Para propostas de mudanças maiores (novos contextos, mudanças de APIs).
- Inclui justificativa, impactos, alternativas.
- Comentários e revisões registrados antes da aprovação.

### ### 15.3 README

- Cada serviço/módulo deve ter README com:
  - Propósito
  - Dependências
  - Como executar localmente
  - Como testar
  - Variáveis de ambiente
  - Exemplo de uso

### ### 15.4 Architecture Docs

- Manter uma visão geral da arquitetura no `docs/architecture/` (diagramas C4, Mermaid).

### ### 15.5 Runbooks

- Procedimentos operacionais para SRE: como reiniciar, escalar, diagnosticar.

### ### 15.6 Playbooks

- Procedimentos de resposta a incidentes: passo a passo para contenção.

### ### 15.7 Threat Models

- Para cada serviço, um modelo de ameaças (STRIDE) registrado.

### ### 15.8 Diagramas C4

- **Contexto:** Visão geral do sistema.
- **Contêineres:** Microserviços, bancos, brokers.
- **Componentes:** Módulos internos.
- **Código:** Classes principais.

### ### 15.9 Mermaid

- Preferir Mermaid para diagramas em Markdown (rápido, versionado).

### ### 15.10 OpenAPI

- Gerado automaticamente pelo FastAPI, mas mantido em `src/api/openapi.yaml` para referência.

---

## ## 16. Git

### ### 16.1 Naming Branches

- **Feature:** `feature/GOV-<id>-short-description`
- **Bugfix:** `bugfix/GOV-<id>-description`
- **Hotfix:** `hotfix/GOV-<id>-description`
- **Release:** `release/vX.Y.Z`
- **Chore:** `chore/description`

### ### 16.2 Commit Convention

- Seguir **Conventional Commits** (já definido em M0.9).
- Exemplo: `feat(incident): add correlation rules endpoint`

### ### 16.3 Semantic Versioning

- Conforme M0.5: MAJOR.MINOR.PATCH.
- Releases com tags `vX.Y.Z`.

### ### 16.4 Pull Requests

- **Template obrigatório:** com checklist de DoR/DoD.
- **Revisores:** mínimo 2 aprovações (incluindo um Tech Lead).
- **Checks:** CI (build, test, lint, security) devem passar.

### ### 16.5 Merge Policy

- **Squash and Merge** para manter histórico linear.
- Branch de origem deve ser deletada após o merge.

### ### 16.6 Tags e Releases

- Tags criadas a partir da `main` após aprovação do release.
- Release notes geradas automaticamente a partir dos commits.

---

## ## 17. CI/CD

### ### 17.1 GitHub Actions

- Pipelines definidos em `.github/workflows/`.
- Workflows: `ci.yml`, `cd.yml`, `security.yml`, `nightly.yml`.

### ### 17.2 Stages

Stage	Descrição
:---	:---
<b>Lint</b>	Black, Ruff, MyPy
<b>Security</b>	Bandit, Safety, Trivy (para containers)
<b>Test</b>	Unit, Integration, Contract
<b>Build</b>	Compilação e criação de imagem Docker

**Deploy Staging**	Deploy via ArgoCD para staging
**Smoke Test**	Teste de sanidade
**Performance**	Teste de carga (se aplicável)
**Deploy Production**	Promoção manual (ou automática com aprovação)

#### ### 17.3 Security Gates

- **SAST**: CodeQL, Semgrep.
- **DAST**: OWASP ZAP (contra staging).
- **Dependency Scan**: Snyk.
- **Container Scan**: Trivy.

#### ### 17.4 Quality Gates

- Cobertura de testes > 80%.
- Sem code smells críticos no SonarQube.
- Complexidade ciclomática < 10.
- Maintainability Index > 65.

#### ### 17.5 Deploy e Rollback

- **Deploy**: ArgoCD sincroniza com o repositório Git (GitOps).
- **Rollback**: ArgoCD reverte para o commit anterior.
- **Feature Flags**: Uso de flags para lançamentos graduais.

---

### ## 18. Checklist Oficial

#### ### 18.1 Checklist para Novo Módulo

- [ ] Estrutura de diretórios criada (domain, application, infrastructure, interfaces)
- [ ] Módulo registrado no `pyproject.toml` (ou estrutura de imports)
- [ ] Dependências adicionais declaradas (se necessário)
- [ ] Testes de unidade para o domínio (cobertura mínima)
- [ ] Testes de integração para infraestrutura
- [ ] Documentação README criada
- [ ] ADR criado (se houver decisão arquitetural)
- [ ] OpenTelemetry instrumentado
- [ ] Rotas/APIs registradas no API Gateway

#### ### 18.2 Checklist para Nova API

- [ ] Rota versionada (`/api/v1/...`)
- [ ] OpenAPI documentado via FastAPI
- [ ] Validação com Pydantic
- [ ] Autenticação exigida (JWT)
- [ ] Autorização verificada (RBAC)
- [ ] Headers obrigatórios (Correlation-ID, Tenant-ID)
- [ ] Erros formatados segundo RFC 7807
- [ ] Testes de integração
- [ ] Testes de contrato (Pact)
- [ ] Métricas e tracing configurados



#### ### 18.3 Checklist para Novo Event

- [ ] Schema definido via Pydantic (ou Avro)
- [ ] Versão atribuída (conforme M0.6)
- [ ] Envelope padrão
- [ ] Partition Key definida (tenant)
- [ ] Teste de serialização/desserialização
- [ ] Teste de contrato (provider/consumer)

#### ### 18.4 Checklist para Novo Command

- [ ] Command DTO definido (Pydantic)
- [ ] Command Handler implementado
- [ ] Handler registrado no Command Bus
- [ ] Validação de domínio
- [ ] Eventos emitidos (se aplicável)
- [ ] Testes unitários (handler)
- [ ] Testes de integração (Command Bus + Handler)
- [ ] Idempotência garantida (via idempotency\_key ou design)
- [ ] Auditoria registrada

#### ### 18.5 Checklist para Novo Connector

- [ ] Conector implementado como serviço separado (ou módulo)
- [ ] Ingestão de dados normalizada para o formato canônico
- [ ] Tratamento de erros (retry, DLQ)
- [ ] Observabilidade (métricas, logs)
- [ ] Testes de integração com fonte externa (mock)
- [ ] Documentação de configuração

#### ### 18.6 Checklist para Novo Scanner (segurança)

- [ ] Scanner implementado (ex: SAST, DAST)
- [ ] Integração com pipeline CI/CD
- [ ] Relatórios gerados (JSON, SARIF)
- [ ] Notificações de vulnerabilidades
- [ ] Testes de validação do scanner

#### ### 18.7 Checklist para Nova Sprint

- [ ] Backlog refinado
- [ ] Estórias com DoR preenchida
- [ ] Capacidade do time alocada
- [ ] Dependências externas resolvidas
- [ ] Riscos identificados
- [ ] Objetivo da sprint claro

#### ### 18.8 Checklist para Nova Release

- [ ] Release branch criada
- [ ] Testes completos (performance, segurança)
- [ ] Release notes preparadas
- [ ] Breaking changes documentadas

- [ ] Aprovação do Architecture Board e CISO
- [ ] Tags e artefatos publicados
- [ ] Rollback planejado

---

## ## 19. Anti Patterns

Anti-Pattern	Descrição	Consequência
**\*\*Big Ball of Mud\*\***	Sem estrutura, misturando domínio com infraestrutura.	Violação da Clean Architecture; difícil de testar e evoluir.
**\*\*God Class\*\***	Uma classe que faz tudo.	Viola SRP; complexidade alta, difícil de manter.
**\*\*Spaghetti Code\*\***	Lógica emaranhada com muitos ifs aninhados.	Baixa legibilidade; alta complexidade ciclomática.
**\*\*Copy-Paste Programming\*\***	Duplicar código em vez de reutilizar.	Viola DRY; correções precisam ser feitas em múltiplos lugares.
**\*\*Magic Numbers/Strings\*\***	Valores literais sem constante.	Dificulta manutenção; risco de erros.
**\*\*Hardcoded Secrets\*\***	Senhas, chaves no código.	Violação grave de segurança.
**\*\*Deep Nesting\*\***	Muitos níveis de indentação (ex: > 4).	Dificulta leitura; sugere necessidade de refatoração.
**\*\*Dependency Hell\*\***	Dependências circulares entre módulos.	Quebra a arquitetura; dificulta testes.
**\*\*Premature Optimization\*\***	Otimizar sem necessidade ou sem medição.	Pode introduzir complexidade desnecessária.
**\*\*Testing the Implementation\*\***	Testar detalhes internos em vez de comportamento.	Testes frágeis; refatoração quebra testes.
**\*\*Ignoring Logs/Metrics\*\***	Não instrumentar código.	Falta de visibilidade; difícil diagnosticar problemas.
**\*\*Monster Pull Request\*\***	PR com muitas alterações.	Dificulta revisão; aumenta risco de bugs.
**\*\*YAGNI Violation\*\***	Implementar funcionalidades "por via das dúvidas".	Aumenta dívida técnica sem valor imediato.

---

## ## 20. Changelog

### ### 20.1 Versão 1.0.0 (2026-07-28)

- **\*\*Criação inicial do documento M1.0.\*\***
- Definição da estrutura de repositório.
- Convenções de nomenclatura para classes, módulos, arquivos.
- Padrões Python: type hints, async, Pydantic, injeção de dependências.
- Estrutura de APIs: FastAPI, versionamento, erros RFC 7807.
- Tratamento de erros hierárquico.
- Logging estruturado com campos obrigatórios.

- Observabilidade com OpenTelemetry.
- Segurança: JWT, OAuth2, RBAC, MFA, TLS, prevenção de vulnerabilidades.
- Persistência: SQLAlchemy, Alembic, Repository Pattern, Unit of Work.
- Eventos: versionamento, envelope, idempotência.
- Testes: frameworks, cobertura, fixtures, contract, mutation, performance.
- Qualidade: Black, Ruff, MyPy, Bandit, SonarQube.
- Documentação: ADR, RFC, README, runbooks, playbooks, diagramas.
- Git: branch naming, commit convention, PR process.
- CI/CD: pipelines, security gates, quality gates, deploy, rollback.
- Checklist oficial para módulos, APIs, eventos, commands, connectors, scanners, sprints e releases.
- Catálogo de anti-patterns.

### ### 20.2 Mudanças Futuras

| Versão | Data Prevista | Mudança |

| :--- | :--- | :--- |

| 1.1.0 | Q4 2026 | Adicionar padrões para migração para Python 3.13 (novos recursos). |

| 1.2.0 | Q1 2027 | Incluir padrões para gRPC (além de REST) e GraphQL. |

| 2.0.0 | Q2 2027 | Atualização para suportar novos requisitos de segurança (ex: PQC). |

---

**\*\*Fim do Documento M1.0\*\***

\*Este documento é normativo e de cumprimento obrigatório. A não observância de qualquer seção constitui violação dos padrões de engenharia do GovSec Shield, sujeita à rejeição de Pull Requests e à necessidade de correção antes da integração. Revisões periódicas serão realizadas para manter os padrões atualizados com as melhores práticas da indústria.\*

# GovSec Shield

## M2 — Master Technical Implementation Specification

**\*\*Versão:\*\* 1.0.0**

**\*\*Status:\*\* APROVADO PELO COMITÊ DE ARQUITETURA E ENGENHARIA**

**\*\*Data de Vigência:\*\* 28 de Julho de 2026**

**\*\*Classificação:\*\* INTERNO – ESTRATÉGICO**

**\*\*Nível de Controle:\*\* IMPLEMENTAÇÃO**

**\*\*Substitui:\*\* N/A**

**\*\*Impacto:\*\* TODAS AS EQUIPES DE DESENVOLVIMENTO, SRE, QA, SECURITY E PLATFORM ENGINEERING**

---

### ## Sumário Executivo

Este documento é o **\*\*Plano Mestre de Implementação Técnica\*\*** do GovSec Shield. Ele transforma os artefatos arquiteturais (M0.1–M1.0) em um guia operacional completo, detalhando a estrutura do monorepo, a construção de cada módulo, os fluxos técnicos, os padrões de código, a gestão de eventos, a observabilidade, a segurança, os testes, a

entrega contínua e o plano de execução. É o documento que orientará dezenas de engenheiros durante todo o ciclo de vida do produto, sem necessidade de decisões arquiteturais adicionais.

Este documento é **normativo e exaustivo**: cada seção contém especificações prontas para implementação, exemplos concretos, diagramas e checklists. Toda Pull Request, toda nova funcionalidade e toda mudança devem estar em conformidade com este guia.

---

## ## 1. Introdução

### ### 1.1 Objetivo

1. **Fornecer um guia único e completo** para a implementação de todos os componentes do GovSec Shield, desde a infraestrutura até a lógica de domínio.
2. **Unificar decisões técnicas** em padrões concretos de código, estrutura e processos, eliminando ambiguidades para a equipe de desenvolvimento.
3. **Servir como referência durante todo o ciclo de vida do produto**, permitindo onboarding rápido de novos membros e consistência ao longo do tempo.
4. **Garantir a rastreabilidade** entre os documentos arquiteturais (M0.x) e a implementação real, assegurando que cada decisão seja refletida no código.

### ### 1.2 Escopo

Este documento abrange:

- **Estrutura do monorepo** e organização de pacotes.
- **Padrões de código** (Python, FastAPI, SQLAlchemy, Pydantic, etc.) com exemplos.
- **Guia de módulos**: especificação detalhada de cada Bounded Context e módulo compartilhado.
- **Fluxos técnicos** (boot, correlação, automação, rollback, etc.) com diagramas Mermaid.
- **Gestão de eventos**: criação, versionamento, serialização, retry e DLQ.
- **APIs**: design, versionamento, validação, erros e segurança.
- **Persistência**: modelos, migrações, repositórios, Unit of Work, otimização.
- **Observabilidade**: logging, métricas, tracing, dashboards e alertas.
- **Segurança**: autenticação, autorização, gestão de secrets, criptografia, auditoria.
- **Testes**: estratégia, ferramentas, cobertura, e integração no pipeline.
- **DevSecOps**: pipelines, quality gates, security gates, deploy, rollback.
- **Plano mestre de implementação**: épicos, capacidades, sprints e dependências.

### ### 1.3 Não Escopo

- **Decisões arquiteturais de alto nível** – já estão definidas em M0.1–M0.9.
- **Especificação de infraestrutura física** (hardware, redes) – coberta por documentos de SRE.
- **Modelagem de ameaças** – realizada em documentos de segurança dedicados.
- **Estratégia de negócios** (precificação, marketing) – fora do escopo de engenharia.

### ### 1.4 Premissas

- A equipe de desenvolvimento possui conhecimento em Python 3.12+, FastAPI, SQLAlchemy, e conceitos de DDD/CQRS.

- O ambiente de desenvolvimento utiliza Docker e Poetry para gerenciamento de dependências.
- O pipeline de CI/CD é baseado em GitHub Actions e ArgoCD.
- O banco de dados é PostgreSQL 16+.
- O broker de eventos é Redpanda (compatível com Kafka API).
- A observabilidade utiliza OpenTelemetry, Prometheus, Grafana, Loki e Tempo.
- A segurança utiliza HashiCorp Vault para secrets, JWT para autenticação e OPA para políticas.

### ### 1.5 Glossário

Termo	Definição
---	---
<b>Command</b>	Instrução para mutação de estado, conforme M0.7.
<b>Event</b>	Fato ocorrido no passado, conforme M0.6.
<b>Handler</b>	Componente que processa um Command ou Query.
<b>Repository</b>	Abstração para acesso a dados.
<b>Unit of Work</b>	Padrão para gerenciar transações e consistência.
<b>Policy</b>	Regra de autorização ou negócio avaliada pelo Policy Engine.
<b>Bounded Context</b>	Limite lógico de um domínio, conforme M0.2.
<b>Aggregate</b>	Grupo de entidades com consistência transacional.
<b>Value Object</b>	Objeto imutável sem identidade própria.
<b>DTO</b>	Data Transfer Object, usado para comunicação entre camadas.
<b>OpenTelemetry</b>	Framework para observabilidade distribuída.
<b>mTLS</b>	TLS mútuo para autenticação entre serviços.

### ### 1.6 Referências e Documentos Relacionados

Documento	Descrição
---	---
M0.1	Visão e Paradigma
M0.2	Bounded Contexts
M0.3	Event Storming
M0.4	Modelo ER
M0.5	Versionamento
M0.6	Contratos Canônicos de Eventos
M0.7	Modelo Canônico de Commands
M0.8	Modelo Operacional e Capacidades
M0.9	Engineering Playbook e Plano Mestre de Desenvolvimento
M1.0	Technical Standards & Engineering Standards

### ### 1.7 Convenções

- **Formato de dados:** JSON para APIs, Avro para eventos de alto throughput (opcional).
- **Codificação:** UTF-8.
- **Fuso horário:** UTC para todos os timestamps.
- **IDs:** UUID v4 (ou v7) para entidades, events e commands.

---

## ## 2. Filosofia Oficial de Desenvolvimento

Os princípios a seguir são a base de toda decisão de implementação.

Princípio	Definição	Implicação na Implementação
**\*\*Architecture First\*\***	A arquitetura (DDD, Hexagonal, CQRS, EDA) guia o código.	Estrutura de pacotes reflete as camadas; dependências apontam para dentro.
**\*\*Domain First\*\***	O modelo de domínio é a principal preocupação.	Entidades, Value Objects, Aggregates, Services e Events são implementados antes da infraestrutura.
**\*\*Security by Default\*\***	Segurança é inerente, não um add-on.	Autenticação e autorização são aplicadas em todas as camadas; validação de entrada obrigatória.
**\*\*Everything as Code\*\***	Configuração, políticas, infraestrutura e documentação são versionadas.	Uso de GitOps, Terraform, e políticas OPA em repositórios.
**\*\*Automation First\*\***	Automatize testes, builds, deploys, e verificações de qualidade.	Pipelines de CI/CD executam todas as etapas sem intervenção manual.
**\*\*Observability by Default\*\***	Toda funcionalidade deve ser mensurável e rastreável.	Todos os serviços exportam metrics, logs e traces; dashboards são criados junto com o código.
**\*\*Zero Trust\*\***	Não confie em nenhuma fonte de input ou serviço.	mTLS, validação contínua de autorização, e auditoria de todas as ações.
**\*\*Defense in Depth\*\***	Múltiplas camadas de segurança.	Validação de entrada, autorização, criptografia, rate limiting, e monitoramento.
**\*\*DDD\*\***	O código reflete o modelo de domínio e a linguagem ubíqua.	Classes e métodos usam nomes do domínio; regras de negócio estão no domínio.
**\*\*CQRS\*\***	Separação estrita entre Commands e Queries.	Commands modificam estado, Queries leem; cada um tem seu próprio modelo e fluxo.
**\*\*EDA (Event-Driven Architecture)\*\***	Comunicação assíncrona baseada em eventos.	Eventos são publicados no Kafka/Redpanda; consumidores reagem a eventos.
**\*\*Hexagonal Architecture\*\***	Isolamento do núcleo de negócio das interfaces externas.	Portas (interfaces) e adaptadores (implementações) separados; domínio desconhece infraestrutura.
**\*\*Clean Architecture\*\***	Camadas concêntricas com dependências direcionadas para dentro.	Domínio → Application → Infrastructure → Interfaces.
**\*\*SOLID\*\***	Princípios de design de código.	Classes pequenas, coesas, com uma única responsabilidade; dependência de abstrações.

---

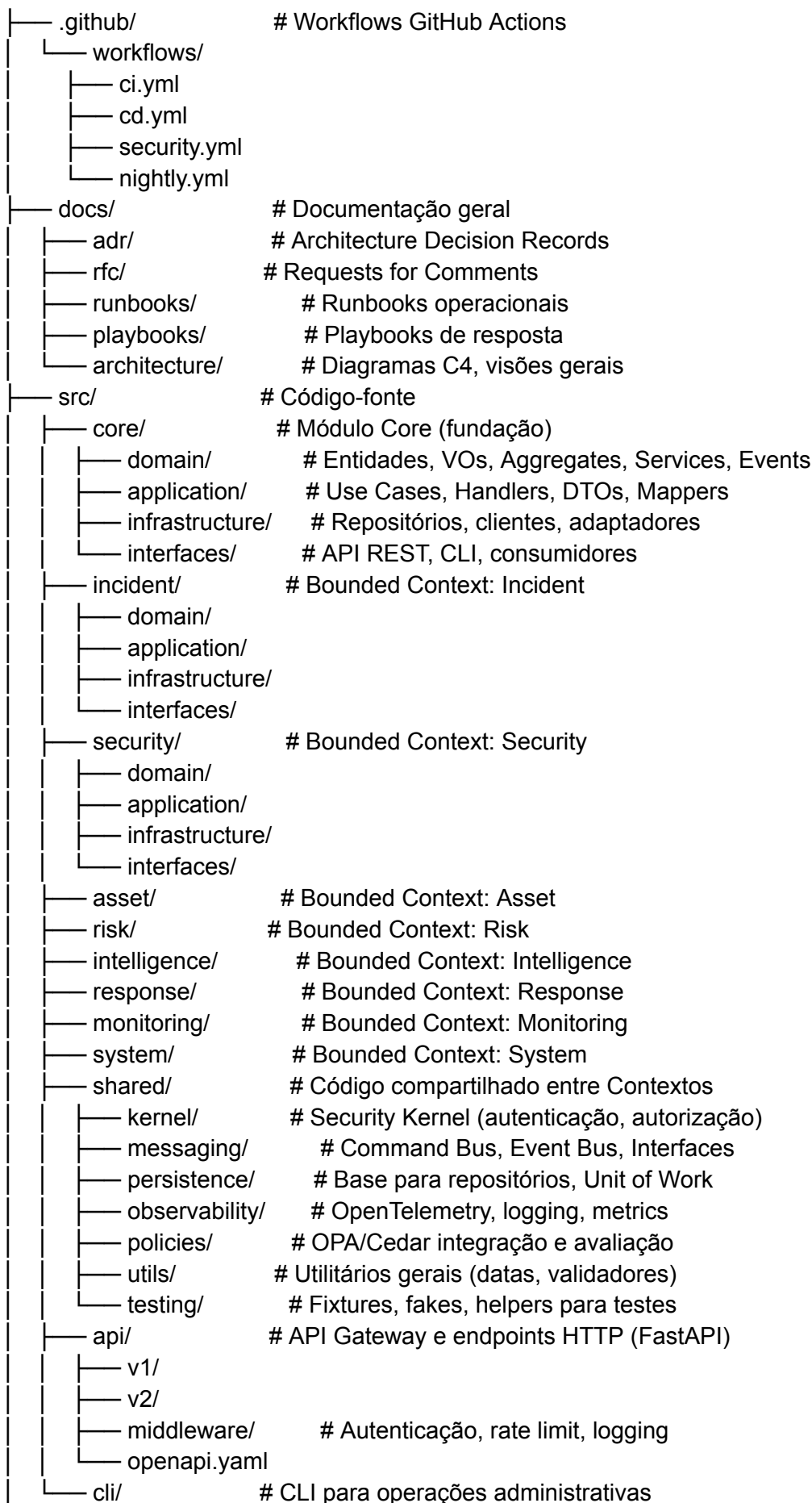
### ## 3. Organização Completa do Monorepo

O GovSec Shield adota um **\*\*monorepo\*\*** para facilitar a coordenação, versionamento integrado e reutilização de código compartilhado. A estrutura abaixo é obrigatória para todos os módulos.

#### ### 3.1 Árvore Completa

...

govsec-shield/



```

├── tests/ # Testes (organizados espelhando src/)
│ ├── unit/
│ ├── integration/
│ ├── contract/
│ └── performance/
├── scripts/ # Scripts auxiliares (migrações, seed, etc.)
│ ├── migrations/ # Alembic migrações
│ ├── seed/ # Dados iniciais
│ └── local-dev/ # Scripts para ambiente local
├── configs/ # Configurações (YAML, JSON, env)
│ ├── dev/
│ ├── staging/
│ └── prod/
├── docker/ # Dockerfiles e docker-compose
│ ├── compose/
│ ├── services/
│ └── Dockerfile.*
├── deploy/ # Manifests Kubernetes / Helm
│ ├── base/
│ ├── overlays/
│ └── helm/
├── tools/ # Ferramentas de desenvolvimento (hooks, linters)
├── .pre-commit-config.yaml # Pre-commit hooks
├── pyproject.toml # Poetry: dependências e configuração de build
├── poetry.lock # Lock de dependências
├── Makefile # Comandos de automação (build, test, run)
└── README.md # Instruções gerais
...

```

### ### 3.2 Responsabilidade de Cada Diretório

```

| Diretório | Responsabilidade | Dependências Permitidas |
| :--- | :--- | :--- |
| `src/<context>/domain/` | Entidades, Value Objects, Aggregates, Domain Services, Domain Events, Repository Interfaces. **Nenhuma dependência externa.** | Apenas módulos `shared/utills` para utilitários genéricos (ex: datetimes, validadores). |
| `src/<context>/application/` | Use Cases (Command/Query Handlers), DTOs, Mappers, Application Services. **Depende do domínio.** | `domain`, `shared/messaging`, `shared/persistence` (interfaces) |
| `src/<context>/infrastructure/` | Implementações concretas de repositórios, clientes HTTP/gRPC, adaptadores de mensageria (Kafka). | `domain`, `application`, `shared/*` |
| `src/<context>/interfaces/` | Adaptadores de entrada: REST controllers, CLI commands, event consumers. | `application`, `domain` (via DTOs) |
| `src/shared/` | Código reutilizável por múltiplos contextos. **Nunca deve conter lógica de domínio.** | Apenas bibliotecas externas padrão. |
| `src/api/` | Definição de rotas HTTP, middlewares, handlers de exceção, OpenAPI. |
| `shared/*`, `src/*/interfaces` |

```



| `tests/` | Todos os testes, organizados por tipo. | Espelhamento de `src/` para testes de unidade/integração. |

### ### 3.3 Regras de Importação

- **Domínio** → **não importa** infraestrutura ou aplicação.
- **Aplicação** → importa domínio, mas não infraestrutura (usa interfaces).
- **Infraestrutura** → importa domínio e aplicação.
- **Interfaces** → importa aplicação.
- **Cross-context** → importações entre Bounded Contexts **são proibidas**; comunicação via eventos e Command Bus.
- **Shared Kernel** → pode ser importado por qualquer módulo, mas com moderação.

### ### 3.4 Shared Kernel e Cross-Cutting Concerns

O diretório `shared/` contém:

- `kernel/`: Security Kernel (autenticação, RBAC, JWT).
- `messaging/`: Command Bus, Event Bus, interfaces para handlers.
- `persistence/`: Base para repositórios (CRUD genérico), Unit of Work.
- `observability/`: Configuração central de logging, métricas, tracing.
- `policies/`: Avaliador de políticas OPA/Cedar.
- `utils/`: Funções auxiliares (ex: `uuid\_generator`, `datetime\_utils`).

### ### 3.5 Bibliotecas Internas (Custom Packages)

Para código que é reutilizado em múltiplos contextos, crie um pacote interno dentro de `shared/`. Exemplo: `shared/validators/` com validação de IP, email, etc.

---

## ## 4. Guia Oficial para Desenvolvimento de Módulos

Cada novo módulo (Bounded Context) deve seguir o processo abaixo para garantir consistência e aderência à arquitetura.

### ### 4.1 Processo de Criação de um Novo Módulo

1. **Aprovação** pelo Architecture Board (AB) com base em análise de domínio.
2. **Criação da estrutura de diretórios** conforme seção 3.
3. **Definição dos contratos iniciais:**
  - Eventos que serão produzidos/consumidos.
  - Commands e Queries públicas.
4. **Implementação do domínio:**
  - Entidades, Value Objects, Aggregates.
  - Repository interfaces, Domain Services, Domain Events.
5. **Implementação da aplicação:**
  - Command/Query Handlers.
  - DTOs, Mappers.

6. **Implementação da infraestrutura**:
  - Repositórios concretos (SQLAlchemy, etc.).
  - Adaptadores de mensageria (Kafka consumers/producers).
7. **Implementação das interfaces**:
  - REST endpoints (FastAPI) ou CLI.
8. **Testes** (unitários, integração, contrato).
9. **Documentação** (README, ADR se necessário).
10. **Integração no pipeline CI/CD**.

#### ### 4.2 Registro no Core

- O módulo deve ser registrado no `core` para injeção de dependências e disponibilização de serviços.
- Utilizar injeção de dependência via construtor.

#### ### 4.3 Publicação e Consumo de Eventos

- Publicar eventos via `EventBus.publish(event)`.
- Consumir eventos via handlers anotados com `@event\_listener(event\_type)`.
- **Nunca** publicar eventos manualmente fora da camada de aplicação.

#### ### 4.4 Registro de Commands e Queries

- Commands: implementar `CommandHandler` e registrar no `CommandBus`.
- Queries: implementar `QueryHandler` e registrar no `QueryBus`.

#### ### 4.5 Templates Completos

##### #### Template para um Command Handler

```
```python
# src/<context>/application/handlers/block_ip_handler.py

from dataclasses import dataclass
from uuid import UUID
from src.shared.messaging.command_bus import CommandHandler, Command
from src.<context>.domain.entities import BlockedIP
from src.<context>.domain.repositories import BlockedIPRepository
from src.<context>.application.dtos import BlockIPDTO

@dataclass(frozen=True)
class BlockIPCommand(Command):
    command_id: UUID
    tenant: str
    correlation_id: str
    payload: BlockIPDTO

class BlockIPCommandHandler(CommandHandler[BlockIPCommand]):
```

```

def __init__(self, repo: BlockedIPRepository, event_bus: EventBus):
    self._repo = repo
    self._event_bus = event_bus

async def handle(self, command: BlockIPCommand) -> None:
    blocked_ip = BlockedIP(
        ip=command.payload.ip,
        cidr=command.payload.cidr,
        reason=command.payload.reason,
    )
    await self._repo.save(blocked_ip)
    event = IPBlockedEvent(
        event_id=uuid4(),
        tenant=command.tenant,
        correlation_id=command.correlation_id,
        payload={"ip": blocked_ip.ip, "cidr": blocked_ip.cidr}
    )
    await self._event_bus.publish(event)
...

```

Template para um Repository

```

```python
src/<context>/domain/repositories.py
from abc import ABC, abstractmethod
from uuid import UUID
from src.<context>.domain.entities import BlockedIP

class BlockedIPRepository(ABC):
 @abstractmethod
 async def save(self, blocked_ip: BlockedIP) -> None:
 ...

 @abstractmethod
 async def get_by_ip(self, ip: str, tenant: str) -> BlockedIP | None:
 ...

 @abstractmethod
 async def delete(self, ip: str, tenant: str) -> None:
 ...
...

```

##### Template para um Event Listener

```

```python
# src/<context>/interfaces/event_listeners.py
from src.shared.messaging.event_bus import EventListener
from src.<context>.domain.events import IPBlockedEvent

class IPBlockedListener(EventListener[IPBlockedEvent]):

```

```

def __init__(self, incident_service: IncidentService):
    self._incident_service = incident_service

    async def on_event(self, event: IPBlockedEvent) -> None:
        # Reage ao evento: atualiza incidente, etc.
        await self._incident_service.notify_block(event.payload.ip)
    ...

```

4.6 Checklist Obrigatório para Novo Módulo

- [] Estrutura de diretórios criada (domain, application, infrastructure, interfaces).
- [] Arquivo `\_\_init\_\_.py` em cada subpacote.
- [] Dependências no `pyproject.toml` atualizadas.
- [] Modelos de domínio (entidades, VOs) implementados.
- [] Interfaces de repositórios definidas.
- [] Handlers de Commands e Queries implementados.
- [] Eventos de domínio definidos.
- [] Adaptadores de infraestrutura (repositórios concretos, clientes) implementados.
- [] Rotas HTTP registradas no `src/api/`.
- [] Testes unitários com cobertura > 80%.
- [] Testes de integração para fluxos críticos.
- [] Documentação README.
- [] ADR (se aplicável) criado e aprovado.
- [] Integração com Security Kernel (autenticação/autorização) verificada.
- [] Observabilidade: logs, métricas, traces configurados.

5. Padrões Oficiais de Código

5.1 Python

- **Versão:** 3.12+.
- **Gerenciador de dependências:** Poetry.
- **Tipagem:** Obrigatória para todos os parâmetros e retornos (mypy em modo estrito).
- **Async:** Todas as operações I/O-bound devem ser assíncronas (`async/await`).

5.2 FastAPI

- **Versão:** 0.100+.
- **Documentação:** OpenAPI automaticamente gerada em `/docs`.
- **Validação:** Pydantic models.
- **Middlewares:** Autenticação, logging, correlation-id, rate-limiting.

5.3 AsyncIO

- **Event loop:** Gerenciado pelo FastAPI.
- **Timeouts:** Usar `asyncio.timeout` ou `asyncio.wait\_for`.
- **Tarefas em background:** Usar `asyncio.create\_task` com tratamento de exceções.

5.4 Pydantic

- **Modelos:** Usar `BaseModel`` para DTOs, schemas de API, schemas de eventos.
- **Validação:** `field_validator`` e `model_validator``.
- **Configuração:** `extra = "forbid"` para evitar campos extras.

5.5 SQLAlchemy

- **Versão:** 2.0+.
- **Modo assíncrono:** `asyncpg`` + `async_sqlalchemy``.
- **Models:** Definidos como `Mapped`` com type hints.

5.6 Alembic

- **Migrações:** Geradas automaticamente via `alembic revision --autogenerate``.
- **Política:** Nunca editar migrações já mergeadas; criar nova.
- **Diretório:** `src/<context>/infrastructure/migrations``.

5.7 Dependency Injection

- **Manual:** Injeção via construtor (preferida) ou via FastAPI DI.
- **Injeção de dependências de infraestrutura:** Feita no momento da composição do aplicativo.

5.8 ABC e Protocols

- Usar `ABC`` para classes base com métodos abstratos.
- Usar `Protocol`` para duck typing estrutural.

5.9 Generics

- Usar `TypeVar`` e `Generic`` para repositórios, serviços e utilitários reutilizáveis.

5.10 Enums

- Usar `StrEnum`` para valores com representação string.

5.11 Value Objects

- Imutáveis, com validação no `__init__`` ou `__post_init__``.
- Implementar `__eq__``, `__hash__`` quando apropriado.

5.12 Factories

- Usar métodos estáticos ou funções para criação complexa de objetos.

5.13 Repositories

- Interfaces no domínio, implementações na infraestrutura.
- Métodos assíncronos.

5.14 Application Services

- Orquestram casos de uso, usam repositórios e serviços de domínio.

5.15 Domain Services

- Encapsulam lógica de negócio que não pertence a uma única entidade.

5.16 Aggregates

- Raiz do agregado gerencia consistência transacional.

- Expor métodos para mutações, em vez de setters públicos.

5.17 Entities

- Têm identidade (UUID) e ciclo de vida.

5.18 DTOs

- Usados para entrada/saída de APIs e entre camadas.
- Podem ser Pydantic models.

5.19 Mappers

- Converter entre entidades de domínio e modelos de infraestrutura/DTOs.

5.20 Validators

- Classes ou funções para validação complexa.

5.21 Serializers

- Para eventos e Commands, usando Pydantic ou Avro.

5.22 Naming Convention

- **Classes:** PascalCase.
- **Funções/métodos:** snake_case (verbos).
- **Variáveis:** snake_case.
- **Constantes:** UPPER_SNAKE_CASE.

5.23 Imports

- Ordem: bibliotecas padrão → bibliotecas de terceiros → módulos internos.
- Evitar importações circulares.

5.24 Logging

- Usar `logging.getLogger(__name__)`.
- Logs estruturados em JSON.
- Campos obrigatórios: `timestamp`, `level`, `logger`, `message`, `correlation_id`, `tenant`.

5.25 Configuration

- Variáveis de ambiente com prefixo `GOVSEC_`.
- Arquivos de configuração em `configs/` por ambiente.
- Uso de Pydantic `BaseSettings` para carregar configurações.

5.26 Error Handling

- Hierarquia de exceções (seção 7 do M1.0).
- Exceções de domínio devem ser capturadas e convertidas em respostas HTTP apropriadas.

5.27 Thread Safety

- Código async é single-threaded por event loop; mas ao usar threads (CPU-bound), usar `concurrent.futures` com cuidado.
- Evitar estado compartilhado mutável entre tarefas.

5.28 Performance

- Evitar loops com I/O bloqueante; usar `asyncio.gather`.
- Usar connection pooling para banco e HTTP.
- Cache com Redis para dados de alta leitura.

6. Guia Oficial dos Módulos

Esta seção fornece uma especificação detalhada para cada módulo (Bounded Context) do GovSec Shield.

6.1 Core Platform

****Objetivo:**** Fornecer a fundação da plataforma, incluindo o Command Bus, Event Bus, Security Kernel, Policy Engine, e serviços de configuração.

****Responsabilidades:****

- Gerenciar o ciclo de vida do sistema.
- Prover mecanismos de autenticação e autorização.
- Avaliar políticas OPA/Cedar.
- Prover infraestrutura de mensageria.

****Limites:**** Não contém lógica de domínio específica de outros contextos.

****Interfaces Públicas:****

- `CommandBus` para enviar Commands.
- `EventBus` para publicar eventos.
- `SecurityKernel` para autenticar e autorizar.
- `PolicyEngine` para avaliar políticas.

****Eventos Produzidos:****

- `TenantCreatedEvent`
- `TenantUpdatedEvent`
- `UserLoggedEvent`
- `PolicyEvaluatedEvent`

****Eventos Consumidos:**** Nenhum.

****Commands:****

- `CreateTenantCommand`
- `UpdateTenantCommand`
- `ActivateTenantCommand`
- `DeactivateTenantCommand`

****Queries:****

- `GetTenantByIdQuery`
- `ListTenantsQuery`

****Dependências:**** Redpanda, PostgreSQL, Vault.

****Fluxos:**** Ver seção 7.

6.2 Security Kernel

****Objetivo:**** Gerenciar identidades, autenticação, autorização e auditoria.

****Responsabilidades:****

- Autenticação (JWT, OAuth2).
- Autorização (RBAC/ABAC) usando OPA.
- Auditoria de ações.
- Gestão de sessões.

****Limites:**** Não conhece detalhes de domínio de outros contextos.

****Interfaces Públicas:****

- `authenticate(credentials) -> JWT`
- `authorize(user, action, resource) -> bool`
- `audit\_log(action, metadata)`

****Eventos Produzidos:****

- `UserAuthenticatedEvent`
- `AuthorizationDeniedEvent`
- `UserRoleChangedEvent`

****Eventos Consumidos:**** Nenhum.

****Commands:****

- `AuthenticateUserCommand`
- `RefreshTokenCommand`
- `UpdateUserRoleCommand`
- `CreateUserCommand`

****Dependências:**** Core Platform, Vault, Redis (sessões).

****Fluxos:**** Ver seção 7.

6.3 Incident Context

****Objetivo:**** Gerenciar incidentes de segurança, incluindo criação, atualização, escalonamento e resolução.

****Responsabilidades:****

- Criação de incidentes a partir de eventos de correlação.
- Atualização de status e atribuição.
- Escalonamento baseado em severidade.

- Encerramento e resolução.

****Limites:**** Não executa ações de resposta (delega ao Response Context via eventos).

****Interfaces Públicas:****

- `create\_incident(command)`
- `update\_incident(command)`
- `escalate\_incident(command)`

****Eventos Produzidos:****

- `IncidentCreatedEvent`
- `IncidentUpdatedEvent`
- `IncidentEscalatedEvent`
- `IncidentResolvedEvent`
- `IncidentClosedEvent`

****Eventos Consumidos:****

- `SecurityAlertEvent` (de Security Context)
- `RiskScoreUpdatedEvent` (de Risk Context)

****Commands:****

- `CreateIncidentCommand`
- `UpdateIncidentCommand`
- `EscalateIncidentCommand`
- `AssignIncidentCommand`
- `ResolveIncidentCommand`
- `CloseIncidentCommand`

****Dependências:**** Core Platform, Security Kernel, Event Bus.

****Fluxos:**** Ver seção 7.

6.4 Security Context

****Objetivo:**** Gerenciar políticas de segurança, regras de firewall, bloqueios e whitelists.

****Responsabilidades:****

- Aplicação de políticas (ex: `BlockIPCommand`).
- Gestão de regras de segurança.
- Aplicação de whitelists e blacklists.

****Limites:**** Não lida com incidentes (apenas com políticas e ações de segurança).

****Interfaces Públicas:****

- `block\_ip(command)`
- `unblock\_ip(command)`
- `apply\_policy(command)`

****Eventos Produzidos:****

- `IPBlockedEvent`
- `IPUnblockedEvent`
- `PolicyAppliedEvent`
- `PolicyRevertedEvent`

****Eventos Consumidos:****

- `IncidentEscalatedEvent` (pode disparar bloqueio automático)

****Commands:****

- `BlockIPCommand`
- `UnblockIPCommand`
- `ApplyPolicyCommand`
- `RevertPolicyCommand`
- `AddToWhitelistCommand`
- `RemoveFromWhitelistCommand`

****Dependências:**** Core Platform, Security Kernel, Asset Context (para isolar ativos).

****Fluxos:**** Ver seção 7.

6.5 Asset Context

****Objetivo:**** Gerenciar o inventário de ativos (servidores, endpoints, aplicações) e sua descoberta.

****Responsabilidades:****

- Descoberta automática de ativos.
- Manutenção do inventário.
- Monitoramento de health dos ativos.

****Limites:**** Não contém lógica de correlação ou resposta.

****Interfaces Públicas:****

- `discover\_assets(command)`
- `get\_asset(query)`

****Eventos Produzidos:****

- `AssetDiscoveredEvent`
- `AssetUpdatedEvent`
- `AssetDeletedEvent`

****Eventos Consumidos:****

- `NetworkScanCompletedEvent` (de Conectores)

****Commands:****

- `DiscoverAssetCommand`
- `ScanNetworkCommand`

- `UpdateAssetCommand`

****Dependências:**** Core Platform, Conectores (para scanning).

****Fluxos:**** Ver seção 7.

6.6 Risk Context

****Objetivo:**** Avaliar riscos de ativos, incidentes e configurações, fornecendo pontuações de risco.

****Responsabilidades:****

- Avaliação de risco dinâmica baseada em vulnerabilidades, ameaças e contexto.
- Atualização de scores de risco.
- Notificação de aumento de risco.

****Limites:**** Não executa ações de mitigação (delega ao Response Context).

****Interfaces Públicas:****

- `assess\_risk(command)`
- `get\_risk\_score(query)`

****Eventos Produzidos:****

- `RiskAssessedEvent`
- `RiskScoreChangedEvent`

****Eventos Consumidos:****

- `AssetDiscoveredEvent`
- `VulnerabilityDetectedEvent`
- `ThreatIntelUpdatedEvent`

****Commands:****

- `AssessRiskCommand`
- `MitigateRiskCommand` (apenas planejamento)
- `RecalculateRiskCommand`

****Dependências:**** Intelligence Context (para threat intel), Asset Context.

****Fluxos:**** Ver seção 7.

6.7 Response Context

****Objetivo:**** Executar ações de resposta a incidentes, incluindo playbooks e automação.

****Responsabilidades:****

- Execução de playbooks (manuais ou automáticos).
- Isolamento de ativos.
- Containment de ameaças.

- Rollback de ações.

****Limites:**** Não define políticas de segurança (usa Security Context).

****Interfaces Públicas:****

- `execute\_playbook(command)`
- `isolate\_asset(command)`
- `reintegrate\_asset(command)`

****Eventos Produzidos:****

- `PlaybookExecutedEvent`
- `AssetIsolatedEvent`
- `AssetReintegratedEvent`
- `ContainmentTriggeredEvent`

****Eventos Consumidos:****

- `IncidentEscalatedEvent`
- `RiskScoreChangedEvent` (se alto risco, pode disparar contenção)

****Commands:****

- `ExecutePlaybookCommand`
- `IsolateAssetCommand`
- `ReintegrateAssetCommand`
- `ContainThreatCommand`

****Dependências:**** Core Platform, Security Kernel, Asset Context, Security Context.

****Fluxos:**** Ver seção 7.

6.8 Intelligence Context

****Objetivo:**** Gerenciar inteligência de ameaças (Threat Intelligence) e indicadores de comprometimento (IoCs).

****Responsabilidades:****

- Ingestão de feeds de ameaças (STIX/TAXII).
- Enriquecimento de eventos com IoCs.
- Atualização da Knowledge Base.

****Limites:**** Não executa correlação (apenas fornece dados enriquecidos).

****Interfaces Públicas:****

- `update\_threat\_intel(command)`
- `store\_indicator(command)`
- `enrich\_event(event)`

****Eventos Produzidos:****

- `ThreatIntelUpdatedEvent`

- `IndicatorMatchedEvent`

****Eventos Consumidos:****

- `AssetDiscoveredEvent` (para enriquecer com dados de TI)

****Commands:****

- `UpdateThreatIntelCommand`

- `StoreIndicatorCommand`

- `EnrichEventCommand`

****Dependências:**** Core Platform, Knowledge Base (PostgreSQL vetorial ou Neo4j).

****Fluxos:**** Ver seção 7.

6.9 AI Context

****Objetivo:**** Fornecer capacidades de IA, incluindo Advisory (sugestões) e Copilot (assistente).

****Responsabilidades:****

- Gerar sugestões para analistas (Suggestion).
- Responder a perguntas via linguagem natural (Copilot).
- Integrar com LLMs e RAG.

****Limites:**** Não executa ações automáticas sem aprovação humana (até M9).

****Interfaces Públicas:****

- `request\_suggestion(command)`

- `apply\_suggestion(command)`

- `chat\_query(query)`

****Eventos Produzidos:****

- `SuggestionGeneratedEvent`

- `SuggestionAcceptedEvent`

- `CopilotInteractionEvent`

****Eventos Consumidos:****

- `IncidentCreatedEvent` (para gerar sugestões)

- `RiskScoreChangedEvent`

****Commands:****

- `RequestSuggestionCommand`

- `ApplySuggestionCommand` (com `requires\_approval=true` até M9)

- `ChatQueryCommand` (para Copilot)

****Dependências:**** Core Platform, Knowledge Base, AI Gateway (LLM).

****Fluxos:**** Ver seção 7.

6.10 Monitoring Context

****Objetivo:**** Coletar métricas, logs e eventos de segurança, e monitorar a saúde do sistema.

****Responsabilidades:****

- Coleta de métricas de ativos e serviços.
- Agregação e armazenamento de logs.
- Geração de alertas baseados em métricas.

****Limites:**** Não realiza correlação (apenas monitoramento).

****Interfaces Públicas:****

- `start\_monitoring(command)`
- `collect\_metrics(command)`
- `get\_alert\_status(query)`

****Eventos Produzidos:****

- `MonitoringStartedEvent`
- `AlertFiredEvent`
- `AlertResolvedEvent`

****Eventos Consumidos:**** Nenhum.

****Commands:****

- `StartMonitoringCommand`
- `StopMonitoringCommand`
- `CollectMetricsCommand`

****Dependências:**** Core Platform, Prometheus, Grafana.

****Fluxos:**** Ver seção 7.

6.11 Connectors (Módulo de Integrações)

****Objetivo:**** Ingerir dados de fontes externas (firewalls, SIEMs, cloud providers) e normalizá-los para o formato canônico.

****Responsabilidades:****

- Conectar-se a sistemas externos via APIs, syslog, etc.
- Normalizar dados para o formato interno (eventos).
- Publicar eventos normalizados.

****Limites:**** Não contém lógica de negócio ou correlação.

****Interfaces Públicas:****

- `ingest(command)` – recebe dados brutos e publica eventos.

****Eventos Produzidos:****

- `IngestionStartedEvent`
- `LogIngestedEvent` (normalizado)
- `ConnectorFailedEvent`

****Eventos Consumidos:**** Nenhum.

****Commands:****

- `IngestLogCommand`
- `StopIngestionCommand`

****Dependências:**** Core Platform, Security Kernel.

****Fluxos:**** Ver seção 7.

6.12 Dashboard (UI)

****Objetivo:**** Prover a interface do analista e administrador.

****Responsabilidades:****

- Exibir incidentes, ativos, riscos e métricas.
- Permitir execução de Commands.
- Interagir com AI Advisor e Copilot.

****Limites:**** Não contém lógica de negócio (apenas apresentação e chamadas de API).

****Interfaces Públicas:**** Frontend (React/Next.js) consumindo APIs REST.

****Eventos Consumidos:**** WebSockets para notificações em tempo real.

****Dependências:**** API Gateway.

****Fluxos:**** Ver seção 7.

7. Fluxos Técnicos

7.1 Inicialização da Plataforma (Boot)

```
```mermaid
sequenceDiagram
 participant Init
 participant Config
 participant Database
 participant Kafka
 participant Services
```

```

Init-->>Config: Carregar configurações
Config-->>Init: OK
Init-->>Database: Verificar conectividade
Database-->>Init: OK
Init-->>Kafka: Verificar conectividade
Kafka-->>Init: OK
Init-->>Services: Inicializar serviços (DI)
Services-->>Init: OK
Init-->>Health: Registrar endpoints /healthz, /ready
Health-->>Init: OK
Init-->>Platform: Sistema pronto
...

```

### ### 7.2 Descoberta de Ativos

```

```mermaid
flowchart TD
    A[Comando DiscoverAssetCommand] --> B[Asset Handler]
    B --> C[Conector de Rede]
    C --> D[Executa scan]
    D --> E[Publica AssetDiscoveredEvent]
    E --> F[Asset Context processa]
    F --> G[Atualiza inventário]
...

```

7.3 Processamento de Eventos e Correlação

```

```mermaid
flowchart LR
 A[Connector] -->|IngestLogCommand| B[Event Bus]
 B --> C[Correlation Engine]
 C --> D{Regras de Correlação}
 D -->|Match| E[Gera IncidentCorrelatedEvent]
 E --> F[Incident Context]
 F --> G[Cria Incidente]
...

```

### ### 7.4 Autorização de Command

```

```mermaid
sequenceDiagram
    participant Client
    participant API
    participant SecurityKernel
    participant PolicyEngine
    participant Handler

```



```

Client->>API: Command
API->>SecurityKernel: authorize(user, command)
SecurityKernel->>PolicyEngine: evaluate(command, context)
PolicyEngine-->>SecurityKernel: Permitido/Negado
alt Permitido
    SecurityKernel->>Handler: execute(command)
    Handler-->>API: Resultado
    API-->>Client: OK
else Negado
    SecurityKernel-->>API: Erro 403
    API-->>Client: Forbidden
end
...

```

7.5 Correlação e Risk Engine

```

```mermaid
flowchart TD
 A[Evento de Log] --> B[Correlation Engine]
 B --> C{Correlação detectada?}
 C -->|Sim| D[Gera Incidente]
 D --> E[Risk Engine]
 E --> F{Avalia Score}
 F -->|Alto| G[Dispara Escalonamento]
 G --> H[Incident Escalado]
...

```

### ### 7.6 AI Gateway e Suggestion

```

```mermaid
sequenceDiagram
    participant User
    participant Dashboard
    participant AI Gateway
    participant KnowledgeBase
    participant IncidentContext

    User->>Dashboard: Solicita sugestão para incidente
    Dashboard->>AI Gateway: RequestSuggestionCommand
    AI Gateway->>KnowledgeBase: Buscar contexto
    KnowledgeBase-->>AI Gateway: Contexto
    AI Gateway->>AI Gateway: Gerar sugestão (LLM)
    AI Gateway-->>Dashboard: Suggestion
    Dashboard-->>User: Exibe sugestão
    User->>Dashboard: Aceita sugestão
    Dashboard->>IncidentContext: ApplySuggestionCommand
    IncidentContext-->>Dashboard: OK
...

```

7.7 Playbooks e Automação

```
```mermaid
flowchart TD
 A[Incident Escalado] --> B{Playbook Automático?}
 B -->|Sim| C[ExecutePlaybookCommand]
 C --> D[Isolar Ativo]
 D --> E[Bloquear IP]
 E --> F[Notificar Equipe]
 F --> G[Registrar Ação]
 G --> H[Incident Atualizado]
 B -->|Não| I[Notificar Analista]
```
```

7.8 Rollback

```
```mermaid
flowchart TD
 A[Command Executado] --> B{Falha?}
 B -->|Sim| C{Compensating Command disponível?}
 C -->|Sim| D[Executar Compensating Command]
 D --> E[Publicar Evento de Rollback]
 C -->|Não| F[Registrar falha e notificar]
 B -->|Não| G[Sucesso]
```
```

7.9 Shutdown e Recovery

```
```mermaid
flowchart TD
 A[Sinal de Shutdown] --> B[Drenar Filas de Kafka]
 B --> C[Finalizar Handlers em andamento]
 C --> D[Fechar Conexões com Banco]
 D --> E[Liberar Recursos]
 E --> F[Pod Terminado]
```
```

7.10 Disaster Recovery

- **Backup diário** de PostgreSQL.
- **Replicação multi-zona** do Redpanda.
- **Restauração**: usar backups + replay de eventos.
- **Procedimento** documentado em runbook.

8. Guia Oficial dos Eventos

8.1 Estrutura do Evento

Todo evento deve seguir o formato canônico definido em M0.6:

```
```json
{
 "id": "uuid",
 "type": "IncidentCreatedEvent",
 "version": "1.0.0",
 "tenant": "gov-abc",
 "correlation_id": "corr-123",
 "timestamp": "2026-07-28T10:30:00Z",
 "payload": { ... }
}
```
```

8.2 Versionamento

- SemVer (MAJOR.MINOR.PATCH).
- Mudanças compatíveis: MINOR.
- Mudanças incompatíveis: MAJOR.

8.3 Schema Validation

- Para eventos, schema Pydantic ou Avro.
- Schema Registry (ex: Apicurio) é recomendado.

8.4 Envelopamento

- Eventos são publicados com metadados (id, type, version, tenant, correlation_id, timestamp) e payload.

8.5 Serialização

- **Padrão:** JSON.
- **Alto throughput:** Avro com Schema Registry.

8.6 Replay e Retry

- Replay de eventos: possível a partir do Event Store.
- Retry: consumidores devem usar retry com backoff.

8.7 Dead Letter Queue (DLQ)

- Eventos que falham após retries vão para `govsec.events.dlq`.
- Monitorar e auditar DLQ.

8.8 Retention

- Eventos armazenados por 30 dias (padrão).
- Configurável por tipo de evento.

8.9 Partition Key

- ****Partition key = tenant**** para garantir ordenação por tenant.
- Se necessário, usar `tenant + correlation\_id` para agrupamento.

8.10 Correlation Key

- Obrigatório em todo evento.
- Usado para rastrear fluxos end-to-end.

8.11 Tracing

- Injetar `trace\_id` e `span\_id` do OpenTelemetry nos metadados do evento.

8.12 Exemplos Completos

Evento de Incidente Criado

```
```json
{
 "id": "123e4567-e89b-12d3-a456-426614174000",
 "type": "IncidentCreatedEvent",
 "version": "1.0.0",
 "tenant": "gov-abc",
 "correlation_id": "corr-123e4567",
 "timestamp": "2026-07-28T10:30:00Z",
 "payload": {
 "incident_id": "inc-123",
 "title": "Alta taxa de falhas de login",
 "severity": "high",
 "status": "open",
 "assets": ["asset-456"],
 "created_by": "user@domain.com"
 }
}
```
```

9. Guia Oficial de APIs

9.1 REST

- **Padrão RESTful** com recursos e verbos HTTP.
- **Versionamento via URL**: `/api/v1/resource`, `/api/v2/resource`.

9.2 OpenAPI

- Gerado automaticamente pelo FastAPI.
- Disponível em `/docs` e `/openapi.json`.
- Deve ser mantido atualizado.

9.3 JSON

- Todas as respostas e requisições em JSON.
- Content-Type: `application/json`.

9.4 Paginação

- `?skip=0&limit=20`.
- Resposta inclui `total`, `skip`, `limit`, `next`.

9.5 Filtros e Ordenação

- Filtros: `?field=value`.
- Ordenação: `?sort_by=created_at&order=desc`.

9.6 Headers

| Header | Descrição |
|-------------------------------|-------------------------------|
| <code>Authorization</code> | Bearer JWT |
| <code>X-Correlation-ID</code> | Obrigatório para rastreamento |
| <code>X-Tenant-ID</code> | Obrigatório |
| <code>Trace-ID</code> | Opcional |

9.7 JWT e OAuth2

- **Autenticação**: JWT Bearer.
- **OAuth2**: suporte para provedores externos (Google, Microsoft, etc.).
- **Refresh Token**: suportado.

9.8 Rate Limit

- Por tenant, por usuário, por endpoint.
- Headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`.

9.9 Status Codes

- 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500.

9.10 RFC 7807 (Problem Details)

- Todos os erros devem seguir este padrão.

9.11 Exemplo de Endpoint

POST /api/v1/incidents

****Request Headers:****

...

Authorization: Bearer <jwt>

X-Correlation-ID: corr-123

X-Tenant-ID: gov-abc

...

****Body:****

``json

{

"title": "Incidente de exemplo",

"severity": "high",

"description": "Descrição do incidente"

}

...

****Response (201 Created):****

``json

{

"id": "inc-123",

"title": "Incidente de exemplo",

"severity": "high",

"status": "open",

"created_at": "2026-07-28T10:30:00Z"

}

...

10. Banco de Dados

10.1 PostgreSQL

- ****Versão:**** 16+.
- ****Driver:**** `asyncpg` (para async).
- ****ORM:**** SQLAlchemy 2.0+.

10.2 SQLAlchemy

- ****Models:**** Definidos em `infrastructure/models.py` ou separados por contexto.

- **Mapping:** Uso de `Mapped` e `mapped\_column`.

10.3 Alembic

- **Migrações:** Geradas automaticamente e versionadas.
- **Diretório:** `src/<context>/infrastructure/migrations`.

10.4 Repository Pattern

- **Interfaces** no domínio.
- **Implementações** na infraestrutura.

10.5 Unit of Work

- **UoW** gerencia transações e repositórios.
- **Flush/Commit** no final do use case.

10.6 Optimistic Locking

- Adicionar campo `version` e verificar no update.

10.7 JSONB

- Para dados semiestruturados (ex: `metadata`, `context`, `extensions`).
- Indexar com GIN.

10.8 Índices

- Criar índices para queries comuns (ex: `tenant`, `status`, `created\_at`).
- Revisar com `EXPLAIN`.

10.9 Performance

- Evitar N+1 queries.
- Usar `selectinload` para eager loading.

10.10 Boas Práticas

- Usar transações para operações de escrita.
- Utilizar migrations para mudanças de schema.
- Nunca fazer alterações manuais no banco em produção.

11. Observabilidade

11.1 Logs

- **Estruturados** (JSON) com campos obrigatórios.
- **Níveis:** DEBUG, INFO, WARNING, ERROR, CRITICAL.
- **Correlação:** incluir `correlation\_id`, `trace\_id`, `span\_id`.

11.2 Tracing

- **OpenTelemetry** para distributed tracing.
- Exportar para Jaeger/Tempo.
- 100% de amostragem para Commands e eventos críticos.

11.3 Metrics

- **Prometheus** para métricas.
- Golden Signals: latência, tráfego, erros, saturação.
- Métricas customizadas: commands processados, eventos emitidos, lag de filas.

11.4 Health Checks

- `/healthz`: liveness.
- `/ready`: readiness (verifica dependências).
- `/startup`: startup.

11.5 Dashboards

- **Grafana** com dashboards padrão para cada serviço.
- Incluir: recursos do sistema, latência, erros, throughput.

11.6 Alertas

- **Alertmanager** para regras de alerta.
- Integração com PagerDuty/Slack.
- Alertas para: serviço down, alta latência, alto erro, lag excessivo, etc.

12. Segurança

12.1 Vault

- Gerenciamento de secrets (credenciais de banco, API keys, certificados).
- Rotação automática.

12.2 Secrets

- **Nunca** armazenar secrets em código ou variáveis de ambiente.
- Usar Vault para injetar secrets no runtime.

12.3 JWT e OAuth2

- ****Autenticação:**** JWT com assinatura RS256/ES256.
- ****Refresh Tokens:**** armazenados em HTTP-only cookies ou em Redis.

12.4 RBAC e ABAC

- ****RBAC**** baseado em roles: viewer, analyst, engineer, security_admin, system_admin.
- ****ABAC**** (opcional) via OPA com políticas dinâmicas.

12.5 OPA

- Políticas de autorização centralizadas.
- Avaliadas antes de cada Command e rota.

12.6 mTLS e TLS

- ****TLS 1.3**** para todas as conexões externas.
- ****mTLS**** para comunicação East-West.

12.7 LGPD/GDPR

- Mascaramento de dados sensíveis em logs.
- Direitos de exclusão e portabilidade suportados.

12.8 Auditoria

- Todos os Commands e ações sensíveis geram eventos de auditoria.
- Logs de auditoria armazenados por 7 anos.

12.9 Criptografia

- Dados sensíveis em repouso criptografados (AES-256).
- Criptografia em trânsito via TLS.

12.10 Hardening

- Remoção de serviços desnecessários.
- Princípio do menor privilégio.
- Atualizações regulares de segurança.

12.11 Threat Modeling

- Realizar STRIDE por serviço.
- Revisar anualmente.

13. Testes

13.1 Unitários

- Framework: `pytest` + `pytest-asyncio`.
- Cobertura mínima: 80% linhas, 70% branches.
- Executar no CI para cada PR.

13.2 Integração

- Usar `testcontainers` para PostgreSQL e Kafka.
- Testar fluxos completos (Command → Handler → DB → Event).

13.3 Contratos (Pact)

- **Consumer-driven contracts**.
- Para APIs HTTP e eventos.
- Executar no CI; falha bloqueia merge.

13.4 Carga e Performance

- Ferramenta: `k6`.
- Cenário: 10k req/s.
- Métricas: latência p95, throughput, erros.

13.5 Stress e Chaos

- **Stress**: além da capacidade.
- **Chaos Engineering**: simular falhas de rede, pods, etc.
- Ferramentas: `chaostoolkit`, `Gremlin`.

13.6 Mutation Testing

- Ferramenta: `mutmut` ou `pytest-mutation`.
- Semanalmente, para avaliar qualidade dos testes.

13.7 Estratégias

- **TDD** encorajado.
- Escrever testes antes do código.
- Usar fakes para repositórios em testes unitários.

13.8 Pipelines

- Testes executados em todos os PRs e no merge para `main`.

14. DevSecOps

14.1 GitHub Actions

- Workflows: `ci.yml`, `cd.yml`, `security.yml`, `nightly.yml`.
- **CI**: lint, test, build, security scan.
- **CD**: deploy via ArgoCD (GitOps).

14.2 Quality Gates

- Cobertura > 80%.
- Sem code smells críticos (SonarQube).
- Complexidade ciclomática < 10.
- Maintainability Index > 65.

14.3 Security Gates

- SAST: CodeQL, Semgrep.
- DAST: OWASP ZAP (em staging).
- Dependency scan: Snyk.
- Container scan: Trivy.

14.4 Deploy

- **Staging**: automático após merge.
- **Production**: promoção manual via ArgoCD (ou com aprovação).

14.5 Rollback

- ArgoCD pode reverter para commit anterior.
- Feature flags para rollback granular.

14.6 Versionamento

- SemVer conforme M0.5.
- Tags Git para releases.

14.7 Containers e Kubernetes

- **Docker**: imagens baseadas em slim/alpine.
- **Kubernetes**: manifests via Helm.
- **GitOps**: ArgoCD sync.

14.8 Terraform

- Infraestrutura em nuvem (VPC, subnets, clusters) provisionada via Terraform.

15. Padrões de Documentação

15.1 README

- Cada módulo/serviço deve ter README com: propósito, dependências, setup, teste, deploy.

15.2 ADR

- Arquitetura Decision Records em `docs/adr`.
- Status: Proposed, Accepted, Deprecated, Superseded.

15.3 RFC

- Request for Comments para mudanças maiores.
- Inclui: motivação, design, alternativas.

15.4 Runbooks

- Procedimentos operacionais: como reiniciar, escalar, diagnosticar.

15.5 Playbooks

- Procedimentos de resposta a incidentes.

15.6 Diagramas C4

- Visão geral do sistema (C1), contêineres (C2), componentes (C3), código (C4).

15.7 Mermaid

- Utilizar Mermaid para diagramas em Markdown.

15.8 OpenAPI

- Gerado automaticamente, manter acessível.

15.9 Comentários e Docstrings

- Docstrings para classes e funções públicas.
- Comentários para lógica complexa.

16. Checklist Oficial

16.1 Checklist para Novo Módulo

- [] Estrutura de diretórios criada.

- ☐ Interfaces de repositórios/domain services definidas.
- ☐ Handlers de Command/Query implementados.
- ☐ Rotas HTTP definidas.
- ☐ Testes unitários > 80%.
- ☐ Testes de integração para fluxos críticos.
- ☐ README.
- ☐ ADR (se houver mudança arquitetural).
- ☐ OpenTelemetry instrumentado.
- ☐ Security kernel integrado.

16.2 Checklist para Nova Sprint

- ☐ Backlog refinado.
- ☐ DoR preenchido para todas as histórias.
- ☐ Capacidade do time alocada.
- ☐ Dependências externas resolvidas.
- ☐ Riscos identificados.
- ☐ Planejamento de testes.

16.3 Checklist para Nova API

- ☐ Versionada.
- ☐ OpenAPI documentado.
- ☐ Validação Pydantic.
- ☐ Autenticação e autorização.
- ☐ Headers obrigatórios.
- ☐ Tratamento de erros RFC 7807.
- ☐ Testes de integração.
- ☐ Testes de contrato (Pact).

16.4 Checklist para Novo Event

- ☐ Schema definido (Pydantic/Avro).
- ☐ Versionado.
- ☐ Envelope padrão.
- ☐ Partition Key definida.
- ☐ Testes de serialização/desserialização.
- ☐ Testes de contrato.

16.5 Checklist para Novo Command

- ☐ DTO definido.
- ☐ Handler implementado.
- ☐ Registrado no Command Bus.
- ☐ Validação de domínio.
- ☐ Eventos emitidos.
- ☐ Testes unitários.
- ☐ Testes de integração.
- ☐ Idempotência garantida.
- ☐ Auditoria registrada.

16.6 Checklist para Nova Query

- [] DTO de resposta definido.
- [] Query Handler implementado.
- [] Registrado no Query Bus.
- [] Testes unitários.
- [] Otimização de índices (se aplicável).

16.7 Checklist para Novo Connector

- [] Integração com fonte externa.
- [] Normalização de dados.
- [] Tratamento de erros (retry, DLQ).
- [] Observabilidade.
- [] Testes de integração com mock.

16.8 Checklist para Novo Scanner (segurança)

- [] Execução no pipeline.
- [] Relatórios gerados.
- [] Notificações configuradas.
- [] Integração com ferramenta (ex: Trivy).

16.9 Checklist para Migration

- [] Alembic revision criada.
- [] Testada localmente.
- [] Verificada com `alembic upgrade head` e `downgrade -1`.
- [] Documentada (se mudança de schema complexa).
- [] Executada em staging antes da produção.

16.10 Checklist para Release

- [] Release branch criada.
- [] Testes completos (performance, segurança, integração).
- [] Release notes preparadas.
- [] Breaking changes documentadas.
- [] Aprovação do AB e CISO.
- [] Tag criada.
- [] Artefatos publicados.

16.11 Checklist para Deploy

- [] Pipeline CI/CD verde.
- [] Testes de smoke executados no staging.
- [] Aprovação manual (se exigido).
- [] Rollback planejado.

17. Catálogo Oficial de Anti-Patterns

| Anti-Pattern | Descrição | Consequência | Mitigação |
|--------------|-----------|--------------|-----------|
| :--- | :--- | :--- | :--- |

| ****Acoplamento entre Contexts**** | Importações diretas entre Bounded Contexts. | Viola DDD; torna mudanças difíceis. | Usar eventos e Command Bus. |
| ****Importações Cruzadas**** | Importações entre camadas (ex: infra importa domínio de outro módulo). | Cria dependências cíclicas. | Seguir regras de importação. |
| ****Dependências Cíclicas**** | Módulo A depende de B, e B depende de A. | Quebra a arquitetura; difícil de testar. | Refatorar para remover dependência. |
| ****Hardcode**** | Valores fixos no código (ex: URLs, timeouts). | Dificulta configuração e manutenção. | Usar variáveis de ambiente e configuração. |
| ****Secrets no Código**** | Chaves e senhas no código-fonte. | Risco de segurança crítico. | Usar Vault e nunca commitar secrets. |
| ****Eventos sem Versionamento**** | Eventos sem versão ou com mudanças breaking. | Quebra consumidores. | Versionar eventos (SemVer). |
| ****Commands sem Autorização**** | Handlers que não verificam permissões. | Vulnerabilidade de segurança. | Usar Security Kernel/OPA. |
| ****Queries Alterando Estado**** | Query que modifica dados. | Viola CQRS. | Separar Commands e Queries. |
| ****Shared Database**** | Múltiplos Contextos usando a mesma tabela. | Viola DDD, cria dependência de schema. | Cada Contexto tem seu próprio schema. |
| ****Lógica de Negócio em Connectors**** | Regras de negócio no conector. | Dificulta manutenção e testes. | Conector apenas normaliza e publica eventos. |
| ****Duplicação**** | Código repetido. | Viola DRY. | Extrair para shared/utils. |
| ****Código Síncrono Bloqueante**** | Uso de `time.sleep`, chamadas bloqueantes em async. | Degrada performance. | Usar `asyncio.sleep` e operações assíncronas. |
| ****Ausência de Telemetria**** | Falta de métricas/traces/logs. | Sistema "cego"; difícil diagnosticar. | Instrumentar com OpenTelemetry. |
| ****Ausência de Testes**** | Código não testado. | Bugs em produção. | Exigir cobertura mínima. |
| ****Magic Numbers**** | Números mágicos sem constante. | Dificulta manutenção. | Usar constantes com nomes descritivos. |
| ****Magic Strings**** | Strings literais sem constante. | Dificulta manutenção. | Usar enums ou constantes. |
| ****Singleton Global**** | Instância global compartilhada. | Dificulta testes e acoplamento. | Usar injeção de dependência. |
| ****God Classes**** | Classes muito grandes com muitas responsabilidades. | Viola SRP. | Dividir em classes menores. |
| ****Spaghetti Code**** | Lógica emaranhada com muitos ifs. | Alta complexidade. | Refatorar com padrões como Strategy, State. |

18. Guia Oficial de Performance

18.1 Concorrência

- Usar `asyncio.gather` para paralelizar tarefas I/O-bound.
- Limitar concorrência com semáforos.

18.2 Connection Pools

- **Banco:** Pool de conexões (ex: `asyncpg` com `min\_size=5`, `max\_size=20`).
- **HTTP:** Pool de clientes HTTP (ex: `httpx.AsyncClient` com limites).

18.3 Cache

- **Redis** para cache de dados de alta leitura (ex: configurações, listas de permissão).
- **Política de invalidação:** TTL ou invalidação por evento.

18.4 Otimizações

- **CPU-bound:** Usar `concurrent.futures.ThreadPoolExecutor` ou `ProcessPoolExecutor`.
- **Memória:** Evitar carregar grandes datasets; usar streaming/paginação.
- **Banco:** Usar índices apropriados; evitar N+1.
- **Kafka:** Ajustar batch size, linger.ms, compression.

18.5 Latência

- **Metas:** p95 < 100ms para Commands, p95 < 500ms para Queries.
- **Monitoramento:** via Prometheus.

18.6 Escalabilidade

- **Horizontal:** Stateless services (escalar via replicação).
- **Vertical:** Aumentar recursos para serviços stateful (ex: PostgreSQL, Kafka).

19. Plano Mestre de Implementação

19.1 Épicos, Capacidades e Sprints

| Fase | Sprints | Capacidade | Épico Principal | Descrição |
|------|---------|-----------------------------|------------------------|--|
| :--- | :--- | :--- | :--- | :--- |
| 1 | 0–4 | M0 (Core + Observability) | Core Platform | Fundação: Command Bus, Event Bus, Security Kernel, PostgreSQL, Redpanda. |
| 2 | 5–6 | M1/M2 (Observability) | Observability | Instrumentação, dashboards, alertas. |
| 3 | 7–10 | M3 (Correlation) | Correlation Engine | Motor de correlação, regras, incidentes. |
| 4 | 11–14 | M4 (Threat Intel) | Threat Intelligence | Feeds STIX/TAXII, enriquecimento, Knowledge Base. |
| 5 | 15–18 | M5/M6 (AI Advisory/Copilot) | AI Gateway | Sugestões, chat, integração com LLM. |
| 6 | 19–22 | M7 (Automation) | Playbooks & Automation | Automação, playbooks, circuit breaker. |
| 7 | 23+ | M8/M9 (Red Team/Autonomy) | Autonomy | Simulação, auto-adaptação, supervisão. |

19.2 Dependências Críticas

| Épico | Depende de | Impacta |
 | :--- | :--- | :--- |
 | Core Platform | – | Todos os outros |
 | Observability | Core Platform | Todos os outros |
 | Correlation Engine | Core Platform, Observability | Incident, Security |
 | Threat Intelligence | Correlation Engine | Risk, Intelligence |
 | AI Gateway | Threat Intelligence, Knowledge Base | Incident, Dashboard |
 | Playbooks | Correlation Engine, Security Context | Response, Asset |

19.3 Critérios de Aceite

Para cada capacidade:

- Todos os testes (unit, integration, contract) passando.
- Observabilidade configurada.
- Documentação atualizada.
- Aprovado pelo Architecture Board.

19.4 Estimativas

- **Core Platform:** 4 sprints (2 pessoas/tarefa).
- **Observability:** 2 sprints (1 pessoa).
- **Correlation Engine:** 4 sprints (3 pessoas).
- **Threat Intelligence:** 4 sprints (2 pessoas).
- **AI Gateway:** 4 sprints (2 pessoas + data scientist).
- **Playbooks:** 4 sprints (2 pessoas).
- **Autonomy:** 6+ sprints (3 pessoas).

19.5 Riscos

| Risco | Probabilidade | Impacto | Mitigação |
 | :--- | :--- | :--- | :--- |
 | Integração com Redpanda | Média | Alto | POCs antecipadas. |
 | Performance de Correlação | Média | Alto | Testes de carga contínuos. |
 | Precisão da IA | Alta | Médio | Validar com dados reais. |
 | Segurança de dados | Baixa | Crítico | Auditoria externa. |
 | Mudança de requisitos | Média | Médio | Sprints curtas, feedback contínuo. |

19.6 Roadmap Técnico

```

```mermaid
graph LR
 title Plano Mestre de Implementação
 dateFormat YYYY-MM-DD
 section Fase 1
 Sprint 0-4 (Core) :a1, 2026-08-01, 8w
 section Fase 2
 Sprint 5-6 (Observability) :a2, after a1, 4w

```

section Fase 3

Sprint 7-10 (Correlation) :a3, after a2, 8w

section Fase 4

Sprint 11-14 (Threat Intel) :a4, after a3, 8w

section Fase 5

Sprint 15-18 (AI) :a5, after a4, 8w

section Fase 6

Sprint 19-22 (Automation) :a6, after a5, 8w

section Fase 7

Sprint 23+ (Autonomy) :a7, after a6, 12w

...

---

## ## 20. Changelog

### ### 20.1 Versão 1.0.0 (2026-07-28)

- \*\*Criação do documento M2\*\* – Master Technical Implementation Specification.
- Consolida todas as diretrizes arquiteturais (M0.x, M1.0) em um guia executável.
- Inclui estrutura de monorepo, padrões de código, guia de módulos, fluxos técnicos, eventos, APIs, banco de dados, observabilidade, segurança, testes, DevSecOps, e planos de implementação.

### ### 20.2 Pendências e Débitos Técnicos

ID	Débito	Prioridade	Plano
TD-IMPL-01	Suporte a Avro para eventos de alto throughput	Média	Implementar no M4
TD-IMPL-02	Automação de testes de carga no pipeline	Baixa	Integrar no CI
TD-IMPL-03	Revisão de segurança do AI Gateway	Alta	Antes do lançamento M5

### ### 20.3 Roadmap Futuro (pós-M9)

- Adoção de \*\*edge computing\*\* para detecção em locais remotos.
- Integração com \*\*sovereign cloud\*\*.
- Melhorias de \*\*XAI\*\* (explicabilidade).

---

**\*\*Fim do Documento M2\*\***

\*Este documento é a referência definitiva para a implementação do GovSec Shield. Qualquer desvio deve ser justificado e aprovado pelo Architecture Board. A equipe de engenharia deve consultar este documento diariamente durante o desenvolvimento. Boa construção!\*